



**Hochschule
Bonn-Rhein-Sieg**
University of Applied Sciences

Fachbereich Informatik
Department of Informatik (M.Sc.)

Masterarbeit

Studiengang Master Informatik (M.Sc.)

Anwendung von Machine Learning-Methoden zur Strukturierung von Freitextdaten am Beispiel von Eignungskriterien von klinischen Studien

von

Erik Autenrieth

Erstprüfer
Zweitprüfer
Betreuer

Prof. Dr. Wolfgang Heiden
Prof. Dr. Juliane Fluck
Johannes Darms M.Sc. (RWTH)
ZB MED

eingereicht am 04. November 2024

Die Arbeit hat mit gleichem Inhalt bzw. in wesentlichen Teilen noch keiner anderen Prüfungsbehörde vorgelegen.

.....
Ort, Datum

.....
Unterschrift

Hilfsmittel

Für die Rechtschreibprüfung und Kommasetzung wurden das KI-basierte LanguageTool sowie GPT-4o eingesetzt. GPT-4o wurde bei der Formatierung von LaTeX-Tabellen und beim Plotten von Ergebnissen genutzt. Zur Identifizierung ergänzender Literatur wurde das Tool Connected Papers verwendet.

Danksagung

An dieser Stelle möchte ich mich bei allen bedanken, die mich während meines Studiums und der Erstellung dieser Arbeit begleitet und unterstützt haben. Besonderer Dank gilt meinem Betreuer Johannes, der mich stets motiviert und mit wertvollen Anregungen unterstützt hat. Seine fachliche Kompetenz und sein Engagement haben maßgeblich zum Gelingen dieser Arbeit beigetragen. Weiterhin danke ich meinen Prüfern Prof. Dr. Wolfgang Heiden und Prof. Dr. Juliane Fluck für ihre konstruktive Begleitung und ihr Interesse an meiner Arbeit. Prof. Dr. Rudolf Berrendorf danke ich für die Bereitstellung des Zugangs zum Rechencluster, der die umfangreichen Berechnungen dieser Arbeit erst ermöglicht hat. Mein ganz persönlicher Dank gilt meiner Freundin Claudia für ihr Verständnis, ihre Geduld und ihre unermüdliche Unterstützung während des Studiums. Ebenso danke ich Marcel für sein wertvolles Feedback zu dieser Arbeit.

Zusammenfassung

Klinische Studien benötigen präzise Eignungskriterien im Studienprotokoll, um geeignete Teilnehmer zu rekrutieren und die Forschungsfrage valide zu beantworten. Die unstrukturierte Dokumentation dieser Kriterien als Freitext verhindert jedoch eine effiziente automatisierte Verarbeitung. Die komplexe und verschachtelte Logik der Kriterien, ausgedrückt durch AND-, OR- und NOT-Operatoren, kann insbesondere bei der Suche nach relevanten Studien nicht systematisch ausgewertet werden. Die fehlende Strukturierung erschwert zudem die automatisierte Identifikation geeigneter Patienten für klinische Studien. Diese Arbeit untersucht den Einsatz von Large Language Models (LLMs) zur automatisierten Strukturierung von Eignungskriterien durch Überführung in eine Abstract Syntax Tree (AST) Struktur, welche die komplexen logischen Beziehungen innerhalb der Kriterien repräsentiert. Durch eine indirekte Strukturierung werden Eignungskriterien mit logischen Operatoren (AND, OR, NOT) angereichert. Für diese Aufgabe wurden die Modelle Llama3-8B und -70B, Gemma2-27B, Qwen2-72B, GPT-4o, GPT-4o-mini und Claude-3.5-Sonnet evaluiert und mit einem regelbasierten Parser verglichen, wobei In-Context Learning- und Fine-Tuning Methoden eingesetzt wurden. Die angereicherten Texte werden über einen Parser in eine AST-Struktur überführt. In 20 Experimenten zur Operatorenextraktion auf dem LCT-Korpus erreicht das Llama3-70B Modell mit Zero-Shot einen F1-Score von 50.2%, vergleichbar mit einem regelbasierten Regex-Parser (50.9%). Mit einem Parameter-Efficient Fine-Tuning übertrifft es mit 77.4% F1-Score das bisherige State-of-the-Art-Modell SciBERT. Die Modelle Llama3, Qwen2 und GPT-4o wurden eingesetzt, um Eignungskriterien aus dem LCT-Korpus direkt in die AST-Struktur zu überführen und Entitäten mittels Few-Shot und Fine-Tuning Methoden zu erkennen. Durch Fine-Tuning erreichte das Llama3-70B Modell bei der Operatorenextraktion über 20 Versuche einen durchschnittlichen F1-Score von 82.4% und bei der Entitätsextraktion 88.8%. Dies stellt eine Verbesserung von 6.5% bzw. 7.5% gegenüber dem SciBERT-Modell dar. Die Few-Shot Methoden erzielten geringere Leistungen als das SOTA-Modell, konnten die AST-Strukturen aber mit variierender Präzision erzeugen. Die Strukturierung von Eignungskriterien als Abstract Syntax Tree im JSON-Format durch Large Language Models bietet gegenüber Systemen wie LeafAI Vorteile in Lesbarkeit und Erhalt des Ursprungstexts. Obwohl für eine effektive Strukturierung ein Fine-Tuning von LLMs nötig ist, kann dies mittels Parameter-Efficient Fine-Tuning ressourceneffizient auf einer GPU erfolgen. Für Inferenzprozesse von Open-Source LLMs sind weiterhin Cluster-Ressourcen mit mehreren leistungsstarken GPUs erforderlich.

Schlüsselwörter— Eignungskriterien, IE, LLMs, ICL, Fine-Tuning

Abstract

Clinical trials require precise eligibility criteria in the study protocol to recruit suitable participants and validly answer the research question. However, the unstructured documentation of these criteria as free text prevents efficient automated processing. The complex and nested logic of the criteria, expressed through AND, OR, and NOT operators, cannot be systematically evaluated, especially when searching for relevant studies. The lack of structure also makes it difficult to automatically identify suitable patients for clinical trials. This work investigates the use of Large Language Models (LLMs) to automatically structure eligibility criteria by transforming them into an Abstract Syntax Tree (AST) structure that represents the complex logical relationships within the criteria. Through indirect structuring, the eligibility criteria are enriched with logical operators (AND, OR, NOT). For this task, the models Llama3-8B and -70B, Gemma2-27B, Qwen2-72B, GPT-4o, GPT-4o-mini, and Claude-3.5-Sonnet were evaluated and compared with a rule-based parser, using In-Context Learning and fine-tuning methods. The processed text files are transformed into a AST-structure by a AST-parser. In 20 operator-extraction experiments on the LCT-corpus, the Llama3-70B model achieved an F1-score of 50.2% with zero-shot, comparable to a rule-based regex-parser (50.9%). With a Parameter-Efficient Fine-Tuning, it surpasses the previous State-of-the-Art model SciBERT with an F1-score of 77.4%. The models Llama3, Qwen2, and GPT-4o were used to directly convert eligibility criteria from the LCT-corpus into the AST-structure and to recognize entities using few-shot and fine-tuning methods. Through fine-tuning, the Llama3-70B model achieved an average F1-score of 82.4% in operator-extraction and 88.8% in entity extraction over 20 trials. This represents an improvement of 6.5% and 7.5% respectively over the SciBERT-model. The few-shot methods achieved lower performance than the SOTA-model but were able to generate the AST structures with varying precision. Structuring eligibility criteria as an Abstract Syntax Tree in JSON-format using Large Language Models offers advantages over systems like LeafAI in terms of readability and preservation of the original text. Although fine-tuning of LLMs is necessary for effective structuring, this can be done resource-efficiently on a GPU using Parameter-Efficient Fine-Tuning. For inference processes of open-source LLMs, cluster resources with multiple high-performance GPUs are still required.

Keywords— Eligibility Criteria , IE, LLMs, ICL, Fine-Tuning

Inhaltsverzeichnis

Abbildungsverzeichnis	x
Tabellenverzeichnis	xii
Abkürzungsverzeichnis	xiii
1. Einleitung	1
1.1. Problemstellung	1
1.2. Motivation	3
1.3. Gliederung	4
2. Grundlagen zur Eignungskriterienanalyse	5
2.1. Klinische Studien	5
2.1.1. Eignungskriterien	5
2.1.2. Studienregister	6
2.1.3. Logische Suche	7
2.2. Natural Language Processing (NLP)	7
2.2.1. Named Entity Recognition (NER)	8
2.2.2. Relation Extraction (RE)	8
2.3. Evaluationsmetriken	10
3. Techniken zur maschinellen Textstrukturierung	12
3.1. Regelbasierte Methoden	12
3.2. Large Language Models (LLMs)	12
3.2.1. Transformerarchitektur	13
3.2.2. Open-Source Modelle	19
3.2.3. Closed-Source Modelle	20
3.3. Anwendung von LLMs	21
3.4. Prompting	22
3.4.1. In-Context Learning (ICL)	22
3.4.2. Zero-Shot Chain-of-Thought (CoT)	24
3.4.3. Emotional Prompting	24
3.4.4. Role Prompting	24
3.5. Fine-Tuning	25
3.5.1. Hyperparameter	26
3.5.2. Low-Rank Adaptation (LoRA)	27
3.5.3. Rank-stabilized LoRA (rsLoRA)	28
3.5.4. Quantized LoRA (QLoRA)	28
3.5.5. LoRA-Hyperparameter	29
3.6. Modellparameter	29
3.7. Herausforderungen	30
4. State-of-the-Art	31
4.1. Forschungsfelder	31
4.2. Verfügbare Korpora	32
4.3. Methoden zur Extraktion und Strukturierung	33
4.3.1. Formelle Repräsentationen	33
4.3.2. Regelbasierte Ansätze	34
4.3.3. Machine Learning-Ansätze	35
4.3.4. BERT-basierte Ansätze	36
4.3.5. Large Language Model-Ansätze	38
4.3.6. End-to-End Matching-Systeme	41
4.4. Zusammenfassung	41
5. Methodik zur Strukturierung von Eignungskriterien	43
5.1. Experimenteller Aufbau	43

5.2. Logische Operatoren: AND, OR, NOT	44
5.2.1. Abstract Syntax Tree (AST)	46
5.2.2. JSON-Format	48
5.2.3. Entitäten	51
5.2.4. Logische Funktion	52
5.3. Modelle	54
5.3.1. Ressourcen	55
5.3.2. Parameterkonfiguration	55
5.4. Indirekte Strukturierung	56
5.4.1. In-Context Learning	57
5.4.2. Fine-Tuning	61
5.4.3. Regelbasierter Parser	62
5.5. Direkte Strukturierung	63
6. Evaluation	64
6.1. Evaluation der indirekten Strukturierung	64
6.1.1. Auswertung des LCT-Korpus	67
6.1.2. Prompting	67
6.1.3. Zero-Shot Methoden	68
6.1.4. Few-Shot Methoden	70
6.1.5. Fine-Tuning	72
6.1.6. Vergleich von LLM-Methoden	73
6.1.7. Ressourcen- und Zeitaufwand	78
6.1.8. Auswertung des Chia-Korpus	79
6.2. Evaluation der direkten Strukturierung	80
6.2.1. Few-Shot	81
6.2.2. Fine-Tuning	81
6.2.3. Operatorenextraktion	81
6.2.4. Einzelnen Operatoren	83
6.2.5. Entitätsextraktion	84
6.2.6. AST-Strukturierung	88
6.2.7. Ressourcen- und Zeitaufwand	89
7. Diskussion	90
7.1. Indirekte Strukturierung	91
7.1.1. Zero-Shot	92
7.1.2. Few-Shot	93
7.1.3. Fine-Tuning	94
7.2. Chia-Korpus	95
7.3. LCT-Korpus	96
7.4. Direkte Strukturierung	97
7.5. Implikationen für die Praxis	98
7.6. Vergleich mit SOTA-Systemen	99
8. Fazit	102
8.1. Zusammenfassung	102
8.2. Beantwortung der Forschungsfragen	103
8.3. Zukünftige Arbeit	104
Literaturverzeichnis	105
Anhang	112
A. SOTA	113
A.1. Strukturierungs-Systeme	113
A.2. LCT-Korpus	115
B. Abstract Syntax Tree im JSON-Format	117
B.1. JSON-Format mit Entitäten	117

B.2. Verbundene Eignungskriterien im JSON-Format	119
C. Prompts	120
D. Ergebnisse	123
D.1. Indirekte Strukturierung	123
D.1.1. Few-Shot Validierung	123
D.1.2. Temperatur-Parameter	123
D.1.3. Chia-Korpus	123
D.2. Direkte Strukturierung	124
D.2.1. JSON-Fehler	124
D.2.2. Fine-Tuning Basisparameter	124
D.2.3. Operatorenextraktion	125
D.2.4. Entitätserkennung mit Llama3-70B	126
D.2.5. Metriken für alle LCT-Entitäten	127
E. Code	128
E.1. AST-Parser	128
E.2. Entitäten-Integration	130
E.3. Verwendung des AST-Parsers	131
E.4. JSON-Parser	132
E.5. AST-Plotter	133
E.6. Prompting	134
E.7. Fine-Tuning	136
E.8. Regex-Parser	138

Abbildungsverzeichnis

1.1. Logisch strukturierte Einschlusskriterien der Studie NCT04412629.	2
2.1. Einschlusskriterien der Studie NCT04412629.	6
2.2. Ausschlusskriterien der Studie NCT04412629.	6
2.3. AND-Verknüpfung zwischen zwei Entitäten.	9
2.4. Konfusionsmatrix.	10
3.1. Attention-Mechanismus.	13
3.2. Transformerarchitektur.	14
3.3. Entwicklungsbaum moderner Pre-trained Language Models.	17
3.4. LLM-Methoden für die Information Extraction.	21
3.5. In-Context Learning vs. Chain-of-Thought Prompting.	23
3.6. Emotionale Stimuli für Emotional Prompting.	24
3.7. Vollständiges Fine-Tuning von Large Language Models.	25
3.8. Fine-Tuning mit Low-Rank Adaptation.	27
4.1. Forschungsschwerpunkte zu Eignungskriterien.	31
4.2. GELLO-Ausdrücke in einem Arden-Syntax Modul.	34
4.3. Exemplarische Systemausgabe von EliE.	35
4.4. Strukturierung mit dem LeafAI-System.	38
4.5. Tabellarische Strukturierung mit dem AutoCriteria-System.	39
4.6. Eignungskriterien der Studie NCT04412629 in Disjunktive Normalform.	39
4.7. Logische DNF-Struktur mit GPT-4 im JSON-Format.	40
4.8. Few-Shot Prompt einer NER-Aufgabe für LLMs.	40
4.9. Logische AST-Struktur mit LeafAI.	42
4.10. Logische DNF-Struktur mit GPT-4.	42
5.1. Eignungskriterien der Studie NCT03860402 mit zusätzlichen AND-Operatoren.	45
5.2. Einschlusskriterien der Studie NCT03860402 als AST.	46
5.3. Ausschlusskriterien der Studie NCT03860402 als AST.	46
5.4. Einschlusskriterien der Studie NCT03860402 als AST mit Entitäten.	47
5.5. Ausschlusskriterien der Studie NCT03860402 als AST mit Entitäten.	47
5.6. Verknüpfte Ein- und Ausschlusskriterien der Studie NCT03860402 als AST.	52
5.7. Indirekte Strukturierung von Eignungskriterien mit LLMs.	56
5.8. Prompt 2.	57
5.9. Direkte Strukturierung von Eignungskriterien mit LLMs.	63
6.1. Eignungskriterien der Studie NCT03860402 mit logischen Operatoren.	64
6.2. Evaluation von Llama3-70B mit Zero-Shot Prompt-Methoden.	69
6.3. Evaluation von Llama3-70B mit k-Shot Methoden.	70
6.4. Evaluation von Llama3-70B mit 5-Shot Varianten.	71
6.5. F1-Scores der Operatorenextraktion mit LLMs auf dem LCT-Korpus.	75
6.6. Evaluation des Temperatur-Parameters für das Llama3-70B Modell.	78
6.7. Auswertung von Llama3-70B Methoden mit Prompt 2 auf dem Chia-Korpus.	79
6.8. F1-Scores der Operatorenextraktion mit JSON-Ausgabe.	82
6.9. Eignungskriterien der Studie NCT03860402 mit zusätzlichen AND*-Operatoren.	83
6.10. F1-Scores der Entitätsextraktion mit JSON-Ausgabe.	85
6.11. Modellevaluation von LLMs bei der Generierung von JSON-Dateien.	88
7.1. Verknüpfte Ein- und Ausschlusskriterien der Studie NCT03860402 als AST.	93
1. Beispiel von unstrukturierten Eignungskriterien.	112
A.1. Beispielhafte Darstellung der Criteria2Query-Anwendung.	113
A.2. Der Criteria2Query-3.0 Workflow.	114
A.3. Eignungskriterien der Studie NCT03860402.	115
A.4. LCT-Annotation der Studie NCT03860402.	115

C.1. Prompt 1.	120
C.2. Role Prompt.	121
C.3. Strukturprompt.	122

Tabellenverzeichnis

4.1. Öffentlich verfügbare Korpora mit annotierten Eignungskriterien.	32
4.2. Auswertung von NER-Modellen auf öffentlichen Eignungskriterien-Korpora.	41
5.1. Übersicht der ausgewählten Large Language Models.	54
5.2. Festgelegte Hyperparameter für ein Fine-Tuning von LLMs.	61
5.3. Wertebereich der Hyperparameter für ein Fine-Tuning von Llama3-8B.	62
6.1. Ausgabefehler von Llama3-70B mit Zero-Shot.	66
6.2. SOTA-Leistung auf dem LCT-Korpus für logische Relationen.	67
6.3. Auswertung der Prompt 1 mit erweiterten Prompt-Methoden.	68
6.4. Auswertung der Prompt 2 mit erweiterten Prompt-Methoden.	68
6.5. Promptevaluation mit Llama3-70B Modell mit Zero-Shot und Few-Shot.	70
6.6. Hyperparameteroptimierung des Fine-Tunings von Llama3-8B.	72
6.7. Hyperparameteroptimierung des Fine-Tunings von Llama3-70B.	73
6.8. Evaluation der Operatorenextraktion mit LLMs auf dem LCT-Korpus.	74
6.9. Evaluation von Llama3-8B Methoden für die Operatorenextraktion.	76
6.10. Evaluation von Llama3-70B Methoden für die Operatorenextraktion.	77
6.11. Kosten proprietärer LLMs für die Textgenerierung.	78
6.12. Auswertung der Operatorenextraktion der LLMs mit JSON-Ausgabe.	82
6.13. Evaluation der Operatorenextraktion für Llama3-70B mit Fine-Tuning.	84
6.14. Evaluation der Entitätsextraktion von LLMs mit JSON-Ausgaben.	84
6.15. Entitätsextraktion für ausgewählte Entitätstypen.	86
6.16. AST-Strukturierung mit Llama3-70B mit Fine-Tuning über 20 Versuche.	89
A.1. Kategorisierung der Entitätstypen des LCT-Korpus.	116
A.2. Few-Shot Beispiele aus dem LCT-Trainingsdatensatz.	116
D.1. Evaluation von Few-Shot Methoden mit Llama3-70B.	123
D.2. Evaluation der Temperatur mit Llama3-70B Methoden.	123
D.3. Vergleich der auf dem LCT-Korpus optimierten Modelle auf dem Chia-Korpus.	123
D.4. JSON-Fehler ausgewählter LLMs bei der Ausgabe von 404 JSON-Dateien.	124
D.5. SFT- und LoRA-Basiswerte für ein Fine-Tuning von LLMs.	124
D.6. Auswertung der Operatorenextraktion für die 2-Shot Modelle.	125
D.7. Auswertung der Operatorenextraktion für die 10-Shot Modelle.	125
D.8. Vergleich zwischen Llama3-70B und SciBERT für die Entitätsextraktion.	126
D.9. Entitätsextraktion mit Llama3-70B für alle Entitäten des LCT-Korpus.	127

Abkürzungsverzeichnis

ADAM	Adaptive Moment Estimation: Ein Optimierungsalgorithmus für neuronale Netze, der adaptive Lernraten verwendet und die Vorteile von RMSprop und Momentum kombiniert. Durch gleitende Durchschnitte der Gradienten und ihrer Quadrate erreicht er schnellere und stabilere Konvergenz.
AST	Abstract Syntax Tree: Eine hierarchische Datenstruktur zur Darstellung der syntaktischen Struktur von formalen Sprachen, bestehend aus Knoten für Operationen und Blättern für Variablen und Literale.
BERT	Bidirectional Encoder Representations from Transformers: Ein von Google entwickeltes Language Model basierend auf der Transformer-Architektur, das durch bidirektionale Kontextverarbeitung und Masked Language Modeling vortrainiert wird. BERT ermöglicht effektives Transfer-Learning für verschiedene NLP-Aufgaben wie Textklassifikation.
Bi-LSTM	Bidirectional Long Short-Term Memory: Eine LSTM-Architektur, die Sequenzdaten vorwärts und rückwärts verarbeitet, um den vollständigen Kontext zu erfassen. Sie wird für NLP-Aufgaben wie Textklassifikation und Spracherkennung eingesetzt.
BIO	Beginning-Inside-Outside: Ein standardisiertes Annotationsschema für Named Entity Recognition, welches Token in Texten mit Markierungen versieht. B (Beginning) kennzeichnet den Beginn einer Entität, I (Inside) die Fortsetzung und O (Outside) Tokens außerhalb von Entitäten.
BRAT	Brat Rapid Annotation Tool: Ein kollaboratives, webbasiertes Werkzeug zur manuellen Textannotation, das die Erstellung von Trainingsdaten für NLP-Systeme unterstützt. BRAT ermöglicht die Markierung von Entitäten, Relationen und Ereignissen in Texten und bietet Funktionen für Qualitätskontrolle und projektübergreifendes Annotationsmanagement.
C2Q	Criteria2Query: Ein NLP-basiertes Tool, das Eignungskriterien klinischer Studien automatisch in ausführbare Datenbankabfragen übersetzt. C2Q analysiert Ein- und Ausschlusskriterien und transformiert diese in strukturierte Abfragen für elektronische Patientenakten, um geeignete Studienpatienten zu identifizieren.
CDM	Common Data Model: Ein Schema zur einheitlichen Organisation von Daten, das Struktur, Beziehungen und Regeln für die Integration verschiedener Datenquellen definiert. CDMs ermöglichen die standardisierte Analyse und den Austausch von Daten über Systeme und Organisationen hinweg.
CFG	Context Free Grammar: Ein formales System zur Beschreibung von Sprachen, bei dem Regeln festlegen, wie Symbole (Nichtterminale) in andere Symbole oder konkrete Zeichen (Terminale) umgewandelt werden. CFGs sind grundlegend für den Bau von Compilern und das Verstehen natürlicher Sprachen, indem sie die Struktur von Programmiersprachen oder mathematischen Ausdrücken definieren.

CNN	Convolutional Neural Networks: Eine spezielle Architektur künstlicher neuronaler Netze, die durch hierarchische Faltungsoperationen (Convolutions) und Pooling-Schichten visuelle Merkmale automatisch extrahiert. CNNs sind optimiert für die Verarbeitung von Daten mit räumlicher Struktur wie Bilder oder Videos und bilden die Grundlage vieler Computer-Vision Anwendungen wie Objekterkennung oder Bildsegmentierung
CoT	Chain-of-Thought: Ein Prompt-Engineering Ansatz für Large Language Models, der komplexe Aufgaben durch explizites schrittweises Denken löst. CoT verbessert die Nachvollziehbarkeit und Genauigkeit von Antworten, indem das Modell seinen Gedankengang in logische Zwischenschritte aufteilt.
CRF	Conditional Random Fields: Ein probabilistisches Modell für die Segmentierung und Annotation sequentieller Daten, das Abhängigkeiten zwischen benachbarten Labels berücksichtigt. CRFs werden besonders für Aufgaben wie Named Entity Recognition und Sequenz-Labeling eingesetzt.
CTP	Clinical Trial Parser: Ein Tool zur automatischen Extraktion und Strukturierung von Informationen aus klinischen Eignungskriterien.
DL	Deep Learning: Ein Bereich des Machine Learnings, basierend auf neuronalen Netzen mit vielen Verarbeitungsschichten. Die hierarchische Architektur ermöglicht das automatische Lernen komplexer Muster aus großen Datenmengen für Aufgaben wie Bild-, Sprach- und Textverarbeitung.
DNF	Disjunktive Normalform: Eine standardisierte Darstellungsform logischer Formeln als Disjunktion von Konjunktionen (mit AND verknüpfte Literale).
EHR	Electronic Health Record: Ein digitales System zur strukturierten Erfassung, Speicherung und Verwaltung medizinischer Patientendaten. EHR-Systeme dokumentieren Diagnosen, Behandlungen, Laborwerte und Medikationen über die Zeit und ermöglichen den sicheren Austausch von Gesundheitsinformationen zwischen autorisierten Gesundheitsdienstleistern.
ElilE	Eligibility Criteria Information Extraction: Ein System zur Extraktion von Informationen aus Eignungskriterien klinischer Studien.
ElIXR	Eligibility Criteria Extraction and Representation: Ein Framework zur Extraktion und Repräsentation von Eignungskriterien für klinische Studien.
EP	EmotionPrompt: Eine Technik zur Verbesserung der Leistung von Large Language Models durch Einbeziehung emotionaler Kontexte in Prompts.
FHIR	Fast Healthcare Interoperability Resources: Ein medizinisches Standardformat für den Datenaustausch im Gesundheitswesen, das Daten als Ressourcen in JSON oder XML beschreibt und flexible Schnittstellen für den Zugriff auf einzelne Informationsfelder bietet.
FRD	Facebook Research Data: Ein von Facebook bereitgestellter Datensatz mit annotierten Eignungskriterien.

GPT	Generative Pre-Trained Transformer: Eine Klasse von Large Language Models, basierend auf der Transformer-Architektur und vortrainiert durch selbstüberwachtes Lernen. GPT-Modelle nutzen eine autoregressive Struktur zur Textgenerierung und können verschiedene sprachbasierte Aufgaben durch Zero-Shot oder Few-Shot Learning lösen, von Textgenerierung über Programmierung bis hin zu Fragenbeantwortung.
HITL	Human-in-the-Loop: Ein Entwicklungsansatz im Bereich der Künstlichen Intelligenz (KI), bei dem menschliche Akteure aktiv in den maschinellen Lernprozess eingebunden sind. Dies umfasst die Überwachung, Korrektur und Verbesserung von Modellen durch menschliche Eingriffe. Human-in-the-Loop wird insbesondere bei sicherheitskritischen Anwendungen eingesetzt, um die Qualität und Zuverlässigkeit der KI-Systeme zu gewährleisten, wie beispielsweise in der medizinischen Diagnostik, bei autonomen Fahrzeugen oder in sicherheitsrelevanten Industrieprozessen.
HPC	High Performance Computing: Die Nutzung von Hochleistungsrechnern und parallelen Systemen zur Durchführung komplexer Berechnungen. HPC kombiniert CPUs, GPUs und spezialisierte Hardware, um rechenintensive Aufgaben wie KI-Training und wissenschaftliche Simulationen effizient zu berechnen.
ICD-10	International Classification of Diseases, 10th Revision: Ein von der WHO standardisiertes Klassifikationssystem für Diagnosen, Symptome und medizinische Prozeduren. ICD-10 ermöglicht die einheitliche Codierung von Krankheiten für Dokumentation, Abrechnung und epidemiologische Statistiken im Gesundheitswesen.
ICL	In-Context Learning: Eine Fähigkeit von Large Language Models, während der Inferenz aus wenigen Beispielen oder Anweisungen innerhalb der Prompt zu lernen und neue Aufgaben zu lösen, ohne dass ein zusätzliches Training erforderlich ist. ICL ermöglicht flexibles Lernen durch Few-Shot (wenige Beispiele) oder Zero-Shot (nur Instruktionen).
IE	Information Extraction: Der Prozess der automatischen Extraktion strukturierter Informationen aus unstrukturierten oder semi-strukturierten Datenquellen.
IFT	Instruction Fine-Tuning: Eine Trainingsmethode zur Optimierung vortrainierter Large Language Models für aufgabenspezifische Anwendungen durch das Training mit Anweisung-Ausgabe Paaren. IFT verbessert die Fähigkeit von LLMs, expliziten Anweisungen zu folgen und ermöglicht eine bessere Steuerbarkeit und Zuverlässigkeit bei spezifischen Aufgaben, ohne das gesamte Modell neu trainieren zu müssen.
JSON	JavaScript Object Notation: Ein leichtgewichtiges, menschen- und maschinenlesbares Datenformat zur Übertragung strukturierter Daten. JSON verwendet eine einfache Syntax aus Key-Value Paaren und Arrays und ist ein Standardformat für Web-APIs und Datenaustausch.

KNF	Konjunktive Normalform: Eine standardisierte Darstellungsform logischer Formeln als Konjunktion von Klauseln (mit OR verknüpfte Literale).
LCT	Leaf Clinical Trials: Ein manuell annotierter Korpus mit Eignungskriterien, der mit detaillierten strukturierten Labels verschiedene biomedizinische Phänomene erfasst. Der Korpus dient als Trainings- und Evaluierungsgrundlage für die automatische Verarbeitung von Ein- und Ausschlusskriterien klinischer Studien.
LLF	Leaf Logical Forms: Formale Repräsentationen oder Strukturen zur Darstellung logischer Aussagen in klinischen Studien oder medizinischen Daten.
LLM	Large Language Model: Ein komplexes, auf der Transformerarchitektur basierendes neuronales Netzwerk, das auf großen Textdatenmengen trainiert wird. Diese Modelle können natürliche Sprache verarbeiten und generieren, indem sie Wahrscheinlichkeitsverteilungen über Sequenzen von Token lernen. LLMs haben die Fähigkeit, Texte zu analysieren, zu übersetzen und zu erstellen, Code zu generieren sowie domänenübergreifendes Wissen in die Ausgaben zu integrieren.
LM	Language Modeling: Eine NLP-Aufgabe, bei der ein Modell die Wahrscheinlichkeitsverteilung von Wortsequenzen erlernt, um das nächste Wort in einem Text basierend auf dem bisherigen Kontext vorherzusagen. Language Modeling bildet die Grundlage für Anwendungen wie Textgenerierung, maschinelle Übersetzung und Sprachverständnis.
LOINC	Logical Observation Identifiers Names and Codes: Ein internationaler Standard zur eindeutigen Identifizierung und Codierung medizinischer und laboratorischer Beobachtungen, einschließlich Tests, Messungen und klinischer Befunde. LOINC ermöglicht die einheitliche Erfassung, den Austausch und die Analyse von Gesundheitsinformationen zwischen verschiedenen Systemen und Einrichtungen.
LoRA	Low-Rank Adaptation: Eine Methode zum effizienten Fine-Tuning von Large Language Models. Die Methode fügt kleine, trainierbare Matrix-Updates zu den vorhandenen Modellgewichten hinzu, anstatt alle Parameter zu aktualisieren.
LSTM	Long Short-Term Memory: Eine fortschrittliche Form rekurrenter neuronaler Netze, die entwickelt wurde, um langfristige Abhängigkeiten in Sequenzdaten effektiv zu verarbeiten. LSTM-Netzwerke nutzen spezielle Steuermechanismen, sogenannte Gates, die den Informationsfluss innerhalb des Netzwerks regulieren. Diese Gates entscheiden, welche Informationen gespeichert, welche vergessen und welche weitergegeben werden sollen. LSTMs sind besonders geeignet für zeitabhängige Aufgaben wie Spracherkennung, Zeitreihenanalyse und maschinelle Übersetzung.
MeSH	Medical Subject Headings: Ein hierarchisch organisiertes Vokabular zur Indexierung, Katalogisierung und Suche von biomedizinischer Literatur und Informationen.

MIMIC-III	Medical Information Mart for Intensive Care III: Eine öffentlich zugängliche Datenbank mit anonymisierten klinischen Daten von über 40000 Intensivpatienten. MIMIC-III enthält detaillierte Aufzeichnungen wie Vitalparameter, Laborwerte, Medikationen und klinische Notizen und wird häufig für die Entwicklung und Evaluation von ML-Algorithmen im medizinischen Bereich verwendet.
ML	Machine Learning: Ein Teilgebiet der künstlichen Intelligenz, das Computersysteme befähigt, durch Erfahrung zu lernen. Es basiert auf der Entwicklung mathematischer Modelle und Algorithmen, die Muster und Gesetzmäßigkeiten in Trainingsdaten erkennen und daraus Regeln ableiten. Diese ermöglichen die automatische Verbesserung bei der Bewältigung komplexer Aufgaben wie Klassifikation, Regression oder Clustering, ohne dass explizite Programmierung erforderlich ist.
MLM	Masked Language Modeling: Eine Self-Supervised Lernmethode für Language Models, bei der zufällig ausgewählte Wörter in einem Text durch Masken ersetzt werden. Das Modell muss diese maskierten Wörter anhand des umgebenden Kontextes vorhersagen. Masked Language Modeling wird insbesondere bei bidirektionalen Transformatoren wie BERT verwendet und ermöglicht es dem Modell, die Beziehungen und Bedeutungen von Wörtern im Kontext zu lernen.
NER	Named Entity Recognition: Eine Aufgabe der Informationsextraktion, bei der benannte Entitäten in Texten automatisch erkannt und vordefinierten Kategorien zugeordnet werden. Dies umfasst allgemeine Entitäten wie Personen, Organisationen und Orte sowie domänenspezifische Entitäten wie Krankheiten, Medikamente oder biologische Marker im medizinischen Kontext.
NLM	National Library of Medicine: Die zentrale biomedizinische Bibliothek der Vereinigten Staaten, die Teil der National Institutes of Health (NIH) ist. Die National Library of Medicine betreibt Datenbanken wie PubMed und entwickelt Standards für Gesundheitsinformationen und Informationssysteme, um den Zugang zu medizinischem Wissen zu erleichtern und die wissenschaftliche Kommunikation zu fördern.
NLP	Natural Language Processing: Ein Forschungsgebiet der künstlichen Intelligenz, das sich mit der automatischen Verarbeitung natürlicher Sprache befasst. NLP kombiniert linguistische, statistische und ML-Methoden für Aufgaben wie Textanalyse, Übersetzung und Sprachverständnis, wobei Syntax, Semantik und Kontext berücksichtigt werden.
NSP	Next Sentence Prediction: Eine Trainingsaufgabe im NLP, bei der ein Modell lernt, die logische Beziehung zwischen zwei Sätzen zu erkennen. NSP wird insbesondere beim Training von Modellen wie BERT verwendet, um Textverständnis und Kohärenzerfassung zu verbessern. Dabei wird das Modell trainiert zu unterscheiden, ob ein Satz tatsächlich auf einen anderen folgt oder zufällig ausgewählt wurde, was für Aufgaben wie Fragenbeantwortung und Textgenerierung wichtig ist.

OMOP	Observational Medical Outcomes Partnership: Ein standardisiertes Datenmodell für die Integration und Analyse klinischer Daten. OMOP-CDM harmonisiert Gesundheitsdaten aus verschiedenen Quellen in ein einheitliches Format mit standardisierten Vokabularen, wodurch institutionsübergreifende Analysen, Forschung und Versorgungsstudien ermöglicht werden.
PEFT	Parameter-Efficient Fine-Tuning: Sammelbegriff für Techniken, die es ermöglichen, Large Language Models effizient an neue Aufgaben anzupassen, ohne alle Parameter zu trainieren.
PLM	Pre-trained Language Model: Ein auf großen Textkorpora trainiertes Language Model, das allgemeines sprachliches und semantisches Wissen erfasst und durch Fine-Tuning für spezifische NLP-Aufgaben angepasst werden kann.
PMC	PubMed Central: Eine frei zugängliche Volltextarchiv der biomedizinischen und biowissenschaftlichen Literatur des U.S. National Institutes of Health's National Library of Medicine.
QLoRA	Quantized LoRA: Eine Variante von LoRA, die Quantisierung verwendet, um den Speicherbedarf während des Trainings zu reduzieren.
RDF	Resource Description Framework: Ein W3C-Standard zur semantischen Beschreibung von Webressourcen mit Tripeln (Subjekt, Prädikat, Objekt). RDF ermöglicht maschinenlesbare Darstellung von Wissen und Beziehungen und bildet die Grundlage für das Semantic Web sowie für Wissensgraphen und verknüpfte Daten.
RE	Relation Extraction: Eine NLP-Aufgabe, die semantische Beziehungen zwischen identifizierten Entitäten in Texten automatisch erkennt und klassifiziert. RE wird oft in Kombination mit Named Entity Recognition eingesetzt.
RegEx	Regular Expression: Ein standardisiertes System von Textmustern, das zur flexiblen Suche, Validierung und Manipulation von Zeichenketten verwendet wird. Reguläre Ausdrücke (RegEx) nutzen spezielle Syntaxelemente wie Quantifizierer, Gruppen und Zeichenklassen, um komplexe Textmuster zu definieren. Sie sind grundlegende Werkzeuge in der Textverarbeitung, Datenvalidierung und beim Parsen von Daten.
RNN	Recurrent Neural Network: Eine Klasse neuronaler Netze, die durch rekurrente Verbindungen zeitliche oder sequentielle Abhängigkeiten in Daten verarbeiten können. RNNs verwenden ihren internen Zustand (Memory) zur Verarbeitung variabler Eingabesequenzen und bilden die Grundlage für fortgeschrittene Architekturen wie LSTMs.
rsLoRA	rank-stabilized LoRA: Eine Weiterentwicklung von LoRA, die die Stabilität des Trainings durch Regularisierung der Rangzerlegung verbessert.
SFT	Supervised Fine-Tuning: Eine Methode zur Anpassung vortrainierter ML-Modelle an spezifische Aufgaben durch gezieltes Training mit annotierten Daten.

SNOMED-CT	Systematized Nomenclature of Medicine – Clinical Terms: Eine umfassende, mehrsprachige klinische Terminologie für die elektronische Dokumentation im Gesundheitswesen. SNOMED-CT standardisiert medizinische Begriffe, ihre Beziehungen und Hierarchien und ermöglicht die Erfassung klinischer Informationen in Gesundheitssystemen. Die Terminologie umfasst Diagnosen, Symptome, Prozeduren und weitere medizinische Konzepte.
SOTA	State-of-the-Art: Bezeichnet den aktuellen wissenschaftlichen oder technischen Entwicklungsstand, der die höchste bisher erreichte Leistungsfähigkeit oder beste verfügbare Methode in einem spezifischen Forschungs- oder Technologiebereich darstellt.
SQL	Structured Query Language: Eine standardisierte Sprache für die Verwaltung und Abfrage relationaler Datenbanken.
SVM	Support Vector Machines: Ein Machine Learning-Algorithmus, der Datenpunkte in einem hochdimensionalen Raum durch eine Hyperebene optimal trennt.
T5	Text-to-Text Transfer Transformer: Ein von Google entwickeltes transformerbasiertes Modell, das alle NLP-Aufgaben in ein einheitliches Text-to-Text Format konvertiert.
TPM	Token per Minute: Ein Maß für die Verarbeitungsgeschwindigkeit von Large Language Models, das angibt, wie viele Token pro Minute verarbeitet werden können.
UMLS	Unified Medical Language System: Ein von der US National Library of Medicine entwickeltes Metathesaurus-System, das über 200 biomedizinische Vokabulare und Standards integriert. UMLS vereinheitlicht medizinische Begriffe, ihre Bedeutungen und Beziehungen und ermöglicht die semantische Interoperabilität zwischen verschiedenen medizinischen Informationssystemen und Terminologien wie SNOMED-CT, ICD und MeSH.
XML	Extensible Markup Language: Ein standardisiertes, textbasiertes Format zur Darstellung und zum Austausch hierarchischer Daten mit selbstbeschreibenden Tags, plattformunabhängig und sowohl für Menschen als auch Maschinen lesbar.

1. Einleitung

Klinische Forschung überführt biomedizinische Grundlagenkenntnisse in praktische Anwendungen zur Verbesserung der Gesundheitsversorgung. Im Mittelpunkt dieses Prozesses stehen klinische Studien, die systematische und kontrollierte Untersuchungen darstellen, um neue Arzneimittel, Medizinprodukte oder innovative Behandlungsmethoden unter standardisierten Bedingungen an sorgfältig ausgewählten menschlichen Probanden zu erproben und zu evaluieren (Chow; J. Liu, 2008). Klinische Studien sind eine der wertvollsten Quellen für den Nachweis der Wirksamkeit von Behandlungen am Menschen. Die evidenzbasierte Medizin zielt darauf ab, die Erkenntnisse aus klinischen Studien zur besseren Bewertung und Behandlung von Patientenpopulationen in allen Bereichen der Medizin zu nutzen (Ross et al., 2010). Die steigende Forschungs- und Entwicklungstätigkeit von Pharma- und Biotechnologieunternehmen hat die Anzahl der weltweit durchgeführten klinischen Studien erheblich erhöht. Der globale Markt für klinische Studien wurde im Jahr 2023 auf über 57 Mrd. USD geschätzt und wird voraussichtlich bis zum Jahr 2032 auf über 100 Mrd. USD anwachsen (Fortune Business Insights, 2023). Für die Durchführung klinischer Studien muss die Teilnehmerpopulation gemäß den im Studienprotokoll festgelegten Eignungskriterien rekrutiert werden, um die Fragestellung der Studie präzise beantworten zu können. Die Eignungskriterien sind dabei ein wesentlicher Bestandteil, da der Erfolg der Studie maßgeblich von deren korrekter Festlegung abhängt (Su et al., 2023). In der Vergangenheit traten bei klinischen Prüfungen zahlreiche Probleme auf, wie eine unzureichende Rekrutierung oder die Auswahl ungeeigneter Populationen (C. Weng et al., 2011). Die Identifizierung geeigneter Patienten für klinische Studien stellt einen wesentlichen Engpass bei der Markteinführung neuer Arzneimittel dar. Etwa ein Drittel der klinischen Studien scheitert aufgrund mangelnder Gewinnung passender Teilnehmer, wobei die Rekrutierungskosten etwa 32% der Gesamtausgaben ausmachen (Wornow et al., 2024). Der Prozess des Patient-Studie Matchings wird in der Praxis durch eine überwältigende Menge unstrukturierter Daten erschwert, die aufseiten der Studien durch unstrukturierte Eignungskriterien und aufseiten der Patienten durch unstrukturierte Electronic Health Record (EHR) Daten repräsentiert werden. Hinzu kommt die nicht skalierbare manuelle Verarbeitung dieser Daten, was die Effizienz erheblich beeinträchtigt (Wong et al., 2023). Die genannten Probleme hängen direkt oder indirekt mit den unstrukturierten Eignungskriterien zusammen, die wesentlich zu einer unzureichenden Rekrutierung beitragen, was ein entscheidender Grund für das Scheitern vieler klinischer Prüfungen darstellt (Su et al., 2023).

1.1. Problemstellung

Eignungskriterien in der klinischen Forschung zeichnen sich oftmals durch eine hohe semantische und logische Komplexität aus, wodurch ihre Interpretation und Anwendung erschwert wird (Chunhua Weng, 2015). Da Ein- und Ausschlusskriterien in Freitextform erfasst werden, ist ihre automatisierte Auswertung aufgrund des unstrukturierten Formats häufig nicht möglich (Wong et al., 2023). Zudem werden Eignungskriterien in verschiedenen Formaten und mit unterschiedlichen Terminologien dokumentiert, was aufgrund mangelnder Interoperabilität zwischen anderen Studien und Datenquellen kritisiert wird (Chunhua Weng, 2015). Die manuelle Extraktion und Verarbeitung von Eignungskriterien ist zudem zeitaufwendig, komplex und mit einer hohen Fehleranfälligkeit verbunden (Penberthy et al., 2010). Aktuelle Methoden und Systeme zur Strukturierung von Eignungskriterien binden die prozessierten Kriterien oft an ein spezifisches medizinisches Format wie das Observational Medical Outcomes Partnership (OMOP)-Common Data Model (CDM) Datenbankenschema. Eine Übertragung in ein weiteres medizinisches Standardformat wie Fast Healthcare Interoperability Resources (FHIR), ist daher oft schwierig. Darüber hinaus berücksichtigen nur wenige der bestehenden Methoden die komplexe und verschachtelte Logik der Eignungskriterien bei der Verarbeitung (N. J. Dobbins et al., 2023). In einer Untersuchung von Ross et al. (2010) wurden Eignungskriterien analysiert, von denen 53% logische Verknüpfungen innerhalb der Kriterien enthielten, die durch AND- und OR-Operatoren repräsentiert werden können, und 26% Negationen, die durch NOT-Operatoren ausgedrückt werden können (Ross et al., 2010). Zudem weisen die Komponenten der Einschluss- und Ausschlusskriterien implizite logische Verknüpfungen auf (Wong et al., 2023). Bei den Einschlusskriterien wird üblicherweise eine AND-Verknüpfung angenommen, sodass ein Patient alle aufgeführten Kriterien erfüllen muss.

Die Ausschlusskriterien basieren hingegen auf einer impliziten NOT-Logik, wonach ein Patient keines der genannten Kriterien aufweisen darf. Abbildung 1.1 veranschaulicht die verschachtelte Logik innerhalb von Einschlusskriterien der Onkologiestudie NCT04412629¹.

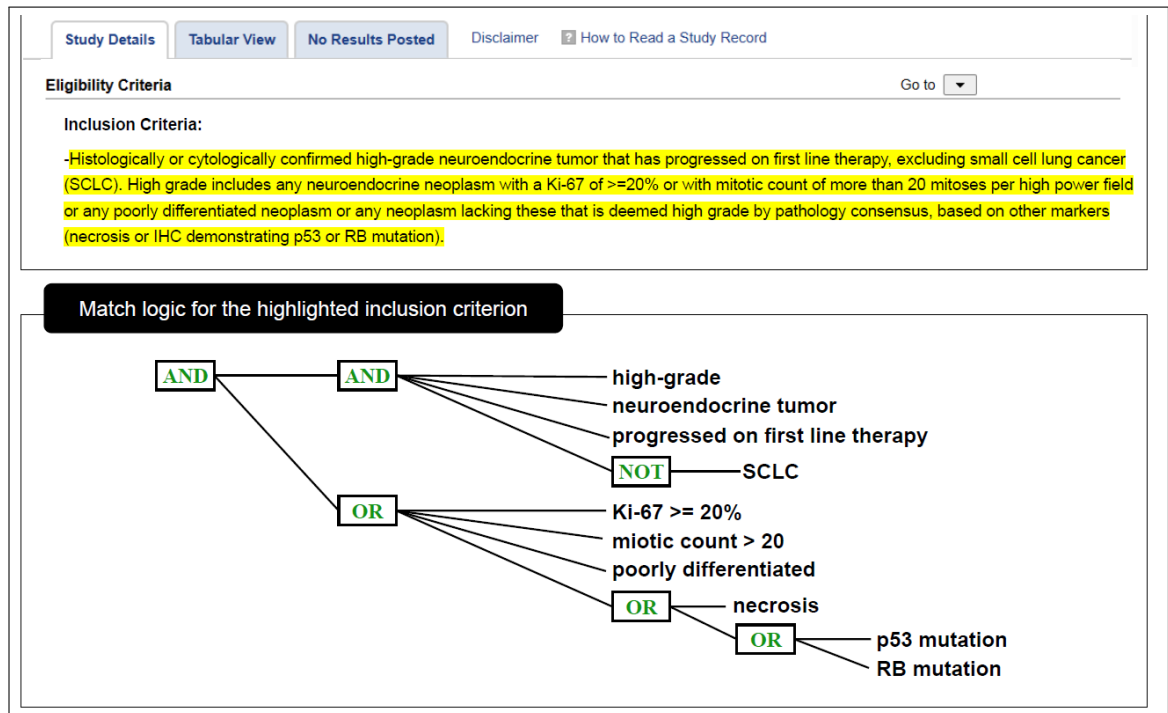


Abbildung 1.1.: Einschlusskriterien der Studie NCT04412629 mit extrahierten Entitäten und hierarchischen, logischen Beziehungen (Wong et al., 2023).

Wenn Entitäten wie *neuroendocrine tumor* und Attribute wie *high-grade* in den Einschlusskriterien erkannt werden, ist es für eine korrekte Auswertung wichtig, die implizite logische Beziehung zwischen diesen Entitäten herzustellen, in diesem Fall über *high-grade* **AND** *neuroendocrine tumor*. Da es sich um Einschlusskriterien handelt, verbindet ein AND in oberer Ebene alle weiteren logischen Verbindungen. Die Formalisierung der Eignungskriterien in eine für Maschinen interpretierbare Syntax mit Berücksichtigung der inhärenten, komplexen logischen Struktur würde die Genauigkeit bei der Suche nach Studien verbessern, indem die verarbeiteten Eignungskriterien mitberücksichtigt werden können (Tseo et al., 2020). Zudem würde das Patient-Studie Matching erheblich vereinfacht werden, wenn Ein- und Ausschlusskriterien in einer verarbeitbaren Form vorliegen (Wong et al., 2023).

Biomedizinische Textdaten können mit Natural Language Processing (NLP) Methoden wie der Information Extraction (IE) analysiert und prozessiert werden. Die Information Extraction zielt darauf ab, strukturierte Informationen aus unstrukturierten Textdaten in natürlicher Sprache zu extrahieren (X. Xu et al., 2023). Für eine Anreicherung von Textdaten mit erkannten Entitäten und Relationen zwischen den Entitäten eignen sich Named Entity Recognition (NER) und Relation Extraction (RE) Methoden. Die aktuellen State-of-the-Art-Modelle (SOTA) für NER- und RE-Aufgaben sind transformerbasierte Pre-trained Language Models (PLMs) wie BERT-Modelle, die durch domänenspezifisches Fine-Tuning auf den zu untersuchenden Datensätzen angepasst wurden. Diese BERT-Varianten haben in der Vergangenheit bei IE-Aufgaben oft deutlich besser abgeschnitten als generative Large Language Models (LLMs) wie GPT-3. In einer Untersuchung von Gutiérrez et al. (2022) auf mehreren biomedizinischen Datensätzen performte das GPT-3 Modell mit Few-Shot bei NER-Aufgaben im Durchschnitt um 16.7% schlechter und bei RE-Aufgaben um 11.4% schlechter gegenüber einem BERT-basierten Modell mit Fine-Tuning. Während das Fine-Tuning von BERT-Modellen mit 345 engl. Million (M) Parametern bis zu 5 Tagen auf einem Cluster mit vier GPUs gedauert hat, war die Nutzung von GPT-3 über die OpenAI-API mit hohen Kosten

¹<https://clinicaltrials.gov/study/NCT04412629>

verbunden. Andererseits haben moderne, generative Large Language Models vielversprechende Leistungen bei allgemeinen Information Extraction-Aufgaben gezeigt (Munnangi et al., 2024). Aufgrund ihres generativen Charakters können Large Language Models verwendet werden, um strukturelle Informationen zu generieren, anstatt sie ausschließlich aus reinem Text zu extrahieren. Dabei kann die Ausgabe effektiv in einem universellen Format modelliert werden (X. Xu et al., 2023). Neuere Architekturen wie GPT-4 zeigen zudem eine deutlich bessere Leistung im Vergleich zu Vorgängermodellen wie GPT-3.5. Beispielsweise erreicht GPT-4 bei einer simulierten Anwaltsprüfung eine Punktzahl, die zu den besten 10% der Testteilnehmer zählt, während GPT-3.5 zu den unteren 10% gehört (OpenAI et al., 2024). Aktuelle Open-Source Modelle wie Llama 3 haben durch ein Training mit hochwertigeren Daten und geringfügige Anpassungen der Architektur deutlich bessere Leistungen im Vergleich zu ihren Vorgängern wie Llama 2 erzielt. Zudem wird ihnen eine vergleichbare Qualität zu führenden proprietären LLMs wie GPT-4 zugeschrieben (Dubey et al., 2024). Open-Source LLMs wie Modelle der Llama-Familie, ermöglichen Forschungseinrichtungen die lokale Ausführung auf eigenen Hardware-Ressourcen, wodurch sowohl der Schutz sensibler Daten als auch die Unabhängigkeit von kostenpflichtigen API-Diensten gewährleistet wird. Die Modelle bieten verschiedene LLM-Methoden zur Bearbeitung von NLP-Aufgaben. Das Zero-Shot Verfahren ermöglicht die Aufgabenbearbeitung ausschließlich durch Instruktionen ohne vorherige Beispiele. Zudem zeigen aktuelle Prompt-Reports wie Schulhoff et al. (2024), dass erweiterte Prompt-Methoden es ermöglichen, durch kurze und einfache Ergänzungen an der ursprünglichen Prompt-Anweisung bessere Ergebnisse bei bestimmten NLP-Aufgaben wie Frage-Antwort-Aufgaben zu erzielen. Mit Few-Shot Szenarien können Aufgaben unter Verwendung einer minimalen Menge annotierter Trainingsdaten bearbeitet werden. Ein domänenspezifisches Fine-Tuning wird bei Verfügbarkeit eines umfangreicheren Korpus annotierter und vorverarbeiteter Beispiele sowie ausreichender Rechenkapazitäten möglich.

1.2. Motivation

Diese Forschungsarbeit hat zum Ziel, ausgewählte Large Language Models hinsichtlich ihrer Eignung zur Strukturierung von Eignungskriterien im freien Textformat durch Information Extraction-Aufgaben zu evaluieren. Die Grundlage der Strukturierung bildet die Extraktion der impliziten, komplexen Logik aus den Eignungskriterien. Dabei sollen folgende Forschungsfragen beantwortet werden. Die Hauptforschungsfrage (**RQ0**) dieser Arbeit lautet:

RQ0: *Wie kann unstrukturierter klinischer Text mit frei verfügbaren Machine Learning-Methoden strukturiert werden?*

Eine systematische Analyse klinischer Eignungskriterien, die als unstrukturierte Texte vorliegen, wird mithilfe öffentlich zugänglicher Korpora durchgeführt. Dabei soll evaluiert werden, welche Strukturen und Formate sich für die Verarbeitung der Kriterien eignen. Der Fokus liegt auf interoperablen Formaten, die sich einfach transformieren lassen und direkt von LLMs ausgegeben werden können. Hierbei wird der Nutzen von Named Entity Recognition- sowie Relation Extraction-Methoden für die Analyse der Eignungskriterien untersucht. Darüber hinaus wird das Potenzial von Open-Source LLMs, einer Untergruppe von Machine Learning-Modellen, für diese Aufgabenstellung evaluiert. Die Hauptforschungsfrage wird durch die folgende Forschungsfrage (**RQ1**) unterstützt:

RQ1: *Welche Methoden eignen sich für die Strukturierung von Eignungskriterien aus klinischen Studien?*

Large Language Models ermöglichen durch Zero-Shot, Few-Shot und Fine-Tuning Methoden vielfältige Anwendungsszenarien. Die unterstützende Forschungsfrage **RQ1** untersucht die Eignung verschiedener LLM-Methoden für die Strukturierung von Eignungskriterien unter Berücksichtigung der erforderlichen Ressourcen. Die Evaluation umfasst dabei einen systematischen Vergleich zwischen Open-Source und proprietären Closed-Source LLMs hinsichtlich ihrer Leistungsfähigkeit und Ressourceneffizienz. Diese Herangehensweise zielt darauf ab, einen reproduzierbaren Workflow für die automatisierte Strukturierung klinischer Eignungskriterien zu entwickeln.

1.3. Gliederung

Die Grundlagen klinischer Studien sowie deren Eignungskriterien werden in Abschnitt 2 definiert. Es wird die Funktion der logischen Suche zur Identifizierung klinischer Studien erläutert und dargestellt, welche Natural Language Processing-Methoden zur Verarbeitung und Nutzung der Eignungskriterien eingesetzt werden können. Darüber hinaus wird erläutert, welche Metriken zur Evaluierung dieser Methoden geeignet sind. Abschnitt 3 stellt Modelle vor, die es ermöglichen, Natural Language Processing-Methoden zur Strukturierung von Eignungskriterien einzusetzen. Dabei werden regelbasierte Methoden sowie transformerbasierte Machine Learning-Methoden präsentiert. Diese Modelle werden in Encoder-only Modelle wie BERT und Decoder-only Large Language Models wie Llama3 oder GPT-4 unterteilt. Zudem werden die in dieser Arbeit verwendeten Open-Source und Closed-Source LLMs vorgestellt und erläutert, wie diese Modelle durch Prompt-Methoden und Fine-Tuning für spezifische Aufgaben angepasst werden können. In Abschnitt 4 werden die State-of-the-Art Methoden und Systeme zur Analyse und Strukturierung von Eignungskriterien diskutiert. Dabei werden aktuelle Forschungsfelder, verfügbare Korpora sowie Methoden zur Extraktion und Strukturierung von Eignungskriterien vorgestellt.

In Abschnitt 5 werden eine indirekte und eine direkte Strukturierungsmethode von Eignungskriterien mit LLMs vorgestellt. Dabei wird der Einsatz von Prompting, In-Context Learning-Methoden wie Zero-Shot und Few-Shot, erweiterte Prompt-Methoden sowie Fine-Tuning Methoden erläutert. Zudem wird ein regelbasierter Ansatz implementiert, um einen Vergleich mit In-Context Learning-Methoden unter Verwendung von LLMs durchzuführen. Die Ergebnisse der Modellevaluation werden im Abschnitt 6 dargestellt. Hier wird die Leistung ausgewählter LLMs anhand verschiedener methodischer Ansätze und der vorgestellten Metriken evaluiert und verglichen. Dabei wird die Performance von erstellten Prompts mit Open-Source Modellen bewertet. In-Context Learning Methoden werden mit Fine-Tuning und aktuellen State-of-the-Art Modellen sowie dem regelbasierten Ansatz auf öffentlichen Korpora mit Eignungskriterien verglichen. Abschnitt 7 diskutiert die Leistungen und die methodischen Ansätze zur Strukturierung mittels ausgewählter Large Language Models. Diese werden mit den Systemen und Formaten aktueller Forschungsarbeiten verglichen. Darüber hinaus wird der praktische Nutzen der entwickelten Methodik analysiert. Das Fazit in Abschnitt 8 fasst die zentralen Erkenntnisse zusammen, zieht Schlussfolgerungen und bietet einen Ausblick auf zukünftige Forschungsrichtungen im Bereich der automatischen Strukturierung von Eignungskriterien für klinische Studien.

2. Grundlagen zur Eignungskriterienanalyse

Die Eignungskriterien klinischer Studien bestehen aus Ein- und Ausschlusskriterien und können über Studienregister wie ClinicalTrials.gov (2000) eingesehen und bezogen werden. Diese Register ermöglichen eine gezielte Suche nach klinischen Studien. Um die Eignungskriterien logisch anzureichern und für eine logische Suche nutzbar zu machen, müssen sie mit Natural Language Processing (NLP) Methoden verarbeitet werden. Hierbei eignen sich insbesondere Named Entity Recognition (NER) und Relation Extraction (RE) Methoden, welche anhand von Machine Learning-Metriken evaluiert werden können.

2.1. Klinische Studien

Klinische Studien sind systematische Untersuchungen, welche die Wirkung und den Nutzen von Interventionen bei Menschen im Vergleich zu einer Kontrollgruppe evaluieren. Dabei werden aktive, standardisierte Maßnahmen wie Medikamente oder Verfahren eingesetzt, und es müssen ethische sowie sicherheitsrelevante Aspekte berücksichtigt werden (Chow; J. Liu, 2008). Die klinischen Studien beginnen mit der Phase I, in der Pharmakologie, Verträglichkeit und Sicherheit an einer kleinen Gruppe von meist gesunden Freiwilligen untersucht werden. Darauf folgt Phase II, in der die Wirksamkeit sowie die optimale Dosierung in einer begrenzten Patientengruppe ermittelt werden. In Phase III werden größere, konfirmatorische Studien durchgeführt, um den klinischen Nutzen des Medikaments nachzuweisen. Nach der Zulassung wird in Phase IV die therapeutische Anwendung in größeren oder speziellen Populationen untersucht und seltene Nebenwirkungen identifiziert (Friedman et al., 2015). Um diese Untersuchungen strukturiert und nachvollziehbar zu gestalten, basieren sie auf Studienprotokollen, die als grundlegende Dokumente und Vereinbarungen zwischen Forschenden, Teilnehmenden und der wissenschaftlichen Gemeinschaft dienen. Ein Studienprotokoll enthält üblicherweise den Hintergrund der Studie, die Ziele, das Studiendesign sowie organisatorische Informationen zu den beteiligten Forschenden und Institutionen. Ein wesentlicher Bestandteil des Studiendesigns sind die Eignungskriterien (engl. *Eligibility Criteria*), die in Ein- und Ausschlusskriterien unterteilt werden (Friedman et al., 2015).

2.1.1. Eignungskriterien

Die Eignungskriterien in klinischen Studien definieren die Studienpopulation und bestimmen, wer für eine Teilnahme in Frage kommt. Der Text der Eignungskriterien umfasst klinisch relevante Merkmale zur Beurteilung der Probandeneignung, einschließlich Krankheiten und Symptomen, Medikation, Laborparameter, demografischer Merkmale, spezifischer Populationscharakteristika sowie der Fähigkeit zur informierten Einwilligung. Die Einschlusskriterien definieren die Anforderungen, die potenzielle Teilnehmer erfüllen müssen, um in die Studie aufgenommen zu werden. Die Ausschlusskriterien hingegen spezifizieren Faktoren, die eine Teilnahme ausschließen (Friedman et al., 2015). Darüber hinaus legen diese Kriterien fest, ob eine Studie gesunde Freiwillige akzeptiert, bestimmte Alters- oder Altersgruppenvorgaben hat oder auf ein spezifisches Geschlecht beschränkt ist (Kim et al., 2017).

Die präzise Definition und Umsetzung von Eignungskriterien in klinischen Studien ist entscheidend, um eine repräsentative Studienpopulation sicherzustellen. Dadurch werden Verzerrungen minimiert, die Validität der Studie erhöht und die Übertragbarkeit der Ergebnisse auf die relevante Patientengruppe ermöglicht (Chunhua Weng, 2015). Eignungskriterien liegen in der Regel als unstrukturierter Freitext ohne standardisierte Gliederung vor und sind häufig in einer semantisch komplexen Fachsprache formuliert, die spezielles Expertenwissen für ein vollständiges Verständnis erfordert. Diese Kombination aus Freitextformat und fachspezifischer Terminologie erschwert die computergestützte Analyse und Verarbeitung der Kriterien (Ross et al., 2010). Abbildung 2.1 zeigt einen exemplarischen Auszug aus den Einschlusskriterien der Onkologiestudie NCT04412629¹, während Abbildung 2.2 einen Auszug aus den Ausschlusskriterien der Studie darstellt. Die Struktur der dargestellten Ein- und Ausschlusskriterien folgt keinem einheitlichen Schema. Zwar sind einige Kriteriensätze als Listen präsentiert, werden jedoch immer wieder durch Ergänzungen im Fließtext unterbrochen. Insgesamt sind die Kriterien somit überwiegend unstrukturiert, mit teilweise semi-strukturierten Passagen.

¹<https://clinicaltrials.gov/study/NCT04412629>

Inclusion Criteria: -Histologically or cytologically confirmed high-grade neuroendocrine tumor that has progressed after at least one line of therapy, excluding small cell lung cancer (SCLC). High grade includes any neuroendocrine neoplasm with a Ki-67 of $\geq 20\%$ or with mitotic count of more than 20 mitoses per high power field or any poorly differentiated neoplasm or any neoplasm lacking these that is deemed high grade by pathology consensus, based on other markers (necrosis or IHC demonstrating p53 or RB mutation). This includes: • High grade well differentiated neuroendocrine neoplasms • Transformed NENs from a lower to a higher grade (patient may have some low grade and some high grade NENs) • High grade neoplasms with significant expression of neuroendocrine markers such as synaptophysin, chromogranin or INSM-1 or unknown origin neoplasms with gene expression signatures consistent with neuroendocrine lineage (as per validated tissue of origin testing, such as CancerType ID, after pathology consensus). • Mixed neuroendocrine and non-neuroendocrine neoplasms (MINEN), including MINEN per WHO and mixed neoplasms not fulfilling criteria of MINEN. The neuroendocrine component would need to be a high-grade neuroendocrine tumor as documented by pathology review. Note: Up to two prostate NEC patients (primary diagnosis, not transformed adenocarcinoma) will be enrolled in the first phase. For the second phase, non gastroenteropancreatic or lung histologies will be approved by PI. Note: For ambiguous cases, will consult with a designated expert pathologist. • Measurable disease defined as lesions that can be accurately measured in at least one dimension (longest diameter to be recorded) as ≥ 10 mm with CT scan, as ≥ 20 mm by chest x-ray, or ≥ 10 mm with calipers by clinical exam. • Concurrent or prior somatostatin analogue therapy is allowed (for well differentiated high grade neoplasms). Prior use of investigational agents is allowed. • At least 18 years of age. • ECOG performance status ≤ 1 (Karnofsky $\geq 80\%$) • Normal bone marrow and organ function as defined below: ◦ Absolute neutrophil count $\geq 1,500/\text{mm}^3$ without granulocyte colony-stimulating factor support ◦ White blood cell count $\geq 2,500/\text{mm}^3$ ◦ Platelets $\geq 100,000/\text{mm}^3$ without transfusion ◦ Hemoglobin ≥ 9.0 g/dL ◦ AST(SGOT), ALT(SGPT), and alkaline phosphatase (ALP) $\leq 3.0 \times \text{IULN}$; ALP $\leq 5.0 \times \text{IULN}$ with documented bone metastases ◦ Total bilirubin $\leq 1.5 \times \text{IULN}$ (for subjects with gilbert's disease $\leq 3.0 \times \text{IULN}$) ◦ Serum albumin ≥ 2.8 g/dL	Exclusion Criteria: • A history of other malignancy with the exception of malignancies for which all treatment was completed at least 2 years before registration and the patient has no evidence of disease. Allowed are superficial skin cancers, or localized, low grade tumors deemed cured and not treated with systemic therapy at any point in the prior year. • Currently receiving any other investigational agents. Prior use of investigational agents is allowed. • Prior treatment with cabozantinib. • Receipt of any small molecule kinase inhibitor (including investigational kinase inhibitor) within 2 weeks before the first dose of study treatment. • Receipt of any type of cytotoxic, biologic, or other systemic anticancer therapy (including investigational) within 4 weeks before first dose of study treatment. • Radiation therapy for bone metastasis within 2 weeks, any other radiation therapy within 4 weeks before the first dose of study treatment. Systemic treatment with radionuclides within 4 weeks before the first dose of study treatment. Patients with clinically relevant ongoing complications from prior radiation therapy are not eligible. • Known brain metastases or cranial epidural disease unless adequately treated with radiotherapy and/or surgery (including radiosurgery) and stable for at least 4 weeks prior to first dose of study treatment after radiotherapy or at least 4 weeks prior to first dose of study treatment after major surgery (e.g., removal or biopsy of brain metastasis). Subjects must have complete wound healing from major surgery or minor surgery before first dose of study treatment. • Inability to swallow tablets. • A history of allergic reactions attributed to compounds of similar chemical or biologic composition to cabozantinib or other agents used in the study. • Concomitant anticoagulation with coumarin agents (e.g. warfarin), direct thrombin inhibitors (e.g., dabigatran), direct factor Xa inhibitor betrixaban, or platelet inhibitors (e.g. clopidogrel). Allowed anticoagulants are the following: ◦ Prophylactic use of low-dose aspirin for cardio-protection (per local applicable guidelines) and low-dose low molecular weight heparins (LMWH) ◦ Therapeutic doses of LMWH or anticoagulation with direct factor Xa inhibitors rivaroxaban, edoxaban, or apixaban in subjects without known brain metastases who are on a stable dose of the anticoagulant for at least 1 week before first dose of study treatment without clinically significant hemorrhagic complications from the anticoagulation regimen or the tumor. • Uncontrolled, significant intercurrent or recent illness including, but not limited to, the following conditions:
---	---

Abbildung 2.1.: Auszug aus den Einschlusskriterien der Onkologiestudie NCT04412629.

Abbildung 2.2.: Auszug aus den Ausschlusskriterien der Onkologiestudie NCT04412629.

In Abbildung 1 im Anhang werden weitere unstrukturierte Eignungskriterien dargestellt, wie sie in der Praxis vorkommen. Die Kriterien wurden nicht korrekt aufgelistet, da die numerische Auflistung fehlerhaft ist oder die Kriterien trotz Aufzählung in einem zusammenhängenden Textblock präsentiert werden. Die Beispiele verdeutlichen, dass Eignungskriterien im besten Fall semi-strukturiert durch Aufzählungen dargestellt werden, jedoch auch vollständig unstrukturiert auftreten können, mit fehlerhaften Formatierungen.

2.1.2. Studienregister

Für eine umfangreiche Datenerhebung von Eignungskriterien bietet sich die Webseite ClinicalTrials.gov (2000) an. Diese zentrale öffentliche Datenbank für klinische Studien wird von der National Library of Medicine (NLM) betrieben. Die Datenbank bietet Zugang zu Informationen über Studienprotokolle und -ergebnisse für eine Vielzahl von klinischen Untersuchungen, einschließlich Interventionsstudien zu Medikamenten, Biologika und Medizinprodukten (Zarin et al., 2011). Derzeit sind 507578 Studien aus 222 Ländern auf ClinicalTrials.gov (2000) registriert, mit über 4.5 Millionen monatlichen Besuchern (NLM, 2024). Diese umfangreiche Datenquelle ermöglicht es Forschern, Ärzten und der Öffentlichkeit sich über laufende und abgeschlossene klinische Studien zu informieren, Trends in der klinischen Forschung zu analysieren und den Zugang zu Studienergebnissen zu verbessern (Zarin et al., 2011). Da jährlich über 32.000 neue Studien hinzugefügt werden, erweist sich der Einsatz automatisierter Methoden zur Analyse der Eignungskriterien als unverzichtbar (Tseo et al., 2020). Studienregister wie ClinicalTrials.gov (2000) ermöglichen eine gezielte Suche nach aktuellen und abgeschlossenen klinischen Studien. Dabei können wesentliche Eignungskriterien wie Alter, Geschlecht, Studienort und Gesundheitszustand der Teilnehmenden sowie spezifischere Kriterien wie Behandlungshistorie und Vorerkrankungen berücksichtigt werden, sofern diese in maschinenlesbarer Form vorliegen (Chunhua Weng, 2015).

2.1.3. Logische Suche

Die logische Suche ist eine einfache und etablierte Form der Informationssuche, bei der Suchbegriffe mittels logischer Operatoren AND, OR und NOT verknüpft werden. Diese Art der Suche ist besonders nützlich im biomedizinischen Bereich, wo oft alle möglichen Bezeichnungen für ein Konzept erfasst werden müssen, was zu sehr langen und komplexen Suchanfragen führen kann (H. Xu; Fushman, 2024). In Studienregistern wie ClinicalTrials.gov (2000) können die Operatoren folgendermaßen für eine Suche nach Studien verwendet werden:

AND Dieser binäre Operator wird verwendet, um Studien abzurufen, die sowohl den linken als auch den rechten Teilausdruck enthalten. Wenn beispielsweise nach "*diabetes AND gestational*" gesucht wird, werden Dokumente mit beiden Begriffen zurückgegeben.

OR Mit dem binären Operator können Studien gefunden werden, die entweder den linken oder rechten Ausdruck enthalten, wobei Treffer mit beiden höher gewichtet werden. Eine Suche mit dem OR Operator wie "*diabetes OR dm OR t1dm OR t2dm*" findet Dokumente, welche "diabetes" oder gängige Akronyme für Diabetes mellitus enthalten.

NOT Als unärer Operator ruft er Studiendaten ab, die den rechten Teilausdruck nicht enthalten. Die Abfrage "*diabetes AND NOT gestational*" liefert Dokumente über Diabetes, schließt jedoch solche mit dem Begriff "gestational" aus (H. Xu; Fushman, 2024).

Die Erstellung komplexerer Suchanfragen kann durch die Verwendung von Klammern zur Darstellung verschachtelter Logik erreicht werden, wie beispielsweise (*a AND (b OR (c AND d AND e))*). Dadurch wird der Vorrang der Operatoren im Suchausdruck festgelegt (H. Xu; Fushman, 2024). Da Eignungskriterien in klinischen Studien häufig als unstrukturierte Freitexte vorliegen, stellt dies eine erhebliche Herausforderung für deren computergestützte Analyse und Verarbeitung dar (Ross et al., 2010). Infolgedessen verzichten Studienregister bei Suchanfragen oft auf eine detaillierte Berücksichtigung dieser schwer auswertbaren Texte (Chunhua Weng, 2015). Diese Vorgehensweise führt zu ungenauen Suchergebnissen und schränkt die Möglichkeiten für spezifische, kriterienbasierte Suchanfragen erheblich ein (Tu et al., 2011).

Um dieses Problem zu lösen, können Natural Language Processing (NLP) Methoden eingesetzt werden, die eine automatische Analyse und Strukturierung von klinischen Eignungskriterien ermöglichen (Chunhua Weng, 2015).

2.2. Natural Language Processing (NLP)

Natural Language Processing (NLP) ist ein interdisziplinäres Forschungsgebiet der Informatik und Linguistik, das sich mit der Interaktion zwischen Computern und menschlicher Sprache befasst. NLP-Techniken ermöglichen es Maschinen, natürliche Sprache zu verstehen, zu interpretieren und zu generieren, um eine Vielzahl von Aufgaben zu unterstützen (H. Xu; Fushman, 2024). Dazu gehören unter anderem maschinelle Übersetzung, Sprachsteuerung, automatische Textzusammenfassung, Sentiment-Analyse, Textklassifikation, Spracherkennung sowie die Entwicklung von Chatbots. Die Information Extraction (IE) ist ein zentraler Bereich des NLP, der darauf abzielt, spezifische Informationen aus unstrukturierten Textquellen zu extrahieren und in eine strukturierte Form zu überführen. Dabei werden wichtige Daten wie Namen, Ereignisse und Beziehungen identifiziert und systematisch erfasst (Dalianis, 2018). Typische Aufgaben der IE umfassen die Named Entity Recognition (NER), die der Identifizierung und Klassifizierung von Entitäten dient, sowie die Relation Extraction (RE), welche die Beziehungen zwischen den erkannten Entitäten bestimmt (H. Xu; Fushman, 2024).

2.2.1. Named Entity Recognition (NER)

Die Named Entity Recognition befasst sich mit der automatischen Identifizierung und Klassifizierung spezifischer Entitäten in Texten. Ursprünglich lag der Schwerpunkt von NER auf der Erkennung von Personennamen, Orten, Organisationen und Zeitangaben in Nachrichtentexten. Heute umfasst sie ein breiteres Spektrum relevanter Informationen. Im klinischen Kontext dient NER sowohl der Anonymisierung von Texten durch die Erkennung persönlicher Daten als auch der Identifizierung medizinisch relevanter Begriffe wie Symptome, Krankheiten und Medikamenten (Dalianis, 2018). Um Entitäten effektiv erkennen zu können, muss der Text zunächst in kleinere, verarbeitbare Einheiten zerlegt werden. Dieser Prozess wird als Tokenisierung (engl. *Tokenization*) bezeichnet (H. Xu; Fushman, 2024). Token repräsentieren typischerweise Wörter, Wortbestandteile oder einzelne Zeichen, können aber auch Zahlen, Datumsangaben oder Satzzeichen umfassen. Nach der Tokenisierung können NER-Methoden angewendet werden, um jedem Token eine bestimmte Kategorie oder Klasse zuzuweisen.

2.2.2. Relation Extraction (RE)

Die Relation Extraction ist ein zentrales Klassifikationsproblem in der Sprachverarbeitung, insbesondere im biomedizinischen und klinischen Kontext. Während die NER darauf abzielt, relevante Entitäten wie Krankheiten, Symptome oder Medikamente im Text zu erkennen, fokussiert sich die RE auf die Bestimmung der Beziehungen zwischen diesen erkannten Entitäten (Dalianis, 2018). Mit Hilfe der RE können komplexe Zusammenhänge aus unstrukturierten Textdaten extrahiert und analysiert werden. Die Relation Extraction ermöglicht es, Wechselwirkungen zwischen Molekülen zu identifizieren, potenzielle Nebenwirkungen von Medikamenten aufzudecken oder kausale Zusammenhänge zwischen Symptomen und Diagnosen herzustellen (Dalianis, 2018). Um die Ergebnisse von Named Entity Recognition- und Relation Extraction-Methoden evaluieren zu können, ist eine präzise und systematische Annotation der identifizierten Entitäten und Beziehungen erforderlich. Hierfür bietet das Brat Rapid Annotation Tool eine standardisierte Methode zur Annotation und Visualisierung von Textdaten.

BRAT-Format

Das Brat Rapid Annotation Tool (BRAT)², entwickelt von Stenetorp et al. (2012), ist ein weitverbreitetes Werkzeug zur Annotation von Entitäten und Relationen in biomedizinischen und klinischen Texten. Es unterstützt Forscher dabei, Texte manuell zu annotieren und bietet benutzerfreundliche Visualisierungsfunktionen. Zudem ermöglicht es die flexible Anpassung von Annotationsschemata, um spezifische Anforderungen verschiedener Forschungsprojekte zu erfüllen (H. Xu; Fushman, 2024). Für die Analyse unstrukturierter Textdaten werden die Rohdaten typischerweise in Textdateien (.txt) gespeichert. Die Annotationen der Daten im BRAT-Format werden in separaten Annotationsdateien (.ann) erzeugt.

Die Annotationsdateien sind folgendermaßen strukturiert:

- T für Entität (entity)
 - Format: T<number> <entity type> <start character index> <stop character index>
- R für Relation (relation)
 - Format: R<number> <entity type> Arg1: <ID> Arg2: <ID>
- E für Events (events)
 - Format: E<number> <entity type>: <ID> <relation type>: <ID>

²<https://brat.nlplab.org/standoff.html>

Die Annotationen T, R und E werden sequenziell nummeriert, wobei `<number>` eine fortlaufende Zahl ist. Die Indizes `<start index>` und `<end index>`, auch als Offsets bezeichnet, geben die exakten Zeichenpositionen in der zugehörigen Textdatei an. Diese präzise Spezifikation ermöglicht eine genaue Lokalisierung der annotierten Textsegmente. Die `<ID>` Werte fungieren als eindeutige Identifikatoren für Entitäten und Ereignisse und können entweder die Form `T<number>` für Entitäten oder `E<number>` für Ereignisse annehmen (Stenetorp et al., 2012).

Abbildung 2.3 zeigt eine AND-Relation zwischen zwei identifizierten Entitäten aus Eignungskriterien. Dabei wurde "Postmenopausal" als Entität des Typs *Condition* und "hormone replacement therapy" als Entität des Typs *Procedure* erkannt und über eine AND-Relation verknüpft.

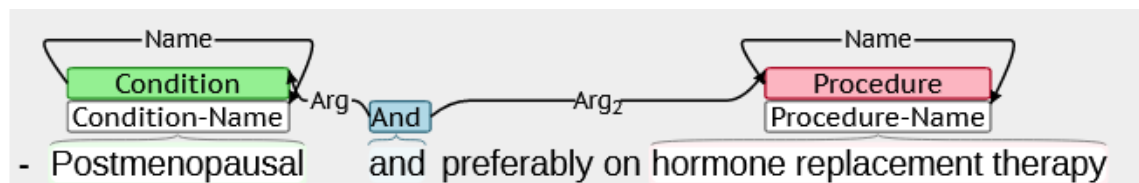


Abbildung 2.3.: Annotierte AND-Verknüpfung zwischen den Entitätstypen *Condition* und *Procedure* im BRAT-Format (N. Dobbins; T. Mullen et al., 2022).

Das BRAT-Format bietet eine effektive Methode zur Annotation logischer AND-, OR- und NOT-Beziehungen zwischen erkannten Entitäten in Eignungskriterien (N. Dobbins; T. Mullen et al., 2022). Durch die Erfassung dieser logischen Verknüpfungen lässt sich eine strukturierte Repräsentation der Freitextkriterien erstellen. Diese logische Strukturierung bildet die Grundlage für eine präzisere und effizientere logische Suche basierend auf den verarbeiteten Eignungskriterien.

2.3. Evaluationsmetriken

Die Evaluierung von Klassifikationen bei Information Extraction-Aufgaben erfolgt durch den Vergleich von manuell annotierten Testdaten (Goldstandard) mit den Vorhersagen von Machine Learning-Modellen. Der Goldstandard dient dabei als Referenz für korrekte Ergebnisse. Für diese überwachten NLP-Aufgaben werden typischerweise die folgenden Evaluationsmetriken verwendet (Biemann et al., 2022).

Konfusionsmatrix

Die Konfusionsmatrix stellt als zweidimensionales Bewertungsschema für Klassifikationsalgorithmen, die vorhergesagten Klassen (Zeilen) den tatsächlichen Klassen (Spalten) gegenüber. Bei binären Klassifikationsproblemen entsteht eine 2×2 Matrix, deren Aufbau in Abbildung 2.4 dargestellt ist (Michelucci, 2024).

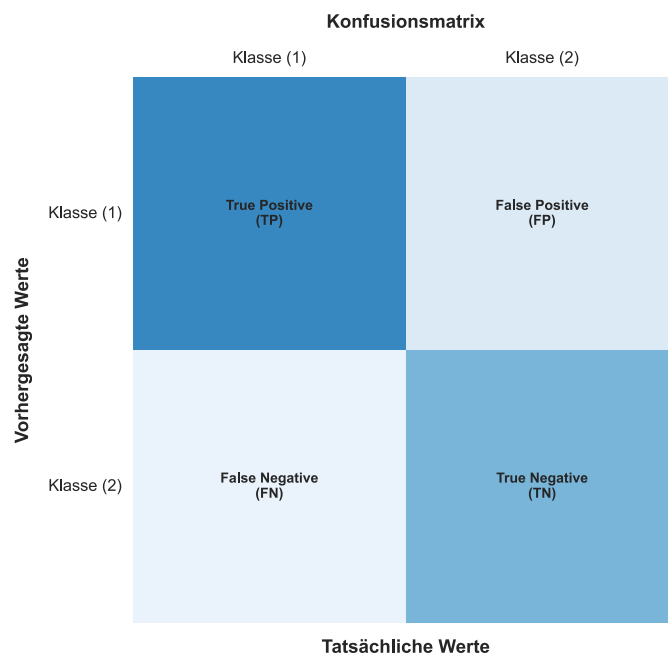


Abbildung 2.4.: Konfusionsmatrix für eine binäre Klassifikation.

Die vorhergesagten Werte von ML-Modellen können in folgende Kategorien klassifiziert werden:

- **True Positive (TP)**: Anzahl der korrekt als Klasse 1 klassifizierten Einträge.
- **True Negative (TN)**: Korrekt der Klasse 2 zugeordnete Instanzen.
- **False Positive (FP)**: Instanzen, die fälschlicherweise Klasse 1 zugeordnet wurden, obwohl sie tatsächlich zu Klasse 2 gehören. In der Statistik als Typ-I Fehler bekannt.
- **False Negative (FN)**: Einträge, die irrtümlich als Klasse 2 klassifiziert wurden, jedoch eigentlich Klasse 1 angehören. Auch als Typ-II Fehler bezeichnet.

Eine Herausforderung bei der Bewertung von Information Extraction-Systemen liegt in der Erfassung von True Negative (TN) Werten, also irrelevante Elemente, die das System korrekterweise nicht zurückgibt. Die Größe von IE-Testdatensätzen macht es unmöglich, alle irrelevanten Elemente für eine Anfrage vollständig aufzulisten. Deshalb werden bei der Evaluation von IE-Aufgaben auf übersehene relevante Inhalte (False Negatives, FN) überprüft. Gleichzeitig werden korrekt gefundene relevante Elemente (True Positives, TP) und fälschlicherweise einbezogene irrelevante Elemente (False Positives, FP) identifiziert (H. Xu; Fushman, 2024). Basierend auf diesen drei Kennzahlen (TP, FN, FP) lassen sich Evaluationsmetriken wie Precision (Genauigkeit) und Recall (Trefferquote bzw. Sensitivität) berechnen (Biemann et al., 2022).

Precision (P)

Die Precision gibt das Verhältnis der korrekten Vorhersagen an, bei denen das Modell True Positiv (TP) Werte vorhersagt.

$$P = \frac{TP}{TP + FP} \quad (2.1)$$

Über die Ermittlung der Precision lässt sich folgende Frage beantworten: Wie viele der als positiv identifizierten Eingaben (z.B. in Klasse 1) sind von allen als positiv identifizierten Eingaben korrekt?

Recall (R)

Der Recall gibt an, wie gut ein Modell die True Positiv (TP) Fälle in jeder Kategorie vorhersagen kann und wie effektiv es dabei ist.

$$R = \frac{TP}{TP + FN} \quad (2.2)$$

Der Recall misst also den Anteil der Elemente aus Klasse 1, die das Modell korrekt identifizieren kann. Bezieht sich Klasse 1 auf Patienten mit einer Erkrankung, gibt der Recall an, wie viele erkrankte Patienten unter allen erkrankten Patienten korrekt identifiziert werden. Sowohl Precision als auch Recall weisen Schwächen auf. Eine hohe Precision ist leicht zu erreichen, indem man nur wenige, sehr sichere Ergebnisse liefert. Ein perfekter Recall hingegen wird erreicht, indem einfach alle möglichen Elemente zurückgegeben werden. Um diese Extreme auszugleichen, wird der F_β -Score verwendet (H. Xu; Fushman, 2024).

 F_β -Score

Der F_β -Score ist eine parametrisierte Metrik zur Evaluation von Klassifikatoren, wobei $\beta \in \mathbb{R}^+$ die relative Gewichtung von Precision und Recall steuert. In der Praxis finden primär die Werte $\beta \in 0.5, 1, 2$ Anwendung (Biemann et al., 2022). Die generalisierte Formel lautet:

$$F_\beta = (1 + \beta^2) \cdot \frac{P \cdot R}{(\beta^2 \cdot P) + R} \quad (2.3)$$

Für $\beta < 1$ wird die Precision stärker gewichtet, für $\beta > 1$ der Recall. Bei $\beta = 1$ resultiert das harmonische Mittel von Precision und Recall (Michelucci, 2024). Der **F1-Score (F1)** ist ein Spezialfall des F_β -Scores mit $\beta = 1$ und ist besonders geeignet für die Evaluation von Modellen bei unbalancierten Datensätzen (Naidu et al., 2023). Er wird wie folgt berechnet:

$$F1 = 2 \cdot \frac{(P \cdot R)}{(P + R)} \quad (2.4)$$

Der F1-Score bietet eine balancierte Bewertung der Modelleistung über alle Klassen hinweg, indem Precision und Recall gleich gewichtet werden (Michelucci, 2024).

Im Folgenden werden Methoden zur Verarbeitung von NLP- und speziell IE-Aufgaben vorgestellt, die mittels der Metriken Precision, Recall und F1-Score auf entsprechenden Goldstandards evaluiert werden können. Neben den traditionell etablierten regelbasierten Ansätzen werden Machine Learning-Modelle vorgestellt, die sich für diese Aufgaben eignen. Während in der Vergangenheit hauptsächlich Pre-trained Language Models wie BERT-Modelle dominierend waren, gewinnen aktuell Large Language Models wie GPT-4 für IE-Aufgaben zunehmend an Bedeutung (X. Xu et al., 2023). Die Arbeit fokussiert sich dabei auf die untersuchten und evaluierten LLMs sowie den Vergleich mit PLMs, die derzeit als SOTA-Modelle in der Analyse und Strukturierung von Eignungskriterien gelten.

3. Techniken zur maschinellen Textstrukturierung

Dieses Kapitel stellt Methoden zur Verarbeitung biomedizinischer Textdaten mittels NER- und RE-Aufgaben vor. Während regelbasierte Ansätze im Bereich des Natural Language Processing traditionell dominieren, verschiebt sich der Forschungsschwerpunkt seit Beginn des 21. Jahrhunderts zunehmend zu Machine Learning- und Deep Learning-Methoden (H. Xu; Fushman, 2024). Als Vertreter der Machine Learning-Methoden werden die in dieser Arbeit untersuchten Large Language Models (LLMs) und deren transformerbasierten Architekturen behandelt. Dabei wird zwischen Pre-trained Language Models (PLMs) mit Encoder-only Architektur wie BERT und Large Language Models mit Decoder-only Architektur wie GPT-4 unterschieden. Die Anwendung von LLMs für Information Extraction-Aufgaben wird durch verschiedene Prompt- und Fine-Tuning Methoden ermöglicht. Eine Prompt ist dabei die Eingabeaufforderung an ein LLM, um spezifische Antworten zu generieren. Zero-Shot Ansätze lösen Aufgaben ohne Beispiele, während Few-Shot Methoden einige wenige Beispiele bereitstellen, um die Modelleleistung zu verbessern. Erweiterte Prompt-Methoden zielen darauf ab, die Leistung der Modelle durch kurze Ergänzungen innerhalb der Prompt zu steigern. Für die domänenspezifische Anpassung der Modelle werden effiziente Parameter-Efficient Fine-Tuning (PEFT) Techniken vorgestellt. Zudem werden Inferenz-Parameter vorgestellt, die angepasst werden können, um die Ausgabe von LLMs zu steuern. Abschließend werden aktuelle Herausforderungen dargestellt, die sich aus der Architektur von Large Language Models ergeben und deren Bearbeitung von NLP-Aufgaben beeinträchtigen können.

3.1. Regelbasierte Methoden

Regelbasierte Programmiermethoden basieren auf vordefinierten Regeln, die das Programmverhalten steuern. Diese Regeln werden von Softwareentwickelnden erstellt und ergeben sich aus der Analyse des Programmablaufs sowie der Beziehung zwischen Ein- und Ausgabe (Biemann et al., 2022). Regelbasierte Ansätze in der Textverarbeitung reichen von einfachen String-Vergleichen bis hin zu komplexen semantischen Grammatiken und sind besonders effektiv bei strukturierten Dokumenten oder wiederkehrenden Mustern (H. Xu; Fushman, 2024). Beispielsweise können reguläre Ausdrücke oder einfache String-Matches verwendet werden, um wiederkehrende Abschnittsüberschriften in Dokumenten zu finden oder Wortfolgen zu identifizieren, die auf eine Kategorie von Interesse hinweisen (H. Xu; Fushman, 2024). Reguläre Ausdrücke, engl. Regular Expression (Regex), sind ein prominentes Beispiel für regelbasierte Methoden und werden in den meisten Programmiersprachen und Betriebssystemen verwendet, um präzise Textmuster zu suchen und zu ersetzen (Biemann et al., 2022).

Regelbasierte Methoden gewährleisten oft eine hohe Präzision bei der Verarbeitung bekannter Datenstrukturen. Sie sind jedoch wenig flexibel im Umgang mit unerwarteten oder unstrukturierten Eingaben und erfordern häufig einen erheblichen manuellen Aufwand sowie ein tiefgehendes Verständnis der zugrunde liegenden Problematik (Dalianis, 2018).

3.2. Large Language Models (LLMs)

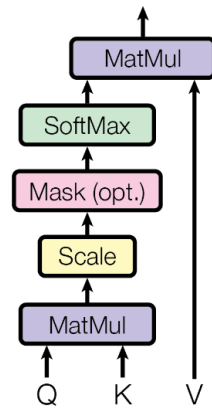
Language Modeling (LM) ist ein grundlegendes NLP-Verfahren zur Vorhersage von Wortsequenzen auf der Basis der Wahrscheinlichkeitsverteilung von Wörtern. Pre-trained Language Models (PLMs) basieren auf Machine Learning-Methoden und insbesondere auf der parallelisierbaren Transformerarchitektur mit Self-Attention-Mechanismus (Hadi et al., 2023). Diese Modelle verwenden neuronale Netze, um eine Vektordarstellung von Wörtern zu erlernen, welche den Kontext der betrachteten Wörter oder Satzteile berücksichtigt. PLMs werden zunächst auf umfangreichen Textkorpora mit engl. Billion (B) oder engl. Trillion (T) Wörtern trainiert und können anschließend für spezifische NLP-Aufgaben über ein Fine-Tuning angepasst werden (Zhao et al., 2023). Transformerbasierte PLMs mit mehreren Billion-Parametern werden allgemein als Large Language Models (LLMs) bezeichnet. Ein Beispiel dafür ist das proprietäre GPT-3.5 Modell mit 175B-Parametern, welches auf einer großen Datenmenge mit mehreren Trillion-Token trainiert wurde (H. Xu; Fushman, 2024). PLMs mit einigen engl. Million (M) Parametern werden als Small Language Models bezeichnet, sofern die Anzahl der Parameter unter einer Billion liegt (Zhao et al., 2023). Das BERT-Modell mit 333M-Parametern ist ein PLM und kann als Small Language Model eingestuft werden, da es im Vergleich zu LLMs eine kleinere Modellgröße aufweist und auf kleineren Datensätzen trainiert wurde (Hadi et al., 2023).

3.2.1. Transformerarchitektur

Die von Vaswani et al. (2017) eingeführte Transformerarchitektur bildet die Grundlage für moderne Large Language Models. Diese Architektur verbessert die Verarbeitung sequenzieller Daten in neuronalen Netzen durch den Einsatz des Self-Attention-Mechanismus (Vaswani et al., 2017). Im Gegensatz zu rekurrenten Deep Learning-Modellen wie Recurrent Neural Networks (RNNs) und Long Short-Term Memorys (LSTMs) ermöglicht der Transformer eine erhöhte Parallelisierung und eine Verarbeitung von Langzeitabhängigkeiten in den Textsequenzen (Bahdanau et al., 2016). Die Fähigkeit, Eingaben variabler Länge zu verarbeiten in Verbindung mit der Skalierbarkeit der Transformerarchitektur, haben zu Fortschritten bei der Verarbeitung natürlicher Sprache geführt (Zhao et al., 2023). Die Transformerarchitektur hat sich als State-of-the-Art in vielen NLP-Aufgaben etabliert und eröffnet neue Forschungsperspektiven in Bereichen wie maschineller Übersetzung und Textgenerierung (Raiaan et al., 2024). Das zentrale Konzept des Transformers ist der Attention-Mechanismus, der dem Modell eine selektive Fokussierung auf verschiedene Segmente des Eingabekontextes ermöglicht. Im Gegensatz zu sequenziellen Verarbeitungsansätzen ermöglicht der Transformer die parallele Verarbeitung mehrerer Kontextabschnitte und berücksichtigt dabei die als relevant eingestuft Teile (Naveed et al., 2024). Diese Funktionalität wird durch die Implementierung von zwei Attention-Mechanismen, wie in Abbildung 3.1 dargestellt, realisiert.

Attention!

Scaled Dot-Product Attention



Multi-Head Attention

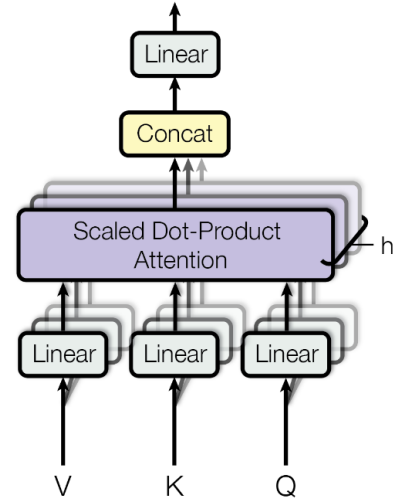


Abbildung 3.1.: Scaled Dot-Product Attention und Multi-Head Attention, die aus mehreren parallel laufenden Attention-Layer besteht (Vaswani et al., 2017).

Der Attention-Mechanismus im Transformermodell basiert auf drei wesentlichen Komponenten: Query Q , Key K und Value V . Diese Matrizen werden aus der Eingabeschicht x durch lineare Transformationen abgeleitet:

$$Q = xW^Q \quad (3.1)$$

$$K = xW^K \quad (3.2)$$

$$V = xW^V \quad (3.3)$$

Hierbei ist x eine Matrix der Dimension $s \times d$, wobei s die Sequenzlänge des Inputs und d die *Embedding-Dimension* repräsentiert. Die Matrizen W^Q , W^K und W^V sind Gewichtsmatrizen, die durch Optimierungsprozesse trainiert werden. Die **Scaled Dot-Product Attention** aus Abbildung 3.1 wird folgendermaßen berechnet:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (3.4)$$

Dabei ist d_k die Dimension der Query- und Key-Vektoren. Die Skalierung mit $\frac{1}{\sqrt{d_k}}$ dient der Stabilisierung des Gradienten bei größeren Dimensionen. Die Softmax-Funktion normalisiert das Ergebnis der Berechnung QK^T zu einer Wahrscheinlichkeitsverteilung. Das Ergebnis wird dann auf die Value-Matrix V angewendet, um die gewichtete Summe der Werte zu erhalten. Die Gewichte werden so trainiert, dass $Q_i^T K_j$ die Relevanz des j -ten Eingabeelements für das i -te Ausgabeelement quantifiziert (Vaswani et al., 2017).

Der **Muli-Head Attention**-Mechanismus erweitert das Konzept der einfachen Attention, indem er multiple parallele Attention-Operationen ermöglicht. Statt einer einzelnen Attention-Funktion mit d_{model} -dimensionalen Keys, Values und Queries, werden diese h -mal linear auf niedrigere Dimensionen d_k , d_k und d_v projiziert:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O \quad (3.5)$$

wobei

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \quad (3.6)$$

Die Projektionsmatrizen sind dabei trainierbare Parameter $W_i^Q \in \mathbb{R}^{d_{model} \times d_k}$, $W_i^K \in \mathbb{R}^{d_{model} \times d_k}$, $W_i^V \in \mathbb{R}^{d_{model} \times d_v}$, $W^O \in \mathbb{R}^{hd_v \times d_{model}}$. Die Verkettung der Ausgabe von jedem Attention-Head, stellt die finale Ausgabe dar (Vaswani et al., 2017). Dieser Mechanismus ermöglicht es dem Transformermodell, komplexe kontextabhängige Repräsentationen zu erzeugen und trägt wesentlich zu seiner Leistungsfähigkeit in verschiedenen NLP-Aufgaben bei (Zhao et al., 2023).

Transformeraufbau und -klassen

Die allgemeine, auf dem Attention-Mechanismus aufbauende Architektur von Transformern ist in Abbildung 3.2 über sieben Hauptkomponenten dargestellt.

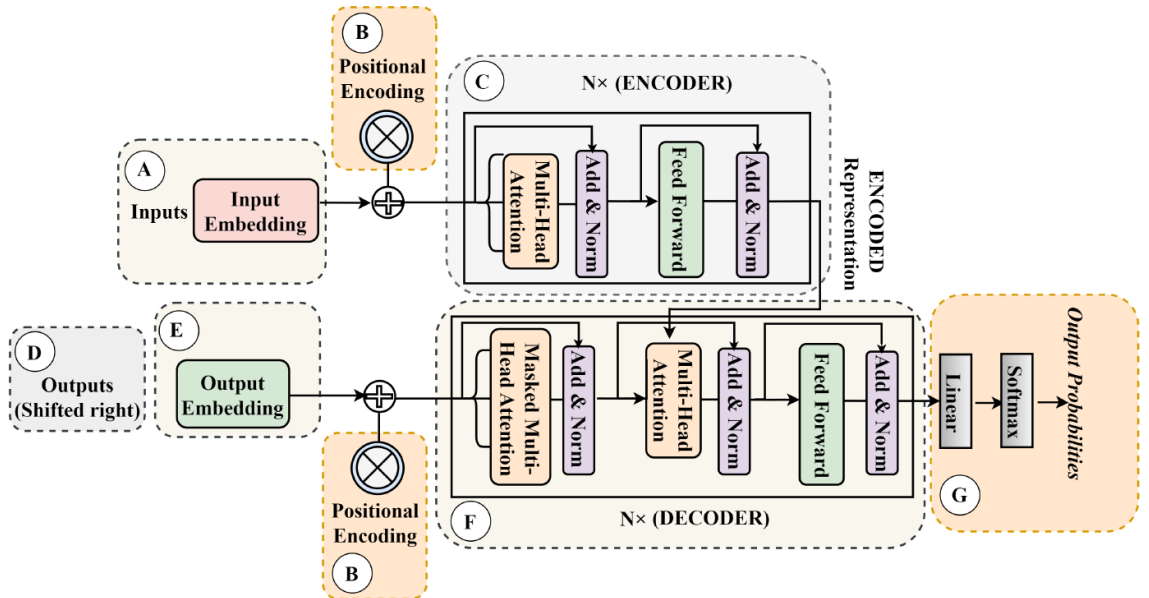


Abbildung 3.2.: Transformerarchitektur mit N Encoder- und Decoderblöcken (Raiaan et al., 2024).

(A) **Input-Embeddings**

Durch *Tokenization* werden Benutzereingaben im Textformat in Token verarbeitet, die von Large Language Models als Trainingsdaten verwendet werden. Diese Token können Zeichen, Subwörter, Symbole oder Wörter sein, abhängig von den Eigenschaften der LLMs (Kudo, 2018). Da diese Modelle nur numerische Daten verarbeiten können, werden die Token in *Input-Embeddings* umgewandelt. Diese Embeddings sind numerische Vektorrepräsentationen von Wörtern im mehrdimensionalen Raum, wobei ähnliche Begriffe nahe beieinander liegen. Sie ermöglichen es dem Modell, Wortbedeutungen und -beziehungen zu verstehen und zu verarbeiten (Hadi et al., 2023).

(B) **Positional Encoding**

Bei der Verarbeitung natürlicher Sprache ist die Erfassung der Wortfolge für die Interpretation der Satzbedeutung zentral. *Positional Encoding* codiert dabei jede Wortposition in der Eingabesequenz als eine Reihe von Ganzzahlen. Diese Technik ermöglicht es neuronalen Netzen in Kombination mit *Input-Embeddings*, die syntaktische Struktur und semantische Nuancen von Sätzen zu erfassen, wodurch Large Language Models in der Lage sind, präzise und kontextadäquate Sprachausgaben zu erzeugen (Raiaan et al., 2024).

(C) **Encoder**

Der Encoder besteht aus einem Stapel von N identischen Schichten (Layer), als Encoder-Blöcke bezeichnet, mit je zwei Unterschichten. Die erste Unterschicht ist ein Multi-Head Self-Attention Mechanismus, die zweite ein neuronales Feed-Forward Netzwerk. Der Encoder generiert *Contextual-Embeddings* $\tilde{x}_1, \dots, \tilde{x}_{T_{\text{src}}}$ für die *Input-Token* $v_1, \dots, v_{T_{\text{src}}}$ (Vaswani et al., 2017). Diese Architektur entspricht genau der des BERT-Modells, wobei das BERT-Basismodell $N = 12$ Encoder-Layer hat und BERT-Large auf $N = 24$ gestapelten Layern aufgebaut ist (Raiaan et al., 2024). Die in dem letzten Layer erzeugten *Text-Embeddings* werden als $\tilde{x}_1, \dots, \tilde{x}_{T_{\text{src}}}$ bezeichnet (Vaswani et al., 2017).

(D) **Outputs**

Während des Trainingsprozesses erwirbt der Decoder die Fähigkeit, die Vorhersage des nächsten Token durch Analyse der vorhergehenden Token zu optimieren. Diese Fähigkeit ist entscheidend für die Generierung kohärenter und kontextsensitiver Sequenzen (Raiaan et al., 2024). Die Ausgabe der Encoder-Blöcke dient in einer *Encoder-Decoder* Transformerarchitektur, wie sie beim T5-Modell verwendet wird, als Eingabe für den Decoder (Roberts et al., 2020). Alleine auf Decodern basierende LLMs mit einer hohen Parameteranzahl (*Decoder-only*) wie GPT-3 erhalten ihre Eingabe während des Trainings basierend auf einer großen Menge von Textdaten (Raiaan et al., 2024). Bei *Decoder-only* Modellen wie BERT wird jeder Encoder-Block auf den Output des vorherigen Encoder-Blocks angewendet (Devlin et al., 2019).

(E) **Output-Embeddings**

LLMs wie GPT-Modelle verarbeiten Text, indem sie ihn in numerische Darstellungen umwandeln. Diese *Input-Embeddings* werden durch Positionskodierung ergänzt, um die Wortstellung zu erfassen. Das Modell erzeugt Vorhersagen, die in *Output-Embeddings* umgewandelt werden. Eine Verlustfunktion bewertet die Genauigkeit dieser Vorhersagen im Vergleich zu Zielwerten und steuert die Optimierung der Modellparameter während des Trainings. In der Anwendung (Inferenz) werden die vom Modell vorhergesagten Wahrscheinlichkeiten für jedes Token verwendet, um Text zu generieren, indem sie auf das Vokabular abgebildet werden (Raiaan et al., 2024).

(F) **Decoder**

Der Decoder besteht wie der Encoder aus einem Stapel von N identischen Layern. Zusätzlich zu den zwei Teilschichten in jedem Encoder-Layer fügt der Decoder eine dritte Teilschicht ein, welche die *Multi-Head Attention* über den Ausgang des Encoder-Layers ausführt (Vaswani et al., 2017). Der Decoder verarbeitet sowohl *Input Embeddings* als auch *Output-Embeddings*, die positionell kodiert sind. Seine Hauptaufgabe ist die Generierung von Ausgabesequenzen, basierend auf codierten Eingabesequenzen. Während des Trainings, lernt der Decoder, das nächste Wort anhand vorheriger Wörter vorherzusagen. PLMs verwenden dabei häufig die Masked Language Modeling (MLM) Methode. Bei dieser werden Token oder Tokensequenzen zufällig maskiert und das Modell wird trainiert, diese maskierten Elemente basierend auf dem umgebenden Kontext vorherzusagen (Naveed et al., 2024). GPT-Modelle, mit Decoder-only Architektur, verwendet *Masked Self-Attention*, um Eingaben ohne separaten Encoder zu verarbeiten. Bei Übersetzungsaufgaben sagt ein GPT-Modell sequenziell das nächste Token in der Zielsprache vorher, basierend auf dem bisher generierten Text und der Eingabesequenz (Raiaan et al., 2024).

(G) **Linear-Layer und Softmax**

Der *Linear-Layer* im Decoder ist eine vollständig vernetzte neuronale Netzwerkschicht. Deren Aufgabe ist es, die *Output-Embeddings* in einen höherdimensionalen Raum zu transformieren. Dieser Schritt ist notwendig, um die *Output-Embeddings* wieder in den ursprünglichen Eingaberaum zurückzuführen. Diese Transformation erhöht die Ausdruckskraft der Repräsentation und ermöglicht es dem Modell, komplexere Muster und Beziehungen in den Daten zu erfassen. Im Anschluss an die lineare Schicht wird die Softmax-Funktion angewendet. Diese erzeugt eine Wahrscheinlichkeitsverteilung für jedes Ausgabtoken im entwickelten Vokabular. Sie normalisiert die Ausgaben der linearen Schicht, sodass sie sich zu 1 addieren und als Wahrscheinlichkeiten interpretiert werden können. Dies ermöglicht es dem Modell, probabilistische Ausgabetoken zu generieren. Jeder Wert in der resultierenden Verteilung repräsentiert die Wahrscheinlichkeit für ein bestimmtes Token das Nächste in der Sequenz zu sein (Raiaan et al., 2024).

In transformerbasierten Pre-trained Language Models werden verschiedene Attention-Strategien angewendet. In Encoder-Decoder Architekturen wird **Cross-Attention** verwendet, wobei die Ausgaben des Encoders als Queries dienen, während Keys und Values vom Decoder stammen (Naveed et al., 2024). **Self-Attention** berechnet die Attention basierend auf Q, K, V aus demselben Encoder- oder Decoder Block (Naveed et al., 2024). Die Decoder-only Architektur nutzt **Masked Self-Attention** wobei die *Self-Attention* so eingeschränkt, dass Abfragen an jeder Position nur für alle Key-Value Paare bis zu dieser Position möglich sind (Raiaan et al., 2024).

Transformer-Klassen

Die Transformerarchitektur ermöglicht die Entwicklung von drei Klassen Pre-trained Language Models. Encoder-only Modelle wie BERT, Encoder-Decoder Modelle wie T5 und Decoder-only Modelle wie GPT-3. Encoder-only und Encoder-Decoder Modelle werden üblicherweise mit Techniken wie Masked Language Modeling (MLM) und Next Sentence Prediction (NSP) trainiert. Decoder-only Modelle basieren auf autoregressiven Language Models, die das nächste Token basierend auf den vorherigen Token vorhersagen (Tian; Jin et al., 2023). Abbildung 3.3 veranschaulicht die Entwicklung aktueller Pre-trained Language Models sowie deren Vorgängermodelle wie Word Embedding Modelle, beispielsweise Word2Vec.

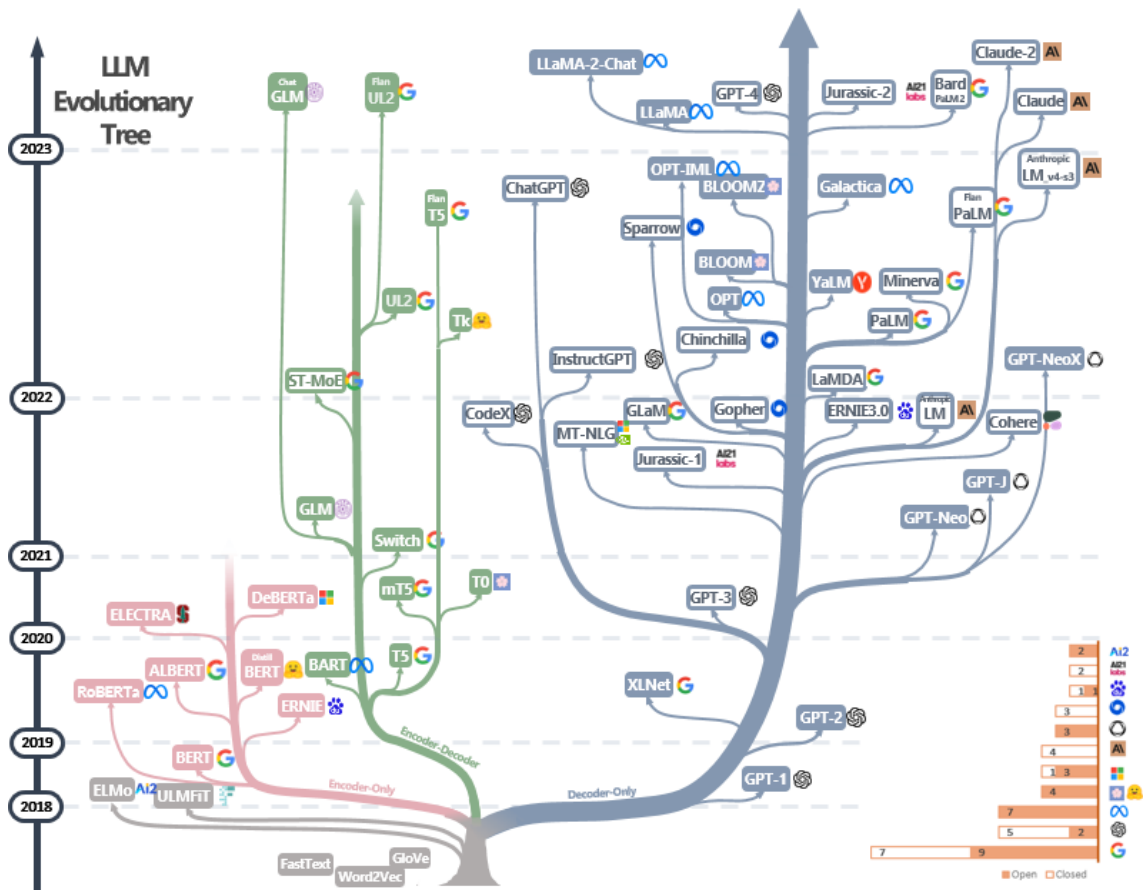


Abbildung 3.3.: Die Entwicklung moderner, transformerbasierter PLMs als Baumstruktur. Die Verzweigungen repräsentieren **Encoder-only**, **Encoder-Decoder** und **Decoder-only** Modelle in separaten Ästen. Open-Source Modelle werden durch gefüllte Quadrate gekennzeichnet, Closed-Source Modelle durch leere Quadrate. Zusätzlich werden Word Embedding Modelle vorgestellt (J. Yang et al., 2023).

Encoder-Decoder

Raffel et al. (2020) haben gezeigt, dass nahezu alle NLP-Aufgaben in Sequenz-zu-Sequenz Formate umformuliert werden können, bei denen eine Eingabesequenz in eine entsprechende Ausgabesequenz übersetzt wird. Modelle wie T5 profitieren von der speziell für diese Aufgaben konzipierten Encoder-Decoder Architektur (Minaee et al., 2024). Das T5 (Text-to-Text Transfer Transformer) Modell wurde auf 750GB englischen Webtext vortrainiert, wobei beschädigter Text wiederhergestellt wurde (Raffel et al., 2020). Diese als Masked Language Modeling bezeichnete Methode maskiert bestimmte Textteile durch Masken ([X], [Y]), welche das Modell rekonstruieren soll. T5 wurde mit 1024 TPU-v3 Einheiten auf 1B-Token trainiert. Die größte Version hat insgesamt 24 Layer und 128 Attention-Heads und hat eine Größe von 11B-Parametern. Nach einem domänenspezifischen Fine-Tuning zeigt es gute Leistungen in verschiedenen NLP-Aufgaben wie Textklassifikation, Übersetzung, Zusammenfassung und Frage-Antwort Systemen und erreicht State-of-the-Art Ergebnisse in einigen Benchmarks (Raiaan et al., 2024).

Encoder-only

Encoder-basierte Pre-trained Language Models werden hauptsächlich für Aufgaben wie die Textklassifikation eingesetzt, bei der Klassenbezeichnungen für Eingabetexte vorhergesagt werden sollen. Prominente Beispiele dieser Modellkategorie sind BERT (Devlin et al., 2019) sowie die Modell-Varianten RoBERTa (Y. Liu et al., 2019) und ALBERT (Lan et al., 2020). Diese Modelle verwenden häufig Masked Language Modeling als Trainingsmethode, bei der sie lernen, maskierte Wörter basierend auf ihrem Kontext vorherzusagen. Durch dieses Verfahren erzeugen sie leistungsfähige Repräsentationen von Textdaten, die für verschiedene nachgelagerte NLP-Aufgaben genutzt werden können (J. Yang et al., 2023).

BERT-Modelle

Bidirectional Encoder Representations from Transformers (BERT) ist ein Encoder-only PLM, das auf einem Korpus von 800M-Wörtern aus BookCorpus und 2.5B-Wörtern aus Wikipedia vortrainiert wurde. Es verwendet einen Self-Attention-Mechanismus und existiert in zwei Varianten. Das Basismodell mit 110M-Parametern (12 Encoder-Blöcke, 8 Attention-Heads) und das große Modell mit 340M-Parametern (24 Encoder-Blöcke, 16 Attention-Heads). Das Vortraining erfolgte über Masked Language Modeling- und Next Sentence Prediction-Techniken auf Nvidia A100- und V100 GPUs. Beim MLM-Methoden wurden 15% der Eingabe-Token zufällig maskiert. Die NSP-Aufgabe bestand darin, die korrekte Reihenfolge von zwei aufeinanderfolgenden Textsegmenten vorherzusagen, wobei das Modell lernen sollte, ob zwei Eingabesätze in der richtigen Reihenfolge stehen (H. Xu; Fushman, 2024). BERT kann durch Fine-Tuning für verschiedene NLP-Aufgaben wie Fragebeantwortung sowie für Named Entity Recognition- und Relation Extraction-Aufgaben optimiert werden (Devlin et al., 2019). Dabei werden annotierte Trainingsdaten für NER-Aufgaben häufig im Beginning-Inside-Outside (BIO) Format prozessiert. Für RE-Aufgaben können in BRAT annotierte Daten beispielsweise in entsprechende JSON-Formate konvertiert werden (N. Dobins; T. Mullen et al., 2022). Im Folgenden sind BERT-Varianten aufgeführt, die für NER- und RE-Aufgaben zur Extraktion und Strukturierung von Informationen aus klinischen Texten eingesetzt wurden (Y. Yang et al., 2023),

- **RoBERTa** (A Robustly Optimized BERT Pretraining Approach) optimiert BERT durch längeres Training mit größerer Batch-size und mehr Daten, eliminiert die NSP-Aufgabe und verwendet dynamische Maskierung der Trainingsdaten (Y. Liu et al., 2019).
- **RoBERTa-MIMIC-Trial** ist ein RoBERTa-Modell mit Fine-Tuning auf klinischen Dokumenten aus dem MIMIC-III Korpus (Johnson et al., 2016), PubMed Abstracts und Eignungskriterien aus klinischen Studien (Tian; Erdengasileng et al., 2021).
- **BioBERT** (Biomedical Language Understanding Evaluation BERT) ist ein BERT-Modell mit Fine-Tuning auf PubMed- und PMC-Artikeln. Es existiert in einer *Base*- (110M-Parameter) und einer *Large*-Variante (345M-Parameter) mit Training auf 21B-Wörtern (Lee et al., 2019).
- **BlueBERT** stellt ein BERT-Modell mit Fine-Tuning auf PubMed-Artikeln und MIMIC-III Daten dar (Peng et al., 2019).
- PubMedBERT ist ein auf PubMed-Artikeln neu trainiertes BERT-Modell (Gu et al., 2021).
- **ClinicalBERT** ist ein BERT-Modell, das durch Fine-Tuning auf dem MIMIC-III Datensatz optimiert wurde (Huang et al., 2020).
- **CT-BERT**(Clinical Trial BERT) ist ein Framework für die Information Extraction aus klinischen Studien, das verschiedene vortrainierte BERT-Modelle (BERT, BioBERT, ClinicalBERT, BlueBERT) auf dem FRD-Datensatz mit annotierten Eignungskriterien (vgl. Abschnitt 4.2) einem Fine-Tuning unterzieht (X. Liu et al., 2021).
- **R-BERT** ist eine Erweiterung des BERT-Modells für Relation Extraction-Aufgaben, die dessen Architektur anpasst, um neben dem Satzkontext auch Informationen über die Beziehungen zwischen Entitäten zu erfassen und zu klassifizieren (S. Wu; He, 2019).
- **SciBERT** ist ein BERT-Modell mit Fine-Tuning auf 1.14M-Texten aus den Bereichen Informatik und Biomedizin aus dem Semantic Scholar (Beltagy et al., 2019).
- **ecBERT** ist ein BERT-Modell mit Fine-Tuning auf annotierten Eignungskriterien aus dem Chia-Datensatz (vgl. Abschnitt 4.2) (K. Wu; Aklilu, 2021).
- **ClinicalTrialBERT** ist ein auf ClinicalBERT basierendes Modell, mit Fine-Tuning auf über 442000 Eignungskriterien (Y. Yang et al., 2023).

Decoder-only

Autoregressive LLMs verwenden eine Decoder-only Architektur und generieren Text sequenziell. In diesen Modellen sind die Attention-Layer so konzipiert, dass sie in jeder Phase ausschließlich auf die vorhergehenden Token der Sequenz zugreifen können. Das Vortraining dieser Modelle basiert typischerweise auf der Vorhersage des nächsten Token in einer Sequenz. Bei der Textgenerierung berechnen diese Modelle das nächste Ausgabtoken auf Grundlage einer Wahrscheinlichkeitsverteilung, wobei jedes neue Token unter Berücksichtigung aller zuvor erzeugten Token vorhergesagt wird (H. Xu; Fushman, 2024). Häufig verwendete Decoder-only LLMs sind GPT-Modelle wie GPT-3 von OpenAI (J. Yang et al., 2023). Daneben existieren auch Open-Source LLMs wie Modelle der Llama-Familie von Meta AI, welche im Gegensatz zu den GPT-Modellen die Modellgewichte der Forschungsgemeinschaft unter einer nicht kommerziellen Lizenz zur Verfügung stellen (Minaee et al., 2024). Im Folgenden werden die in dieser Arbeit für IE-Aufgaben verwendeten Open-Source LLMs mit Decoder-only Architektur vorgestellt. Weiterhin werden Closed-Source Modelle wie GPT-4o aufgeführt, die zum Vergleich herangezogen wurden.

3.2.2. Open-Source Modelle

Zunächst werden die Basismodelle der in dieser Arbeit untersuchten Open-Source LLMs hinsichtlich Eigenschaften wie Kontextlänge und Architektur beschrieben. Anschließend folgt eine Beschreibung der jeweiligen instruktionsoptimierten Varianten, die durch Fine-Tuning auf Instruktionsdatensätzen entwickelt wurden.

- **Llama2-70B** von Meta AI hat eine Kontextlänge von 4000 Token. Das Modell basiert auf der Standard Decoder-only Architektur aus Vaswani et al. (2017) und wurde auf 2000 NVIDIA A100 (80GB) GPUs mit einer Mischung aus öffentlichen Webdaten mit 2T-Token über 25 Tage trainiert (Zhao et al., 2023). Das Llama2-70B Chat-Modell mit Fine-Tuning wurde auf öffentlichen Datensätzen mit Instruktionen und über 1M neu annotierten Beispielen optimiert. Der Schwerpunkt lag dabei auf der Verbesserung der Dialogfähigkeit und der Anpassung an menschliche Präferenzen (Touvron et al., 2023).
- **Llama3-8B** von Meta AI ist ein aktuelles Open-Source LLM mit einer Kontextlänge von 8000 Token. Die Architektur unterscheidet sich nicht wesentlich von Llama2. Das 8B-Modell besteht aus 32 Layern, einer Modelldimension von 4096 und 32 Attention-Heads. Im Vergleich zu Llama2 wurde Llama3 auf einem erweiterten und qualitativ verbesserten und aktuellen Datenkorpus trainiert, der sich aus 50% Allgemeinwissen, 25% mathematischen und logischen Texten, 17% Code und 8% mehrsprachigen Quellen zusammensetzt. Das Training erfolgte auf einem Cluster mit 16000 NVIDIA H100 (80GB) GPUs und umfasste insgesamt 1.3M GPU-Stunden. Für das Fine-Tuning des Llama3-8B Instruktionsmodells wurden öffentlich verfügbare Instruktionsdatensätze sowie über 10M manuell annotierte Beispiele verwendet (Dubey et al., 2024).
- **Llama3-70B** von Meta AI hat eine Kontextlänge von 8000 Token. Es wurde mit 6.4M GPU-Stunden auf der gleichen Datenbasis wie das 8B-Modell trainiert. Die Architektur besteht aus 80 Layern, einer Modelldimension von 8192 mit 64 Attention-Heads. Auch das Instruktionsmodell mit Fine-Tuning unterscheidet sich lediglich in der Parameteranzahl von dem 8B-Modell (Dubey et al., 2024).
- **Gemma2-27B** von Google AI hat eine Kontextlänge von 8192 Token. Das Modell wurde auf 13T-Token mit englischen Datensätzen trainiert, darunter Webdaten, Code und wissenschaftliche Artikel. Die Architektur besteht aus 46 Layern, einer Modelldimension von 4608 und 32 Attention-Heads. Das Gemma2 Instruktionsmodell wurde einem Fine-Tuning auf einem Datensatz mit englischen Textdaten und annotierten Eingabe-Ausgabe Instruktionen unterzogen (Team et al., 2024).

- **Qwen2-72B** hat eine Kontextlänge von 131072 Token. Das Modell wurde auf einem Datensatz mit 7T-Token trainiert, der Code, Mathematik und mehrsprachige Daten enthielt. Der Versuch, Qwen2 auf 12T-Token zu trainieren, brachte keine Verbesserung. Die Transformerarchitektur besteht aus 80 Layern, einer Modelldimension von 8192 und 64 Attention-Heads. Für das Instruktionsmodell mit Fine-Tuning wurde ein Datensatz von 500000 Beispielen erstellt, der Instruktionen, Programmieraufgaben, Mathematikaufgaben, Logikaufgaben, Rollenspiele, Mehrsprachigkeit und Sicherheitsaspekte abdeckt (A. Yang et al., 2024).

3.2.3. Closed-Source Modelle

In diesem Unterkapitel werden die in dieser Arbeit verwendeten proprietären Large Language Models vorgestellt. Dabei werden die Kontextlängen der Modelle beschrieben und ihre Nutzung über die entsprechende API erläutert.

- **GPT-4o** verfügt über eine Kontextlänge von 128000 Token. Das Modell ist jedoch auf 30000 Token per Minute (TPM) limitiert. Während das Vorgängermodell GPT-4 auf öffentlichen Internetdaten und lizenzierten Daten von Drittanbietern bis September 2021 trainiert wurde, erweitert GPT-4o den Trainingshorizont bis Oktober 2023 (OpenAI et al., 2024). Das Training von GPT-4o umfasst neben Webdaten und Code auch mathematische Inhalte sowie multimodale Daten wie Bilder, Audio und Videos. Da sowohl GPT-4 als auch GPT-4o als Closed-Source Modelle entwickelt wurden, liegen keine offiziellen Angaben zu ihrer Architektur, Parameteranzahl oder dem Umfang des Trainingsdatensatzes vor. Die GPT-4 Modelle performen deutlich besser als das Vorgängermodell GPT-3 mit 175B Parameter, welches auf einem Datensatz von 300B-Token trainiert wurde (Zhao et al., 2023). Ein weiteres aktuelles Modell von OpenAI mit einer geringeren Parameteranzahl ist **GPT-4o mini**. Das Modell hat eine Kontextlänge von 128000 Token und ist mit 200000 TPM limitiert. Aufgrund der geringeren Größe ist GPT-4o-mini etwa zehnmal günstiger in der API-Nutzung als GPT4-o (OpenAI, 2023).
- **Claude-3.5-Sonnet** hat eine Kontextlänge von 200000 Token und ist auf 8000 Token als maximale Ausgabe beschränkt. Die Claude3-Modelle wurden auf öffentlichen Internetdaten bis August 2023 und auf proprietären Daten von Drittanbietern trainiert (Anthropic, 2024).

3.3. Anwendung von LLMs

Die vorgestellten Decoder-only Large Language Models, darunter GPT- und Llama3-Modelle, können durch verschiedene Information Extraction-Techniken für NLP-Aufgaben wie Named Entity Recognition und Relation Extraction eingesetzt werden. Abbildung 3.4 bietet einen Überblick über IE-Methoden zur Erzeugung strukturierter Ausgaben mit LLMs. Durch Prompting mit aufgaben-spezifischen Instruktionen und Beispielen können die Modelle direkt strukturierte Ausgaben generieren. Die Verfügbarkeit umfangreicher annotierter Datensätze ermöglicht zusätzlich ein Fine-Tuning der LLMs. Basierend auf diesen Prompting- und Fine-Tuning Methoden lassen sich spezialisierte Frameworks sowohl mit Open-Source als auch mit Closed-Source LLMs für NLP-Aufgaben entwickeln (D. Xu et al., 2023).

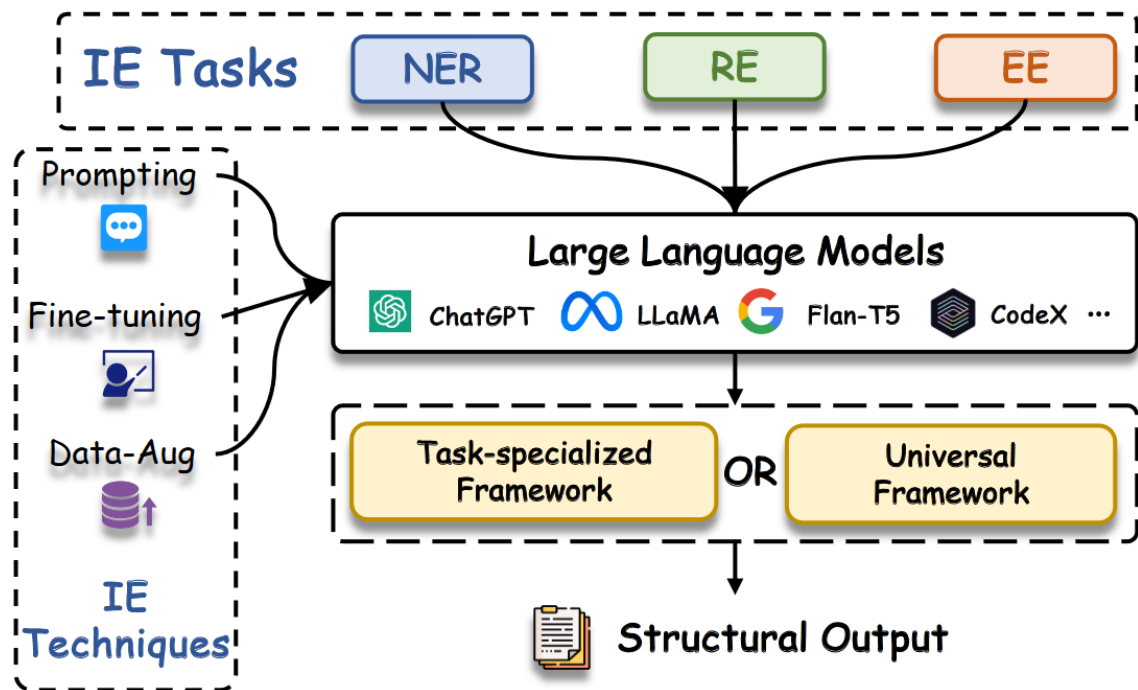


Abbildung 3.4.: Methoden von Decoder-only Large Language Models wie GPT für Information Extraction-Aufgaben (D. Xu et al., 2023).

Im Folgenden werden Prompt-Methoden vorgestellt, bei denen LLMs unstrukturierte Textdaten entweder durch eine einzelne Instruktion (Zero-Shot) oder durch die Verwendung von Beispielen (Few-Shot) in eine strukturierte Form überführen können. Darüber hinaus werden erweiterte Prompt-Methoden wie Emotional Prompting und Role Prompting vorgestellt, die darauf abzielen, Zero-Shot Prompting durch gezielte Ergänzungen in den Prompts zu optimieren. Bei Verfügbarkeit umfangreicher annotierter Datensätze ist ein Fine-Tuning von LLMs möglich. Dabei werden zunächst die klassischen Deep Learning-Parameter für das Supervised Fine-Tuning von LLMs erläutert. Ergänzend werden moderne Parameter-Efficient Fine-Tuning Methoden vorgestellt, die ein effizientes Training von LLMs ermöglichen. Die erforderlichen Parameter werden definiert und deren typische Wertebereiche spezifiziert.

Für den produktiven Einsatz der LLMs werden die konfigurierbaren Inferenz-Parameter der Modelle definiert. Diese ermöglichen eine Feinjustierung der Modellausgaben. Trotz der aktuellen Fortschritte und vielversprechenden Ergebnisse bei NLP-Aufgaben stehen Large Language Models vor Herausforderungen wie der Halluzination ungewollter Ausgaben, die ihren breiten Einsatz einschränken (Hadi et al., 2023). Diese Limitierungen, die insbesondere bei der Verarbeitung von NER- und RE-Aufgaben auftreten können, werden im Folgenden aufgezeigt.

3.4. Prompting

Prompt-Engineering ist ein aktuelles Forschungsfeld, das sich mit dem systematischen Entwurf, der Optimierung und der Implementierung von Prompts oder Anweisungen beschäftigt. Diese Prompts, welche aus verschiedenen Modalitäten wie Text, Bild, Ton oder anderen Medien bestehen können, dienen der gezielten Steuerung der LLM-Ausgabe für diverse Anwendungsbereiche. Der Prozess ist charakterisiert durch einen iterativen Ansatz zur Verfeinerung der Prompts mittels präziser Modifikationen und kontextspezifischer Anpassungen (Schulhoff et al., 2024). Wesentliche Bestandteile von Prompts sind Anweisungen, Beispiele, Formatierungshinweise und zusätzliche Kontextinformationen. Werden dem Modell k Beispiele in Form von Ein- und Ausgabe-paaren (x_k, y_k) über die Prompteingabe mitgegeben, spricht man von Few-Shot Prompting (Zhao et al., 2023). Werden dem Modell während der Inferenz lediglich Anweisungen, Formatierungshinweise oder Kontextinformationen gegeben, so spricht man von Zero-Shot Prompting. Da die Qualität der Prompts maßgeblich die Leistung von Large Language Models bei spezifischen Aufgaben beeinflusst, wurden verschiedene Optimierungsmethoden entwickelt (Schulhoff et al., 2024). Die folgenden Abschnitte erläutern die in dieser Arbeit angewandten Prompt-Methoden.

3.4.1. In-Context Learning (ICL)

In-Context Learning wurde erstmals mit GPT-3 eingeführt und verwendet formatierte Prompts in natürlicher Sprache, welche eine Aufgabenbeschreibung und einige Beispiele enthalten, um LLMs ohne Anpassung der Gewichte und damit ohne erneutem Training anzuwenden. Kritisch anzumerken ist, dass der Begriff *Learning* in diesem Kontext potenziell irreführend ist. ICL stellt primär eine Methode der Aufgabenspezifikation dar, bei der die gezeigten Fähigkeiten nicht zwangsläufig neu erlernt werden, sondern möglicherweise bereits implizit in den Trainingsdaten des Modells enthalten sein können (Schulhoff et al., 2024). Die formale Darstellung des In-Context Learning lässt sich wie folgt beschreiben.

Sei $D_k = \{f(x_1, y_1), \dots, f(x_k, y_k)\}$ eine Menge von k Demonstrationen, wobei $f(x_k, y_k)$ eine Funktion repräsentiert, die das k -te Aufgabenbeispiel innerhalb einer Promptanweisung transformiert. Gegeben eine Aufgabenbeschreibung I , die Demonstrationen D_k und eine neue Eingabeabfrage x_{k+1} , kann die Vorhersage der Ausgabe $\hat{y}_{k+1}$ durch ein Large Language Model folgendermaßen formuliert werden.

$$LLM(I, \underbrace{f(x_1, y_1), \dots, f(x_k, y_k)}_{\text{Demonstrationen}}, \underbrace{f(x_{k+1}, \text{---})}_{\text{Eingabe Antwort}}) \rightarrow \hat{y}_{k+1} \quad (3.7)$$

Hierbei bleibt die tatsächliche Antwort y_{k+1} zunächst unbekannt und soll vom Large Language Model vorhergesagt werden. Die Leistungsfähigkeit des ICL-Ansatzes ist in hohem Maße von der Qualität und Komposition der Demonstrationsbeispiele abhängig. Die Effektivität wird maßgeblich durch die sorgfältige Auswahl und Anordnung dieser Beispiele sowie durch die präzise Gestaltung der Prompts beeinflusst. Die Herausforderung besteht darin, Demonstrationen zu konstruieren, die sowohl repräsentativ für die Aufgabendomäne sind als auch die Generalisierungsfähigkeit des Modells auf neue, ungesehene Eingaben maximieren (Zhao et al., 2023).

Zero-Shot

Eine Methode, bei der Large Language Models Texte zu spezifischen Themen ohne vorherige domänenspezifische Trainingsbeispiele generieren. Dabei wird dem Modell während der Inferenz ausschließlich eine Promptanweisung mit vordefinierten Regeln übergeben (Kojima et al., 2023). Das Zero-Shot Prompting stellt einen Spezialfall des In-Context Learning dar, bei dem die Anzahl der Demonstrationsbeispiele $k = 0$ gesetzt wird (Min et al., 2022). Dieser Ansatz nutzt die durch umfangreiches Vortraining erworbene Generalisierungsfähigkeit der Modelle. LLMs greifen dabei auf ihr internalisiertes Wissen und linguistische Kompetenz zurück, um kohärente und kontextuell relevante Ausgaben zu produzieren. Empirische Studien belegen die Effektivität dieser Methode, insbesondere bei der Umformulierung von Anfragen (Zhu et al., 2024).

Few-Shot

Beschreibt eine Methode, bei der Large Language Models eine begrenzte Anzahl von aufgaben-spezifischen Beispielen oder Demonstrationen als Teil des Prompts präsentiert werden. Diese Beispielinstanzen fungieren als implizite Anweisungen, die es dem Modell ermöglichen, seine Sprachgenerierung an die spezifische Aufgabe oder Domäne anzupassen ohne die Modellparameter zu verändern (Zhu et al., 2024). Typischerweise wird Few-Shot Prompting mit k -Beispielen durchgeführt, wobei k so groß werden kann, wie es die Kontextlänge des Modells zulässt. Bei Experimenten mit dem GPT-3 Modell wurde k zwischen 10 und 100 gewählt (Brown et al., 2020). Trotz der beobachteten Effektivität von Few-Shot gegenüber Zero-Shot Prompting in verschiedenen Aufgabenstellungen, bleiben die zugrundeliegenden Mechanismen und die spezifischen Beiträge der Demonstrationskomponenten zur Aufgabenperformanz weitgehend ungeklärt (Min et al., 2022). Few-Shot Prompting steht, insbesondere bei der Erfassung komplexer Zusammenhänge, aufgrund der begrenzten Anzahl annotierter Beispiele vor Herausforderungen (D. Xu et al., 2023). Trotz der geringen Anzahl an Beispielen, kann es zu einem Overfitting kommen, was die Verwendung robuster Generalisierungsstrategien nötig macht. Die Forschung zeigt dabei einen positiven Zusammenhang zwischen der Skalierung der Modellparameter in LLMs und deren Generalisierungsfähigkeit, was die Few-Shot Leistung verbessert (D. Xu et al., 2023).

Chain-of-Thought (CoT)

Die Chain-of-Thought Technik basiert auf iterativem Prompting, bei dem das Large Language Model eine Folge von Anweisungen oder Teilausgaben erhält, die einen schrittweisen Denkprozess simulieren (Zhu et al., 2024). Ein Kernaspekt von CoT ist die Aufforderung an das Modell, bei komplexen Problemen explizite Zwischenschritte oder Argumentationspfade zu generieren. Dies erhöht die Genauigkeit und logische Konsistenz der Modellausgabe. Durch die Kombination mit Few-Shot Prompting, bei dem Beispiele für die Argumentation gegeben werden, kann die Fähigkeit des Modells zur logischen Argumentation weiter verbessert werden (Zhao et al., 2023). Diese Technik wird auch für Large Language Model-Anwendungen in anspruchsvollen Bereichen wie der Medizin eingesetzt (Zhou et al., 2023).

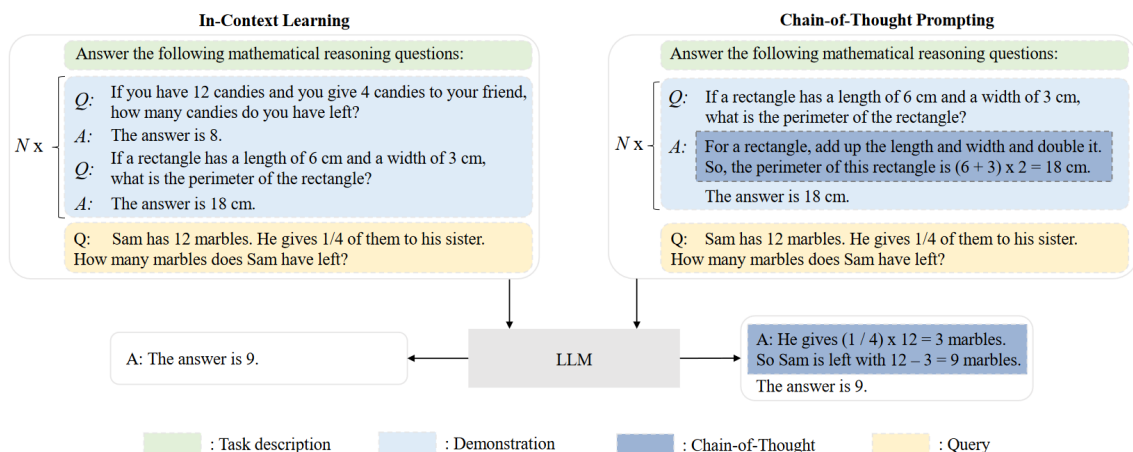


Abbildung 3.5.: Ein Vergleich der Prompting-Methoden In-Context Learning (ICL) und Chain-of-Thought (CoT) bei einer logischen Denkaufgabe für ein Large Language Model (Zhao et al., 2023).

Abbildung 3.5 vergleicht zwei vorgestellte Prompt-Methoden, um LLMs für logische Aufgaben anzuleiten. Links wird In-Context Learning mit zwei Demonstrationsbeispielen (2-Shot) gezeigt. Die rechte Seite zeigt die Chain-of-Thought Methode für dieselbe Anfrage. Hier wird ein 1-Shot CoT-Beispiel verwendet, das aus einem Input-Output Paar besteht. Dieses Paar besteht aus einer Anfrage und der dazugehörigen Antwort, wobei der Lösungsweg schrittweise durch logisches Denken dargestellt wird. Die Ausgabe des LLMs zeigt, dass das Modell bei der ICL-Methode die Antwort direkt liefert, während die CoT-Methode die Lösung iterativ durch logische Schlussfolgerungen erzeugt (Zhao et al., 2023).

3.4.2. Zero-Shot Chain-of-Thought (CoT)

Die Zero-Shot CoT Methode verzichtet auf Beispiele in der Anfrage. Stattdessen wird die ursprüngliche Aufforderung durch spezifische Formulierungen ergänzt, die darauf abzielen, den logischen Denkprozess im LLM anzuregen (Ye et al., 2024). Eine von Kojima et al. (2023) vorgestellte Zero-Shot CoT Phrase lautet: *“Let’s think step by step.”* Diese Ergänzung kann an das Ende der ursprünglichen Promptanfrage angehängt werden. Zero-Shot CoT Ansätze kommen ohne annotierte Beispiele für logische Schlussfolgerungen aus und lassen sich flexibel auf verschiedene Aufgaben übertragen (Zhou et al., 2023).

3.4.3. Emotional Prompting

Emotional Prompting ist eine Methode zur Verbesserung der LLM-Leistung, die auf psychologischen Prinzipien basiert. Dabei werden elf speziell entwickelte emotionale Stimuli in Eingabeaufforderungen integriert, um Aspekte wie Erwartung, Selbstvertrauen und sozialen Einfluss anzusprechen. Die Methode wird angewendet, indem Sätze wie *“This is very important to my career.”* an die Aufgabenbeschreibung angehängt werden (C. Li et al., 2023).

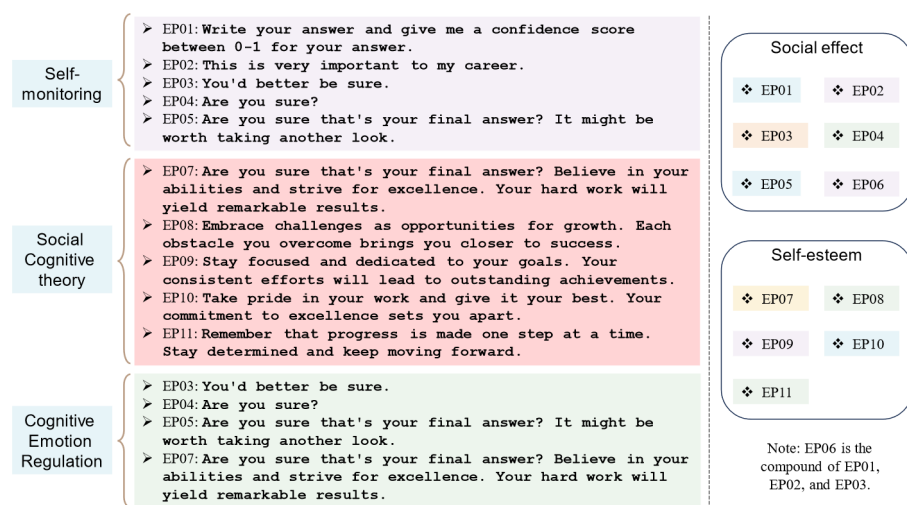


Abbildung 3.6.: Emotionale Stimuli für Emotional Prompting (C. Li et al., 2023).

Abbildung 3.6 zeigt die elf Prompt-Erweiterungen, die in C. Li et al. (2023) entwickelt wurden, sowie deren mögliche Kombinationen. Die emotionalen Stimuli (EP) basieren auf drei psychologischen Theorien. Self-Monitoring (EP01-EP05) bezieht sich auf die Anpassung des Verhaltens in sozialen Situationen, basierend auf den Reaktionen anderer, um sich besser an verschiedene soziale Kontexte anzupassen. Social-Cognitive Theory (EP07-EP11) betont, dass Lernen durch die Beobachtung anderer in sozialen Kontexten sowie durch persönliche Erfahrungen und den Zugang zu Informationen erfolgt. Cognitive-Emotion-Regulation Theory (EP03-EP05, EP07) erklärt, wie Menschen durch Techniken wie die Neubewertung von Situationen ihre Emotionen positiv beeinflussen können, um ihre Motivation und Ausdauer bei Herausforderungen zu steigern. Eine Untersuchung von C. Li et al. (2023) konnte zeigen, dass die Leistung von LLMs mit Zero-Shot über emotionale Stimuli verbessert werden kann (C. Li et al., 2023).

3.4.4. Role Prompting

Role Prompting oder Persona Prompting bezeichnet eine Technik, bei der LLMs durch die Prompt eine spezifische Rolle zugewiesen wird. Die Rollenzuweisung erfolgt typischerweise am Anfang der Prompteingabe durch einen einleitenden Satz wie: *“You are a/an {role}”* (Zheng et al., 2023). Die gewählte Rolle wird gezielt an die jeweilige Aufgabenstellung angepasst, um optimale Ergebnisse zu erzielen. Diese Technik ermöglicht es, das Verhalten des LLMs zu steuern, indem es beispielsweise angewiesen wird, sich wie eine bestimmte Persönlichkeit wie “Madonna” oder ein spezifischer Berufstyp wie “Reiseschriftsteller” zu verhalten. Wissenschaftliche Untersuchungen zeigen, dass Role Prompting die Leistung von LLMs in verschiedenen NLP-Aufgaben verbessert, insbesondere bei kreativen und fachspezifischen Anwendungen (Schulhoff et al., 2024).

3.5. Fine-Tuning

Obwohl Large Language Models mit großen, domänenübergreifenden Datensätzen vortrainiert werden, besteht oft die Notwendigkeit, sie an neue Domänen oder spezifische Aufgaben anzupassen, die im ursprünglichen Training möglicherweise nicht ausreichend berücksichtigt wurden. Dies kann etwa die Spezialisierung auf juristische oder medizinische Texte oder die Anpassung an bestimmte Anwendungen betreffen. Um diese Anpassung zu erreichen, wird das bereits vortrainierte Modell mit zusätzlichen, relevanten Daten aus dem neuen Anwendungsbereich weiter trainiert (Zhou et al., 2023). Dieser Prozess, bei dem ein vollständig vortrainiertes Modell zusätzlichen Trainingsdurchläufen mit neuen Daten unterzogen wird, wird als Fine-Tuning bezeichnet. Diese Methode ermöglicht es, bestehende LLMs effizient an neue Anwendungsgebiete anzupassen, ohne sie komplett neu trainieren zu müssen (Zhu et al., 2024). Beim Supervised Fine-Tuning (SFT) werden dem Modell während des Trainings annotierte Datensätze in Form von Input-Output Paaren übergeben. Wenn zusätzlich Instruktionen in Form von Prompts bereitgestellt werden, spricht man von Instruction Fine-Tuning (IFT). Diese Technik ermöglicht eine gezielte Anpassung des Modells an spezifische Aufgaben, indem das Modell lernt, wie es auf bestimmte Eingaben reagieren soll. In Abbildung 3.7 ist ein vollständiges Fine-Tuning über SFT- oder IFT-Methoden gezeigt, wobei alle Parameter der Gewichtsmatrix W über den Trainingsprozess angepasst werden.

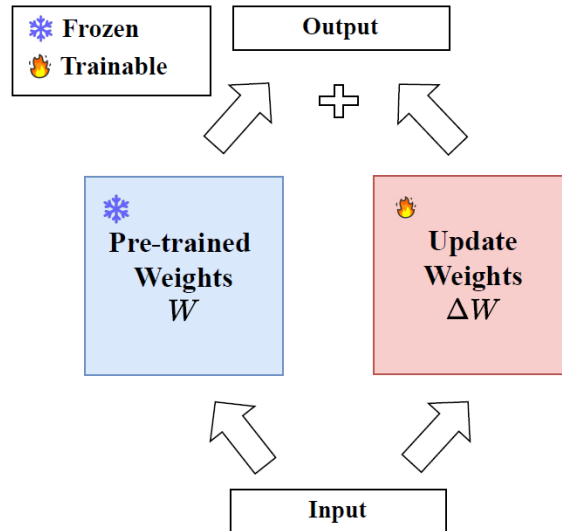


Abbildung 3.7.: Vollständiges Fine-Tuning von Large Language Models, wobei die Gewichtsmatrix W vollständig aktualisiert wird (Mao et al., 2024).

Die vortrainierte Gewichtsmatrix eines LLM kann durch $W_0 \in \mathbb{R}^{d \times k}$ parametrisiert werden. Um sie an eine nachgelagerte Aufgabe anzupassen, kann diese durch die trainierbare Matrix $\Delta W \in \mathbb{R}^{d \times k}$ aktualisiert werden. Als Ergebnis des Fine-Tunings wird die aktualisierte Gewichtsmatrix erhalten, parametrisiert durch $W = W_0 + \Delta W$. Die Vorwärtsberechnung h zur Berechnung der Ausgabe basierend auf der Eingabe x lautet dann:

$$h = Wx = (W_0 + \Delta W)x \quad (3.8)$$

Für das vollständige Fine-Tuning wird ΔW basierend auf den Gradienten aller $d \times k$ Parameter der Schicht berechnet. Dies ist rechnerisch aufwendig und erfordert für LLMs mit Milliarden von Parametern eine große Menge an GPU-Speicher (Mao et al., 2024). Zur Optimierung eines Supervised Fine-Tuning können verschiedene Deep Learning-Hyperparameter angepasst werden.

3.5.1. Hyperparameter

- **Epochs:** Bestimmt die Anzahl der Trainingsdurchläufe eines Deep Learning-Modells. Jede Epoche aktualisiert die Modellgewichte basierend auf den berechneten lokalen Gradienten. Mehr Epochen verbessern oft die Modellqualität, verlängern aber die Trainingszeit und können zu Overfitting führen. Epochen können über die Batch-size in weitere Einzelschritte unterteilt werden (Bengio, 2012).
- **Batch-size:** Die Batchgröße bestimmt, wie viele Trainingsbeispiele gleichzeitig verarbeitet werden, bevor die Modellparameter aktualisiert werden. Kleinere Batchgrößen führen zu Gewichtsaktualisierungen auf Basis weniger Beispiele, was zu höheren Schwankungen der Gradienten führen kann. Der Standardwert liegt oft bei 32, jedoch variiert die optimale Größe abhängig von Datensatz, verfügbarer Hardware und Modellarchitektur. In der Praxis werden häufig Größen zwischen zehn und mehreren Hundert verwendet, in speziellen Fällen sogar bis zu mehreren Tausend (Bartz et al., 2023). Für das Training von LLMs gibt die Batch-size die Anzahl der Beispiele an, die während der Evaluierungsphase gleichzeitig auf jeder verfügbaren GPU verarbeitet werden. Der Parameter *Gradient accumulation steps* gibt die Anzahl der Trainingsschritte an, über die alle Gradienten akkumuliert werden, bevor eine Gewichtsaktualisierung des Modells erfolgt (Zhao et al., 2023).
- **Learning Rate (η):** Die Lernrate, engl. Learning Rate (LR), bestimmt die Schrittweite der Gewichtsadjustierungen bei jedem Trainingsschritt eines neuronalen Netzwerks. Sie beeinflusst maßgeblich die Trainingsgeschwindigkeit und -stabilität. Eine zu hohe Lernrate kann zu Instabilität und Divergenz führen, während eine zu niedrige Rate die Konvergenz verlangsamt. Die optimale Lernrate balanciert zwischen schnellem Fortschritt und stabiler Konvergenz des Modells. Typischerweise liegt die Lernrate im Bereich $\eta \in [10^{-6}, 10^{-2}]$, kann aber über LR-Scheduling auch dynamisch angepasst werden (Bartz et al., 2023).
- **LR-Scheduling:** Learning Rate-Scheduling ist eine Technik, bei der die Lernrate während des Trainingsprozesses systematisch angepasst wird, um die Konvergenz und Leistung des Modells zu optimieren. LLMs verwenden üblicherweise eine konsistente LR-Scheduling Strategie, die sich aus einer Aufwärmphase (Warm-up) und einer Abkühlphase (Decay) zusammensetzt. In der **Warm-up** Phase wird die Lernrate zu Beginn des Trainings langsam erhöht, typischerweise in den ersten 0.1% bis 0.5% der Trainingsschritte. Dies fördert eine stabilere Konvergenz und verhindert eine zu schnelle Anpassung der Modellparameter in frühen Trainingsphasen. Die Lernrate steigt dabei auf ihren Maximalwert an, der meist zwischen 5×10^{-5} bis 1×10^{-4} liegt. Der LR-Scheduler bestimmt die mathematische Funktion dieser Anpassung, wobei lineare oder Cosinus-Funktionen gängige Optionen sind. In der anschließenden Decay-Phase wird die Lernrate allmählich auf etwa 10% ihres Maximalwerts reduziert (Zhao et al., 2023).
- **Optimizer:** Passen die Gewichte von Deep Learning-Modellen an, um den Vorhersagefehler zu minimieren. Dies erfolgt durch schrittweise Anpassung der Parameter, basierend auf dem Gradienten der Verlustfunktion. Für Hyperparameteroptimierung wird oft eine Grundmenge von Optimizer wie RMSProp, ADAM (Adaptive Moment Estimation) und NADAM empfohlen (Bartz et al., 2023). Bei Large Language Models wie GPT-3 werden ADAM und seine Variante *AdamW* häufig eingesetzt (Zhao et al., 2023). ADAM berechnet für jeden Parameter eine individuelle, adaptive Lernrate. Er kombiniert einen gleitenden Durchschnitt der Gradienten mit einem gleitenden Durchschnitt der quadrierten Gradienten zur Lernratenanpassung. AdamW, eine Weiterentwicklung von ADAM, implementiert ein verbessertes Weight-Decay, was Überanpassung reduziert und die Generalisierungsfähigkeit des Modells steigert (Loshchilov; Hutter, 2019).
- **Gradient-Checkpointing:** Ist eine Methode zur Reduzierung des Speicherbedarfs beim Training neuronaler Netzwerke, insbesondere bei der Berechnung von Gradienten. Dabei werden selektiv Zwischenergebnisse gespeichert und Gradienten bei Bedarf teilweise neu berechnet, anstatt alle Zwischenwerte für die *Backpropagation* im Speicher zu halten. Dieser Ansatz ermöglicht einen Ausgleich zwischen Rechenaufwand und Speichernutzung, wodurch das Training größerer Modelle oder längerer Sequenzen mit begrenztem GPU-Speicher möglich wird (T. Chen et al., 2016).

- **Trainingsstabilisierung:** Beim Fine-Tuning von LLMs kommen verschiedene Techniken zur Stabilisierung des Trainings zum Einsatz. **Weight-Decay** wird typischerweise mit einem Wert von 0.1 im AdamW Optimizer verwendet (Zhao et al., 2023). Dieser Mechanismus reduziert die Modellkomplexität durch graduelle Gewichtsreduktion, indem vor der adaptiven Lernratenanpassung mit LR-Scheduling ein kleiner Bruchteil von allen Modellgewichten subtrahiert wird (Loshchilov; Hutter, 2019). Komplementär dazu dient **Gradient-Clipping**, üblicherweise auf 1.0 gesetzt, zur Begrenzung der Gradientennorm (Zhao et al., 2023). Überschreitet die berechnete Norm den Schwellenwert, wird der gesamte Gradientenvektor skaliert, was explodierende Gradienten verhindert und kontrollierte Gewichtsaktualisierungen gewährleistet. In Kombination wirken diese Methoden dem Overfitting entgegen, fördern die Generalisierungsfähigkeit und beugen abrupten Leistungseinbrüchen (Modellkollaps) vor, indem sie ein ausgewogenes Verhältnis zwischen Modellkapazität und Trainingsstabilität herstellen (Loshchilov; Hutter, 2019).

Das Fine-Tuning von LLMs auf spezifische Aufgaben und Datensätze kann deren Leistung erheblich steigern und den Aufwand für Prompt-Engineering reduzieren (Zhao et al., 2023). Das klassische Fine-Tuning erfordert jedoch beträchtliche Rechen- und Speicherressourcen, was diese Technik für viele Organisationen unzugänglich macht (Dettmers et al., 2023). Parameter-Efficient Fine-Tuning (PEFT) bietet hier einen effizienten Lösungsansatz. Durch die selektive Anpassung weniger Modellparameter wird der Ressourcenbedarf drastisch gesenkt, während vergleichbare Leistungsverbesserungen erzielt werden können (Hu et al., 2021).

3.5.2. Low-Rank Adaptation (LoRA)

Die von Hu et al. (2021) vorgestellte Low-Rank Adaptation Methode ist eine Parameter-Efficient Fine-Tuning Variante, die den Fine-Tuning Prozess von LLMs optimiert. Diese Methode behält die ursprünglichen Parameter des Modells bei und fügt trainierbare Matrizen mit niedrigem Rang in die Self-Attention-Mechanismen jeder Transformerschicht ein. Dadurch wird die Anzahl der anzupassenden Parameter erheblich reduziert, was die Effizienz des Fine-Tuning erhöht. Trotz dieser Vereinfachung bleibt das LLM mit Fine-Tuning in der Lage, die spezifischen Eigenschaften der Zielaufgaben effektiv zu erfassen (Zhou et al., 2023).

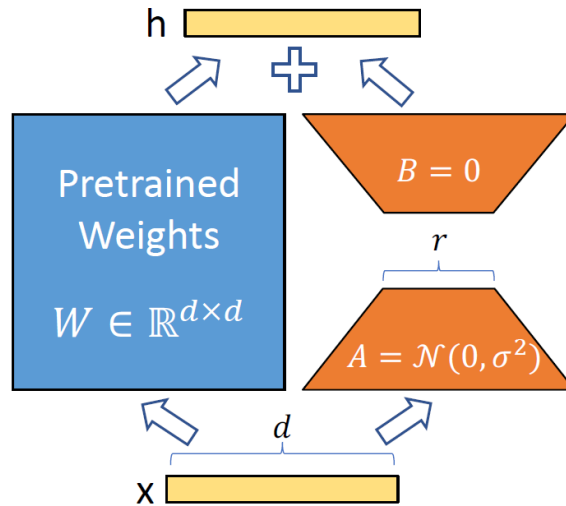


Abbildung 3.8.: Fine-Tuning von LLMs mit Low-Rank Adaptation (Hu et al., 2021).

LoRA ist eine effiziente Fine-Tuning Methode für LLMs, wie in Abbildung 3.8 dargestellt ist. Anstatt die gesamte Gewichtsmatrix $W_0 \in \mathbb{R}^{d \times k}$ zu aktualisieren, führt LoRA zwei kleinere Matrizen $B \in \mathbb{R}^{d \times r}$ und $A \in \mathbb{R}^{r \times k}$ ein, die zusammen eine Niedrigrangaktualisierung $\Delta W = BA$ bilden. Die ursprüngliche Matrix W_0 bleibt unverändert, während nur A und B trainiert werden. Die modifizierte Vorwärtsberechnung aus Gleichung 3.8 lautet:

$$h = W_0 x + \gamma_r B A x \quad (3.9)$$

Dabei ist $\gamma_r \in \mathbb{R}^+$ der Skalierungsfaktor, welcher in der LoRA-Methode als $\gamma_r = \frac{\alpha}{r}$ definiert ist. r ist der LoRA-Rang, der die Dimension von A und B bestimmt und deutlich kleiner als $\min(d, k)$ gewählt wird, um die Anzahl der trainierbaren Parameter zu reduzieren. α ist ein Hyperparameter, der die Stärke der LoRA-Anpassung kontrolliert und oft so gewählt wird, dass $\gamma_r = \frac{\alpha}{r} = 1$ gilt. A wird typischerweise mit einer Gaußschen-Verteilung initialisiert, während B mit Nullen initialisiert wird, sodass ΔW zu Trainingsbeginn Null ist. Diese Methode ermöglicht eine effiziente Modellanpassung bei reduziertem Trainingsaufwand (Hu et al., 2021).

Der Hauptvorteil von LoRA liegt in der Reduzierung des Speicherbedarfs und der Speichernutzung, insbesondere des VRAM-Verbrauchs. Beispielsweise müssen bei einem vollständigen Fine-Tuning von Llama2-7B $11.008 \times 4.096 = 45.088.768$ Parameter angepasst werden, während bei dem Verwenden der LoRA-Methode mit $r = 4$ nur $(11.008 \times 4) + (4 \times 4.096) = 60.416$ Parameter angepasst werden müssen. Dabei werden nur etwa ein Tausendstel der Parameter im Vergleich zum vollständigen Fine-Tuning angepasst (Mao et al., 2024). Diese Effizienz ermöglicht es, LLMs mit deutlich weniger Ressourcen zu trainieren und anzupassen, was sowohl die Kosten senkt als auch die Zugänglichkeit für Forschung und Anwendung erhöht (Zhao et al., 2023). In Mao et al. (2024) werden weitere, aktuelle LoRA-basierte PEFT-Methoden gelistet.

3.5.3. Rank-stabilized LoRA (rsLoRA)

Das von Kalajdzievski (2023) vorgestellte Rank-stabilized LoRA (rsLoRA) ist eine Weiterentwicklung der ursprünglichen LoRA-Methode, welche die Instabilität und das Kollabieren des Lernprozesses bei höheren Rängen r adressiert. Der Hauptunterschied zwischen LoRA und rsLoRA liegt in der Wahl des Skalierungsfaktors $\gamma_r \in \mathbb{R}^+$. In der ursprünglichen LoRA-Methode wird γ_r mit $\gamma_r = \frac{\alpha}{r}$ definiert, wobei α ein Skalierung-Hyperparameter und r der Rang der Adaptermatrizen ist. Diese Skalierung führt jedoch zu einer Verlangsamung des Lernens und einer eingeschränkten Leistung bei höheren Rängen (Kalajdzievski, 2023). Der Skalierungsfaktor wird über rsLoRA mit $\gamma_r = \frac{\alpha}{\sqrt{r}}$ modifiziert. Diese Änderung stabilisiert den Lernprozess über verschiedene Ränge hinweg und ermöglicht effektives Training auch bei höheren Rängen. Die modifizierte Vorwärtsberechnung mit rsLoRA lautet:

$$h = W_0 x + \frac{\alpha}{\sqrt{r}} B A x \quad (3.10)$$

wobei $W_0 \in \mathbb{R}^{d \times k}$ die ursprüngliche Gewichtsmatrix, $B \in \mathbb{R}^{d \times r}$ und $A \in \mathbb{R}^{r \times k}$ die Adaptermatrizen sind. Ein Fine-Tuning mit rsLoRA kann auch mit höheren Rangwerten durchgeführt werden, was zu potenziell besseren Ergebnissen führt. Die Methode ermöglicht einen flexiblen Kompromiss zwischen Rechenaufwand und Modellleistung, während sie die Parametereffizienz der LoRA-Methode beibehält (Kalajdzievski, 2023).

3.5.4. Quantized LoRA (QLoRA)

In LLMs werden die Parameter typischerweise als 32-bit oder 16-bit Fließkommazahlen gespeichert. QLoRA ist eine Methode, die ein vortrainiertes LLM auf 4-bit quantisiert und durch Hinzufügen kleiner, trainierbarer LoRA-Gewichte ein effizientes Fine-Tuning bei erhaltener Modellleistung ermöglicht. Quantisierung ist ein Prozess, bei dem Daten mit hoher Präzision (z.B. 32-bit Floats) in Daten mit niedrigerer Präzision (z.B. 4-bit Integers) umgewandelt werden, um den Speicherbedarf zu reduzieren. QLoRA nutzt eine spezielle 4-bit Quantisierungstechnik *4-bit NormalFloat*, die für normalverteilte Daten optimiert ist. Ein LLM wie Llama2-70B benötigt bei einem regulären 16-bit Fine-Tuning über 780GB VRAM. Mit QLoRA benötigt das 70B-Modell unter 48GB VRAM, wodurch ein Fine-Tuning aktueller Large Language Models auf nur einer GPU möglich wird (Detmers et al., 2023). Im Folgenden werden die LoRA-Parameter beschrieben, die für erweiterte LoRA-Methoden wie rsLoRA und QLoRA angepasst werden können, um ein effizientes Fine-Tuning von Large Language Models zu ermöglichen.

3.5.5. LoRA-Hyperparameter

- **LoRA-Rang (r):** Der LoRA-Rang r bezieht sich auf die Dimension der niederrangigen Matrizen, die für das Fine-Tuning verwendet werden. Ein niedriger Rang reduziert die Anzahl der trainierbaren Parameter, was das Fine-Tuning effizienter macht. Ein höherer Rang kann potenziell zu besseren Ergebnissen führen, erhöht aber den Rechenaufwand. In der Praxis wird r typischerweise als Zweierpotenz gewählt mit $r \in \{2^k \mid k \in \mathbb{N}, 2 \leq k \leq 11\} = \{4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048\}$ (Kalajdzievski, 2023). LoRA-Implementierungen verwenden häufig Werte für r im Bereich von 4 bis 32, während rsLoRA effektiv höhere Werte ($r \geq 128$) nutzen kann, was ein verbessertes Fine-Tuning ermöglicht (Kalajdzievski, 2023).
- **Skalierungsparameter (α):** Der LoRA-Parameter α ist ein Hyperparameter, der die Gesamtstärke der LoRA-Anpassung kontrolliert. In der ursprünglichen LoRA-Implementierung wird α mit $\alpha = r$ gewählt, um die Initialisierung zu normalisieren (Hu et al., 2021).
- **LoRA-Dropout:** LoRA-Dropout ist eine Regularisierungstechnik wobei zufälliges Rauschen in die trainierbaren Niedrigrangmatrizen eingeführt wird. Dies erfolgt durch das zufällige Nullsetzen von Zeilen und Spalten dieser Matrizen während des Trainings, was dazu beiträgt, Overfitting zu kontrollieren. Typische Werte für die LoRA-Dropout Rate liegen zwischen 0.1 und 0.5. In den meisten Experimenten hat sich ein Wert von 0.5 als effektiv erwiesen, da er einen guten Kompromiss zwischen Regularisierung und der Beibehaltung der Modellkapazität bietet (Lin et al., 2024).

3.6. Modellparameter

Für Large Language Models ist eine Dekodierungsstrategie, für die Erzeugung aufgabenspezifischer Ausgaben nötig. Die Textgenerierung kann dabei durch Modellparameter wie Temperatur-Sampling und Top- p Sampling gesteuert werden (Zhao et al., 2023). Die grundlegende Decodierungsmethode der Ausgabe ist die *Greedy Search*. Bei diesem Ansatz wird in jedem Schritt das wahrscheinlichste Token basierend auf den zuvor generierten Token ausgewählt.

$$x_i = \arg \max_x P(x \mid x_{<i}) \quad (3.11)$$

wo x_i das Token mit der höchsten Wahrscheinlichkeit im i -ten Schritt der Generierung darstellt, bedingt durch den Kontext $x_{<i}$. Eine Alternative ist *Beam Search*, die bei jedem Decodierungsschritt die n wahrscheinlichsten Sätze beibehält und am Ende die Antwort mit der höchsten Wahrscheinlichkeit auswählt. Moderne LLMs verwenden für die Inferenz optimierte *Random Sampling*-Dekodierungsmethoden. Dabei werden Token aus dem gesamten Vokabular ausgewählt, was die Vielfalt der Ausgaben erhöht, aber auch zu kontextuell unangemessenen Ergebnissen führen kann. Zur Verbesserung der Generierungsqualität wurden Strategien entwickelt, welche die Auswahl unwahrscheinlicher Wörter einschränken. Diese Methoden sind als anpassbare Inferenz-Parameter in LLMs implementiert (Zhao et al., 2023).

- **Temperature Sampling:** Um das Random Sampling zu modulieren, kann der Temperaturkoeffizient der Softmax-Funktion angepasst werden. Dieser berechnet die Wahrscheinlichkeit für das j -te Token im Vokabular über den vorherigen Kontext $\mathbf{x}_{<i}$:

$$P(x_j \mid \mathbf{x}_{<i}) = \frac{\exp(l_j/t)}{\sum_{j'} \exp(l_{j'}/t)} \quad (3.12)$$

Wobei l_j der Logit der einzelnen Token und t der Temperaturkoeffizient ist. Die Temperatur kann beliebige positive Werte annehmen. Kleinere Werte für t erhöhen die Wahrscheinlichkeit, Wörter mit hohen Wahrscheinlichkeiten auszuwählen, wodurch vorhersehbare und konservative Texte entstehen. Wird t auf 1 gesetzt, entspricht dies dem Random-Sampling. Nähert sich t dem Wert 0, verhält sich die Methode äquivalent zur Greedy Search, bei der stets das wahrscheinlichste Token gewählt wird (Zhao et al., 2023). Niedrigere Temperaturen erzeugen somit vorhersehbare und konservative Texte, während höhere Temperaturen zu kreativen und unerwarteten Texten führen (Minaee et al., 2024).

- **Top- p Sampling:** Wählt Token aus der kleinsten Menge aus, deren kumulierte Wahrscheinlichkeit einen Schwellenwert p erreicht oder überschreitet. Diese Menge wird gebildet, indem Token nach absteigender Wahrscheinlichkeit sortiert und nacheinander hinzugefügt werden, bis der Schwellenwert überschritten wird. Typische Werte für p liegen zwischen 0.9 und 0.95 (Zhao et al., 2023). Das Nutzen von Top- p führt oft zu vielfältigeren und kreativeren Ergebnissen (Minaee et al., 2024).

3.7. Herausforderungen

Autoregressive Large Language Models haben bei diversen nachgelagerten NLP-Aufgaben gute Leistungen erzielt (X. Xu et al., 2023). Dennoch weisen diese Modelle weiterhin spezifische Limitationen auf, die ihre Anwendbarkeit und Zuverlässigkeit in bestimmten Kontexten einschränken. Im Folgenden werden zentrale Herausforderungen von LLMs bei NLP-Aufgaben aufgeführt.

- **Aktualität:** Die Verarbeitung von Aufgaben, die aktuelles Wissen jenseits des ursprünglichen Trainingsdatensatzes erfordern, stellt eine Herausforderung für LLMs dar. Ein regelmäßiges Aktualisierungstraining mit neuen Daten ist nicht nur ressourcenintensiv und kostspielig, sondern birgt auch das Risiko des katastrophalen Vergessens, bei dem zuvor erlernte Informationen verloren gehen können (Zhao et al., 2023).
- **Stochastische Natur:** Eine charakteristische Eigenschaft von Large Language Models ist ihr stochastischer oder probabilistischer Charakter. Bei wiederholter Eingabe der gleichen Anfrage erzeugen sie in der Regel unterschiedliche Antworten. Es gibt zwar Kontrollmöglichkeiten, insbesondere durch den Temperatur-Parameter, um die Variabilität der Ausgaben zu begrenzen. Dennoch bleibt diese Eigenschaft ein grundlegender Bestandteil ihrer Funktionsweise, die sich aus dem Lernprozess ergibt (Minaee et al., 2024).
- **Halluzinationen:** Large Language Models erzeugen häufig faktisch falsche oder nicht überprüfbare Informationen, ein Phänomen, das als Halluzination bezeichnet wird. Halluzination führt dazu, dass LLMs unerwünschte Ausgaben generieren und die Leistung verschlechtert wird (Zhao et al., 2023). Selbst fortgeschrittene Modelle wie GPT-4 weisen Defizite bei der Erkennung halluzinierter Inhalte auf (OpenAI et al., 2024).
- **Numerische Berechnungen:** LLMs zeigen Schwächen bei komplexen numerischen Berechnungen, insbesondere wenn es sich um Symbole oder Zahlen handelt, die beim Training selten vorkamen. Diese Einschränkung zeigt sich besonders deutlich bei Rechenoperationen mit großen Zahlen. Eine mögliche Erklärung ist, dass die Tokenisierung von Zahlen zu inkonsistenten Sequenzen führen können. Beispielsweise kann die Zahl 7481 als 7_481 und 74815 als 748_15 tokenisiert werden, obwohl beide dieselben numerischen Substrings enthalten. Diese Schwäche limitiert die Anwendbarkeit von LLMs in Bereichen, die mathematische Präzision erfordern (Zhao et al., 2023).
- **Endliche Kontextlänge:** Die Kontextlänge ist entscheidend für die Leistungsfähigkeit von LLMs. Sie ermöglicht eine effektive Interpretation von Modelleingaben sowie eine präzise semantische Analyse. Eine ausreichende Kontextlänge gewährleistet qualitativ hochwertige Modellantworten, während eine zu geringe Kontextlänge die Leistung von LLMs erheblich beeinträchtigen kann. Die Fähigkeit, längere Kontexte zu verarbeiten, ist daher ein kritischer Faktor für die Effektivität und Vielseitigkeit dieser Modelle in verschiedenen Anwendungsszenarien (Hadi et al., 2023).
- **Ressourcenverbrauch:** Die enorme Größe von Large Language Models stellt eine Herausforderung dar. Sowohl für das Training als auch für die Inferenz werden leistungsstarke und kostspielige GPU-Systeme benötigt, was hohe Anforderungen an die Infrastruktur stellt. Die Bereitstellung und Skalierung der Modelle ist komplex und kann zu Leistungsproblemen führen (Minaee et al., 2024).

Weitere Herausforderungen im Umgang mit Large Language Models können in Hadi et al. (2023), Zhao et al. (2023) und Minaee et al. (2024) nachgelesen werden.

4. State-of-the-Art

Das folgende Kapitel beschreibt aktuelle Forschungsfelder in der Analyse von Eignungskriterien klinischer Studien. Es werden öffentlich verfügbare, annotierte Korpora für Named Entity Recognition (NER) und Relation Extraction (RE) Aufgaben vorgestellt. Zudem werden State-of-the-Art (SOTA) NLP-Modelle und -Methoden zur Analyse von Eignungskriterien verglichen.

4.1. Forschungsfelder

Die existierenden Forschungsarbeiten zur Analyse von Eignungskriterien lassen sich in drei Hauptkategorien einteilen. Die erste Kategorie implementiert vollständige End-to-End Systeme für das Patient-Studie Matching. Diese Systeme sind in der Lage, Patienten direkt den für sie geeigneten Studien zuzuordnen. Die wesentlichen Komponenten solcher Systeme sind in Abbildung 4.1 dargestellt.

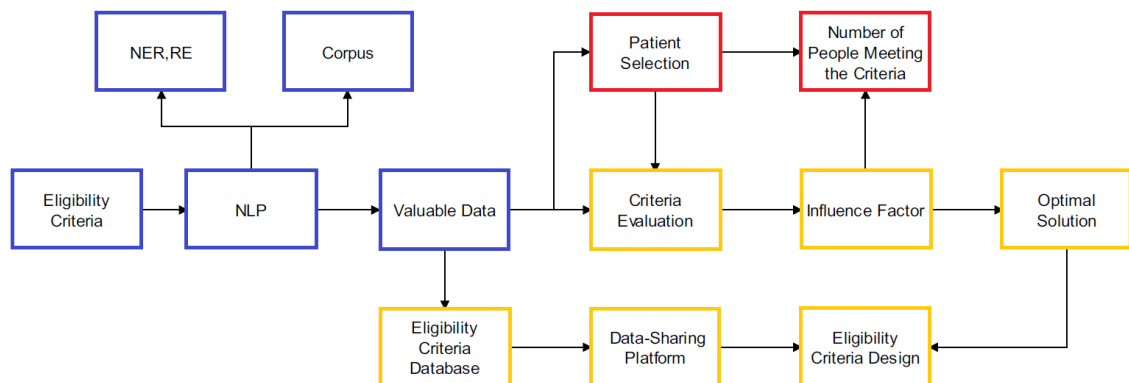


Abbildung 4.1.: Darstellung der Forschungsschwerpunkte zu Eignungskriterien nach Su et al. (2023). Methoden bezogen auf Eignungskriterien klinischer Studien (blau), potenzielle Erweiterungen der Datenspeicherung über Datenbanken und Plattformen (gelb) sowie Ergänzungen durch Patienteninformationen (rot).

Zunächst werden Eignungskriteriendaten über NLP-Methoden in eine Form codiert, mit der sie mit Patientendaten abgeglichen werden können. Dabei werden die Eignungskriterien und Patientendaten aus den Electronic Health Records (EHR) in einen gemeinsamen Vektorraum transformiert. Die Ermittlung der Anzahl der Patienten, welche die Eignungskriterien einer klinischen Studie erfüllen, erfolgt durch die Berechnung der Ähnlichkeit zwischen den Einbettungen der Kriterien und der Patientendaten (Wong et al., 2023). Die primär für die interne Verarbeitung optimierte Datenverarbeitung führt zu einer eingeschränkten Interoperabilität und limitierten externen Nutzbarkeit der transformierten Eignungskriterien, da diese in einem systemspezifischen Format vorliegen.

Methoden der zweiten Kategorie optimieren das Patienten-Studie Matching über eine strukturierte Aufbereitung der Eignungskriterien, um deren Verarbeitbarkeit und Interoperabilität zu erhöhen. Dabei werden sowohl Eignungskriterien klinischer Studien als auch individuelle Patientendaten strukturiert einbezogen. Aus den Eignungskriterien werden Entitäten wie *Krankheiten* oder *demographische Merkmale* und deren Relationen mittels Named Entity Recognition und Relation Extraction extrahiert. Um die extrahierten Daten weiter anzureichern, können die Entitäten durch Terminologiesysteme wie UMLS, SNOMED-CT, LOINC oder ICD-10 normalisiert werden. Beispielsweise würde "Arthritis" in SNOMED-CT dem Code 3723001 zugeordnet werden (Bodenreider et al., 2018). Die angereicherten Eignungskriterien können dann in interoperable Formate wie JavaScript Object Notation (JSON), Extensible Markup Language (XML) oder dem medizinischen Standardformat Fast Healthcare Interoperability Resources (FHIR) prozessiert werden (N. J. Dobbins et al., 2023). Da Patientendaten (EHR-Daten) üblicherweise in standardisierten Datenformaten wie Observational Medical Outcomes Partnership (OMOP) - Common Data Model (CDM) oder FHIR gespeichert sind, werden die angereicherten Eignungskriterien über NLP-Methoden oft ebenfalls in ein solches Format gebracht (Park et al., 2023).

Die dritte Kategorie umfasst Publikationen, die sich auf die Strukturierung von Eignungskriterien mittels NER- und RE-Methoden konzentrieren. Der Schwerpunkt liegt dabei auf der Optimierung von Precision, Recall und F1-Score bei der Erkennung von Entitäten und deren Beziehungen innerhalb der Eignungskriterien (Z. Chen et al., 2023). Die Eignungskriterien werden also ausschließlich über Information Extraction-Aufgaben strukturiert, ohne Patientendaten zu berücksichtigen. Die angereicherten Kriterien können anschließend für eine weitere Verarbeitung gespeichert und für ein Patient-Studie Matching verwendet werden, sofern Patientendaten in gleicher Form vorliegen (Wong et al., 2023). Für eine effektive Verarbeitung der Eignungskriterien nutzen einige Systeme öffentlich verfügbare Korpora für das Training und die Validierung von NLP-Modellen

4.2. Verfügbare Korpora

Die in der Fachliteratur verwendeten Daten zu Eignungskriterien stammen überwiegend aus der öffentlichen Datenbank ClinicalTrials.gov (2000) für klinische Studien Su et al. (2023). Einige Publikationen wie Tian; Erdengasileng et al. (2021) oder K. Wu; Aklilu (2021) annotieren Datensätze mit Eignungskriterien aus ClinicalTrials.gov (2000), stellen diese aber nicht zur Verfügung. Zusätzlich existieren Datensätze von Eignungskriterien, welche auf Anfrage von den jeweiligen Forschungsgruppen bereitgestellt werden. Darunter der EliE-Korpus (Kang et al., 2017), Criteria2Query-Datensatz (C. Yuan et al., 2019), Covance-Datensatz (M. Chen et al., 2020), Criteria2SQL-Datensatz (Yu et al., 2020) und PROTECTOR1-Datensatz (Y. Yang et al., 2023). Trotz formeller Anfragen war es jedoch innerhalb dieser Arbeit nicht möglich, Zugang zu den genannten Datensätzen zu erhalten. Weiterhin gibt es einige öffentlich zugängliche Korpora mit annotierten Eignungskriterien. Diese Datensätze sind in Tabelle 4.1 mit der Domäne, der Anzahl der Datensätze mit Eignungskriterien, die Anzahl der Annotationen insgesamt und die Anzahl der unterschiedlichen Entitäts- und Relationstypen aufgeführt. Die Datensätze sind über die angegebene Referenz verfügbar.

Anzahl	Chia	FRD	COVIC	LCT	LLF
Domäne	alle	alle	COVID-19	alle	alle
Eignungskriterien	1.000	3.314	700	1.006	2.000*
Annotationen	68.174	121.221	28.153	105.816	—
Entitäten	15	15	8	50	44
Relationen	12	1	11	51	20
Quelle	(Chia, 2020)	(CTP, 2020)	(WengLab, 2020)	(Bio-NLP, 2020)	(Bio-NLP, 2023)

Tabelle 4.1.: Öffentlich verfügbare Korpora mit annotierten Eignungskriterien. *Angabe in Kriterien-Zeilen, ansonsten wird die Anzahl der annotierten Dateien angegeben.

Im Folgenden werden die öffentlichen Korpora detailliert beschrieben.

- **Chia-Korpus:** Ein annotierter Korpus mit 12409 Eignungskriterien aus 1000 Phase-IV Studien. Diese wurde von zwei Mediziner*innen annotiert und enthalten 41487 unterscheidbare Entitäten aus 15 Entitätstypen, die mit den Datendomänen im OMOP-CDM Format abgestimmt sind. Entitäten und Relationen sind im BRAT-Format annotiert. Entitätstypen sind unter anderem "Condition", "Qualifier", "Drug" und "Procedure". Relationstypen sind unter anderem "AND", "OR", "HAS_NEGATION" und "HAS_VALUE" (Kury et al., 2020).
- **Facebook Research Data (FRD):** Bietet einen annotierten Korpus aus 3314 zufällig ausgewählten Studien. Er enthält 121221 annotierte klinische Entitäten, Attribute und Grenzwerte, welche im CSV-Format vorliegen. Es wurden 15 Entitätstypen identifiziert, darunter "treatment", "chronic_disease", "age" und "upper_bound". In der Relationsextraktion wurden Negationen von Entitäten erkannt (Tseo et al., 2020).

- **COVIC:** Aus 700 annotierten COVID-19 Studiendatensätzen wurde der COVIC-Korpus gebildet. Der Datensatz enthält 11710 annotierte Ein- und Ausschlusskriterien mit insgesamt 44622 Annotationen im BRAT-Format. Das Annotationsschema basiert auf dem OMOP-CDM, wobei acht Entitätstypen wie “Condition”, “Measurement” und “Procedure” identifiziert wurden sowie elf Relationen, darunter “AND”, “OR” und “Has_Negation” (Sun et al., 2021).
- **Leaf Clinical Trials (LCT) Korpus:** Der LCT-Korpus baut auf den vorherigen Arbeiten von EliIE (Kang et al., 2017) und Chia (Kury et al., 2020) auf und enthält Eignungskriterien aus 1006 klinischen Studien über alle Phasen und Krankheiten hinweg. Es wurden spezifische Annotationsrichtlinien¹ genutzt, um Entitäten und Relationen im BRAT-Format zu annotieren. Die insgesamt 50 Entitäten werden sechs Kategorien zugeordnet, darunter Clinical, Demographic und Qualifier. Die 51 Relationen werden zu sechs übergeordneten Kategorien zugeordnet. Die Relationskategorie “Logical” umfasst die Relationen “And”, “IF-Then”, “Negates” und “Or”. Der LCT-Korpus ist nicht an ein bestimmtes Datenmodell gebunden, ermöglicht aber die Transformation der enthaltenen Entitäten und Relationen in verschiedene Zielformate wie OMOP-CDM (N. Dobbins; Tony Mullen et al., 2022).
- **Leaf Logical Forms (LLF) Korpus:** Im LLF-Korpus wurden 2000 Zeilen an Eignungskriterien aus klinischen Studien in eine logische Form transformiert. Jedes Kriterium enthält mindestens eine annotierte Entität, wobei der Korpus insgesamt 44 Funktionen und 20 Prädikate umfasst. Der Datensatz verwendet eine Abstract Syntax Tree (AST) Repräsentation der Kriterien, welche komplexe logische Strukturen wie Verschachtelungen und zeitliche Sequenzen abbilden kann (N. J. Dobbins et al., 2023).

Die folgenden Methoden zur Extraktion und Strukturierung von Eignungskriterien nutzen sowohl frei verfügbare Korpora als auch selbst annotierte Daten für die Optimierung und die Evaluation der erstellten Methoden.

4.3. Methoden zur Extraktion und Strukturierung

Die Extraktion und Strukturierung von Eignungskriterien haben mehrere Entwicklungsphasen durchlaufen. Zunächst wurden formale Repräsentationen und regelbasierte Ansätze genutzt, um Eignungskriterien strukturiert abzubilden, wie in Unterabschnitt 4.3.1 und Unterabschnitt 4.3.2 erläutert. Mit dem Aufkommen von Machine Learning-Methoden kamen vermehrt ML-Algorithmen zur Strukturierung zum Einsatz oder wurden in Kombination mit regelbasierten Ansätzen angewandt, wie in Unterabschnitt 4.3.3 beschrieben. Die Einführung transformerbasierter Deep Learning-Modelle veränderte die Modellnutzung für Information Extraction-Aufgaben. Pre-trained Language Models, insbesondere BERT-basierte Modelle, die durch domänenspezifisches Fine-Tuning optimiert sind, wurden zu einem Standardwerkzeug für IE-Aufgaben, wie in Unterabschnitt 4.3.4 gezeigt. Aktuelle Forschungsarbeiten nutzen Large Language Models wie GPT-3 für NER- und RE-Aufgaben, um Eignungskriterien zu strukturieren. Diese Modelle werden sowohl einzeln als auch in Kombination mit anderen Methoden angewendet, wie in Unterabschnitt 4.3.5 erläutert. Methoden, die ein End-to-End Patient-Studie Matching implementieren, sind in Unterabschnitt 4.3.6 beschrieben.

4.3.1. Formelle Repräsentationen

Ein wesentliches Hindernis für die gemeinsame Nutzung berechenbaren klinischen Wissens ist das Fehlen einer einheitlichen Sprache zur Definition von Ausdrücken und Kriterien (Sordo et al., 2004). Frühere Projekte haben dabei formelle Repräsentationen eingesetzt, um Eignungskriterien für klinische Studien maschinenlesbar darzustellen.

Die **Arden-Syntax** ist eine Sprache zur Darstellung medizinischer Entscheidungsregeln in modularen Einheiten, sogenannten *Medical Logic Modules*, die den Austausch von klinischem Wissen zwischen verschiedenen Gesundheitseinrichtungen erleichtert (Hripcsak et al., 1993).

¹<https://github.com/uw-bionlp/clinical-trials-gov-annotation/wiki>

GELLO ist eine objektorientierte Ausdruckssprache, die entwickelt wurde, um logische Bedingungen und Berechnungen für die klinische Entscheidungsunterstützung auszudrücken (Sordo et al., 2004). Abbildung 4.2 zeigt die Integration von GELLO-Ausdrücke in ein Arden-Syntax Modul.

```

let renal_failure_threshold :=
    PhysicalQuantity.new(2.0, "mg/dl")
let renal_failure :=
if creatinine->notEmpty() and
    creatine.observed_quantity() > renal_failure_threshold
then
    true
else
    false
endif
let threshold_for_osteodystrophy := 70
calcium_phosphate_product :=
    if calcium->notEmpty() and phosphate->notEmpty() then
        calcium.observed_quantity.value *
        phosphate.observed_quantity.value
    else
        -1
    endif

```

Abbildung 4.2.: GELLO-Ausdrücke, integriert in die Arden-Syntax (Sordo et al., 2004).

GELLO-Ausdrücke und -Klassen wie *PhysicalQuantity* definieren medizinische Werte und Kriterien, während die Arden-Syntax mit *let*-Anweisungen und *if-else* Blöcken den Rahmen für die Auswertung und klinische Entscheidungslogik bereitstellt (Sordo et al., 2004). Laut Tu et al. (2011) weisen diese Ansätze mehrere Limitationen auf. Zu den Hauptproblemen zählt der Mangel an einheitlichen Standards, der die Interoperabilität beeinträchtigt. Darüber hinaus stellt die präzise Codierung komplexer Eignungskriterien wie *“Histologically or cytologically confirmed extensive stage small cell lung cancer”* eine Herausforderung dar. Zudem führt der zeitaufwendige Formalisierungsprozess zu Skalierungsproblemen bei der Anwendung auf ein breites Spektrum von Studien (Tu et al., 2011).

4.3.2. Regelbasierte Ansätze

Regelbasierte Ansätze sind Methoden zur Automatisierung klinischer Aufgaben, die auf vordefinierten Regeln basieren. Sie nutzen eine Vielzahl formaler Regeln und Muster, um Entscheidungen zu treffen oder Informationen zu extrahieren. Aufgrund der begrenzten Abdeckung durch vordefinierte Regeln und Ontologien weisen regelbasierte Systeme hinsichtlich der Information Extraction-Aufgabe bei Eignungskriterien häufig eine hohe Precision, jedoch einen geringeren Recall auf (Tian; Erdengasileng et al., 2021).

In der Studie von C. Weng et al. (2011) wird **EliXR** ein Netzwerk zur semantischen Repräsentation von Eignungskriterien für klinische Studien vorgestellt. Das Netzwerk kombiniert syntaktische Analysen mit der Erkennung von Mustern in Abhängigkeitsbaumstrukturen, die die Beziehungen zwischen den Wörtern und Phrasen eines Kriterien-Satzes darstellen. Ziel ist es, häufige semantische Muster in 1000 zufällig ausgewählten Eignungskriterien zu identifizieren. Dabei wurden 175 semantische Muster erkannt, die zu zwölf übergeordneten Rollenbezeichnungen wie medizinischer Zustand, Therapie oder Medikation zusammengefasst wurden (C. Weng et al., 2011).

EliXR-TIME, eine spezialisierte Erweiterung des EliXR-Frameworks, fokussiert sich auf die Extraktion und formale Repräsentation temporaler Ausdrücke in klinischen Eignungskriterien. Das System zielt auf die präzise Annotation und Formalisierung zeitlicher Spezifikationen wie *“täglich”* oder *“5 Tage pro Woche”*, die charakteristisch für Eignungskriterien sind. Die Verarbeitung basiert auf einer semi-automatisierten Pipeline, welche manuelle Validierungsschritte integriert, um optimale Annotations- und Analyseergebnisse sicherzustellen (Boland et al., 2012).

Die Studie von Hao et al. (2016) entwickelte **Valx**, ein automatisiertes System zur Extraktion und Strukturierung numerischer Laborwertvergleiche aus klinischen Studientexten. Das System integriert semantisches Wissen aus dem UMLS sowie domänenspezifische Expertise und implementiert einen regelbasierten Prozess mit sieben Verarbeitungsstufen. Dieser ermöglicht die Erkennung, Extraktion und Normalisierung von Laborwertangaben wie *“HbA1c $\geq$ 7,5 %”*. Die Evaluation der Systemleistung erfolgte anhand manuell annotierter Datensätze, die spezifisch aus Eignungskriterien diabetologischer Studien erstellt wurden (Hao et al., 2016).

4.3.3. Machine Learning-Ansätze

Für die Strukturierung klinischer Daten finden Machine Learning-Methoden wie Support Vector Machines (SVM) und Conditional Random Fields (CRF) Anwendung, wobei diese häufig mit regelbasierten Parsern kombiniert werden. Die Untersuchungen von Tian; Erdengasileng et al. (2021) belegen, dass solche hybriden Ansätze höhere Werte für Precision und Recall erreichen und damit eine robustere Verarbeitung ermöglichen (Tian; Erdengasileng et al., 2021).

In der Forschungsarbeit von Kang et al. (2017) wurde das Information Extraction-System **ElIIE** für klinische Eignungskriterien vorgestellt (ElIIE, 2017). Aufbauend auf früheren Arbeiten automatisiert das System die Umwandlung von Eignungskriterien in strukturierte EHR-Datenanfragen mittels NLP- und Machine Learning-Methoden. Es nutzt das OMOP-CDM zur Darstellung von Eignungskriterien, um die semantische Interoperabilität mit EHR-Daten zu verbessern. ElIIE übertrifft bestehende Systeme wie CliNER (Boag et al., 2015) durch den Einsatz von Machine Learning-Methoden, die bei NER-Aufgaben F1-Scores von 0.79 und bei RE-Aufgaben 0.98 erzielen. Die Negationserkennung basiert dabei auf dem NegEx-Algorithmus in Kombination mit vordefinierten Regeln, während für die Relationsextraktion ein SVM-Klassifikator eingesetzt wird.

Example 3	- Within the previous 2 years , unstable or clinically significant cardiovascular disease
ElIIE	<pre> <sent> <text>- Within the previous 2 years , unstable or clinically significant cardiovascular disease .</text> <attribute class="Temporal_measurement" index="T1" start="1"> Within the previous 2 years </attribute> <attribute class="Qualifier" index="T2" start="7"> unstable </attribute> <attribute class="Qualifier" index="T3" start="9"> clinically significant </attribute> <entity class="Condition" encoding="40568109" index="T4" negation="N" relation="T2:modified_by T3:modified_by T1:has_tempMea" start="11"> cardiovascular disease </entity> </sent> </pre>
CliNER	<pre> Within the previous 2 years , unstable or <problem> clinically significant cardiovascular disease </problem> </pre>
EliXR	<pre> study 1 Condition Occurrence exclusion within the previous 2 years , unstable or clinically significant cardiovascular disease C0443343{unstable;False;False;lower 2 previous years;;;} C0007222{cardiovascular disease;False;False;;;} </pre>

Abbildung 4.3.: Eine exemplarische Ausgabe des ElIIE-Systems (Kang et al., 2017).

In Abbildung 4.3 werden die Ergebnisse des ElIIE-Systems im Vergleich zu dem NER-Tool CliNER (Boag et al., 2015) und dem EliXR-System (C. Weng et al., 2011) anhand der Analyse eines exemplarischen Eignungskriteriums veranschaulicht. Dabei wird gezeigt, wie die unterschiedlichen Modelle einen Beispielsatz verarbeiten und strukturieren (Kang et al., 2017).

Criteria2Query (C2Q) wurde von C. Yuan et al. (2019) entwickelt und bietet ein NLP-Interface, welches freitextliche Eignungskriterien für klinische Studien in strukturierte SQL-Abfragen umwandelt, die dem OMOP-CDM entsprechen. Das System kombiniert Machine Learning-Modelle wie Conditional Random Fields für NER-Aufgaben mit regelbasierten Methoden wie NegEx für die Negationsdetektion. Der halbautomatisierte Hybridansatz ermöglicht es Benutzern, generierte Datenbankabfragen interaktiv zu überprüfen und anzupassen (C. Yuan et al., 2019). Die Benutzeroberfläche von Criteria2Query wird im Anhang in Abbildung A.1 dargestellt. Nutzer können Ein- und Ausschlusskriterien über die Plattform eingeben und erhalten eine annotierte Version mit verschiedenen Tags, beispielsweise "AD" für Alzheimer. Diese Kriterien können anschließend durch manuelle Anpassungen weiter verfeinert werden (C. Yuan et al., 2019).

Tseo et al. (2020) entwickelten den **Clinical Trial Parser** (CTP, 2020). Dieser Ansatz kombiniert Deep Learning-Methoden mit einer Context Free Grammar (CFG) für die Entitätserkennung bei Eignungskriterien. Die zugrundeliegende Wissensbasis integriert Medical Subject Headings (MeSH) ein standardisiertes Vokabular zur Klassifizierung medizinischer Begriffe. Der Ansatz nutzt eine Attention-Bi-LSTM + CRF-Architektur für die NER-Aufgabe sowie die Anwendung von Word2Vec Embedding-Clustering, um die extrahierten Entitäten mit den entsprechenden Konzepten in der Wissensbasis zu verknüpfen. Für die Extraktion spezifischer Attributkriterien wie Alter, BMI oder klinischer Variablen wird ein CFG-Parser verwendet. Über regelbasierte Relation-Extraction-Methoden wird ermittelt, ob Studienteilnehmer mit spezifischen Eigenschaften akzeptiert oder ablehnt werden, indem primär Negationen erfasst werden. Auf dem Goldstandard von Criteria2Query (C. Yuan et al., 2019) erreicht Clinical Trial Parser einen F1-Score von 0.802 für die Entitätserkennung. Das bisherige SOTA-Modell Criteria2Query erreicht auf dem Datensatz einen F1-Score von 0.804 (Tseo et al., 2020).

4.3.4. BERT-basierte Ansätze

Das von Devlin et al. (2019) entwickelte BERT-Modell hat bei verschiedenen NLP-Aufgaben, insbesondere im Bereich der Named Entity Recognition, vielversprechende Ergebnisse erzielt (J. Li et al., 2022). Im Kontext der Entitäts- und Relationsextraktion aus Eignungskriterien wurden verschiedene vortrainierte BERT-Varianten angewendet und mit etablierten Deep Learning- und Machine Learning-Methoden verglichen. Mehrere Studien untersuchten die Leistungsfähigkeit von BERT-Modellen mit domänenspezifischem Fine-Tuning auf Korpora mit Eignungskriterien.

In einer Untersuchung von Zeng et al. (2021) wird ein auf Transformer-basierendes **Ensemble Lernmodell** für die Klassifizierung von Eignungskriterien in klinischen Studien vorgeschlagen. Dieses Modell kombiniert eine Auswahl von Pre-trained Language Models einschließlich BERT, RoBERTa, XLNet, ELECTRA und ERN1 und vergleicht diese, mit Deep Learning-Modellen, darunter TextCNN, TextRNN und FastText. Evaluiert wurde eine CHIP-Klassifizierungsaufgabe, bei der unstrukturierte, chinesische Eignungskriterientexte in 44 Kategorien eingeordnet werden müssen. Das Ensemble-Modell performt mit einem F1-Score von 0.81 um 0.84% besser als die auf Deep Learning-basierenden Vergleichsmodelle (Zeng et al., 2021).

Ein transformerbasiertes NER-Modell zur Analyse von Eignungskriterien in klinischen Studien wurde in der Studie von Tian; Erdengasileng et al. (2021) evaluiert. Die Forschenden verglichen vier Modelle (BERT, ALBERT, RoBERTa, ELECTRA), welche sowohl mit allgemeinen englischen Korpora als auch mit spezialisierten klinischen Korpora aus PubMed-Zitaten, klinischen Notizen von MIMIC-III und Eignungskriterien aus ClinicalTrials.gov (2000) vortrainiert wurden. Das **RoBERTa**-Modell, welches mit MIMIC-III Daten und Eignungskriterien vortrainiert wurde, erzielte den höchsten F1-Score mit 0.65 bei NER-Aufgaben auf dem Chia-Korpus und mit 0.78 F1-Score die beste Leistung auf dem FRD-Korpus (Tian; Erdengasileng et al., 2021).

K. Wu; Aklilu (2021) entwickelten **ecBERT** (eligibility criteria-BERT), ein auf BERT-basierendes Modell zur Named Entity Recognition auf Eignungskriterien in klinischen Studien. Das Modell wurde aus einem vortrainierten BERT-Modell abgeleitet und anschließend durch zusätzliches Fine-Tuning auf einem umfangreichen Korpus von unannotierten Eignungskriterien weiter spezialisiert. In einem Vergleich der NER-Leistung mit anderen spezialisierten BERT-Modellen wie BlueBERT und Clinical-bioBERT, angewendet auf den Chia-Datensatz (Kury et al., 2020), erreichte ecBERT den höchsten F1-Score von 0.54. Im Vergleich dazu erzielte das blueBERT-Modell einen F1-Score von 0.43 und Clinical-bioBERT einen Wert von 0.52 (K. Wu; Aklilu, 2021).

(X. Liu et al., 2021) entwickelten **CT-BERT**, ein Framework zur Information Extraction aus klinischen Studientexten, das auf verschiedenen BERT-Varianten wie BERT, BioBERT, ClinicalBERT und BlueBERT basiert (CTBERT, 2021). Diese Modelle wurden auf einem breiten Spektrum von Textkorpora vortrainiert von allgemeinen englischen Texten bis hin zu spezialisierten medizinischen Dokumenten. Innerhalb des Frameworks wurden die BERT-Modelle einem Fine-Tuning auf annotierten Eignungskriterien aus dem FRD-Korpus unterzogen. CT-BERT nutzt NER- und RE-Methoden, um aus Eignungskriterien klinischer Studien relevante Entitätstypen und Relationen zu extrahieren. Auf dem FRD-Testdatensatz erzielt die CT-BERT Methode mit einem Fine-Tuning von BioBERT-Large mit 0.84 den höchsten F1-Score und übertrifft das Attention Bi-LSTM aus Tseo et al. (2020) und das Criteria2Query-System aus X. Liu et al. (2021) um je 4% F1-Score.

J. Li et al. (2022) vergleicht die Leistungen mehrerer PLMs für die NER-Aufgabe auf drei Korpora mit Eignungskriterien. Sechs BERT-Varianten wurden untersucht, darunter zwei aus allgemeinen Domänen (BERT und SpanBERT) und vier speziell für den biomedizinischen Bereich (BioBERT, BlueBERT, PubMedBERT und SciBERT). Die Modelle wurden auf den Korporas Covance (M. Chen et al., 2020), EliE (Kang et al., 2017) und Chia (Kury et al., 2020) mit NER-Aufgaben evaluiert. Dabei schnitt **PubMedBERT** mit F1-Scores von 0.71, 0.83 und 0.62 über die drei Korpora hinweg am besten ab. Zudem konnte gezeigt werden, dass BERT-Varianten mit domänenspezifischem Fine-Tuning besser performen als die allgemeinen Modelle (J. Li et al., 2022).

Eine Erweiterung von Criteria2Query stellt Fang et al. (2022) mit **Criteria2Query-2.0** vor. Extrahierte Informationen aus den Eignungskriterien werden in einem JSON-Format repräsentiert. Erkannte Entitäten, Kategorien und zugeordnete Konzepte werden dem Benutzer zur Überprüfung über ein webbasiertes Interface bereitgestellt. Die über den Nutzer vorgenommenen Modifikationen können in Echtzeit gespeichert werden. Ein Fine-Tuning des PubMedBERT-Modells verbesserte die Negationsdetektion, wie Evaluierungen auf dem COVIC-Korpus (Sun et al., 2021) zeigen. Dabei erreichte C2Q-2.0 mit einem F1-Score von 0.922 eine Leistungssteigerung gegenüber dem ursprünglichen C2Q-Modell mit einem F1-Score von 0.77 (Fang et al., 2022).

N. Dobbins; Tony Mullen et al. (2022) untersuchen die Leistung von BERT-Modellen für NER- und RE-Aufgaben auf dem selbst entwickelten LCT-Korpus. Für die NER-Aufgabe wurden auf dem biomedizinischen-Bereich spezialisierte Modelle wie **SciBERT** und PubMedBERT evaluiert. Neben diesen transformerbasierten-Ansätzen wurde auch ein Bi-LSTM + CRF-Modell evaluiert. Bei der NER-Aufgabe erzielt das SciBERT-Modell mit einem F1-Score von 81.3% die höchste Leistung gegenüber PubMedBERT mit 77.0% und dem Bi-LSTM + CRF-Modell mit 79.9%. Für die Extraktion von Relationen wurde eine Kombination aus einer angepassten BERT-Architektur (R-BERT) (S. Wu; He, 2019) und SciBERT verwendet und mit SciBERT verglichen. Die R-BERT + SciBERT-Kombination performt mit einem F1-Score von 85.2% im Durchschnitt besser als das SciBERT-Modell mit 84.0% F1-Score (N. Dobbins; Tony Mullen et al., 2022).

Das von N. J. Dobbins et al. (2023) entwickelte **LeafAI**-System, strukturiert Eignungskriteriensätze über einen hybriden-Ansatz, der mehrere Pre-trained Language Models mit regelbasierten Ansätzen kombiniert. Das System ist in Abbildung 4.4 vorgestellt. Zunächst werden zwei BERT-Varianten auf dem LCT-Korpus für NER- und RE-Aufgaben einem Fine-Tuning unterzogen. Die Modelle extrahieren Entitäten und deren Relationen aus den Kriteriensätzen. Die extrahierten Entitäten und Relationen werden von einem T5-Modell in eine logische Form überführt, die in Abbildung 4.4 als *Prepare logical form input* dargestellt ist. Das T5-Modell hat für die logische Transformation ein Fine-Tuning auf dem erstellten LLF-Korpus erhalten. Die erzeugte logische Struktur wird mit einem rekursiven Parser in eine Abstract Syntax Tree (AST) Struktur transformiert. Innerhalb dieser AST-Objekte wurden die zuvor extrahierten Entitäten mit einem weiteren Framework in Codierungssysteme wie UMLS, ICD-10, SNOMED-CT und LOINC normalisiert. Dafür wird unter anderem MetaMapLite (2016) und ein BERT-Modell mit Fine-Tuning verwendet. Die Entitäten werden über RDF-Tripel verarbeitet, um die ursprüngliche extrahierte Entität beizubehalten. Die so verarbeiteten AST-Objekte werden in SQL-Datenbanken gespeichert und können über SQL-Abfragen mit Patientendaten abgeglichen werden, wenn die EHR-Daten in der gleichen Form vorliegen.

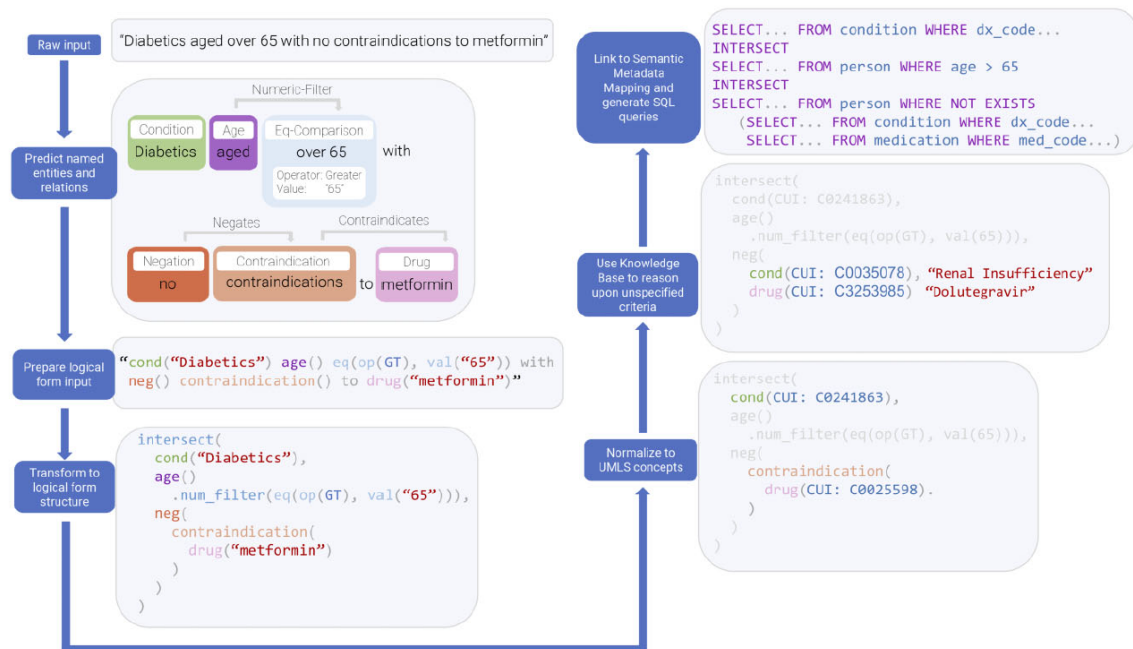


Abbildung 4.4.: Logische Strukturierung von Eignungskriterien in SQL-Abfragen mit LeafAI (N. J. Dobbins et al., 2023).

Das LeafAI-System konnte in einem direkten Patient-Studie Matching 43% der in klinischen Studien registrierten Patienten identifizieren, ein signifikanter Vorteil gegenüber den 27%, die von einem Datenbankspezialisten gefunden wurden (N. J. Dobbins et al., 2023).

4.3.5. Large Language Model-Ansätze

Mit dem Aufkommen von modernen Large Language Models (LLMs) mit mehreren Milliarden Parametern wie GPT-3, wurden diese auch für NER- und RE-Aufgaben bei der Strukturierung von Eignungskriterien eingesetzt. Aktuelle Studien untersuchen die Effektivität von Zero-Shot und Few-Shot Ansätzen bei der Analyse dieser Kriterien, wobei sowohl proprietäre als auch Open-Source LLMs evaluiert werden. Zudem werden LLMs in übergeordnete Analysepipelines integriert.

Park et al. (2023) erweiterten Criteria2Query (C. Yuan et al., 2019) zu **Criteria2Query-3.0** durch die Integration von **GPT-4**. Diese aktualisierte Version integriert speziell entwickelte Prompts in einen Prozess, welcher aus einer Konzeptextraktion und der anschließenden Strukturierung über SQL-Abfragen besteht. Abbildung A.2 im Anhang zeigt den Gesamtprozess von Criteria2Query-3.0 unterstützt durch GPT-4. Interaktionen zwischen Anwendern und dem System sind durch orange Pfeile dargestellt. Diese signalisieren, dass bei jeglichen Anpassungen der von GPT-4 erstellten Ergebnisse durch die Benutzer das System die Änderungen erkennt und die betreffenden Module automatisch neu ausführt. Das System kann über die Nutzung von GPT-4 in der Konzeptextraktion einen F1-Score von 0.891 erreichen, wobei der Testdatensatz aus Eignungskriterien von über 20 klinischen Studien bestand. Die Effektivität der LLM-Ausgabe wurde anhand von Kriterien wie Lesbarkeit, Korrektheit, Kohärenz und Nützlichkeit bewertet, wobei C2Q-3.0 in allen Bereichen Verbesserungen gegenüber dem Vorgängermodell zeigt (Park et al., 2023).

In der Studie von Datta et al. (2023) wurden Eignungskriterien mit **AutoCriteria**, einem auf GPT-4 basierendem System prozessiert. Dabei wurden über NER-Aufgaben die Eignungskriterien tabellarisch strukturiert. Ein eigener Datensatz wurde aus über 180 klinischen Studien aus ClinicalTrials.gov (2000) für neun verschiedene Krankheiten erstellt und annotiert. Es wurde eine Prompt entwickelt, die das LLM anweist, Entitäten aus mehreren Entitätstypen wie *Diagnosis* oder *Score*, inklusive deren *Attribut*-, *Value*-, *Modifier*- und *Temporal*-Eigenschaften in tabellarischer Form auszugeben. Die Prompt für das GPT-4 Modell wurde in mehreren Iterationen innerhalb des Systems entwickelt und verfeinert. Das AutoCriteria-System ist über mehrere Phasen in Abbildung 4.5 dargestellt.

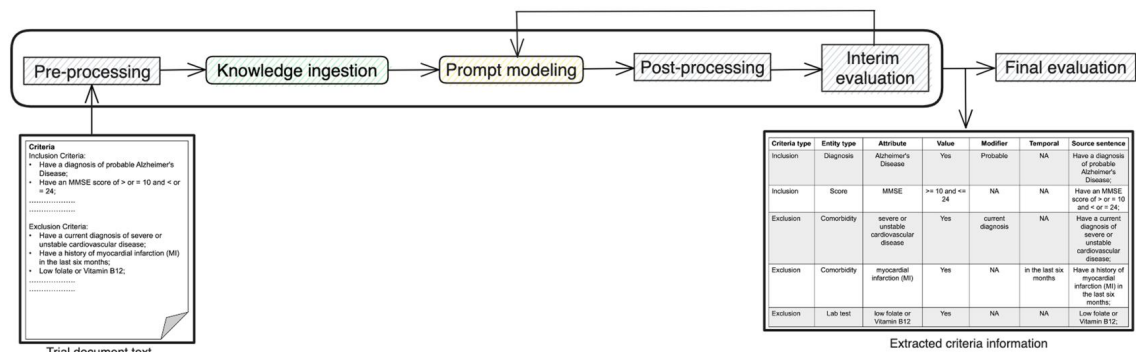


Abbildung 4.5.: Tabellarische Strukturierung von Eignungskriterien mit dem AutoCriteria-System (Datta et al., 2023).

In der Vorverarbeitung werden Eignungskriterien aufbereitet und segmentiert. Die Wissensengabe dient der Definition relevanter medizinischer Begriffe. Während der Anfragemodellierung werden spezifische Prompts für das GPT-4 Modell entwickelt. Die Nachbearbeitung soll die Konsistenz der Antworten von GPT-4 verbessern. Schließlich werden im Rahmen der Zwischen- und Endbewertung die Genauigkeit und Zuverlässigkeit des Systems beurteilt. AutoCriteria hat einen F1-Score von 89.42% für neun Krankheiten mit einer Accuracy von 78.95% bei der Few-Shot Entitätsextraktion erzielt (Datta et al., 2023).

Wong et al. (2023) haben ein **Expert System** entwickelt und dieses im Rahmen einer Analyse mit den LLMs GPT-4 und GPT-3.5 sowie den bestehenden Lösungen Criteria2Query und SciSpaCy für das End-to-End Patienten-Studie Matching in selbst annotierten Onkologiestudien verglichen. Der Fokus der Bewertung lag auf der Extraktion von Entitäten, insbesondere auf der Identifizierung von Histologie und Biomarkern, um die Zuordnung von Patienten zu Onkologiestudien zu optimieren. Das Expert System umfasst die Strukturierung von Eignungskriterien, Abgleich mit Patientendaten und eine Benutzeroberfläche mit menschlicher Prüfung über die Human-in-the-Loop (HITL) Methode. Die Strukturierung von Eignungskriterien umfasst Ein- und Ausschlusskriterien, welche implizit durch OR-, AND- und NOT-Logiken verknüpft sind. Das System nutzt für die Strukturierung eine Promptanweisung, welche das GPT-4 Modell anleitet, relevante Informationen aus klinischen Studien zu extrahieren und in JSON zu strukturieren. Dabei sollen Informationen aus den Kategorien (*disease state*, *histology inclusion*, *biomarker inclusion*, *histology exclusion* und *biomarker exclusion*) extrahiert werden. Die extrahierten Entitäten sollen so implizit in einer logischen Disjunktive Normalform (DNF) gespeichert werden. Ein Beispiel für die Strukturierung ist in Abbildung 4.6 dargestellt.

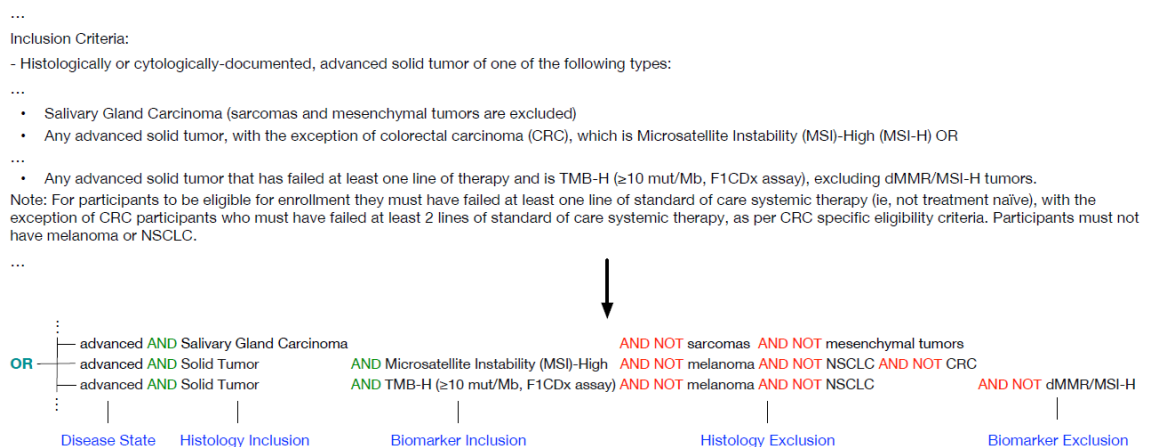


Abbildung 4.6.: Eignungskriterien der Onkologiestudie NCT04412629 in Disjunktive Normalform (Wong et al., 2023).

Die logische Form wird bei der Extraktion implizit über die jeweilige Zuordnung zu der entsprechenden Kategorie repräsentiert. Entitäten, die der Kategorie *Biomarker Inklusion* zugeordnet werden, sind über AND-Relationen verknüpft, während Entitäten in *Biomarker Exclusion* über NOT-AND verbunden sind. Die Kategorien werden in einem JSON-Format als Keys angegeben und die Entitäten werden als Values in Arrays gespeichert, wie exemplarisch in Abbildung 4.7 gezeigt.

```
[
  {
    "cohort": "Dose Escalation Phase (Part 1)",
    "disease_state": "advanced or metastatic",
    "histology_inclusion": "Solid Tumor",
    "biomarker_inclusion": ["RAS mutation"],
    "histology_exclusion": [],
    "biomarker_exclusion": []
  },
  {
    "cohort": "Dose Escalation Phase (Part 1)",
    "disease_state": "advanced or metastatic",
    "histology_inclusion": "Solid Tumor",
    "biomarker_inclusion": ["NF1 mutation"],
    "histology_exclusion": [],
    "biomarker_exclusion": []
  },
  {
    "cohort": "Dose Escalation Phase (Part 1)",
    "disease_state": "advanced or metastatic",
    "histology_inclusion": "Solid Tumor",
    "biomarker_inclusion": ["RAF mutation"],
    "histology_exclusion": [],
    "biomarker_exclusion": []
  },
  (... more cohorts are omitted for space saving)
]
```

Abbildung 4.7.: Logische DNF Struktur mit GPT-4 im JSON-Format (Wong et al., 2023).

Bei der Entitätserkennung auf einem erstellten Evaluations-Korpus erreicht GPT-4 mit Zero-Shot, unter Berücksichtigung der extrahierten Logik in DNF, einen F1-Score von durchschnittlich 24% und übertrifft damit Criteria2Query mit einem F1-Score von 12.4%. GPT-4 mit 3-Shot konnte ein F1-Score von 29.6% erreichen. Bei der alleinigen Evaluation der *Biomarker*-Extraktion erreichte GPT-4 mit Zero-Shot einen F1-Score von 66.7%. Die Leistung sank auf 22.2% wenn die extrahierten *Biomarker* zusätzlich in DNF strukturiert werden mussten (Wong et al., 2023).

Eine aktuelle Studie von Munnangi et al. (2024) untersucht die Leistung von LLMs bei der Named Entity Recognition-Aufgabe auf biomedizinischen-Korpora unter Verwendung von In-Context Learning Methoden. Die Autoren evaluieren die proprietäre LLMs **GPT-3.5**, GPT-4 und Claude2 sowie Llama2 und ein Flan-T5 Modell auf sechs biomedizinischen NER-Datensätzen, einschließlich dem Chia-Datensatz. Mit einem 5-Shot Ansatz, bei dem der Input der Chia-Daten als Text übergeben wurde und die Entitäten im JSON-Format ausgegeben wurden, erzielte GPT-3.5 die beste Leistung mit einem F1-Score von 36.47% mit einer Standardabweichung von 0.6 über drei Versuche. Das Llama2-Modell erzielte einen F1-Score von 35.44% mit einer höheren Standardabweichung von 1.85 (Munnangi et al., 2024). Abbildung 4.8 zeigt ein Few-Shot Beispiel für die Entitätsextraktion mit LLMs aus einem biomedizinischen-Korpus.



Given the sentence from an abstract, extract all the chemicals and diseases and return as a json.

Sentence: BE-Induced seizures occurred more frequently and had significantly longer latencies than those induced by equimolar amounts of cocaine.'},

Output : {"chemicals": ["BE", "cocaine"], "diseases": ["seizures"]}



Sentence: Famotidine-associated delirium.

Output: {"chemicals": ["Famotidine"], "diseases": ["delirium"]}

Abbildung 4.8.: Exemplarische Few-Shot Prompt einer NER-Aufgabe für LLMs mit strukturierter JSON-Ausgabe (Munnangi et al., 2024).

Im Auswertungsprozess werden die Datensätze sequenziell an das LLM übergeben, das chemische Substanzen und Krankheiten identifiziert und die Ergebnisse in einem strukturierten JSON-Format ausgibt. Dabei dienen die Entitätstypen als Keys, während die erkannten Entitäten als Elemente in Arrays den entsprechenden Entitätstypen zugeordnet werden (Munnangi et al., 2024).

4.3.6. End-to-End Matching-Systeme

Aktuelle Forschungsarbeiten konzentrieren sich zunehmend auf die Entwicklung von End-to-End-Systemen, die Patientendaten direkt mit den Eignungskriterien klinischer Studien abgleichen. Diese Systeme nutzen überwachte Lernmethoden, um Patienten basierend auf vordefinierten Matching-Annotationen direkt bestimmten Studien oder Kriterien zuzuordnen (Wornow et al., 2024). Dabei verarbeiten sie die Eignungskriterien in ihrer ursprünglichen, unstrukturierten Textform ohne vorherige Aufbereitung oder Strukturierung der Kriterien (Wong et al., 2023). Methoden wie *DeepEnroll* (Zhang et al., 2020) und *COMPOSE* (Gao et al., 2020) verwenden BERT-Varianten, um Eignungskriterien klinischer Studien und Patientendaten aus Electronic Health Records in einen gemeinsamen Vektorraum zu überführen. Large Language Model-Ansätze bauen auf diesen Grundlagen auf und setzen insbesondere GPT-Modelle ein, um das Patient-Studie Matching zu optimieren. Beispiele hierfür sind *LLM-PTM* (J. Yuan et al., 2023), *FairPM* (Chang et al., 2023), *TrialGPT* (Jin et al., 2023) und *InstructGPT* (Hamer et al., 2023). Zudem werden auch Open-Source LLMs für das End-to-End Patient-Studie Matching eingesetzt.

TrialLLaMa von (Nievas et al., 2023) führen ein Fine-Tuning von Open-Source LLMs wie Llama2 in den Varianten 7B, 13B und 70B für das Patient-Studie Matching aus. In dem entwickelten Ansatz werden den Modellen sowohl Eignungskriterien klinischer Studien als auch Patienteninformationen als Eingabe übergeben, wobei die Aufgabe darin besteht, die Eignung des Patienten für die jeweilige Studie zu beurteilen. Der rechenintensive Trainingsprozess wurde auf 16 NVIDIA A100 GPUs durchgeführt (Nievas et al., 2023). Wornow et al. (2024) evaluierten sowohl proprietäre Modelle (GPT-3.5 und GPT-4) als auch Open-Source Modelle (Llama2-70B und Mixtral-8x7B) für die Zuordnung von Patienten zu klinischen Studien. Dabei konnte ein **Zero-Shot** Ansatz auf der n2c2 2018 Cohort-Selection Benchmark aus Oleynik et al. (2019) implementiert werden. Mit GPT-4 konnten 86 Patienten in etwa zwei Stunden gescreent werden, während herkömmliche Methoden etwa eine Stunde pro Patient benötigen. Die Kosten betrugen dabei \$1.55 pro Patient für GPT-4 API-Anfragen (Wornow et al., 2024).

4.4. Zusammenfassung

Die Forschungsgebiete zur Analyse von Eignungskriterien aus klinischen Studien lassen sich in drei Hauptkategorien einteilen: Strukturierung über NER- und RE-Methoden, Patienten-Studie Matching Systeme mit verwertbarer Strukturierung der Kriterien und End-to-End Matching Systeme, bei denen die prozessierten Kriterien nicht weiterverwendet werden können. Zum Zeitpunkt der Recherche erzielten BERT-Varianten mit einem domänenspezifischen Fine-Tuning die besten Ergebnisse bei NER- und RE-Aufgaben zur Strukturierung von Eignungskriterien. Large Language Models wie GPT-3 oder Llama2 wurden aufgrund ihrer jüngeren Entwicklung bisher seltener bei der Analyse von Eignungskriterien eingesetzt. Die Integration von LLMs in komplexere Architekturen wie Criteria2Query, welche zusätzlich regelbasierte, ML- und BERT-Methoden kombinieren, konnte die logische Strukturierung der Kriterien verbessern (Park et al., 2023). Bei NER-Aufgaben liegt die Leistung von LLMs mit Few-Shot Methoden im Vergleich unter den F1-Scores von BERT-Modellen mit domänenspezifischem Fine-Tuning (Munnangi et al., 2024).

Datensatz	Modelle	Entitäten	F1	Publikation
Chia	RoBERTa-MIMIC-Trial	11	0.65	(Tian; Erdengasileng et al., 2021)
	ecBERT	30	0.54	(K. Wu; Aklilu, 2021)
	PubMedBERT	15	0.62	(J. Li et al., 2022)
	GPT-3.5 (5-Shot)	30	0.36	(Munnangi et al., 2024)
FRD	RoBERTa-MIMIC-Trial	15	0.78	(Tian; Erdengasileng et al., 2021)
	CT-BERT (BioBERT)	15	0.84	(X. Liu et al., 2021)
LCT	SciBERT	50	0.81	(N. Dobbins; Tony Mullen et al., 2022)

Tabelle 4.2.: Auswertung der am besten performenden NER-Modelle der jeweiligen Studie auf öffentlich verfügbaren Korpora mit Eignungskriterien. Die F1-Scores beziehen sich auf den Durchschnitt der erkannten Entitäten.

In Tabelle 4.2 wird eine Übersicht der Leistung von NER-Methoden auf den frei verfügbaren Korpora Chia, FRD und LCT, aus Tabelle 4.1, gegeben. Dabei konnten BERT-Modelle auf dem Chia-Datensatz F1-Scores von über 60% erreichen, während ein 5-Shot Ansatz mit GPT-3.5 einen F1-Score von 36% erzielte. Auf den FRD- und LCT-Datensätzen erreichten BERT-Modelle mit Fine-Tuning, F1-Scores von über 80% für NER-Aufgaben.

Für die Strukturierung von Eignungskriterien wurden Relation Extraction-Aufgaben, die extrahierte Relationen einzeln evaluieren, bisher nur von N. Dobbins; T. Mullen et al. (2022) unter Einsatz von BERT-Varianten auf dem LCT-Korpus durchgeführt und veröffentlicht.

Logische Strukturierung

Einige transformerbasierte-Systeme haben die Strukturierung von Eignungskriterien weiter optimiert, indem die implizite logische Struktur der Kriterien mit AND-, OR- und NOT-Operatoren nutzbar gemacht wurde. Das LeafAI-System extrahiert mit BERT-Modellen Entitäten, welche über ein T5-Modell logisch verknüpft und über einen Parser in eine Abstract Syntax Tree-Struktur gebracht werden (N. J. Dobbins et al., 2023). Abbildung 4.9 zeigt exemplarisch eine solche logische AST-Struktur der Kriterien.

```
intersect(
  cond("Diabetics"),
  age()
    .num_filter(eq(op(GT), val("65"))),
  neg(
    contraindication(
      drug("metformin")
    )
  )
)
```

Abbildung 4.9.: Logische AST-Struktur mit LeafAI (N. J. Dobbins et al., 2023).

Ein von Wong et al. (2023) entwickelte In-Context Learning Methodik mit GPT-4 ermöglicht die strukturierte Repräsentation von Entitäten aus klinischen Onkologiestudien im JSON-Format. Die extrahierten Entitäten werden dabei implizit in der logischen Disjunktive Normalform (DNF) dargestellt. Abbildung 4.10 veranschaulicht diese Methode anhand der strukturierten Darstellung der Eignungskriterien der klinischen Studie NCT04412629 (Wong et al., 2023).

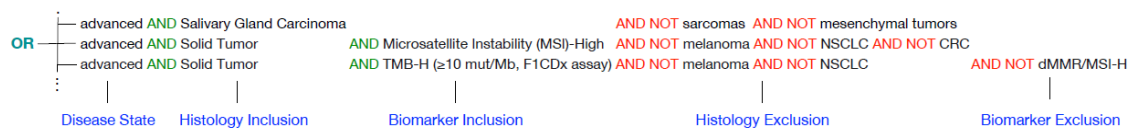


Abbildung 4.10.: Logische Few-Shot Strukturierung mit GPT-4 in DNF (Wong et al., 2023).

In diesem Kontext werden Entitäten der Typen *Disease State*, *Histology* und *Biomarker* für Einschlusskriterien mit AND-Operatoren verknüpft und für Ausschlusskriterien mit AND-NOT. Die Logik wird dabei implizit zwischen den extrahierten Entitäten im JSON-Format repräsentiert.

5. Methodik zur Strukturierung von Eignungskriterien

Die Strukturierung von Freitextkriterien in klinischen Studien mithilfe der Erkennung von Entitäten und Relationen ermöglicht eine maschinell verarbeitbare Darstellung der Informationen. Ein zentraler Aspekt dabei ist die Extraktion der logischen Operatoren AND, OR und NOT, welche die inhärente logische Struktur der Ein- und Ausschlusskriterien abbilden. Diese komplexen und verschachtelten Strukturen sind entscheidend für die präzise Interpretation der Kriterien. Ohne eine logische Verknüpfung der extrahierten Entitäten aus den Kriterientexten können diese nicht korrekt interpretiert werden. Zudem müssen die über NER- und RE-Aufgaben verarbeiteten Eignungskriterien in einem interoperablen, maschinenlesbaren Format gespeichert werden, um ihre Nutzung in logischen Suchanfragen nach klinischen Studien in verschiedenen klinischen Systemen zu erleichtern (N. J. Dobbins et al., 2023). Dies ermöglicht weiterhin die Übertragung der angereicherten Daten in etablierte klinische Standardformate wie OMOP-CDM oder FHIR und fördert die Interoperabilität zwischen verschiedenen Systemen. Durch die Einbeziehung der logischen Verknüpfungen zwischen den extrahierten Entitäten wird eine Grundlage geschaffen, um einen effektiven Abgleich mit Patientendaten zu ermöglichen und somit die präzise Identifizierung geeigneter Patienten für spezifische Studien zu unterstützen (Wong et al., 2023).

5.1. Experimenteller Aufbau

Die bisherigen State-of-the-Art Methoden für die Extraktion von logischen Relationen zwischen Entitäten basieren auf BERT-Varianten, die durch Fine-Tuning auf großen, domänenspezifischen Datensätzen optimiert wurden. Für die Relationserkennung ist es erforderlich, die miteinander verbundenen Entitäten gleichzeitig zu extrahieren. Die Ausgaben für NER-Aufgaben mit BERT-Modellen erfolgen dabei üblicherweise im Beginning-Inside-Outside (BIO) Format, das die Positionen der Entitäten im Text angibt. Um die erkannten Entitäten für eine weitere Verarbeitung nutzbar zu machen, muss die Ausgabe von BERT-Modellen in Standardformate wie JSON oder XML transformiert werden, um eine interoperable Speicherung zu ermöglichen. Wenn der ursprüngliche Kriterientext im Parsing-Prozess nicht mit übertragen wird, können Informationen verloren gehen oder müssen nachträglich ergänzt werden. In der klassischen Entitätsextraktion werden der Anfang und das Ende einer Entität über Offsets im Text bestimmt und ausgegeben. BERT-Modelle können diese Offsets präzise ermitteln und liefern somit genaue Positionsinformationen der identifizierten Entitäten. Im Gegensatz dazu haben LLMs Schwierigkeiten bei der präzisen Wortlokalisierung und Offset-Extraktion, was mit den in Abschnitt 3.7 diskutierten Problemen bei numerischen Berechnungen zusammenhängt. Der generative Charakter von LLMs ermöglicht es, logische Operatoren wie AND, OR und NOT direkt in die ursprünglichen Texte einzufügen. Dadurch wird eine logische Strukturierung der Eignungskriterien erreicht, ohne dass die verknüpften Entitäten separat extrahiert werden müssen. Die inhärent verschachtelte und komplexe logische Struktur dieser Kriterien kann mittels eines Abstract Syntax Tree (AST) abgebildet werden, indem die Operatoren als Knoten und die Texte als Blätter fungieren. Zudem kann diese Struktur in interoperablen Formaten wie JSON-Dateien repräsentiert werden. Diese Methode erlaubt eine beliebig tiefe Verschachtelung der ursprünglichen Texte durch logische Operatoren und bietet die Flexibilität zusätzlich extrahierte Entitäten einzubinden.

Um die Hauptforschungsfrage **RQ0**: *“Wie kann unstrukturierter klinischer Text mit frei verfügbaren Machine Learning-Methoden strukturiert werden?”* zu beantworten, untersucht diese Arbeit die Anwendung aktueller Open-Source LLMs zur Extraktion logischer Operatoren aus Freitexten klinischer Eignungskriterien. Das Ziel besteht darin, die Kriterien in ein strukturiertes Format zu überführen, das sowohl eine präzise logische Suche als auch ein effektives Patient-Studie Matching ermöglicht. Die Identifizierung und Extraktion der logischen Operatoren AND, OR und NOT bilden die Grundlage für diese Strukturierung. Diese Operatoren ermöglichen den Aufbau einer komplexen, hierarchischen logischen Struktur, die eine genaue Analyse und Interpretation der Kriterien erlaubt. Durch die Verknüpfung von Satzteilen oder Entitäten mittels dieser Operatoren entsteht eine AST-Struktur, welche sich durch logische Formeln evaluieren lässt.

Im Rahmen der unterstützenden Forschungsfrage **RQ1**: “*Welche Methoden eignen sich zur Strukturierung von Eignungskriterien aus klinischen Studien?*” sollen die in Abschnitt 3.3 vorgestellten Anwendungsmethoden von LLMs für NER- und RE-Aufgaben evaluiert werden. Dazu zählen Prompt-Methoden, bei denen lediglich eine einzelne Anweisung zum Einfügen von Operatoren in Kriterientexte erstellt werden muss (Zero-Shot). Zusätzlich werden erweiterte Prompt-Methoden evaluiert, die darauf abzielen, das Zero-Shot Verfahren zu verbessern. Durch die Verarbeitung von annotierten Dateien aus öffentlichen Korpora werden zusätzlich Few-Shot und Fine-Tuning Methoden evaluiert. Der Fokus liegt dabei auf Open-Source LLMs wie Llama3, die mit Closed-Source Modellen wie GPT-4o verglichen werden. Zur Beantwortung der Forschungsfragen wurden zwei methodische Ansätze entwickelt. Beide Ansätze basieren auf der Erkennung logischer Operatoren in Eignungskriterien, um diese mithilfe eines Abstract Syntax Tree in einem interoperablen Format wie JSON zu strukturieren.

1. **Indirekte Strukturierung:** Ausgewählte Large Language Models setzen logische Operatoren über [AND]-, [OR]- und [NOT]-Tags in Eignungskriterientexte ein, indem ICL- und Fine-Tuning Methoden verwendet werden. Ein RegEx-Parser dient dabei als Baseline für die Operatorenextraktion. Die ausgegebenen Daten können mittels eines Parsers in eine Abstract Syntax Tree (AST) Struktur im JSON-Format überführt werden. Die Methodik ist in Abschnitt 5.4 dargestellt.
2. **Direkte Strukturierung:** Large Language Models werden mit Few-Shot und Fine-Tuning Methoden optimiert, um Eignungskriterien direkt in eine AST-Struktur zu verarbeiten. Dabei werden zusätzlich Entitäten aus den Kriterientexten erkannt und in die AST-Struktur integriert. Der direkte Prozess wird in Abschnitt 5.5 erläutert.

Wie die Zusammenfassung des State-of-the-Art zur Strukturierung von Eignungskriterien in Abschnitt 4.4 zeigt, wurden bereits Ansätze zur logischen Strukturierung entwickelt. Es wurde eine logische Strukturierung von Eignungskriterien als Abstract Syntax Tree im LeafAI-System durch eine Kombination aus verschiedenen Methoden wie BERT- und T5-Modellen und einem AST-Parser vorgenommen (N. J. Dobbins et al., 2023). Zusätzlich wurden Eignungskriterien mithilfe von GPT-4 mit Few-Shot Methoden in ein JSON-Format mit logischer Disjunktive Normalform überführt (Wong et al., 2023). Diese Arbeit kombiniert beide Ansätze und entwickelt eine Lösung, die Eignungskriterien sowohl als Abstract Syntax Tree repräsentiert als auch im JSON-Format darstellt. Dies verbessert die Lesbarkeit und ermöglicht es, den Ursprungstext zu erhalten. Eine Strukturierung kann so mithilfe von einzelnen LLMs automatisiert werden. Dies bietet zwei zentrale Vorteile. Erstens entfällt die Notwendigkeit, ein komplexes Framework aus verschiedenen Modellen zu entwickeln und zweitens ist das öffentlich verfügbare Modell für alle Forschenden zugänglich.

5.2. Logische Operatoren: AND, OR, NOT

Eignungskriterien weisen eine komplexe, hierarchische Logik auf, die durch die Operatoren AND, OR und NOT repräsentiert wird und eine oder mehrere Entitäten in den Kriterientexten verknüpft. Abbildung 1.1 zeigt einen Ausschnitt der Einschlusskriterien einer Onkologiestudie, die mit erkannten Entitäten und logischen Operatoren angereichert wurden. Extrahierte Entitäten wie *neuroendocrine tumor* werden dabei hierarchisch mit weiteren Entitäten verknüpft. Die extrahierte Struktur kann anschließend in Normalformen wie der Konjunktive Normalform (KNF) oder der Disjunktive Normalform (DNF) überführt werden (Wong et al., 2023). Nur durch die Extraktion der logischen Beziehungen zwischen Entitäten aus Eignungskriterien können diese in logischen Suchanfragen nach klinischen Studien integriert werden. Zudem ist ein korrektes Patient-Studie Matching nur unter Einbeziehung der logischen Verknüpfungen von Entitäten wie Krankheiten in den Eignungskriterien möglich (N. J. Dobbins et al., 2023).

Ross et al. (2010) zeigten, dass grammatikalische Elemente wie Klammern “()”, Kommata (“,”) und Schrägstriche (“/”) häufig als implizite Repräsentationen für logische AND- und OR-Verknüpfungen dienen. NOT-Relationen werden durch Signalwörter wie “not” oder “without” repräsentiert. Zudem wurden weitere Regeln für das Erkennen von logischen Operatoren ausgearbeitet.

AND Wenn Wörter wie “with” ohne Informationsverlust in ein AND umgewandelt werden konnten. Der Satz “*Cirrhosis with a history of ascites*” wurde beispielsweise umgeschrieben zu “*Cirrhosis **AND** a history of ascites*”. “*No history of CHF and CAD*” konnte logisch zu “*No history of CHF **AND** No history of CAD*” verarbeitet werden.

OR Unvollständige Listen wurden als implizite OR-Verknüpfungen ihrer Elemente interpretiert. “*History of cardiac disease (e.g., MI, CHF)*” wurde ergänzt zu “*History of cardiac disease **OR** MI **OR** CHF*”.

NOT Phrasen mit den Verknüpfungen “without”, “except”, “unless” und “other than” wurden als NOT interpretiert.

Diese Regeln können verwendet werden, um Trainings- und Evaluationskorpora zu erstellen, wobei die logischen Operatoren in die Kriterientexte vor Signalwörtern wie “and”, “or” oder typischen Satzzeichen wie Kommata eingefügt werden. Wenn Eignungskriterien aus öffentlichen Korporas, wie in Tabelle 4.1, im BRAT-Format vorliegen und die logischen Relationen zwischen den Entitäten annotiert sind, können die logischen Operatoren durch Datenverarbeitung in die Kriterientexte eingefügt werden. Abbildung 5.1 zeigt exemplarisch die Ein- und Ausschlusskriterien der Studie NCT03860402 mit eingefügten AND-, OR- und NOT-Operatoren.

NCT03860402 (mit AND, OR, NOT)

Inclusion Criteria:

- Among the pregnant women **[AND]** who were administered cesarean section in this study, those who agreed to participate in this study.

Exclusion Criteria:

- Fetal distress requiring fast delivery. **[AND]**
- Patients with hypersensitivity, allergic response, **[OR]** resistance to the drugs used in this study (fentanyl, **[OR]** ropivacaine, **[OR]** lidocaine). **[AND]**
- Patients who it is **[AND] [NOT]** not possible for regional anesthesia. **[AND]**
- Patients who refused to participate in this study. **[AND]**
- Patients who cannot read or understand the agreement. **[AND]**
- Patients whose weight is less than 50kg **[OR]** or exceeds 100kg.

Abbildung 5.1.: Eignungskriterien der Studie NCT03860402 mit zusätzlichen AND-Operatoren.

Die Operatoren AND und OR sind binäre Operatoren, die jeweils zwei getrennte Entitäten oder Satzteile miteinander verknüpfen. Der NOT-Operator bezieht sich auf eine einzelne Entität. Um Folgesätze zu verbinden, werden AND-Operatoren zusätzlich am Satzende eingefügt. Gleiches gilt für den NOT-Operator, vor dem ebenfalls ein AND gesetzt wird. Um die hierarchische Logik abzubilden, können die logisch angereicherten Textdaten in eine Abstract Syntax Tree-Struktur in JSON-Dateien überführt werden. Im Folgenden wird aufgezeigt, wie die AST-Struktur mithilfe eines AST-Parsers aus den Textdaten mit eingefügten Operatoren erstellt werden kann.

5.2.1. Abstract Syntax Tree (AST)

Eignungskriterien weisen häufig eine verschachtelte und komplexe logische Struktur auf, wie in Abbildung 1.1 veranschaulicht wird (Wong et al., 2023). Um diese Strukturen in einem von modernen Large Language Models verarbeitbaren Format zu repräsentieren, wurde ein spezifisches JSON-Format entwickelt, das gleichermaßen auf Ein- und Ausschlusskriterien angewendet wird. Dieses Format ermöglicht die Darstellung der Kriterientexte als Abstract Syntax Tree (AST), indem die logischen Operatoren extrahiert und die Satzteile der Kriterien rekursiv über die Operatoren AND, OR und NOT beliebig tief verschachtelt werden. Die Knoten des AST repräsentieren dabei die logischen Operatoren, während die Textsegmente als Blätter dienen. Die Eignungskriterien der Studie NCT03860402 lassen sich über einen Parsingprozess mit den logischen Operatoren als Knoten und die verbundenen Satzteilen als Blätter des Abstract Syntax Tree darstellen. Die als AST prozessierten Einschlusskriterien werden Abbildung 5.2 gezeigt.

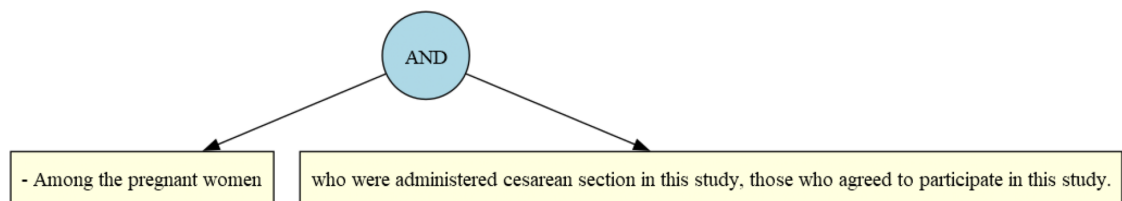


Abbildung 5.2.: Einschlusskriterien der Studie NCT03860402 als AST.

Dabei bilden die Satzteile die Blätter des Baumes, welche über einen AND-Knoten verbunden sind. Der Erzeugte AST der zugehörigen Ausschlusskriterien ist in Abbildung 5.3 dargestellt.

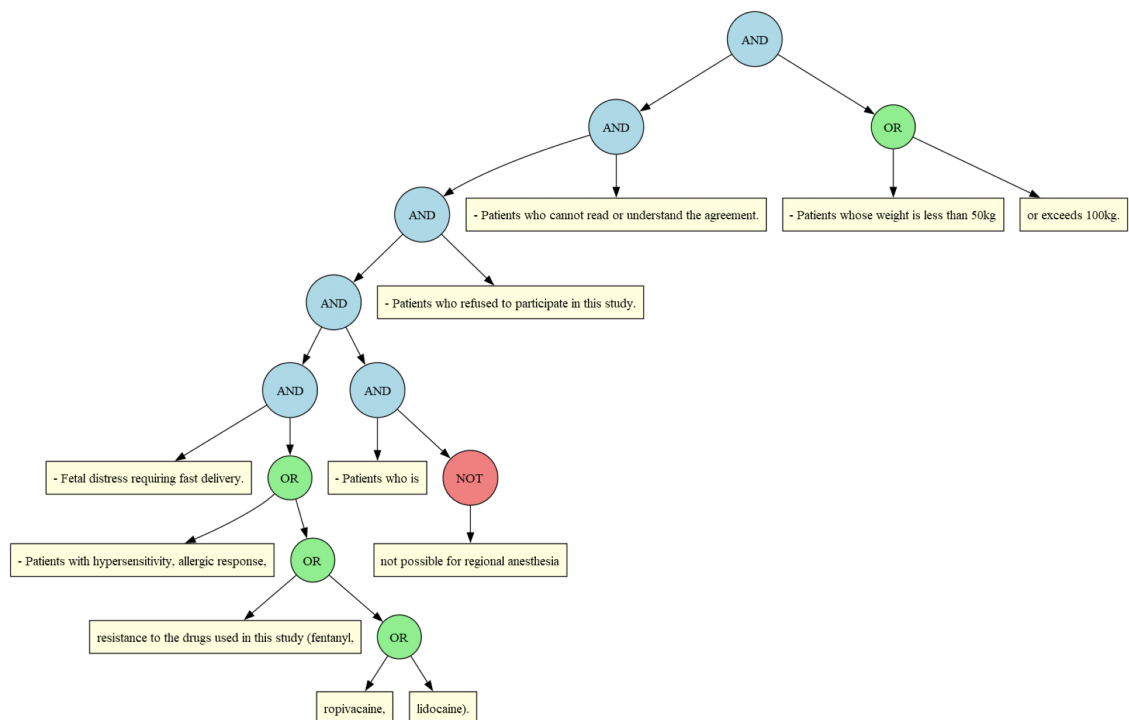


Abbildung 5.3.: Ausschlusskriterien der Studie NCT03860402 als AST.

Ein Vorteil dieser Struktur ist, dass die Ursprungstexte so vollständig übernommen werden können. Zusätzlich kann die entwickelte AST-Struktur flexibel erweitert werden. Sie ermöglicht die Integration zusätzlicher Metadaten aus dem Originaltext, wie zum Beispiel extrahierte Entitäten, die als weitere Blätter des Abstract Syntax Tree fungieren. Die um Entitäten erweiterte AST-Struktur wird anhand der prozessierten Eignungskriterien der Studie NCT03860402 veranschaulicht. Die Einschlusskriterien als AST inklusive Entitäten sind in Abbildung 5.4 gezeigt. Die entsprechenden Ausschlusskriterien werden in Abbildung 5.5 dargestellt. Die Entitäten werden zusammen mit

Die Texte der Eingangskriterien werden über die logischen Operatoren AND, OR und NOT verbunden. Die extrahierten Entitäten werden mit ihren Entitätstypen als Blätter unter dem Ursprungstext dargestellt. Die AST-Strukturierung der Ausschlusskriterien in Abbildung 5.5 ermöglicht eine übersichtliche und lesbare Darstellung komplex verschachtelter Informationen. Diese übersichtliche Struktur ermöglicht Fachkräften eine einfache Überprüfung der Entitätsextraktion und logischen Verknüpfungen. Logische Operatoren sowie fehlerhafte Entitäten lassen sich bei Bedarf leicht anpassen oder löschen. Die manuelle Nachbearbeitung kann über ein zu implementierendes Web-Framework erfolgen, das einen Human-in-the-Loop Ansatz umsetzt.

Da die ursprünglichen Textteile erhalten bleiben, können Metadaten, die aus nachfolgenden Schritten hinzugefügt wurden, wie erkannte Entitäten innerhalb der Texte, einfach überprüft werden. Diese Abstract Syntax Tree-Struktur wird als hierarchisches JSON-Format repräsentiert, welches sich flexibel in weitere Formate übertragen lässt. Ein weiterer Vorteil dieser Struktur ist ihre Interoperabilität, da sie mithilfe geeigneter Parser in standardisierte medizinische Formate wie FHIR oder OMOP-CDM transformiert werden kann. Im Folgenden wird das JSON-Format mit AST-Struktur detailliert beschrieben.

5.2.2. JSON-Format

Die Eignungskriterientexte mit eingefügten Operatoren wie in Abbildung 5.1 exemplarisch gezeigt, können über einen Parser weiter in ein JSON-Format mit AST-Struktur verarbeitet werden. Dafür ist es notwendig, sowohl die verschachtelte Logik innerhalb der Kriterientexte einzubeziehen als auch die übergeordnete Logik zu berücksichtigen, welche Eignungskriteriensätze über AND-Operatoren verbindet. Der Quellcode zur Erzeugung solcher AST-Strukturen im JSON-Format ist im Folgenden dargestellt. Der Code implementiert einen Parser zur Strukturierung von Eignungskriterien mittels markierter komplexer logischer Ausdrücke innerhalb der Kriterien. Ein- und Ausschlusskriterien werden so zunächst separat verarbeitet.

```

1 class Node:
2     """ Represents a node in the logic tree.
3     Attributes:
4         operator: The logical operator ('AND', 'OR', 'NOT', NONE for leafs)
5         criteria: The criteria text for leaf nodes
6         left: Left child node
7         right: Right child node
8     """
9     def __init__(self, operator: Optional[str] = None,
10                  criteria: Optional[str] = None):
11         self.operator: Optional[str] = operator
12         self.criteria: Optional[str] = criteria
13         self.left: Optional[Node] = None
14         self.right: Optional[Node] = None
15
16     def to_dict(self) -> Dict[str, Any]:
17         """ Converts the node and its children to a dictionary.
18         Returns:
19             Dict[str, Any]: A dictionary representing the node structure
20         """
21         if self.operator:
22             children: Dict[str, Any] = {}
23             if self.left:
24                 children["left"] = self.left.to_dict()
25             if self.right:
26                 children["right"] = self.right.to_dict()
27             return {self.operator: children}
28         else:
29             return {"raw_text": self.criteria if self.criteria is not None
30                     else "empty set"}

```

Listing 5.1: Node-Klasse des logischen AST-Parsers mit der Funktion to_dict() welche den erzeugten AST als Dictionary ausgibt.

Die *Node*-Klasse repräsentiert einen Knoten im logischen Baum und kann entweder einen Operator (AND, OR, NOT) oder ein Kriterium als Text enthalten. Die Funktion `parse_text()` zerlegt einen Textstring mit logischen Kriterien rekursiv in eine Baumstruktur, während `build_tree()` die Teilbäume der einzelnen Kriterien zu einem Gesamtbaum kombiniert. Der AST-Parser besteht aus einer *Node*-Klasse und zwei Hauptfunktionen `parse_text()` und `build_tree()`. Die *Node*-Klasse repräsentiert einen Knoten eines Abstract Syntax Tree, wobei jeder Knoten entweder einen logischen Operator (AND, OR, NOT) oder ein spezifisches Kriterium als Blatt enthält. Die Methode `to_dict()` konvertiert die Baumstruktur rekursiv in eine Dictionary-Darstellung, die sich für die Serialisierung in JSON eignet. In der JSON-Repräsentation werden die Verbindungen zwischen Knoten durch *left* und *right* Eigenschaften implementiert. Binäre Operatoren (AND, OR) haben beide Verbindungen, während der NOT-Operator nur eine *left*-Verbindung besitzt. Der ursprüngliche Text vor einem Operator wird als *raw_text* gespeichert und bildet ein Blatt im Baum.

Die Funktion `parse_text()` im Anhang in Code E.2 nimmt einen einzelnen Textstring als Eingabe, der logische Kriterien mit markierten Operatoren wie *[AND]*, *[OR]* und *[NOT]* enthält. Sie verwendet reguläre Ausdrücke, um diese Operatoren zu identifizieren, und zerlegt den Text rekursiv in eine Baumstruktur. Dabei werden die Operatoren als innere Knoten und die Kriterien als Blätter des Baums dargestellt. Wenn ein Operator im Text gefunden wird, teilt die Funktion den Text an der Stelle des ersten Operators in zwei Teile, den Text vor dem Operator und den Text nach dem Operator. Für AND- und OR-Operatoren werden beide Teile als separate Teilbäume behandelt und als linke und rechte Kindknoten dem Operator-knoten hinzugefügt. Beim NOT-Operator wird der nachfolgende Text als linker Kindknoten des NOT-Knotens hinzugefügt. Falls vor dem NOT-Operator noch Text vorhanden ist, wird dieser nicht ignoriert. Stattdessen wird ein AND-Knoten erstellt, der den Text vor dem NOT-Operator und den NOT-Ausdruck verbindet. Falls im Text keine logischen Operatoren gefunden werden, erstellt die Funktion einen Blattknoten, der das gesamte Kriterium enthält. Dies ermöglicht es, auch einfache Kriterien ohne logische Verknüpfungen korrekt in die Baumstruktur einzubinden. Die Funktion `build_tree()`, im Anhang in Quellcode E.1 gezeigt, initiiert die Verarbeitung mehrerer Kriterien. Die Funktion akzeptiert ein Dictionary als Eingabe, dessen Werte Eignungskriterientexte mit eingefügten logischen Relationen sind. Jedes dieser Kriterien wird einzeln mit der Funktion `parse_text()` verarbeitet, wodurch separate Teilbäume entstehen. Die resultierenden Teilbäume werden anschließend zu einem Gesamtbaum kombiniert, wobei sie durch AND-Operatoren verbunden werden. Dadurch wird die übergeordnete Logik, die die einzelnen Kriteriensätze miteinander verbindet, korrekt abgebildet. Die Funktion berücksichtigt auch Sonderfälle. Wenn das Eingabe-Dictionary leer ist oder keine Kriterien enthält, wird ein leerer-Knoten mit dem Kriterium "*empty set*" erstellt.

AST-Strukturierung im JSON-Format

Um die Anwendung der Parsingmethode bei Eignungskriterien zu demonstrieren, zeigt Abbildung 5.1 die Ein- und Ausschlusskriterien der Studie NCT03860402 mit eingefügten Operatoren. Die verarbeiteten Eignungskriterien können mithilfe des AST-Parsers in verschachtelte JSON-Formate mit Abstract Syntax Tree-Struktur überführt werden. Die Eingabe des Parsers ist ein Dictionary, das die Eignungskriteriensätze zusammen mit den eingefügten Operator-Tags enthält. In Code 5.2 werden die Einschlusskriterien als Dictionary dargestellt und über die Parser-Funktionen aus Unterabschnitt 5.2.2 verarbeitet.

```
1 import json
2
3 data = {"IC1": "- Among the pregnant women [AND] who were administered ..."}
4 tree = build_tree(data)
5 tree_dict = tree.to_dict()
6 tree_json = json.dumps(tree_dict, indent=2)
```

Listing 5.2: Nutzen des AST-Parsers für die Einschlusskriterien der Studie NCT03860402.

Die folgenden Darstellungen demonstrieren das Ergebnis des Parsingprozesses für den in Abbildung 5.1 gezeigten, prozessierten Kriterientext im JSON-Format. Für die Einschlusskriterien wird im Folgenden die vollständige Parsingausgabe gezeigt.

Strukturierte Einschlusskriterien der Studie NCT03860402 (.json):

```

1 {
2   "AND": {
3     "left": {
4       "raw_text": "- Among the pregnant women"
5     },
6     "right": {
7       "raw_text": "who were administered ..."
8     }
9   }
10 }

```

Ein Auszug aus den verarbeiteten Ausschlusskriterien der Studie ist im Folgenden dargestellt.

Strukturierte Ausschlusskriterien der Studie NCT03860402 (.json):

```

1 {
2   "AND": {
3     "left": {
4       "AND": {
5         "left": {
6           "AND": {
7             "left": {...
8               "left": {
9                 "raw_text": "- Fetal distress requiring
10                   fast delivery."
11               },
12               "right": {
13                 "OR": {
14                   "left": {
15                     "raw_text": "- Patients with ..."
16                   }, ...
17                 "right": {
18                   "AND": {
19                     "left": {
20                       "raw_text": "- Patients who is"
21                     },
22                     "right": {
23                       "NOT": {
24                         "left": {
25                           "raw_text": "not possible for ..."
26                         } ...
27                     "right": {
28                       "OR": { ...
29                     }
30                 }
31             }
32           }
33         }
34       }
35     }
36   }
37 }

```

Die AST-Struktur der verarbeiteten Kriterien bietet eine flexible Möglichkeit zur Erweiterung. An die Blattknoten können beliebige zusätzliche *Key:Value* Paare angehängt werden. Diese Eigenschaft ist besonders nützlich für das Hinzufügen von Entitäten, die aus den Ursprungstexten extrahiert wurden.

5.2.3. Entitäten

Zur Evaluation für eine Entitätsextraktion über Large Language Models kann das bestehende JSON-Format um annotierte Entitäten aus Korporas im BRAT-Format erweitert werden. Diese Erweiterung integriert annotierte Entitäten aus definierten Entitätstypen als Blätter innerhalb der *left* und *right* Verbindungen. Die Entitätstypen werden als *Keys* in die Struktur integriert, während die extrahierten Entitätsinstanzen als *Values* in einem Array implementiert werden. Dabei werden die Entitäten in der Form *Entitätstyp:[Entitäten]* an die bestehenden Blattknoten unter dem extrahierten *raw_text* angefügt. Die Funktion `add_entities_to_tree()`, dargestellt im Anhang in Codeauszug E.3, erweitert einen gegebenen Abstract Syntax Tree (*node*) in Form eines verschachtelten Dictionaries um Entitätsinformationen aus annotierten Korporas mit Eignungskriterien. Sie verwendet dafür eine Liste ausgewählter Entitätstypen (*selected_types*) und eine Liste konkreter Entitäten (*entities*) welche über ein Preprocessing der Annotationsdateien zu den Kriteriendateien erstellt wurde. Die Funktion traversiert den Baum rekursiv und untersucht die Blattknoten, die durch den Key *raw_text* gekennzeichnet sind. Für jeden Blattknoten wird geprüft, ob Textfragmente aus der Entitätenliste im *raw_text* vorkommen und zu den ausgewählten Entitätstypen gehören. Bei einer Übereinstimmung fügt die Funktion die entsprechende Entität samt Typ als neues Key-Value Paar unter dem *raw_text* Eintrag hinzu. Die Funktion gibt nichts zurück, sondern modifiziert den übergebenen Baum direkt.

Durch diese Methodik lassen sich die Eignungskriterien aus Abbildung 5.1, die mithilfe des AST-Parsers aus Unterabschnitt 5.2.2 in JSON-Dateien strukturiert wurden, mit zusätzlichen Entitäten anreichern, um einen Goldstandard zu erzeugen. Diese Entitäten werden dabei unter den Ursprungstexten an den Blättern des Baums angehängt. Dies ermöglicht insbesondere einen Vergleich der Entitätsextraktion auf Korporas welche sowohl logische Operatoren als auch Entitäten im BRAT-Format annotiert haben. Die Modellausgabe von LLMs erfolgt analog dazu und kann mit einem erzeugten Goldstandard verglichen werden. Die JSON-Dateien des Goldstandards mit eingefügten Entitäten als AST sind folgendermaßen aufgebaut.

Strukturierte Einschlusskriterien der Studie NCT03860402 mit Entitäten (.json):

```

1 {
2   "AND": {
3     "left": {
4       "raw_text": "- Among the pregnant women",
5       "Condition": ["pregnant"],
6       "LifeStageAndGender": ["women"]
7     },
8     "right": {
9       "raw_text": "who were administered cesarean section in this
10                  study, those who agreed to participate in this
11                  study.",
12       "Procedure": ["cesarean section"],
13       "Study": ["study"],
14       "EqComparison": ["administered"],
15       "EqTemporalPeriod": ["administered"]
16     }
17   }
18 }
```

Eine verkürzte Repräsentation der strukturierten Ausschlusskriterien der Studie, einschließlich der zugehörigen Entitäten, ist im Anhang unter Abschnitt B.1 zu finden. Bei der Annotation der Einschlusskriterien der Studie NCT03860402 ist zu beachten, dass die eigentliche Relation über den AND-Operator genau zwei Entitäten verbindet. In diesem Beispiel wurde die Relation folgendermaßen annotiert: *women AND cesarean section*. Die Methodik der Extraktion der logischen Operatoren im JSON-Format als AST verbindet die Relation wie folgt: *(pregnant,women) AND (cesarean section, study, administered)*.

5.2.4. Logische Funktion

Eine erweiterte Parsingmethode (JSON-Parser) ermöglicht die logische Verknüpfung der Einschlusskriterien mit den Ausschlusskriterien durch die Verwendung eines AND-Operators als Wurzelknoten des Abstract Syntax Tree. Um die inhärente logische Struktur adäquat abzubilden, werden die Ausschlusskriterien auf der obersten Hierarchieebene zusätzlich mit einem NOT-Operator versehen. Im Anhang in Quellcode E.5 setzt die Funktion *negate_exclusion_criteria()* ein NOT als Wurzelknoten für Ausschlusskriterien in AST-Struktur und die Funktion *merge_criteria()* verbindet Ein- und Ausschlusskriterien über ein AND als gemeinsamen Wurzelknoten.

Abbildung 5.6 veranschaulicht, wie die verarbeiteten Eignungskriterien der Studie NCT03860402 mithilfe der erweiterten Parsermethode zusammen mit den erkannten Entitäten in einer AST-Struktur kombiniert wurden. Eine komprimierte Darstellung der verarbeiteten Eignungskriterien im JSON-Format findet sich im Anhang in Abschnitt B.2.

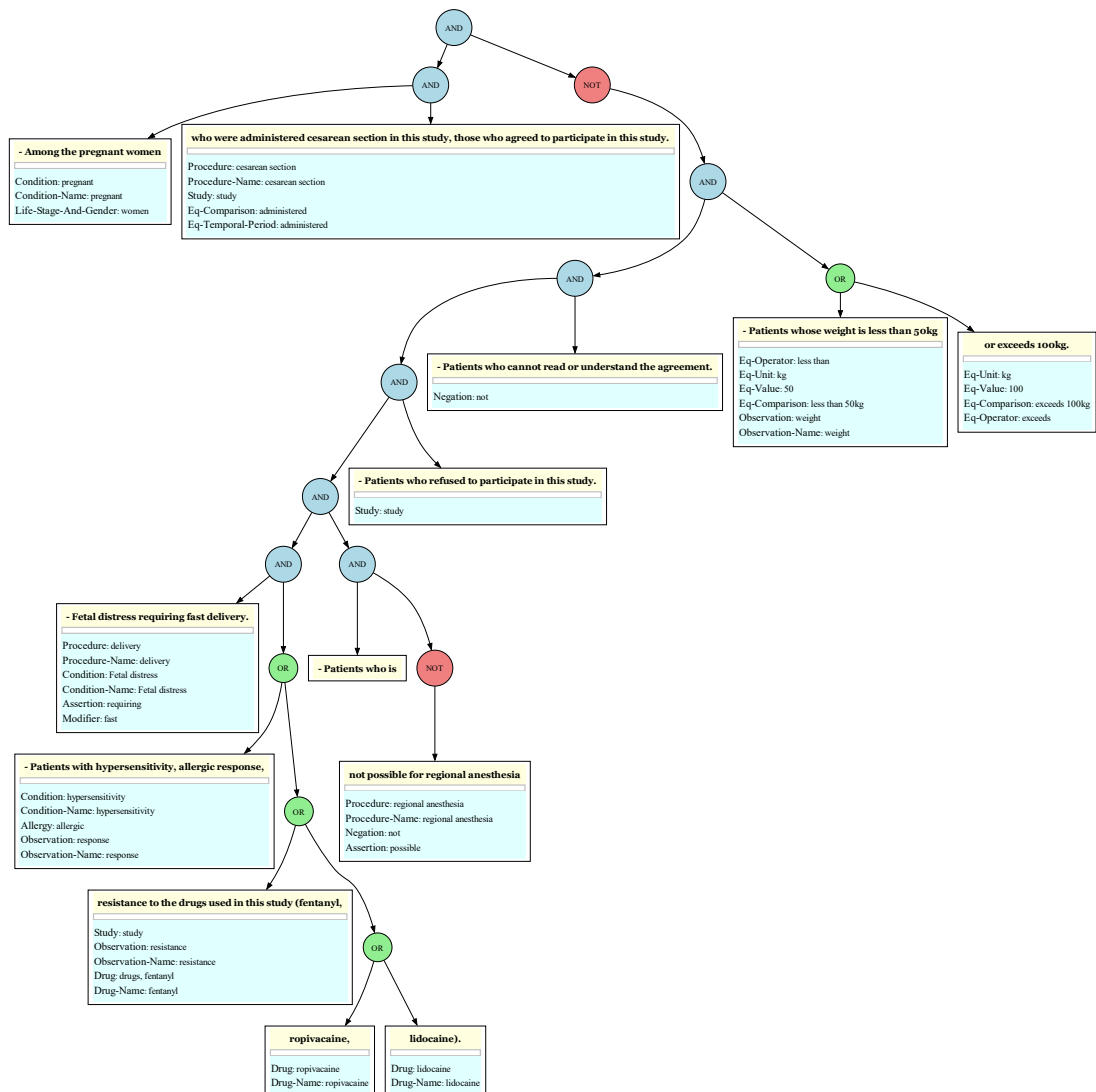


Abbildung 5.6.: Verknüpfte Ein- und Ausschlusskriterien der Studie NCT03860402 als AST.

Die logischen Operatoren als Knoten verbinden zunächst den Ursprungstext der Kriterien als Blätter. Erkannte Entitäten aus diesen verbundenen Textsequenzen werden als weitere Blätter unter den Texten visualisiert. Die Operatoren verbinden somit auch die Gruppen der erkannten Entitäten in den Blättern. Die AST-Struktur ermöglicht es, über logische Formeln, die Entitäten als Gruppen über eine verschachtelte Logik zu verknüpfen. Im Folgenden wird eine logische Formel für die erkannten Entitätsgruppen aus Abbildung 5.6 dargestellt.

$$I : (\text{pregnant, women}) \wedge (\text{cesarean section, study, administered}) \wedge \\ \neg(((((((\text{Fetal distress, requiring, fast delivery}) \wedge ((\text{hypersensitivity, allergic response}) \vee \\ (\text{study, resistance, drugs, (fentanyl} \vee \text{ropivacaine} \vee \text{lidocaine)})))))) \wedge \\ ((\text{Patients who is}) \wedge \neg(\text{not, possible, regional anesthesia}))) \wedge (\text{refused to participate})) \wedge \\ (\text{cannot read or understand the agreement})) \wedge ((\text{weight less than 50kg}) \vee \\ (\text{exceeds 100kg}))))$$

Die oben dargestellte logische Formel kann durch das Ersetzen von Entitätsgruppen durch Symbole übersichtlicher gestaltet werden. Es ist jedoch zu beachten, dass diese Darstellung keine formal korrekte logische Funktion ist, sondern lediglich eine symbolische Vereinfachung der Bedingungen. Die logische Struktur ist so nicht vollständig maschinell interpretierbar, da teilweise mehr als zwei Entitäten durch AND- und OR-Operatoren verknüpft sind und NOT-Operatoren direkt auf Satzteile angewendet wurden. Dies führt zu einer Repräsentation, die zwar als Annäherung an eine logische Notation dient, jedoch noch formale Ungenauigkeiten aufweist und den Anforderungen einer systematischen, maschinellen Verarbeitung nicht genügt.

$$I : (P \wedge C) \wedge \neg((((((F \wedge (H \vee D)) \wedge (PA \wedge \neg A)) \wedge R) \wedge U) \wedge (W_{<} \vee W_{>})))$$

mit:

P : pregnant, women

F : Fetal distress, requiring, fast delivery

D : study, resistance, drugs, (fentanyl $\vee$ ropivacaine $\vee$ lidocaine)

PA : Patients who is

R : refused to participate

$W_{<}$: weight less than 50kg

C : cesarean section, study, administered

H : hypersensitivity, allergic response

A : not, possible, regional anesthesia

U : cannot read or understand the agreement

$W_{>}$: exceeds 100kg

Zur weiteren Vereinfachung kann die mit Symbolen ersetzte Funktion in standardisierte Formate wie Konjunktive Normalform (KNF) oder Disjunktive Normalform (DNF) übertragen werden.

5.3. Modelle

Zur Strukturierung von Eignungskriterien klinischer Studien wurden aktuelle Open-Source Large Language Models ausgewählt und mit Closed-Source LLMs verglichen. Der Zugriff der Open-Source LLMs erfolgte über die `transformer`¹-Bibliothek von Huggingface. Diese bietet die Möglichkeit zur Nutzung und zum Fine-Tuning moderner Pre-trained Language Models. Die verwendeten Open-Source Modelle sind in Tabelle 5.1 nach Veröffentlichungsdatum aufgelistet. Die Tabelle enthält zudem den Zugang der Modelle über die entsprechenden Modellbezeichnungen von Huggingface. Zudem werden für einen Vergleich verwendete proprietäre Modelle gezeigt. Die GPT-Modelle wurden über die OpenAI (2023)-API genutzt, während das Claude-Modell über die Anthropic (2021)-API eingesetzt wurde. Diese Arbeit untersucht den Einsatz von Open-Source LLMs zur Strukturierung von Eignungskriterien. Der Fokus liegt dabei auf den zu Beginn dieser Arbeit veröffentlichten Llama3-Modellvarianten mit 8B- und 70B-Parametern, die mittels Prompting- und Fine-Tuning Techniken analysiert werden. Die Auswahl des Llama3-Modells mit 70B-Parametern basiert auf verbesserten Leistungen in diversen Benchmarks im Vergleich zu größeren Open-Source Modellen wie Mixtral 8×22B oder dem Vorgängermodell Llama2-70B. Das Llama3-8B Modell weist in der Regel eine geringere Leistung als die 70B-Variante auf, benötigt jedoch für Inferenz und Fine-Tuning signifikant weniger Ressourcen (Dubey et al., 2024).

Modell	Veröffentlichung	Zugang
<i>Open-Source</i>		
Llama2-70B	Juli 2023	meta-llama/Llama-2-70b-chat-hf
Llama3-8B	April 2024	meta-llama/Meta-Llama3-8B-Instruct
Llama3-70B	April 2024	meta-llama/Meta-Llama-3-70B-Instruct
Gemma2-27B	Juli 2024	google/gemma-2-27b-it
Qwen2-72B	Juli 2024	Qwen/Qwen2-72B-Instruct
<i>Closed-Source</i>		
GPT-4o	März 2023	gpt-4o
GPT-4o-mini	Juli 2024	gpt-4o-mini
Claude-3.5-Sonnet	Juli 2024	claude-3-5-sonnet-20240620

Tabelle 5.1.: Übersicht der ausgewählten Large Language Models zur Strukturierung von Eignungskriterien nach Veröffentlichungsdatum und Zugang.

Für einen umfassenden Vergleich wurde zusätzlich das mittelgroße Gemma2-27B Modell sowie das Qwen2-72B Modell in die Analyse einbezogen. Gemäß dem Huggingface-Leaderbord (2024) zeigt das Qwen2-Modell in einigen Benchmarks eine höhere Leistung als das Llama3-Modell und zeichnet sich zudem durch eine erweiterte Kontextlänge aus. Die proprietäre LLMs GPT-4o und Claude-3.5-Sonnet wurden ausgewählt, da sie derzeit die leistungsstärksten Modelle in ihrer Modellreihe darstellen (Anthropic, 2024). Der Zugriff auf diese Modelle erfolgt kostenpflichtig über die entsprechenden APIs der Anbieter. Zusätzlich wurde das GPT-4o-mini Modell in die Auswahl aufgenommen, da es bei vergleichbarer Leistung geringere Kosten für die Inferenz aufweist.

Für beide methodischen Ansätze, der indirekten- und der direkten Strukturierung, standen die in Unterabschnitt 5.3.1 aufgeführten finanziellen- und Rechenressourcen zur Verfügung. Die Parameterkonfiguration der Large Language Models wurde ebenfalls für beide Methoden und alle Modelle gemäß Unterabschnitt 5.3.2 vorgenommen.

¹<https://huggingface.co/docs/transformers/index>

5.3.1. Ressourcen

Das Fine-Tuning von Open-Source LLMs wurde mit Hilfe des Huggingface SFTTrainer (2024) vorgenommen. Ein Parameter-Efficient Fine-Tuning (PEFT) wurde über Low-Rank Adaptation (LoRA) mit der unsloth (2024)-Bibliothek implementiert. Für die Nutzung von Open- und Closed-Source LLMs wurde dabei auf folgende Ressourcen zugegriffen.

- **Cluster-Ressourcen:** Die Inferenz- und Fine-Tuning Prozesse von Open-Source LLMs wurden auf dem High Performance Computing (HPC) Cluster² der Hochschule Bonn-Rhein-Sieg (H-BRS) durchgeführt. Diese Infrastruktur bietet sechs Rechenknoten, von denen jeder mit vier NVIDIA A100 SXM4 GPUs (80GB) ausgestattet ist. Somit stehen sechs Knoten mit je 320GB VRAM zur Verfügung.
- **Finanzierung von OpenAI:** Über das "OpenAI Researcher Access Program"³ konnte ein Betrag von \$600 für die Nutzung von OpenAI Modellen wie GPT-4o über die OpenAI-API erworben werden.

5.3.2. Parameterkonfiguration

Für die Inferenz der ausgewählten LLMs wurden Open- und Closed-Source Modelle gemäß den in Abschnitt 3.6 beschriebenen Modellparametern für alle Experimente wie folgt konfiguriert.

Die Ausgabelänge der Large Language Models wurde auf `max_new_tokens = 2048` festgelegt. Diese Länge erwies sich als ausreichend, um die Text- und JSON-Ausgaben ohne Abschneidung auszugeben. Die Modellparameter Temperatur und Top-p aus Abschnitt 3.6 wurden für alle Open- und Closed-Source Modelle bei Prompting-Methoden wie Zero-Shot und Few-Shot auf `temperature = 0.5` und `top_p = 0.95` gesetzt. Die niedrigere Temperatur von 0.5 führt zu konsistenten und weniger zufälligen Ausgaben. Für Llama3-8B und -70B Modelle, welche ein aufgabenspezifisches Fine-Tuning erhalten haben, wurde bei der Inferenz der Standardwert von `temperature = 1.0` und `top_p = 1.0` beibehalten. Eine Variation des Temperatur-Parameters bei Llama3-70B unter Verwendung von ICL- und Fine-Tuning Methoden, in Unterabschnitt 6.1.6 beschrieben, führte zu keiner signifikanten Verbesserung der Modellergebnisse. Deshalb wurde für die weiteren Experimente der Standardwert des Parameters beibehalten. Dasselbe gilt für den `top_p`-Wert, der in initialen Versuchen mit Parameter-Variationen die Standardeinstellung nicht verbessern konnte.

²<https://wr0.wr.inf.h-brs.de/wr/index.html>

³https://openai.smapply.org/prog/openai_researcher_access_program/

5.4. Indirekte Strukturierung

Ein methodischer Ansatz zur Strukturierung von Eignungskriterien besteht darin, die logischen Operatoren AND, OR und NOT direkt in die unstrukturierten Kriterientexte einzufügen. Hierzu werden Large Language Models eingesetzt, die diese Operatoren in die Originaltexte integrieren. Dieser Ansatz ermöglicht insbesondere die Anwendung von Zero-Shot Methoden, bei denen lediglich eine Anweisung zum Einfügen der Operatoren erstellt werden muss, um die LLMs zu nutzen. Liegen annotierte Eignungskriterien als Trainingsdaten vor, können zusätzlich Few-Shot und Fine-Tuning Methoden angewendet werden.

Indirekte Strukturierung über die Operatorenextraktion mit LLMs

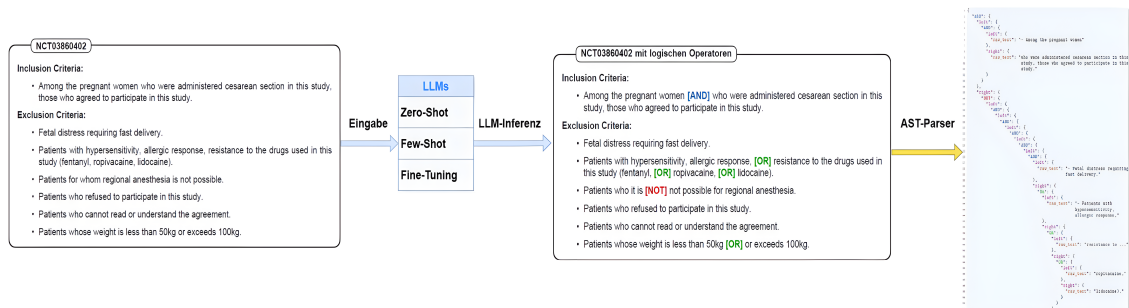


Abbildung 5.7.: Indirekte Strukturierung von Eignungskriterien über die Extraktion von logischen Operatoren mit LLMs mit Zero-Shot, Few-Shot und Fine-Tuning Methoden. Die erzeugten Daten können anschließend in eine AST-Struktur geparkt werden.

Die in Abbildung 5.7 dargestellte Methodik beschreibt den Prozess zur Strukturierung klinischer Eignungskriterien durch die Integration der logischen Operatoren in die Kriterientexte über LLM-Methoden. Die Ein- und Ausschlusskriterien, in Form von Textdateien, dienen als Eingabe für Open- und Closed-Source LLMs. Diese LLMs integrieren die logischen Operatoren AND, OR und NOT über verschiedene Prompt und Fine-Tuning Methoden in den Eingabetext. Durch das Einfügen der Operatoren in die Kriterientexte kann auf die Extraktion der verknüpften Entitäten, wie beispielsweise *Krankheiten*, verzichtet werden. Die von LLMs erzeugten Textdaten mit eingefügten Operatoren, können mithilfe des in Unterabschnitt 5.2.2 beschriebenen AST-Parsers in JSON-Dateien mit AST-Struktur umgewandelt werden. Der Quellcode E.4 im Anhang demonstriert die Anwendung des AST-Parsers für LLM-Ausgaben. Die AST-Struktur ermöglicht darüber hinaus verschiedene Darstellungen der verknüpften Inhalte. Diese können wie in Abbildung 5.6 gezeigt als Baumstruktur visualisiert werden. Der Code E.6 im Anhang stellt einen AST-Plotter dar, der JSON-Dateien mit AST-Struktur visualisiert. Um die logischen Operatoren mittels Zero-Shot Prompting zu extrahieren, wurden verschiedene Promptanweisungen entwickelt. Diese wurden iterativ in mehreren Versuchen auf einem kleinen Testdatensatz evaluiert, bis schließlich zwei Prompts erstellt wurden, die anschließend auf öffentlichen Korpora weiter evaluiert wurden. Die Leistung dieser Prompts wurde durch Zero-Shot Inferenz mit den Open-Source Modellen Llama3-8B und 70B an einem Testdatensatz evaluiert. Um das Zero-Shot Prompting mit Llama3-70B weiter zu optimieren, wurden verschiedene Methoden wie Zero-Shot Chain-of-Thought (CoT), Emotional Prompting und Role Prompting implementiert und getestet. Als Benchmark für die Leistungsfähigkeit der LLM-basierten Ansätze diente ein regelbasierter Parser, der die effektivste Promptvariante mit regulären Ausdrücken (RegEx) abbildet und als *Baseline* fungierte. Um die Leistungen zur Operatorenextraktion von unterschiedlich großen Open-Source und Closed-Source Modellen zu vergleichen, wurde die in Tabelle 5.1 aufgeführten Modelle in Zero-Shot und Few-Shot Szenarien getestet. Dabei wurden ausgewählte Beispiele aus dem Trainingsdatensatz als Kontext bereitgestellt, um die Anpassungsfähigkeit und Generalisierungsleistung der Modelle zu bewerten. Weiterhin wurde ein effizientes Fine-Tuning Verfahren auf die Modelle Llama3-8B und 70B angewandt, um die Extraktionsleistung der Open-Source Modelle weiter zu optimieren. Im Folgenden werden die verwendeten Methoden für die Operatorenextraktion mit LLMs beschrieben.

5.4.1. In-Context Learning

Durch den Einsatz von Prompt-Methoden soll eine ressourceneffiziente und einfache Strukturierung der Eignungskriterien mittels logischer Operatoren ermöglicht werden. Zu diesem Zweck wurden zunächst auf die Korpora angepasste Promptanweisungen entwickelt und evaluiert. Die Anweisungen basieren dabei auf der Annahme, dass das Einfügen von logischen Operatoren nach Signalwörtern wie “and”, “or” und “not” in den Eignungskriterientexten erfolgen kann, wie in Abschnitt 5.2 beschrieben. Anschließend wird geprüft, ob die ursprünglich entwickelten Prompts mithilfe aktueller Prompt-Techniken wie Zero-Shot CoT, Emotional Prompting oder Role Prompting optimiert werden können. Die Auswahl dieser Methoden erfolgte mit dem Ziel, bei minimalem Aufwand eine Leistungssteigerung zu erzielen. Hierfür wurden in den meisten Fällen lediglich ein oder mehrere vordefinierte Sätze an die ursprünglichen Prompts angefügt.

Zero-Shot

Für das Zero-Shot Prompting wurden den Modellen bei der Inferenz ausschließlich die entwickelten Prompts übergeben. Um logische Operatoren wie AND, OR und NOT zu integrieren, wurden zwei Prompts erstellt. Exemplarisch wird Prompt 2 vorgestellt, die in Abbildung 5.8 gezeigt ist und Instruktionen enthält, logische Operatoren als Tags vor Signalwörtern wie “and”, “or” oder Satzzeichen wie “,” zu platzieren.

Prompt 2: Logical Operator Insertion for Eligibility Criteria

Create an edited version of the following inclusion and exclusion criteria that incorporates the logical operators [AND], [OR], [NOT]. Apply these rules for inserting the logical operators:

- Always write an [OR] before an “or”, “and/or”, “and / or” in the text.
- Always write an [OR] after a comma “,” and after “/” in the text.
- Always write an [AND] before words like “with”, “who”, “in addition”, “plus”, “and”, “but”, “that”, “despite”, “having”.
- Always write a [NOT] before “no”, “not”, “none”, “don’t”, “free”, “prevent”, “Inability”, “lack”, “impossible”, “off”, “without”, “unable”, “naive”, “excluded”, “absence”.

General rules:

- The operators [OR], [AND], [NOT] should be placed before words such as or, and, not.
- An [AND], [OR], [NOT] must not come before a line segment (\n).
- Please carefully read through the given criteria text, then output only an edited version with the logical operators incorporated, enclosed in <EDITED_CRITERIA> tags.
- Do not include any other explanations or output.

Abbildung 5.8.: Prompt 2 mit Anweisungen zum Einfügen logischer Operatoren in Ein- und Ausschlusskriterien des LCT-Datensatzes.

Der folgende Quellcode 5.3 implementiert die Inferenz von Llama3-Modellen als Zero-Shot Ansatz, wobei dem Modell lediglich eine Promptanweisung übergeben wird, wie beispielsweise die Prompt aus Abbildung 5.8. Dabei wird das Python-Skript *llama3_zero_shot.py* im HPC-Cluster über eine Bash-Datei 5.4 als Pipeline ausgeführt. Der Llama3-Modelltyp kann als Parameter der Pipeline übergeben werden. Das im Bash-Skript 5.4 definierte Llama3-Modell wird über die *transformers*-Bibliothek geladen. Durch das Setzen von *device_map*=“auto” wird die automatische Verteilung des Modells auf alle verfügbaren GPUs innerhalb eines Rechenknotens ermöglicht. Für die Textgenerierung mit dem Zero-Shot Ansatz wird die vordefinierte Prompt als Zeichenkette aus einer Textdatei eingelesen. Nach der Erstellung einer Textgenerierungspipeline mit dem geladenen Modell und dem Tokenizer wird der Testdatensatz iterativ durchlaufen. Dabei werden für jeden Eintrag die aktuellen Eignungskriterien mit dem Prompt kombiniert und als Eingabe der Modellpipeline übergeben.


```

1 import argparse
2 import os
3 import transformers
4 from helper_functions import read_file, save_file
5
6 parser = argparse.ArgumentParser()
7 parser.add_argument("--model", type=str)
8 args = parser.parse_args()
9 model_name = args.model # load model
10 prompt = read_file(f"PATH_TO_PROMPT") # load prompt
11
12 pipeline = transformers.pipeline(
13     "text-generation",
14     model=model_name,
15     model_kwargs={"torch_dtype": torch.bfloat16},
16     device_map="auto",
17 )
18
19 for file_name in os.listdir("PATH_TO_TEST_FILES"):
20     test_file = read_file(study_path + file_name.split(".")[0])
21
22     messages = [{"role": "system", "content": prompt},
23                 {"role": "user", "content": prompt + test_file}]
24
25     prompt = pipeline.tokenizer.apply_chat_template(
26         messages,
27         tokenize=False,
28         add_generation_prompt=True
29     )
30
31     outputs = pipeline(
32         prompt,
33         max_new_tokens=2048,
34         eos_token_id=[...(terminators)...],
35         do_sample=True,
36         temperature=0.5,
37         top_p=0.95
38     )
39
40     output = outputs[0]["generated_text"][len(prompt):]
41     save_file(output, f"{OUTPUT_PATH}/{FILE_NAME}.txt")

```

Listing 5.3: Quellcode für das Einfügen von Operatoren mit Llama3-70B Zero-Shot.

Die Textgenerierung wird durch spezifische LLM-Parameter wie Temperatur- und Top-p Sampling gesteuert, die in Abschnitt 5.3.2 definiert sind. Die generierten Ausgaben werden in einer strukturierten Textdatei gespeichert. Jede Ausgabe besteht aus den ursprünglichen Eignungskriterien, die durch das Modell mit logischen Operatoren über [AND]-, [OR]- und [NOT]-Tags angereichert wurden. Um andere Llama3-Modelle für Zero-Shot Inferenzen zu nutzen, muss lediglich der Modellname geändert werden. Weitere Open-Source Modelle können mit geringfügigen Anpassungen im Code integriert werden. Zur Ausführung auf dem HPC-Cluster wird das Python-Skript über ein konfiguriertes Bash-Skript aufgerufen, welches die erforderlichen Umgebungseinstellungen vornimmt und die Ressourcenzuweisung konfiguriert.

Das Bash-Skript 5.4 ermöglicht die Ausführung eines Python-Skripts auf einem HPC-Knoten mit vier GPUs. Durch den Parameter `--partition=gpu4` wird der Job einer speziellen Partition zugewiesen, die mit vier NVIDIA A100 GPUs ausgestattet ist. Mit der Option `--mem=260G` wird ein Arbeitsspeicher von 260GB reserviert. Die Ausführungszeit wurde zunächst auf vier Stunden festgelegt (`--time=04:00:00`), kann jedoch bei Bedarf auf maximal 72 Stunden erweitert werden. Das modular strukturierte Python-Skript lässt sich flexibel an verschiedene LLM-Implementierungen anpassen.

```

1 #!/bin/bash
2 #SBATCH --partition=gpu4           # GPU partition
3 #SBATCH --nodes=1                 # Number of nodes
4 #SBATCH --mem=260G                # Memory per node
5 #SBATCH --gres=gpu:4              # Number of GPUs
6 #SBATCH --time=04:00:00           # Time limit hrs:min:sec
7
8 module load cuda
9 MODEL_NAME="meta-llama/Meta-Llama-3-8B-Instruct" # specify model
10
11 python llama3_zero_shot.py --model "$MODEL_NAME"

```

Listing 5.4: Bash-Skript zur Konfiguration von Python basierten GPU-Jobs für LLM-Experimente.

Es werden zusätzlich verschiedene erweiterte Prompt-Methoden implementiert, darunter Zero-Shot Chain-of-Thought (CoT), Emotional Prompting und Role Prompting. Diese Ansätze erweitern den Zero-Shot Ansatz durch das Hinzufügen spezifischer Textelemente, die in früheren Anwendungen Leistungssteigerungen zeigten (Schulhoff et al., 2024). Während diese erweiterten Prompts hauptsächlich für Fragebeantwortungsaufgaben evaluiert wurden, steht eine Untersuchung für NER- und RE-Aufgaben noch aus. Die Anpassung kann durch Modifikation der Prompteingabe in Zeile 5 im Code 5.3 erfolgen, wobei der restliche Prozess und der Quellcode unverändert bleiben.

Zero-Shot CoT

Bei der Zero-Shot CoT Methode, in Unterabschnitt 3.4.2 definiert, wird das Modell durch einen ergänzenden Satz dazu angeregt, in einem iterativen Prozess die eigene Ausgabe kritisch zu reflektieren und zu überarbeiten. Für die erweiterte Promptmethode wird eine initiale Prompt P_0 als Basis verwendet. Folgender Ausdruck, wurde an das Ende der erstellten Prompt angehängen.

CoT: *“Let’s think through this carefully, step by step.”*

Die finale Prompt ergibt sich aus der Konkatenation der CoT-Prompt an die Ursprungsprompt mit $P_{cot} = P_0 \oplus \text{CoT}$.

Emotional Prompting

Des Weiteren soll untersucht werden, ob der Einsatz emotionaler Stimuli (EPs) die Leistung von LLM-Modellen bei NER-Aufgaben optimieren kann. Zu diesem Zweck wurden drei Stimuli, die in Unterabschnitt 3.4.3 vorgestellt wurden, in Zero-Shot Szenarien kombiniert. Die Effektivität der sequenziellen Anwendung mehrerer Stimuli bei Fragebeantwortungen wurde in der Studie von C. Li et al. (2023) nachgewiesen. Die verwendeten emotionalen Prompts sind wie folgt definiert:

EP02 : *“This is very important to my career”*

EP07 : *“Are you sure that’s your final answer? Believe in your abilities and strive for excellence. Your hard work will yield remarkable results.”*

EP09 : *“Stay focused and dedicated to your goals. Your consistent efforts will lead to outstanding achievements.”*

Die Emotional Prompting-Sequenz wurde durch Konkatenation der spezifischen emotionalen Stimuli generiert: $EP = EP02 \oplus EP07 \oplus EP09$ Die finale Prompt P_{ep} wurde durch Anhängen von EP an den jeweiligen initialen Prompt P_0 erstellt mit $P_{ep} = P_0 \oplus EP$.

Role Prompting

Das Role Prompting, in Unterabschnitt 3.4.4 beschrieben, kombiniert eine Promptanweisung mit einer erstellten Persona oder Rolle, welche die Anweisung ausführen soll. Zheng et al. (2023) konnten zeigen, dass die Integration sozialer Rollen in Prompts die Leistung von LLMs über verschiedene Fragetypen hinweg konsistent verbessert. Daher wurde untersucht, ob die Ergänzung einer Rollenbeschreibung (RP) als professioneller *Medical Research Assistant*, die Extraktion von Operatoren verbessern kann.

RP : “*You are a highly skilled medical resarch assistant ...*”

Die finale Prompt P_{rp} wurde durch Voranstellen der Role Prompt an den initialen Prompt P_0 erstellt mit $P_{rp} = RP \oplus P_0$. Die im Anhang in Abbildung C.2 dargestellte Role Prompt definiert die Aufgabe eines spezialisierten medizinischen Forschungsassistenten. Die Hauptfunktion besteht in der Präzisierung von Ein- und Ausschlusskriterien in klinischen Studientexten durch die Integration logischer Operatoren AND, OR und NOT. Die Prompt erläutert logische Konzepte und spezifiziert die erforderlichen Kompetenzen des Assistenten, einschließlich medizinischer Fachkenntnisse, ausgeprägter logischer Denkfähigkeit, präziser Ausdrucksfähigkeit und eines strukturierten Ansatzes zur Textanalyse und -optimierung.

Few-Shot

Weiterhin soll mit Hilfe von Few-Shot Methoden evaluiert werden, ob Large Language Models durch die Bereitstellung von annotierten Beispielen eine Leistungssteigerung gegenüber diesen einfachen Prompt-Methoden erzielen können.

Beim Few-Shot Prompting, wie in Unterabschnitt 3.4.1 beschrieben, werden dem Modell k -Beispiele präsentiert. Diese Beispiele setzen sich aus Eingabe und zugehöriger optimaler Ausgabe zusammen. Die Anzahl der Beispiele wird durch die maximale Kontextlänge des Modells begrenzt. Analog zum Zero-Shot Ansatz wird dem Modell zusätzlich eine Promptinstruktion bereitgestellt.

Anwendung der Prompt-Methoden

Der Quellcode E.7 für die Anwendung der Prompt-Methoden mit Llama3-Modellen ist im Anhang in Abschnitt E.6 gezeigt. Bei der Inferenz über die Testdaten werden den ausgewählten Large Language Models die Anfragen, also die zu strukturierenden Eignungskriterientexte in Verbindung mit den Prompttechniken übergeben. Dabei wird in jedem Fall eine initiale Promptanweisung P_0 mitgegeben. In Zeile 28 kann die Instruktionsprompt flexibel mit den erweiterten Prompt-Methoden Zero-Shot CoT und Emotional Prompt erweitert werden. Im Falle von Zero-Shot und Few-Shot wird die Prompt sowohl als `system_content` als auch in der Anfrage `user_content` mitgegeben. Bei Few-Shot Prompting werden k -Beispiele mit Eingabetext *criteria*s und annotiertem Ausgabertext *labels* übergeben. Der `assistant_content` repräsentiert dabei die Modellausgabe, welche dem Modell bei Few-Shot vorgegeben wird.

Template für Prompttechniken für LLMs:

```
1 [
2   {"role": "system", "content":  $P_0 \mid P_{rp}$ },
3   {"role": "user", "content":  $P_0 \mid P_x + \text{criteria}[k]$ },
4   {"role": "assistant", "content":  $\text{labels}[k]$ }
5 ]
```

Die so zusammengefügte Prompts werden als Array aufgebaut. Die eigentliche Anfrage an das Modell enthält P_0 oder P_x in Kombination mit dem zu strukturierenden Text `test_file[i]`, dem i -ten File aus dem Testdatensatz.

Anfrage:

```
1 {"role": "user", "content":  $P_0 \mid P_x + \text{nct\_file}[i]$ }
```

Die i -te Anfrage wird vor der Inferenz an das Ende des erstellten Eingabe-Arrays angehängt. Basierend auf dieser Eingabe erstellt das Modell eine Vorhersage der Eignungskriterien mit eingefügten Operatoren im String-Format, die als Textdatei gespeichert werden kann.

5.4.2. Fine-Tuning

Für die Aufgabe der Integration logischer Operatoren in Eignungskriterientexte wurde ein Fine-Tuning der Llama3-8B und -70B Modelle durchgeführt. Aufgrund des geringeren Ressourcenbedarfs im Vergleich zu 70B-Modellen erfolgte eine Hyperparameteroptimierung für SFT- und LoRA-Parameter auf dem zunächst auf dem 8B-Modell. Ein SFT wurde über ein Instruction Fine-Tuning realisiert, indem die entwickelte Prompt als Instruktionen mit den annotierten Trainingsdaten übergeben wurde. Für ein Parameter-Efficient Fine-Tuning wurde LoRA in Verbindung mit rsLoRA und QLoRA aus Unterabschnitt 3.5.2 verwendet. Die Parameterkonfiguration für ein Supervised Fine-Tuning mit LoRA mit Standardwerten ohne Parameter anzupassen ist im Anhang in Tabelle D.5 gezeigt. Die LoRA-Basiswerte wurden aus der unsloth (2024)-Bibliothek übernommen. Für die Hyperparameterkonfiguration des Supervised Fine-Tuning dienen die Standardeinstellungen des Huggingface-SFTTrainer (2024). Diese Standardkonfiguration für das Fine-Tuning von Large Language Models soll als Vergleichsgrundlage dienen gegenüber einer Parameteranpassung und Hyperparameteroptimierung. Um das Fine-Tuning zu optimieren, wurde eine Auswahl an SFT- und LoRA-Parametern einmalig neu initialisiert. Diese sind in Tabelle 5.2 aufgeführt. Die Auswahl dieser Parameter basiert aus den in Abschnitt 3.5 empfohlenen Einstellungen und wurde durch initiale Tests validiert.

Kategorie	Parameter
LoRA-Parameter	lora_dropout = 0.05 use_rslora = True
Batch-Konfiguration	train_batch_size = 2 gradient_accumulation_steps = 8 eval_batch_size = 4
Trainingssteuerung	warmup_ratio = 0.1 weight_decay = 0.05 max_grad_norm = 1.0
Optimierung	optim = AdamW (8-bit) gradient_checkpointing = True
Evaluierung	eval_steps = 100 metric_for_best_model = eval_loss

Tabelle 5.2.: Festgelegte Hyperparameter für das Fine-Tuning von LLMs.

Dabei wurde der LoRA-Dropout von 0 auf 0.5 erhöht, um Overfitting zu reduzieren und die Generalisierungsfähigkeit zu verbessern. Um auch größere Werte für r und α zu testen, wurden rsLoRA und QLoRA mit einer 4-Bit Quantisierung eingesetzt. Die Batch-size wurde niedrig gesetzt, da der Trainingsdatensatz verhältnismäßig klein ist. Ein Warmup-Ratio von 0.1 ermöglicht eine graduelle Erhöhung der Lernrate in den ersten 10% des Trainings, was die Stabilität verbessert und abrupte Modellanpassungen verhindert. Der Weight-Decay Wert von 0.05 dient der Regularisierung, indem er die Gewichte konstant klein hält und somit Overfitting reduziert. Das Gradient Clipping bei 1.0 wurde beibehalten, um extreme Gradientenwerte zu begrenzen und eine stabile Optimierung zu gewährleisten. Als Optimierungsalgorithmus wurde AdamW in 8-bit Präzision gewählt. Zusätzlich wurde Gradient Checkpointing aktiviert, um den Speicherbedarf während des Trainings zu reduzieren und somit das Training größerer Modelle oder längerer Sequenzen zu ermöglichen. In der Standardkonfiguration für SFT wird das Large Language Model während der Trainingsphase nicht kontinuierlich evaluiert. Um das leistungsfähigste Modell zu identifizieren, wurde eine regelmäßige Evaluationsstrategie implementiert. Hierbei wird das Modell alle 100 Trainingsschritte anhand des Evaluationsdatensatzes bewertet und der aktuelle Stand gespeichert. Nach Abschluss des Trainings wird automatisch das Modell mit der besten Leistung geladen.

Weitere Parameter wie die Anzahl der Epochen oder der LoRA-Rang wurden über eine Hyperparameteroptimierung festgelegt. Dabei wurden die Parameter aus Tabelle 5.3 über den entsprechenden Wertebereich optimiert.

Kategorie	Parameter	Wertebereich
LoRA-Parameter	r	{16, 32, 64, 128, 256, 512, 2048}
	α	{16, 32, 64, 128, 256, 512, 1024}
SFT-Parameter	Epochs	{10, 20, 30, 40}
	η	$\{2 \times 10^{-4}, 5 \times 10^{-5}, 5 \times 10^{-6}\}$
	LR-Scheduler	{linear, cosine}

Tabelle 5.3.: Hyperparameter und deren Wertebereiche für die Optimierung des Fine-Tunings von Llama3-8B zur Operatoreinfügung in Eignungskriterientexte.

Die LoRA-Parameter r und α konnten mit der Verwendung von rsLoRA und QLoRA auf Zweierpotenzwerte von 16 bis 2048 für r und 16 bis 1024 für α getestet werden. Die SFT-Parameter, die während des Fine-Tunings variiert wurden, sind die Anzahl der Epochen, die Learning Rate η und der LR-Scheduler. Nach einer Hyperparameteroptimierung mit dem Llama3-8B Modell wurde ein Fine-Tuning des Llama3-70B Modells durchgeführt. Dabei wurden die Anzahl der Epochen sowie der LoRA-Rang und LoRA- α optimiert.

5.4.3. Regelbasierter Parser

Für die Anwendung von Few-Shot und Fine-Tuning Methoden bei Large Language Models ist es notwendig, zunächst Instruktionen in Form von Prompts zu formulieren und annotierte Beispiele durch Preprocessing bereitzustellen. Die Einfügung logischer Operatoren in Kriterientexte kann jedoch auch mittels regelbasierter Methoden realisiert werden. Um den Nutzen des Einsatzes von LLMs sowie die Erstellung von Beispielen für Few-Shot und Fine-Tuning zu evaluieren, wurde daher ein regelbasierter Parser implementiert, der die Instruktionen zur Operatoreneinfügung abbildet. Insbesondere dient dies auch dem Vergleich zum Zero-Shot Verfahren. Die in Abbildung 5.8 dargestellte Prompt 2, welche Instruktionen zum Einfügen von logischen Operatoren nach Signalwörtern wie “and”, “or” und “not” in Eignungskriterientexte enthält, kann auch durch einen regelbasierten Ansatz mit einem Python-Parser realisiert werden. Dabei werden die Regeln zum Einsetzen der Operatoren über reguläre Ausdrücke (RegEx) umgesetzt.

Der regelbasierte Parser wurde als Funktion `naive_greedy_match()` implementiert und ist im Anhang im Quellcode E.9 dargestellt. Für die Anwendung des Parsers müssen die Eignungskriterientexte im Rohformat als Eingabe übergeben werden. Die Ausgabe erfolgt analog zur LLM-Ausgabe als Textdatei mit den prozessierten Eignungskriterien. Die Funktion verwendet reguläre Ausdrücke, um nach bestimmten Schlüsselwörtern und Satzzeichen zu suchen, die auf logische Verknüpfungen wie [AND], [OR] und [NOT] hindeuten. Sobald solche Muster gefunden werden, fügt die Funktion die entsprechenden logischen Operatoren in den Text ein.

5.5. Direkte Strukturierung

Die Aufgabe des Einfügens logischer Operatoren in Eignungskriterien mittels Large Language Models ermöglicht die Evaluation von Zero-Shot und erweiterten Prompt-Methoden. Für eine Weiterverarbeitung in maschinenlesbare und interoperable Formate wie JSON ist dabei die Entwicklung eines spezifischen Parsers erforderlich. Um zu untersuchen, ob aktuelle Open-Source LLMs die komplexe Logik von Eignungskriterien effizient in ein verschachteltes JSON-Format als Abstract Syntax Tree überführen können, wurde ein direkter Prozess für die Strukturierung entwickelt. In diesem Ansatz generieren LLMs unter Anwendung von Few-Shot und Fine-Tuning die AST-Repräsentationen im JSON-Format direkt aus den ursprünglichen Eignungskriterien.

Direkte Strukturierung im JSON-Format als AST

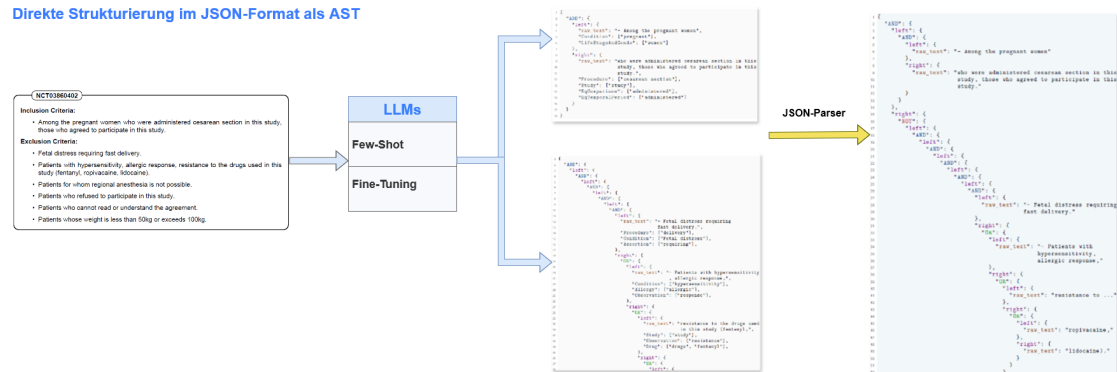


Abbildung 5.9.: Direkte Strukturierung von Ein- und Ausschlusskriterien mit LLMs mit Few-Shot und Fine-Tuning. Die Kriterien werden als AST im JSON-Format ausgegeben. Die Ausgaben können über einen JSON-Parser zusammengefügt werden.

Der direkte Strukturierungsprozess mithilfe von LLM-basierten Methoden wird in Abbildung 5.9 dargestellt. Um die Anzahl der Eingabetoken zu reduzieren, werden Ein- und Ausschlusskriterien den Modellen separat übergeben. Für Open-Source LLMs wie Llama3 ist es aufgrund der begrenzten Kontextlänge von 4000 Token nicht möglich, die Eignungskriterien vollständig zu verarbeiten. Es wurde eine spezifische Strukturprompt entwickelt, welche die ausgewählten Large Language Models dazu anleitet, zunächst die logischen Operatoren aus den Kriterientexten zu extrahieren und diese zur Erstellung der AST-Struktur zu verwenden. Die zugehörigen Textpassagen werden dabei als Blätter extrahiert. Darüber hinaus sollen die Modelle relevante Entitäten identifizieren und diese den extrahierten Textpassagen zuordnen. Die im JSON-Format erzeugten Ein- und Ausschlusskriterien können mithilfe eines einfachen JSON-Parsers, wie in Unterabschnitt 5.2.4 beschrieben, zusammengeführt werden. Dabei fungiert ein AND-Operator als Wurzelknoten, der die Ein- und Ausschlusskriterien verbindet. Um die inhärente Negationseigenschaft der Ausschlusskriterien korrekt zu repräsentieren, werden diese zusätzlich mit einem NOT-Knoten als direktem Vorgängerknoten versehen, bevor sie in die Gesamtstruktur integriert werden. Die alleinige Verwendung von Prompts als Eingabe für LLMs mit Zero-Shot erwies sich als unzureichend, um die gewünschten AST-Strukturen zu erzeugen. Die Komplexität der AST-Strukturen erschwerte es, diese ausschließlich in Worten zu beschreiben. Erste Versuche zeigten, dass LLMs Beispiele benötigen, um die gewünschte Struktur effektiv zu erzeugen. Dafür wurden annotierte Eignungskriterien aus einem öffentlichen Korpus in das JSON-Format prozessiert, um als Beispiele für Few-Shot und Fine-Tuning Methoden zu dienen. Die Strukturprompt, die den Modellen für ICL- und Fine-Tuning Methoden übergeben wird, hat ebenfalls Einfluss auf die Modelleistung. Deshalb wurde sie über mehrere Versuche hinweg iterativ entwickelt und verbessert.

Eine Evaluation mittels Few-Shot Prompting wurde für die Modelle Llama3-70B und Qwen2-72B in 2-Shot und 10-Shot Szenarien durchgeführt. Als Referenz diente ein Vergleich mit dem Closed-Source Modell GPT-4o. Die Bewertung umfasste die korrekte Ausgabe des JSON-Formats sowie die Erkennung logischer Operatoren und relevanter Entitäten. Weiterhin wurde ein Fine-Tuning des Llama3-70B Modells vorgenommen. Hierfür wurden die entwickelte Instruktion und der in JSON-transformierte Trainingsdatensatz verwendet. Für das SFT unter Verwendung von IFT- und LoRA-Methoden wurden die Hyperparameter eingesetzt, die im vorherigen Fine-Tuning Prozess aus Abschnitt 5.4.2 zu den besten Ergebnissen führten.

6. Evaluation

In Abschnitt 6.1 wird die *indirekte Strukturierung* untersucht. Dabei erkennen Large Language Model-Methoden, wie in Abschnitt 5.4 beschrieben, logische Operatoren, um eine anschließende Transformation in eine AST-Struktur zu ermöglichen. Die Leistungsfähigkeit dieser Methode wird anhand eines öffentlichen Korpus mit den aktuellen State-of-the-Art-Modellen verglichen. In Abschnitt 6.2 wird die *direkte Strukturierung* evaluiert, bei der LLMs die Eignungskriterien direkt in eine AST-Struktur überführen.

6.1. Evaluation der indirekten Strukturierung

Bei der indirekten Strukturierung findet zunächst eine Information Extraction durch Large Language Models statt, um in einem zweiten nachfolgenden Schritt eine Transformation, durch einen Parser, in eine AST-Struktur zu ermöglichen. Für dieses Vorgehen wird der öffentliche verfügbare LCT-Datensatz als Datengrundlage verwendet. Dieser dient als Trainings- und Testdatensatz. Um die Generalisierbarkeit des Ansatzes zu testen, wird zusätzlich der Chia-Datensatz für eine Evaluation herangezogen. Beide Datensätze werden in Abbildung 4.1 gezeigt. Im Folgenden wird beschrieben, wie die Korpora mit Eignungskriterien für die Operatorenextraktion verarbeitet wurden, um eine Evaluation zu ermöglichen. Anschließend wird erläutert, wie die Modelle evaluiert wurden.

Aufbereitung und Evaluation der Korpora

Die LCT- und Chia-Datensätze liegen im BRAT-Format vor. In diesem Format wird der Originaltext in einer Textdatei gespeichert, während die Entitäten und Relationen in separaten Annotationsdateien abgelegt werden, wie in Unterabschnitt 2.2.2 beschrieben. Trotz des gleichen Formats sind die Entitäten und Relationen in beiden Datensätzen unterschiedlich codiert. Durch die Konvertierung werden diese Unterschiede ausgeglichen. Abbildung A.4 im Anhang zeigt einen Auszug der im BRAT-Format erstellten Annotationen der Studie NCT03860402. Die annotierten logischen Operatoren werden in die zugehörigen Eignungskriterien integriert, die im Anhang in Abbildung A.3 dargestellt sind. Die so erzeugte, angereicherte Textdatei ist in Abbildung 6.1 gezeigt.

NCT03860402 (mit AND, OR, NOT)

Inclusion Criteria:

- Among the pregnant women **[AND]** who were administered cesarean section in this study, those who agreed to participate in this study.

Exclusion Criteria:

- Fetal distress requiring fast delivery.
- Patients with hypersensitivity, allergic response, **[OR]** resistance to the drugs used in this study (fentanyl, **[OR]** ropivacaine, **[OR]** lidocaine).
- Patients who it is **[NOT]** not possible for regional anesthesia.
- Patients who refused to participate in this study.
- Patients who cannot read or understand the agreement.
- Patients whose weight is less than 50kg **[OR]** or exceeds 100kg.

Abbildung 6.1.: Eignungskriterien der Studie NCT03860402 aus dem LCT-Datensatz mit eingefügten logischen Operatoren.

Die Integration der Operatoren in den Text ist notwendig, um ICL-Methoden anwenden und Ungleichheiten bei der Codierung zwischen den beiden Datensätzen ausgleichen zu können. Zudem erlaubt dies, die Erkennung der logischen Operatoren unabhängig von den erkannten Entitäten zu evaluieren. Dies ermöglicht eine zweistufige Evaluation der logischen inhärenten Struktur sowie der Entitäts- und Relationsextraktion. Der Prozess der Konvertierung ist wie folgt.

- **Einsetzen von AND, OR:** Die binären Operatoren (AND und OR) werden in den beiden Datensätzen als Relation zwischen zwei Entitäten codiert. Die Relation kann in den Ursprungstext durch die Offsets (`<stop character index>`) der beiden Argumente (Arg1 und Arg2) der jeweiligen Relation eingefügt werden. Dabei wurde ein logischer Operator (AND oder OR) zwischen den beiden Argumenten im Text eingefügt, also nach dem `<stop character index>` des Argumentes, welches als erstes im Text vorkommt. Formal: Sei P_{op} die Position des Operators im Text, S_{Arg1} der `<stop character index>` des ersten Arguments und S_{Arg2} der `<stop character index>` des zweiten Arguments. Dann lässt sich die Position des Operators anhand folgender Vorschrift bestimmen: $P_{op} = \min(S_{Arg1}, S_{Arg2}) + 2$. Anschließend wird der Operator um zwei Positionen im Text nach rechts verschoben, damit der eingefügte Operator durch Leerzeichen von anderen Wörtern getrennt ist. Die Anwendung dieser Methodik führte zu charakteristischen und korrekten Positionierungen im Text, die in allen Datensätzen manuell überprüft wurden. Dabei wurde festgestellt, dass in beiden Datensätzen selektiv einige Relationen bzw. Entitäten annotiert wurden, was zu nicht nachvollziehbaren Diskrepanzen führte. So ist beispielsweise in Abbildung 6.1 eine OR-Relation zwischen *allergic response* und *resistance* codiert, jedoch nicht zwischen *hypersensitivity* und *allergic response*.
- **Einsetzen von NOT:** In BRAT-Annotationen können NOT-Relation über ein Event E definiert werden, welches einen Entitätentyp *Negation* und ein Relationstyp *Negates* verbindet. Die NOT-Relation aus der Annotationsdatei aus Abbildung A.4 beschreibt die Negierung folgendermaßen: Die Entität *not* (negiert) *Negates* die Entität *regional anesthesia*. Bei der Integration in den Eignungskriterientext wurde der NOT-Operator systematisch vor die negierende Entität platziert.

Evaluierung der Operatoren

Anstatt die logischen Operatoren zusammen mit den verknüpften Entitäten zu extrahieren und in Formaten wie dem BIO-Format auszugeben, besteht das methodische Vorgehen darin, die Operatoren als Tags *[AND]*, *[OR]* und *[NOT]* direkt in die Eignungskriterientexte einzufügen. Dieses Vorgehen hängt mit den in Abschnitt 5.1 diskutierten Problemen von Large Language Models zusammen. Diese Modelle haben Schwierigkeiten, die genauen Positionen von Entitäten im Text zu ermitteln. Selbst einfache Aufgaben wie das Bestimmen der Positionen von Textsegmenten können für LLMs herausfordernd sein. Dies ist auf die in Abschnitt 3.7 beschriebenen Probleme bei numerischen Berechnungen und das Auftreten von Halluzinationen zurückzuführen. Bei Relation Extraction-Aufgaben mit LLMs werden Entitäten extrahiert und können beispielsweise als Tripel wie *(Steve, works for, Apple)* dargestellt werden (X. Xu et al., 2023). Die genauen Positionen der Entitäten und Relationen im Text werden dabei jedoch nicht erfasst. Wie in Abbildung 4.8 gezeigt, extrahieren LLMs Entitäten aus einzelnen Sätzen und geben die Ergebnisse satzbezogen an, um die Positionen zu verdeutlichen. Logische Relationen existieren implizit zwischen einer oder mehreren Entitäten und haben keine eindeutige Position im Text. Sie werden häufig aufgrund von Signalwörtern wie “and”, “or” und “not” annotiert (N. Dobbins; T. Mullen et al., 2022). Dies zeigt sich auch, wenn im BRAT-Format annotierte logische Operatoren zwischen den verbundenen Entitäten in den Text eingefügt werden. Die Operatoren stehen dann in der Regel direkt vor diesen Signalwörtern. Daher können die logischen Relationen als eigenständige Entitäten im Text betrachtet werden, ohne dass die verbundenen Entitäten selbst extrahiert werden müssen. Für die Extraktion der Operatoren mittels Large Language Models ist es notwendig, die Operatoren direkt in den Text einzufügen, um ausschließlich die Operatoren zu extrahieren, ohne die verknüpften Entitäten zu berücksichtigen. Die Operatoren werden somit als Entitäten behandelt und erhalten eine Position im Text. Bei *[AND]* und *[OR]* wird der Text vor und nach dem Operator verbunden, während bei *[NOT]* nur der nachfolgende Text relevant ist. Die Operatorenextraktion mit LLMs kann mittels der Metriken Precision (P), Recall (R) und F1-Score (F1) evaluiert werden. Ein Operator AND, OR oder NOT wird als korrekt eingefügt betrachtet, wenn das unmittelbar vorhergehende Wort sowohl in der Modellausgabe als auch im annotierten Text identisch ist. Dadurch können die Positionen der Operatoren im Text exakt evaluiert werden.

Seien W_L die Wörter vor dem Operator im annotierten Text des Goldstandards (Label) und W_M die Wörter vor dem Operator im vom Modell generierten Text. Die Klassifikationsergebnisse können wie folgt bestimmt werden.

- **True Positives (TP)** repräsentieren die Anzahl der korrekt extrahierten Operatoren. Sie werden durch die Kardinalität des Durchschnitts der beiden Wörtermengen bestimmt, mit $TP = |W_L \cap W_M|$, also die Summe der übereinstimmenden Wörter pro Operator.
- **False Positives (FP)** quantifizieren die Anzahl der fälschlicherweise eingefügten Operatoren. Sie werden durch die Kardinalität der Differenzmenge berechnet mit $FP = |W_M \setminus W_L|$. Wobei die Differenzmenge alle Elemente enthält, die in W_M , aber nicht in W_L vorkommen. Dies repräsentiert Operatoren, die vom Modell eingefügt wurden, aber nicht im annotierten Text existieren.
- **False Negatives (FN)** erfassen die Anzahl der nicht erkannten Operatoren mit $FN = |W_L \setminus W_M|$. Sie werden durch die Kardinalität der Differenzmenge ermittelt, welche alle Elemente umfasst, die in W_L , aber nicht in W_M vorkommen. Dies repräsentiert Operatoren, die im annotierten Text vorhanden sind, aber vom Modell nicht extrahiert wurden.

Dieser Ansatz ermöglicht es, die Einfügung der logischen Operatoren mit Large Language Models in den Rohtext der Eignungskriterien mit den aktuellen SOTA-Modellen zu vergleichen. Ein wesentlicher Vorteil dieses Ansatzes besteht darin, dass eine Auswertung ausschließlich auf den drei Operatoren AND, OR und NOT möglich ist.

Ausgabefehler

Die Large Language Models verarbeiten die Eignungskriterien als Textdateien und geben diese mit integrierten logischen Operatoren erneut als Textdateien aus. Aufgrund des generativen probabilistischen Ansatzes der LLMs können jedoch Halluzinationen in der Ausgabe auftreten. Diese treten auf, wenn das Modell von den vorgegebenen Instruktionen abweicht und die ursprünglichen Kriterientexte nicht unverändert inklusive der Operatoren zurückgibt, sondern zusätzliche, zuvor nicht vorhandene Wörter oder Satzteile einfügt. Die Tags *[AND]*, *[OR]*, *[NOT]* werden dabei nicht als neu eingefügte Wörter gezählt. Typische Ausgabefehler sind in Tabelle 6.1 dargestellt. Beispielsweise kann es vorkommen, dass Sätze doppelt eingefügt oder Wörter durch Operatoren

Goldstandard (LCT-Korpus)	LLM-Ausgabe
Beispiel 1: Ein Satz wird doppelt eingefügt	
No history of immunosuppression.	2 × <i>[NOT]</i> No history of immunosuppression.
Beispiel 2: Wort wird durch einen Operator ersetzt	
ages 2-21 years	ages 2 <i>[AND]</i> 21 years
Beispiel 3: Nicht existierende Operatoren und Wortersetzungen	
Patients in which it is <i>[NOT]</i> not possible to place the esophageal catheter <i>[OR]</i> it is <i>[NOT]</i> not possible to detect the diaphragm by ultrasound <i>[OR]</i> it is <i>[NOT]</i> not possible ...	Patients <i>[IN WHICH]</i> <i>[NOT]</i> possible to place the esophageal catheter <i>[OR]</i> <i>[NOT]</i> possible to detect the diaphragm by ultrasound <i>[OR]</i> <i>[NOT]</i> possible ...

Tabelle 6.1.: Beispiele von Ausgabefehlern von Llama3-70B mit Zero-Shot auf dem LCT-Testdatensatz.

ersetzt werden. In einigen Fällen fügt das Modell sogar nicht existierende Operatoren ein oder nimmt unerwartete Wortersetzungen vor. In Beispiel 2 wird 2-21 im Originaltext durch den verbindenden Bindestrich als eine einzelne Einheit behandelt. Durch das Ersetzen des Bindestrichs durch das Modell mit dem Operator *[AND]* entstehen jedoch zwei separate Wörter 2 und 21. Diese werden als neu eingefügte Wörter gezählt, was den Ausgabefehler erhöht. Um diese Halluzinationen zu quantifizieren, wird ein Ausgabefehler berechnet. Dieser Fehler misst den Anteil neu eingefügter Wörter in den Kriterientexten, wobei die logischen Operatoren selbst nicht berücksichtigt werden. Der Ausgabefehler wird als prozentualer Mittelwert der neu eingefügten Wörter über alle erzeugten Dateien ausgedrückt. Der Ausgabefehler dient dazu, die Qualität der von den LLMs generierten Texte zu bewerten, indem er zeigt, wie häufig und in welchem Ausmaß unerwünschte Ergänzungen auftreten. Ein hoher Ausgabefehler deutet darauf hin, dass das Modell die Anweisungen nicht zuverlässig befolgt und häufig Halluzinationen produziert.

6.1.1. Auswertung des LCT-Korpus

Der Leaf Clinical Trials-Korpus, in Unterabschnitt 4.2 beschrieben, wurde zur Evaluation der Strukturierung von Eignungskriterien mit Large Language Models ausgewählt, da unter 51 annotierten Relationstypen auch die logischen Relationen And (AND), Or (OR) und Negates (NOT) annotiert wurden. Zudem wurden aktuelle SOTA-Modelle, SciBERT und R-BERT auf den drei Operatoren über Precision, Recall und F1-Score evaluiert, wodurch eine Vergleichsgrundlage geschaffen wird. Tabelle 6.2 zeigt die Leistung der SOTA-Modelle SciBERT und einer Kombination aus R-BERT und SciBERT bei der Extraktion von Operatoren. Die Evaluierung wurde auf 20% der LCT-Daten durchgeführt, wobei Precision, Recall und F1-Score als Metriken verwendet wurden (N. Dobbins; T. Mullen et al., 2022). Der Durchschnitt ($\emptyset$) stellt den Mittelwert der jeweiligen Metrik über alle drei Operatoren dar. Die Modellausgabe wird durch Precision, Recall und F1-Score über die Mittelwerte der drei Operatoren AND, OR, NOT bewertet.

Relation	SciBERT	R-BERT + SciBERT
And	54.1 / 60.0 / 56.9	53.8 / 53.8 / 53.8
Negates	74.3 / 91.0 / 81.8	74.5 / 88.7 / 81.0
Or	85.1 / 93.2 / 89.0	88.4 / 92.2 / 90.2
$\emptyset$	71.2 / 81.4 / 75.9	72.2 / 78.2 / 75.0

Tabelle 6.2.: Leistungsvergleich (Precision / Recall / F1-Score in %) von SOTA-Modellen für logische Operatoren auf dem LCT-Korpus (N. Dobbins; T. Mullen et al., 2022).

Um die ausgewählten LLMs ausschließlich für die Extraktion der drei logischen Operatoren zu evaluieren, wurde aus dem LCT-Datensatz ein Trainings- und Testdatensatz erstellt. Dabei wurden die annotierten Operatoren gemäß der Methodik in Abschnitt 6.1 in die Eignungskriterien eingefügt. Von den insgesamt 1006 Textdateien des LCT-Datensatz wurden 80% (804 Beispiele) als Trainingsdaten für das Fine-Tuning und als Few-Shot Beispiele verwendet, während die verbleibenden 20% (202 Beispiele) als Testdatensatz zurückgehalten wurden.

6.1.2. Prompting

Das Zero-Shot Verfahren erfordert lediglich die Erstellung einer spezifischen Instruktion (Prompt) für das Einfügen von Operatoren in die Eignungskriterien. Mithilfe des Llama3-70B Modells wurden zwei solcher Prompts evaluiert. Zusätzlich wurden darauf basierend weitere Prompttechniken untersucht, darunter Role Prompting, Zero-Shot (CoT) und Emotional Prompting (EP). Folgende Prompts wurden für die Operatorenextraktion mit LLMs entwickelt.

- Prompt 1: Beinhaltet Teile der LCT-Annotationsrichtlinien¹ für die drei Operatoren, welche beschreiben, wann ein logischer Operator gesetzt werden muss. Dabei wurden auch explizite Beispiele für die unterschiedlichen Anwendungsfälle entwickelt. Zudem werden häufige Wörter oder Satzzeichen genannt, nach denen typischerweise Operatoren geschrieben werden. Um Halluzinationen vorzubeugen, soll die Ausgabe in <EDITED_CRITERIA> tags geschrieben werden. Dabei soll neben den Kriterien inklusive der eingesetzten Operatoren kein zusätzlicher Text ausgegeben werden. Die Prompt ist im Anhang in Abbildung C.1 gezeigt.
- Prompt 2: Diese Variante weist das Modell lediglich an, Operatoren nach häufigen Signalwörtern wie "and", "or" oder Satzzeichen wie Kommata (",") zu platzieren. Die allgemeinen Regeln für die Ausgabe bleiben identisch zu denen von Prompt 1. In Abbildung 5.8 wird Prompt 2 dargestellt, welche entwickelt wurde, um einen einfachen und generalisierbaren Ansatz für die logischen Strukturierung von Eignungskriterien bereitzustellen.

Die Entwicklung der Prompts erfolgte in einem iterativen Prozess, bei dem verschiedene Varianten systematisch auf 50 Dateien des LCT-Testdatensatzes getestet wurden. Durch diese iterative Evaluierung konnten die Prompts kontinuierlich optimiert werden. Unter den getesteten Varianten

¹<https://github.com/uw-bionlp/clinical-trials-gov-annotation/wiki>

zeigten Prompt 1 und Prompt 2 die besten Ergebnisse hinsichtlich der Genauigkeit bei der Extraktion der logischen Operatoren und wurden daher für die weiteren Experimente ausgewählt. Die in Abbildung 5.8 dargestellte Prompt 2 enthält Anweisungen zum Einfügen von AND-, OR- und NOT-Operatoren vor Signalwörtern wie “and”, “or” und “not” oder nach Satzzeichen wie Kommas. Diese Instruktionen können über einen regelbasierten Parser (Unterabschnitt 5.4.3) implementiert werden, welcher die Regeln zum Einsetzen der Operatoren über reguläre Ausdrücke (RegEx) umsetzt. Der Regex-Parser, als *Naive Greedy Match* bezeichnet, wurde als Baseline entwickelt, um zu evaluieren, ob sich der teilweise ressourcenintensive Einsatz von Large Language Models mit Zero-Shot oder Few-Shot Methoden gegenüber einem klassischen Ansatz mit geringem Ressourcenverbrauch lohnt.

6.1.3. Zero-Shot Methoden

Die initiale Evaluation der beiden erstellten Prompts sowie der erweiterten Prompt-Methoden erfolgte mit dem Llama3-70B Modell. Ziel war es, die Leistungsfähigkeit der Prompts zu vergleichen und zu untersuchen, ob ergänzende Prompt-Methoden zu einer Verbesserung führen. Die Auswertung des Modells Llama3-70B mit Zero-Shot Methoden mit Prompt 1 ist in Tabelle 6.3 gezeigt. Dabei werden Precision (P), Recall (R), F1-Score (F1) sowie der durchschnittliche Ausgabefehler in Prozent angegeben.

Modell	Prompting	P	R	F1	Ausgabefehler ($\bar{x}$, %)
Llama3-70B (Zero-Shot)	Prompt 1	42.9	59.5	47.2	3.7
	Role	40.2	61.3	46.5	6.2
	CoT	41.9	58.9	46.0	4.2
	EP	41.0	56.5	44.9	4.1

Tabelle 6.3.: Evaluation von erweiterten Prompt-Methoden mit Llama3-70B auf der Grundlage von Prompt 1.

Die Ergebnisse zeigen, dass die Zero-Shot Methode mit Prompt 1 ohne zusätzliche Promptanweisungen die beste Leistung auf dem Testdatensatz von 202 Einträgen erzielte, mit einem F1-Score von 47.2%. Die Optimierungsversuche mittels erweiterter Prompt-Methoden führen nicht zu einer Verbesserung der Leistung. Dabei erhöht sich zudem der Ausgabefehler unter der Verwendung erweiterter Prompt-Methoden. Exemplarischer Ausgabefehler von Llama3-70B mit Prompt 1 sind im Anhang in Tabelle 6.1 dargestellt.

Ergänzend wurde auch Prompt 2 aus Abbildung 5.8 unter Verwendung verschiedener Prompt-Methoden untersucht. Die Ergebnisse dieser Untersuchung sind in Tabelle 6.4 gezeigt.

Modell	Prompting	P	R	F1	Ausgabefehler ($\bar{x}$, %)
Naive Greedy Match	–	45.0	74.4	50.9	–
Llama3-70B (Zero-Shot)	Prompt 2	43.4	69.5	49.7	3.9
	Role	40.4	69.1	48.7	6.5
	CoT	43.2	70.6	50.0	3.8
	EP	42.7	70.7	49.9	14.7

Tabelle 6.4.: Evaluation von erweiterten Prompt-Methoden mit Llama3-70B auf der Grundlage von Prompt 2.

Prompt 2 stellt eine verkürzte und vereinfachte Version von Prompt 1 dar, die lediglich Regeln für die Platzierung von Operatoren vor oder nach typischen Signalwörtern implementiert. Diese ist auch als Regex-Parser *Naive Greedy Match* implementiert und ausgewertet worden. Die Evaluation des Llama3-70B Modells mit Zero-Shot unter der Verwendung von Prompt 2 zeigt, dass die Leistung mit einem F1-Score von 49.7% durch zusätzliche Prompt-Methoden nicht signifikant verbessert werden kann. Die Anwendung von Zero-Shot CoT führt zu einem marginal höheren F1-Score von 50.0% und einem niedrigeren Ausgabefehler im Vergleich zur alleinigen Nutzung von Prompt 2. Bei der Verwendung des Emotional Prompting (EP) zeigt das Llama3-Modell eine deutlich erhöhte Tendenz zur Halluzination, mit einem Ausgabefehler von 14.7%. In Abbildung 6.2 werden die Zero-Shot Verfahren unter Verwendung von Llama3-70B mit den beiden Prompts und erweiterten Prompt-Methoden dem Regex-Parser auf Basis von Prompt 2 gegenübergestellt. Die Ergebnisse für Precision, Recall und F1-Score sind in Prozent angegeben.

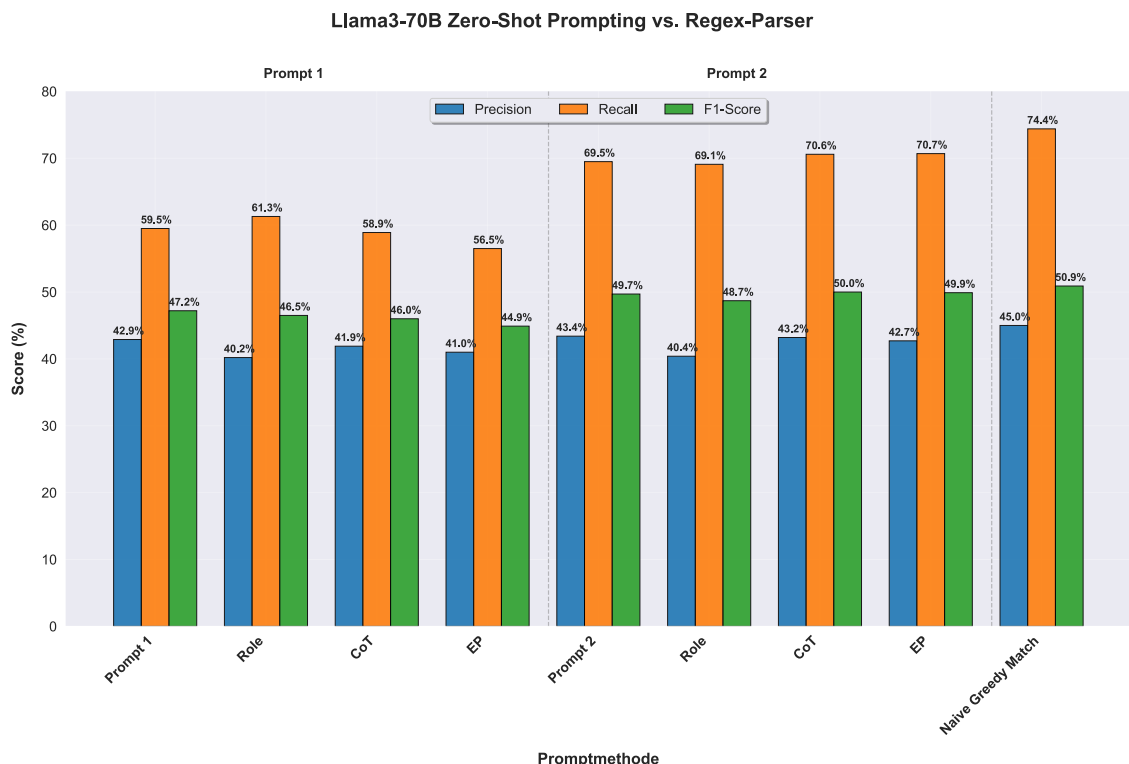


Abbildung 6.2.: Evaluation von Llama3-70B mit Zero-Shot Prompt-Methoden. Evaluiert werden zwei erstellte Prompts mit erweiterten Prompt-Methoden Role Prompting (Role), Zero-Shot CoT (CoT) und Emotional Prompting (EP). Diese werden mit dem Regex-Parser *Naive Greedy Match* verglichen.

Die Methoden unter Verwendung von Prompt 2 weisen konsistent höhere Recall-Scores auf, bei ähnlichen oder leicht verbesserten Precision Werten im Vergleich zu erweiterten Prompt-Methoden mit Prompt 1. Der Ausgabefehler bei den meisten Prompt 2 Varianten bleibt vergleichbar mit den der Prompt 1 Methoden, wobei das Emotional Prompting (EP) einen signifikant höheren Ausgabefehler aufweist. Die Ergebnisse zeigen, dass eine generelle Leistungsverbesserung im Nutzen von Prompt 2 zu erwarten ist. Die mögliche Erhöhung der Ausgabefehler bei der Verwendung erweiterter Prompt-Methoden weist auf mögliche Instabilitäten bei der Nutzung hin. Der Regex-Parser hat den höchsten Recall mit 74.4% und den höchsten F1-Score von 50.9%. Das Llama3-70B Modell erzielt mit Zero-Shot unter Verwendung von Prompt 2 einen nur um 1.2 Prozentpunkte niedrigeren F1-Score, liegt jedoch im Recall um 4.9 Prozentpunkte zurück. Dies zeigt, dass Fachkräfte durch die Nutzung von Open-Source LLMs und einfachen Promptanweisungen eine Leistung erzielen können, die der eines programmierten Parsers nahekommt. Es ist jedoch anzunehmen, dass der Parser mehr Operatoren identifiziert. Zudem ist zu beachten, dass der Zero-Shot Ansatz einen Ausgabefehler aufweist, bei dem der Ursprungstext fehlerhaft zurückgegeben werden kann.

6.1.4. Few-Shot Methoden

Um die Leistung der beiden entwickelten Prompts in Few-Shot Szenarien zu evaluieren, wurde dem Llama3-70B Modell während der Inferenz fünf ausgewählte, annotierte Beispiele übergeben. Tabelle 6.5 zeigt den Vergleich der Metriken Precision, Recall und F1-Score des 5-Shot Llama3-Modells im Vergleich zu dem Zero-Shot Modell.

Modell	Prompt	P	R	F1
Llama3-70B (0-Shot)	Prompt 1	42.9	59.5	47.2
	Prompt 2	43.4	69.5	49.7
Llama3-70B (5-Shot)	Prompt 1	51.2	56.7	51.7
	Prompt 2	49.1	73.8	55.8

Tabelle 6.5.: Promptevaluation mit Llama3-70B Modell mit Zero-Shot und Few-Shot.

Die Evaluation der 5-Shot Methode zeigt, dass die Verwendung von Prompt 2 für die Operatorenextraktion auch mit Few-Shot zu besseren Ergebnissen führt als das Nutzen von Prompt 1. Durch die Integration von fünf annotierten Beispielen in den Inferenzprozess konnte der F1-Score für die Operatorenextraktion im Vergleich zu Zero-Shot um ca. 5 Prozentpunkte gesteigert werden. Alle folgenden Untersuchungen basieren aufgrund der Leistungsunterschiede auf Prompt 2.

Zur Untersuchung des Einflusses der Shot-Anzahl auf die Modellleistung wurde das Llama3-70B Modell mit 1 bis 5 zufällig gewählten Beispielen aus den Trainingsdaten getestet. Für jedes k-Shot Szenario wurden 20 separate Versuche durchgeführt, wobei jeweils zufällig ausgewählte Beispiele aus den LCT-Trainingsdaten verwendet wurden. Die Leistung des Modells wurde anschließend auf dem Testdatensatz evaluiert und über 20 Versuche gemittelt. Das Ergebnis der Auswertung ist in Abbildung 6.3 dargestellt, wobei die gemittelten Werte für Precision, Recall und F1-Score über 20 Versuche gezeigt werden, einschließlich der Standardabweichung, welche als Fehlerbalken visualisiert ist.

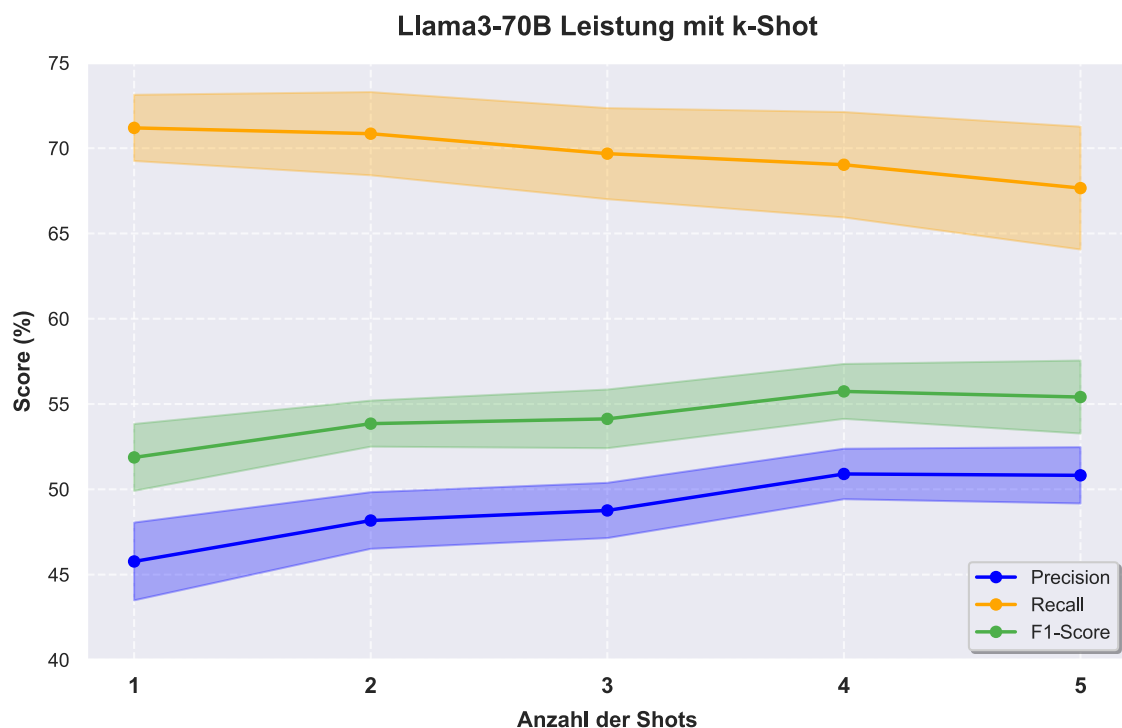


Abbildung 6.3.: Evaluation von Llama3-70B mit k-Shot Methoden basierend auf gemittelten Precision, Recall und F1-Scores über 20 Versuche.

Die Ergebnisse zeigen, dass die Precision mit steigender Shot-Anzahl von 45.7% bei 1-Shot auf 50.8% bei 5-Shot zunimmt, während der F1-Score von 51.8% auf 55.4% ansteigt. Die Recall-Werte hingegen sinken von 71.1% bei einem Shot auf 67.6% bei 5-Shots. Die Standardabweichung der Precision sinkt mit zunehmender Shot-Anzahl von 2.3 auf 1.7. Die Standardabweichung des Recalls steigt von 1.9 auf 3.6, während sie für den F1-Score von 2.0 auf 2.1 leicht zunimmt. Weiterhin wurde untersucht, ob 5-Shot Methoden mit ausgewählten Beispielen die Nutzung von zufälligen Beispielen optimieren können. Zu diesem Zweck wurden die 5-Shot Ergebnisse mit zufälligen Beispielen aus dem Trainingsdatensatz als *5-Zufall* übernommen und mit selbstständig ausgewählten Beispielen *5-Auswahl* verglichen. Zusätzlich wurden die fünf Dateien mit den meisten annotierten Operatoren als *5-Max* und jene mit den wenigsten Operatoren als *5-Min* evaluiert. Die Ergebnisse der 5-Shot Untersuchung werden in Abbildung 6.4 dargestellt. Für alle 5-Shot Methoden wurden die Ergebnisse über 20 Durchläufe gemittelt und die Standardabweichung berechnet.

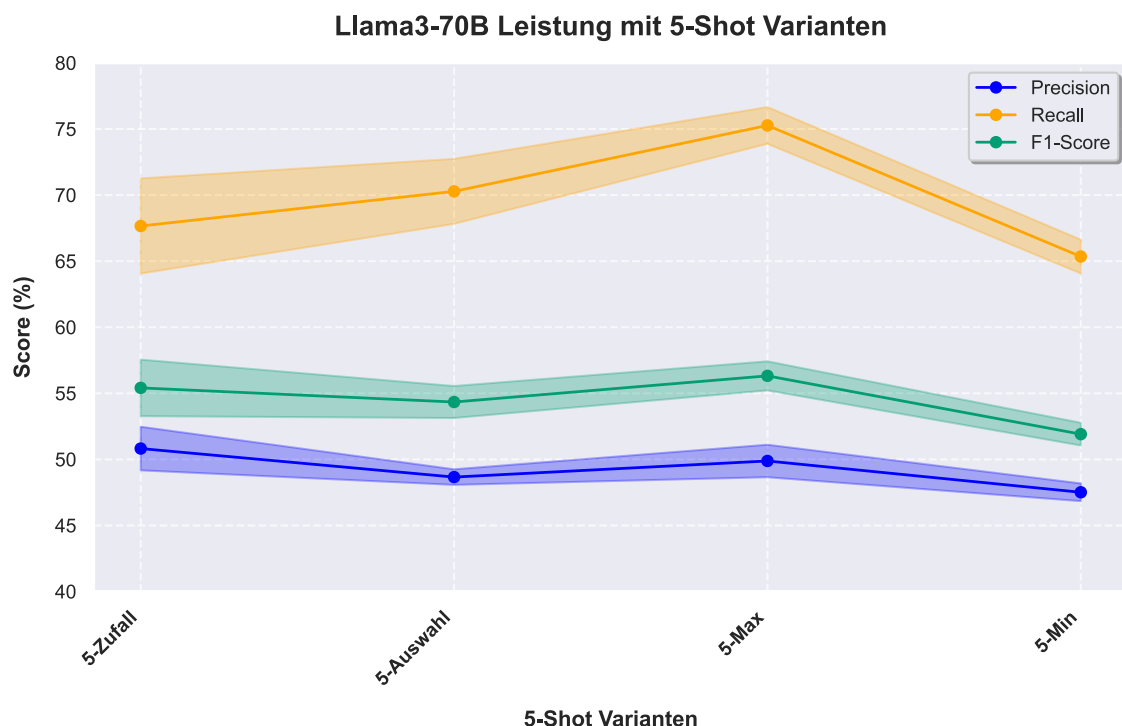


Abbildung 6.4.: Evaluation von Llama3-70B mit 5-Shot Varianten basierend auf gemittelten Precision, Recall und F1-Scores über 20 Versuche.

Die zufällige Auswahl erreicht eine mittlere Leistung mit einem F1-Score von 55.4%. Die gezielte Auswahl *5-Auswahl* wurde evaluiert, um zu simulieren, ob die intuitive Auswahl von kurzen Eignungskriterien mit möglichst vielen annotierten Operatoren die zufällige Auswahl von Beispielen verbessern kann. Dabei führt die Nutzung der *5-Auswahl* zu einem leicht niedrigeren F1-Score von 54.3%, zeigt aber einen höheren Recall von 70.3% und eine stabilere Precision gegenüber der zufälligen Auswahl. Die *5-Max* Methode erzielt die besten Gesamtergebnisse mit dem höchsten F1-Score von 56.3% und einem Recall von 75.3%. Die *5-Min* Methode zeigt die schwächste Leistung mit einem F1-Score von 51.9%. Die für den Vergleich der 5-Shot Methoden extrahierten Beispiele aus den LCT-Trainingsdaten mit den Kategorien *Ausgewählt*, *Max* und *Min* sind im Anhang in Tabelle A.2 aufgeführt. Die Zusammenfassung der Few-Shot Experimente, gemittelt über 20 Versuche, wird im Anhang in Tabelle D.1 dargestellt. Die Tabelle zeigt die Evaluation von 1 bis 5 Shots mit Llama3-70B unter Verwendung zufälliger Beispiele sowie die Auswertung verschiedener 5-Shot Szenarien. Die Evaluation von Few-Shot Szenarien mit Open-Source LLMs für die Operatorenextraktion zeigt, dass mehr Beispiele potenziell bessere Ergebnisse liefern, jedoch auch zu einer höheren Standardabweichung in den Ausgaben führen können. Die Verwendung ausgewählter Beispiele führt im Vergleich zu zufälligen Beispielen zu einem höheren Recall und geringerer Standardabweichung in allen Metriken.

6.1.5. Fine-Tuning

Die aktuelle State-of-the-Art Leistung für NER- und RE-Aufgaben auf dem LCT-Datensatz wird von BERT-Varianten erreicht, mit einem Fine-Tuning auf 80% der Korpusdaten. Um zu evaluieren, ob Large Language Models durch Fine-Tuning im Vergleich zu Prompting-Methoden verbessert werden können, wurden die Open-Source Modelle Llama3-8B und -70B einem Parameter-Efficient Fine-Tuning für die Aufgabe der Einfügung von Operatoren in Eignungskriterientexte unterzogen. Eine Hyperparameteroptimierung wurde dabei insbesondere beim Llama3-8B Modell vorgenommen. Der LCT-Korpus wurde zufällig in einen Trainings-, Evaluations- und Testdatensatz unterteilt. Der Evaluationsdatensatz wurde erstellt, um die Leistung des Modells während des Trainingsprozesses zu überwachen und zu evaluieren. Für das Fine-Tuning der LLMs wurden 723 Dateien (72%) als Trainingsdaten verwendet. Zusätzlich dienten 81 Dateien (8%) während des Trainings als Evaluationsdatensatz, und 202 Dateien (20%) wurden als unabhängiger Testdatensatz eingesetzt. Für ein Instruction Fine-Tuning wurde Prompt 2 zusammen mit den Trainingsdaten während des Fine-Tunings übergeben. Bei der Hyperparameteroptimierung wird auf den 202 Testdaten evaluiert und der F1-Score des Durchschnitts über die Operatoren AND, OR und NOT gebildet.

Hyperparameteroptimierung von Llama3-8B

Epochen	Rang r	Skalierungsparameter α	Lerning Rate η	LR-Scheduler	F1
Baseline Parameter					
3, 10	16	16	5e-5	linear	28.2
Optimierung von r					
10	16	256	5e-5	cosinus	74.5
	32				74.5
	64				75.7
	128				77.1
	256				76.3
	512				76.1
	2048				77.1
Optimierung von α					
10	128	16	5e-5	cosinus	69.3
		32			71.4
		64			71.6
		128			75.0
		256			77.1
		512			74.4
		1024			74.7
Optimierung der Learning Rate / LR-Scheduler					
10	128	2e-4	256	cosinus	50.7
		5e-6		cosinus	69.1
		5e-5		linear	75.5
Optimierung der Epochen					
20	128	256	5e-5	cosinus	74.6
30					77.0
40					75.6

Tabelle 6.6.: Hyperparameteroptimierung von Llama3-8B mit Fine-Tuning auf den LCT-Testdatensatz. Der F1-Score (F1) wird über den Durchschnitt der AND-, OR- und NOT-Operatoren angegeben.

Tabelle 6.6 zeigt die Ergebnisse der vollständigen Hyperparameteroptimierung für das Llama3-8B Modell mit Fine-Tuning wobei alle SFT- und LoRA-Hyperparameter aus Tabelle 5.2 über den gegebenen Wertebereich angepasst und evaluiert werden. Als Vergleich dient eine Baseline mit einem Fine-Tuning des 8B-Modells mit den Standardeinstellungen aus Tabelle D.5. Bei den LoRA-Parametern zeigte sich, dass eine Erhöhung des LoRA-Rangs r über 128 hinaus keine wesentliche Verbesserung der Ergebnisse brachte. Der höchste F1-Score von 77.1% wurde mit einem α Wert von 256 erreicht, wobei größere α -Werte keine weitere Steigerung bewirkten. Bei den SFT-Parametern erwies sich eine Lernrate von $\eta = 5 \times 10^{-5}$ in Kombination mit einem Cosinus LR-Scheduler als optimal. Es hat sich gezeigt, dass 10 Epochen für ein effektives Fine-Tuning ausreichen. Die Optimierung der Hyperparameter führte zu einer Steigerung des F1-Scores von 28.2% (*Baseline*) auf 77.1%. Eine Erhöhung der Epochenanzahl von 3 auf 10 bei den Standardeinstellungen brachte keine Verbesserung, was die Bedeutung der Parameteroptimierung hervorhebt. Diese Erkenntnisse bilden die Grundlage für das anschließende Fine-Tuning des Llama3-70B Modells. In initialen Tests zeigte das größere 70B-Modell mit den optimalen Einstellungen des 8B-Modells eine bessere Leistung wie mit den Standardeinstellungen.

Hyperparameteroptimierung von Llama3-70B

Bei Fine-Tuning Experimenten mit Llama3-70B auf dem LCT-Trainingsdatensatz wurden die Anzahl der Epochen sowie der LoRA-Rang und der LoRA-Skalierungsparameter α weiter optimiert. Das Ergebnis der Hyperparameteroptimierung ist in Tabelle 6.7 gezeigt.

Epochen	Rang r	Skalierungsparameter α	F1
10	128	256	77.3
		512	77.3
	256	256	78.0
		512	75.7
20	256	256	75.3
40	128	256	76.8
	256	256	75.1

Tabelle 6.7.: Hyperparameteroptimierung von Llama3-70B mit Fine-Tuning auf den LCT-Testdatensatz. Der F1-Score (F1) wird über den Durchschnitt der AND-, OR- und NOT-Operatoren angegeben.

Die Evaluation des Fine-Tunings von Llama3-70B zeigt, dass die besten Ergebnisse mit 10 Epochen, einem LoRA-Rang r von 256 und einem α Wert von 256 erzielt wurden. Diese Konfiguration erreichte den höchsten F1-Score von 78.0%. Eine Erhöhung der Epochenanzahl auf 20 oder 40 führt zu keiner Verbesserung der Ergebnisse. Ebenso zeigte die Variation des α -Parameters von 256 auf 512 keine konsistenten Leistungssteigerungen. Für das Fine-Tuning wurde ein Evaluationsdatensatz gebildet. Ein Fine-Tuning ohne diesen Evaluationsdatensatz führte zu keiner Verbesserung.

6.1.6. Vergleich von LLM-Methoden

Die Auswertung auf dem LCT-Testdatensatz umfasst neben den Llama3-Modellen mit Prompt-Methoden und Fine-Tuning auch weitere Open-Source Modelle wie Gemma2, Qwen2 und Llama2 welche über Zero-Shot und Few-Shot mit den Closed-Source Modellen GPT-4o, GPT-4o-mini und Claude-3.5-Sonnet verglichen wurden. Initiale Zero-Shot- und Few-Shot Versuche mit Llama3 zeigten, dass Prompt 2 bessere Ergebnisse liefert, daher wurde diese als Grundlage für weitere Auswertungen verwendet. Um die Effektivität von Few-Shot Lernmethoden unter Verwendung von Prompt 2 zu bewerten, wurde ein Vergleich mit einer Baseline *Naive Greedy Match* durchgeführt. Bei Few-Shot Methoden wurden k -ausgewählte Beispiele mit den Werten $k \in 3, 5, 15$ evaluiert, wobei die maximale Anzahl der Beispiele modellabhängig begrenzt war. Das Llama2-70B Modell konnte maximal 3-Shot verarbeiten, während Llama3-8B und -70B bis zu 5-Shot prozessieren

konnten. Das Qwen2-72B Modell mit einer größeren Kontextlänge konnte mit den Modellen GPT-4o, GPT-4o-mini und Claude-3.5-Sonnet in 5-Shot und 15-Shot Szenarien getestet werden. Die ausgewählten LLMs werden mit den aktuellen State-of-the-Art Modellen SciBERT und R-BERT in Kombination mit SciBERT verglichen. Diese Referenzmodelle wurden einem Fine-Tuning auf 80% zufällig gewählten, annotierten LCT-Daten unterzogen. Die Leistung der angewandten LLM-Methoden wurde anhand von Precision, Recall und F1-Score für die Durchschnittswerte der Operatorenextraktion evaluiert. Die Ergebnisse der Auswertung auf dem LCT-Testdatensatz mit ausgewählten LLMs sind in Tabelle 6.8 dargestellt.

Modell	Parameter	Shots	Ausgabefehler ($\bar{x}$, %)	P	R	F1	Δ F1 SOTA	Δ F1 Baseline
State-of-the-Art								
SciBERT	110M	804	–	71.2	81.4	75.9	–	↑25.0
R-BERT + SciBERT		804	–	72.2	78.2	75.0	↓0.9	↑24.1
Baseline								
Naive Greedy Match	–	–	–	45.0	74.4	50.9	↓25.0	–
Open-Source								
			Zero-Shot					
Llama3	8B	0	40.4	19.2	59.6	27.5	↓48.4	↓23.4
	70B	0	3.9	43.4	69.5	49.7	↓26.2	↓1.2
Gemma2	27B	0	2.8	37.7	50.6	39.8	↓36.1	↓11.1
Qwen2	72B	0	8.6	28.3	46.0	31.5	↓44.4	↓19.4
			Few-Shot					
Llama2	70B	3	50.8	15.6	17.8	15.3	↓60.6	↓35.6
Llama3	8B	5	19.1	27.2	49.2	33.5	↓42.4	↓17.4
	70B	5	1.5	49.1	73.8	55.8	↓20.1	↑4.9
Gemma2	27B	5	4.7	36.2	60.9	41.7	↓34.2	↓9.2
Qwen2	72B	5	4.9	43.8	63.1	49.6	↓26.3	↓1.3
		15	8.8	51.7	65.8	56.3	↓19.6	↑5.4
			Fine-Tuning					
Llama3	8B	723	0.4	79.0	75.6	77.1	↑1.2	↑26.2
	70B	723	0.5	72.6	84.5	78.0	↑2.1	↑27.1
Closed-Source								
			Zero-Shot					
GPT-4o	k.A.	0	1.5	38.8	69.6	46.8	↓29.1	↓4.1
			Few-Shot					
GPT-4o	k.A.	5	1.3	43.3	79.4	52.9	↓23.0	↑2.0
		15	1.4	46.6	77.8	55.2	↓20.7	↑4.3
GPT-4o mini	k.A.	5	1.1	49.0	70.3	53.5	↓22.4	↑2.6
		15	1.6	51.3	68.9	55.6	↓20.3	↑4.7
Claude-3.5-Sonnet	k.A.	5	2.2	47.7	79.7	56.5	↓19.4	↑5.6
		15	2.9	54.1	74.7	60.7	↓15.2	↑9.8

Tabelle 6.8.: Auswertung der Operatorenextraktion von LLMs mit Few-Shot und Fine-Tuning Methoden auf dem LCT-Testdatensatz. Evaluiert wird über die durchschnittlichen Metriken, Precision (P), Recall (R) und F1-Score (F1) und dem durchschnittlichen Ausgabefehler in Prozent.

Die Differenz der F1-Scores der LLMs zum F1-Score von SciBERT wird als $\Delta F1$ -SOTA angegeben. Die Bewertung der LLM-Leistung erfolgt farblich. Eine grüne Differenz zeigt an, dass das LLM besser als das SOTA-Modell abschneidet, was bei einem $\Delta F1$ -Score > 0 der Fall ist. In orange werden Leistungen markiert, wobei $\Delta F1$ zwischen -22 und -5 liegt, also die LLM-Ausgabe bis zu 22% schlechter als das SOTA-Modell abschneidet. Fällt der $\Delta F1$ -Score unter -22, wird die Diffe-

renz rot markiert, wobei das Modell unterdurchschnittlich performt. Die Differenz zum F1-Score des Regex-Parsers ist als $\Delta F1\text{-Baseline}$ angegeben. Ein Modell wird positiv bewertet, wenn es eine bessere Leistung als der regelbasierte Parser erzielt. Für die Large Language Models wird zusätzlich der Ausgabefehler dargestellt. Dieser gilt als akzeptabel, wenn er unter 3% liegt, wobei davon auszugehen ist, dass keine wichtigen Informationen verloren gehen. Bei einer Halluzinationsrate zwischen 3% und 5% kann das Modell noch sinnvoll eingesetzt werden. Übersteigt der Fehler 5%, kann die Korrektheit der ausgegebenen Informationen nicht mehr gewährleistet werden. Die Zero-Shot Evaluation unter Verwendung von Prompt 2 zeigt, dass Llama3-70B mit einem F1-Score von 49.7% über alle Operatoren eine höhere Leistung erzielt als die Open-Source Modelle Gemma2 und Qwen2. Llama3-70B übertrifft auch GPT-4o, welches einen F1-Score von 46.8% erreicht, weist jedoch mit 3.9% eine höhere Ausgabefehlerrate auf im Vergleich zu GPT-4o mit 1.5%. Das Llama3-8B Modell hatte mit Zero-Shot und Few-Shot hohe Ausgabefehlerraten von 40% und 19.1%. Gemma2-27B hatte mit Zero-Shot eine bessere Leistung als das größer Qwen2-72B Modell und den geringsten Ausgabefehler unter den Open-Source Zero-Shot Modellen. Das Vorgängermodell Llama2-70B konnte bei Zero-Shot Versuchen mit Prompt 2 keine verwertbaren Ausgaben erzeugen. Aufgrund der geringeren Kontextlänge im Vergleich zu aktuellen LLMs wie Llama3 konnte Llama2 nur bis maximal 3-Shot genutzt werden. Dabei war der Ausgabefehler mit 50.8% am höchsten unter den LLM-Modellen und die Leistung lag 35.6% unter dem F1-Score des Regex-Parsers. Mit 5-Shot zeigt Llama3-70B unter den Open-Source Modellen mit einem F1-Score von 55.8% und einer Fehlerrate von 1.5% die beste Leistung. Unter den Closed-Source Modellen performt Claude-3.5-Sonnet mit 5-Shot als einziges Modell leicht besser als Llama3-70B, weist aber einen leicht höheren Ausgabefehler auf. Auch im 15-Shot weist Claude-3.5-Sonnet unter allen LLMs mit einem F1-Score von 60.7% die beste Leistung auf, wobei es um 15.2% unter der SOTA-Leistung liegt. Das Qwen2-Modell war das einzige Open-Source LLM, was aufgrund der Kontextlänge ein 15-Shot Szenario ermöglichte. Mit erhöhter Anzahl von Beispielen stieg der Ausgabefehler des Qwen2-Modells auf 8.8% an und das Modell schneidet etwas besser als Llama3-70B mit 5-Shot ab. Über ein Fine-Tuning auf dem LCT-Trainingsdatensatz haben die Llama3-Modelle mit 8B und 70B Parametern die beste Leistung erzielt mit F1-Scores, die um 1.2% und 2.1% höher als beim SOTA-Modell SciBERT liegen.

Vergleich ausgewählter Modelle für die Operatorenextraktion

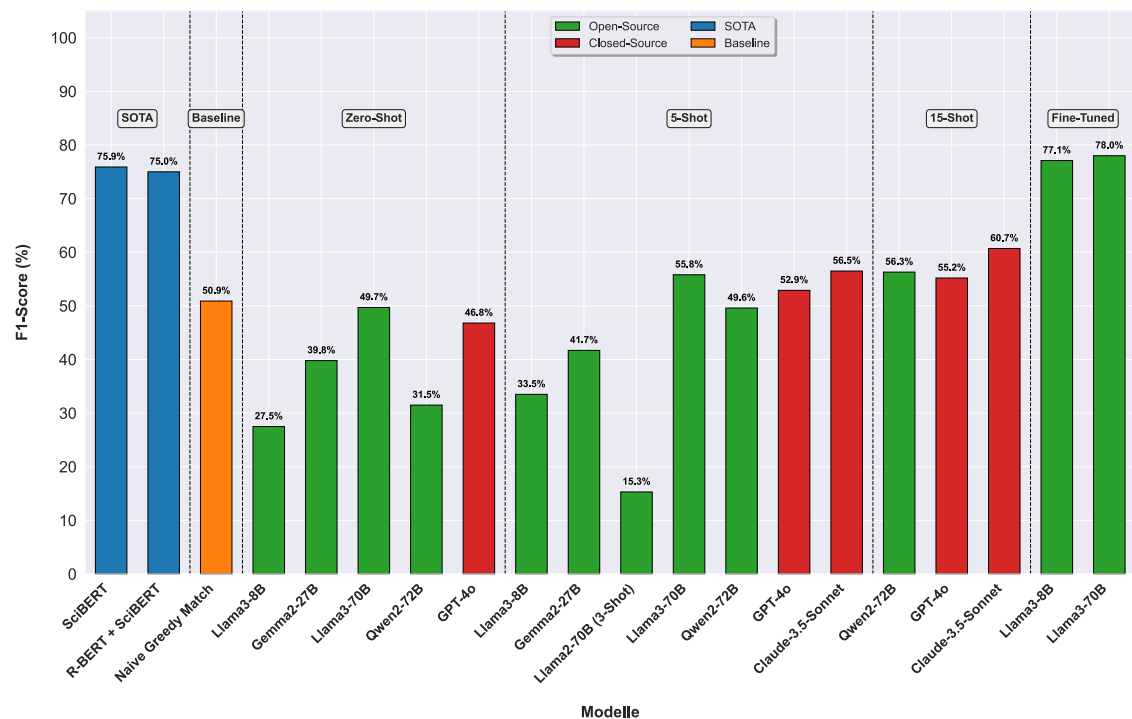


Abbildung 6.5.: F1-Score der Operatorenextraktion mit aktuellen LLMs mit Few-Shot und Fine-Tuning Methoden im Vergleich zu SOTA-Modellen und einem Regex-Parser.

Abbildung 6.5 fasst die Auswertung auf dem LCT-Datensatz zusammen, indem die erreichten F1-Scores bei der Operatorenextraktion der LLMs mit den SOTA-Modellen SciBERT und R-BERT + SciBERT verglichen werden. Zudem wird die Leistung dem regelbasierten Parser *Naive Greedy Match* gegenübergestellt. Die Ergebnisse der verwendeten Large Language Models sind nach Zero-Shot, Few-Shot und Fine-Tuning gruppiert. Zusätzlich wurden die Open-Source Modelle nach ihrer Parametergröße aufsteigend angeordnet, um den potenziellen Zusammenhang zwischen Modellgröße und Leistungsfähigkeit zu analysieren. Bei den verwendeten Closed-Source Modellen wie GPT-4o ist die Parameterzahl nicht bekannt. Da aber bereits Vorgängermodelle wie GPT-3 mehrere Milliarden-Parameter aufweisen, liegt die Parameteranzahl weit über den Open-Source Modellen (Brown et al., 2020). Bei der Zero-Shot-Evaluation zeigt sich ein genereller Trend zur Verbesserung des F1-Scores mit zunehmender Modellgröße, wobei Llama3-70B die beste Leistung erzielt. Allerdings spiegeln die Ergebnisse von Qwen2 mit 72B-Parametern und GPT-4o diesen Trend nicht durchgängig wider. Die Leistung von Qwen2 liegt unter der des kleineren Modells Gemma2, während GPT-4o hinter dem Llama3-70B Modell zurückbleibt. Llama3-70B weist als einziges Zero-Shot Modell eine ähnliche Leistung wie der Regex-Parser *Naive Greedy Match* auf, wobei es mit 3.9% einen hohen Ausgabefehler aufweist. Mit 5-Shot übertrifft das Llama3-70B Modell die Baseline, mit einem geringen Ausgabefehler von 1.5%. Das Closed-Source Modell Claude-3.5-Sonnet kann mit 15-Shot die Leistung auf 60.7% steigern, hat dabei aber einen höheren Ausgabefehler. Die Few-Shot Methoden mit größeren aktuellen Modellen liegen dabei 15 - 25% unter der State-of-the-Art Leistung. Ein Fine-Tuning von Open-Source LLMs kann die Leistung bei der Operatorenextraktion im Vergleich zu SOTA-Modellen verbessern. Während das Llama3-8B Modell mit Zero-Shot und Few-Shot Methoden im Vergleich zu den weiteren untersuchten LLMs eine geringere Leistung zeigt, performt es mit Fine-Tuning, mit einem F1-Score von 77.1%, besser als State-of-the-Art Modelle. Ein Fine-Tuning des größeren 70B-Modells verbessert die Leistung dabei nicht wesentlich.

Um einen detaillierten Vergleich der Operatorenextraktion zwischen dem SOTA-Modell SciBERT und dem **Llama3-8B** Modell zu ermöglichen, werden in Tabelle 6.9 die Precision-, Recall- und F1-Scores für die einzelnen Operatoren sowie den Durchschnitt über alle Operatoren gegenübergestellt. Die Ergebnisse sind als Mittelwerte über 20 Versuche mit 5-Shot (Max) und dem 8B-Modell mit Fine-Tuning dargestellt, wobei die Standardabweichung ebenfalls angegeben ist.

	Metrik	SOTA	5-Shot (Max) Δ		Fine-Tuned Δ	
AND	P	54.1	10.7 $\pm$ 1.1	↓ 43.4	67.3 $\pm$ 1.0	↑ 13.2
	R	60.0	51.7 $\pm$ 2.5	↓ 8.3	57.7 $\pm$ 1.2	↓ 2.3
	F1	56.9	17.8 $\pm$ 1.7	↓ 39.1	62.1 $\pm$ 1.0	↑ 5.2
OR	P	85.1	46.1 $\pm$ 4.9	↓ 39.0	86.1 $\pm$ 0.2	↑ 1.0
	R	93.2	63.2 $\pm$ 5.1	↓ 30.0	87.3 $\pm$ 0.4	↓ 5.9
	F1	89.0	53.2 $\pm$ 4.9	↓ 35.8	86.7 $\pm$ 0.2	↓ 2.3
NOT	P	74.3	13.7 $\pm$ 2.8	↓ 60.6	80.4 $\pm$ 0.3	↑ 6.1
	R	91.0	23.5 $\pm$ 4.4	↓ 67.5	79.8 $\pm$ 0.6	↓ 11.2
	F1	81.8	17.3 $\pm$ 3.4	↓ 64.5	80.1 $\pm$ 0.3	↓ 1.7
$\emptyset$	P	71.2	23.5 $\pm$ 2.9	↓ 47.7	77.9 $\pm$ 0.5	↑ 6.7
	R	81.4	46.1 $\pm$ 4.0	↓ 35.3	74.9 $\pm$ 0.7	↓ 6.5
	F1	75.9	29.4 $\pm$ 3.3	↓ 46.5	76.3 $\pm$ 0.5	↑ 0.4

Tabelle 6.9.: Vergleich der Leistung der Operatorenextraktion zwischen dem SOTA-Modell SciBERT und Llama3-8B mit 5-Shot (Max) und Fine-Tuning. Die Scores sind als Mittelwerte über 20 Versuche mit Standardabweichung angegeben. Zusätzlich sind die Differenzen (Δ) der Metriken zwischen SciBERT und den LLMs aufgeführt.

Im initialen Zero-Shot Versuch erzielte das Llama3-8B Modell einen durchschnittlichen F1-Score von 27.5% über alle Operatoren, was deutlich unter der SOTA-Leistung liegt und deshalb nicht weiter analysiert wurde. Eine optimierte 5-Shot (Max) Variante, bei der Beispiele mit der größten annotierten Operatorenanzahl verwendet wurden, führte zu keiner wesentlichen Verbesserung, mit einem durchschnittlichen F1-Score von 29.4% über 20 Versuche. Bei der Erkennung von AND-

und NOT-Operatoren weist das Modell eine geringe Precision auf. Die Standardabweichung ist insbesondere bei OR-Operatoren mit über 4% in allen drei Metriken hoch. Das Fine-Tuning verbesserte die Leistung und reduzierte die Standardabweichungen im Vergleich zum Few-Shot Szenario deutlich. Im Durchschnitt erreicht das Llama3-8B Modell mit Fine-Tuning einen F1-Score von 76.3%, was leicht über dem SOTA-Modell SciBERT mit 75.9% liegt. Allerdings könnte unter Berücksichtigung der Standardabweichung im ungünstigsten Fall Llama3-8B mit einem F1-Score von 75.8% minimal hinter SciBERT zurückfallen.

Darüber hinaus wurde das **Llama3-70B** Modell, welches mit Fine-Tuning die besten Ergebnisse erzielte, in Zero-Shot, 5-Shot (Max) und Fine-Tuning Szenarien evaluiert. Die Leistungen dieses Modells über 20 Versuche sind in Tabelle 6.10 dargestellt.

	Metrik	SOTA	0-Shot	Δ	5-Shot (Max)	Δ	Fine-Tuned	Δ
AND	P	54.1	16.4 $\pm$ 0.2	↓ 37.7	20.1 $\pm$ 0.5	↓ 34.0	58.2 $\pm$ 0.6	↑ 4.1
	R	60.0	73.8 $\pm$ 1.2	↑ 13.8	80.8 $\pm$ 1.9	↑ 20.8	70.5 $\pm$ 0.5	↑ 10.5
	F1	56.9	26.8 $\pm$ 0.4	↓ 30.1	32.2 $\pm$ 0.7	↓ 24.7	63.8 $\pm$ 0.5	↑ 6.9
OR	P	85.1	67.9 $\pm$ 0.6	↓ 17.2	72.5 $\pm$ 0.5	↓ 12.6	84.4 $\pm$ 0.2	↓ 0.7
	R	93.2	69.4 $\pm$ 0.8	↓ 23.8	79.8 $\pm$ 2.4	↓ 13.4	91.6 $\pm$ 0.2	↓ 1.6
	F1	89.0	68.6 $\pm$ 0.6	↓ 20.4	76.0 $\pm$ 1.2	↓ 13.0	87.9 $\pm$ 0.2	↓ 1.1
NOT	P	74.3	46.3 $\pm$ 1.2	↓ 28.0	56.9 $\pm$ 3.2	↓ 17.4	72.5 $\pm$ 0.4	↓ 1.8
	R	91.0	68.4 $\pm$ 1.5	↓ 22.6	65.1 $\pm$ 2.0	↓ 25.9	90.1 $\pm$ 0.8	↓ 0.9
	F1	81.8	55.2 $\pm$ 1.2	↓ 26.6	60.6 $\pm$ 1.8	↓ 21.2	80.4 $\pm$ 0.5	↓ 1.4
$\emptyset$	P	71.2	43.5 $\pm$ 0.7	↓ 27.7	49.8 $\pm$ 1.4	↓ 21.4	71.7 $\pm$ 0.4	↑ 0.5
	R	81.4	70.5 $\pm$ 1.2	↓ 10.9	75.2 $\pm$ 2.1	↓ 6.2	84.1 $\pm$ 0.5	↑ 2.7
	F1	75.9	50.2 $\pm$ 0.9	↓ 25.7	56.3 $\pm$ 1.5	↓ 19.6	77.4 $\pm$ 0.4	↑ 1.5

Tabelle 6.10.: Vergleich der Leistung der Operatorenextraktion zwischen dem SOTA-Modell SciBERT und Llama3-70B mit Zero-Shot, 5-Shot (Max) und Fine-Tuning. Die Scores sind als Mittelwerte über 20 Versuche mit Standardabweichung angegeben. Zusätzlich werden die Differenzen (Δ) der Metriken zwischen SciBERT und den LLMs aufgeführt.

Das Llama3-70B Modell zeigt bereits im Zero-Shot-Szenario mit einem mittleren F1-Score von 50.2% eine bessere Leistung als das 8B Modell mit 5-Shot (Max). Die Standardabweichungen sind in allen Lernmethoden signifikant geringer als bei Llama3-8B. Die 5-Shot (Max) Methode führte zu einer moderaten Verbesserung von 5.1 Prozentpunkten gegenüber Zero-Shot, mit einem durchschnittlichen F1-Score von 56.5%. Ein Fine-Tuning des 70B-Modells verbesserte die Extraktion des Operators AND, mit höheren Precision- und Recall-Scores im Vergleich zu SciBERT. Bei der Erkennung von OR- und NOT-Operatoren ist die Leistung etwas geringer. Das Modell 70B mit Fine-Tuning ist selbst im schlechtesten Fall mit einem F1-Score von 77% für alle Operatoren besser als das SOTA-Modell.

Temperatur-Evaluation

Um den Einfluss des Temperaturparameters auf die Leistung des Llama3-70B Modells mit In-Context Learning- und Fine-Tuning Methoden zu evaluieren, wurde die Temperatur in Schritten von 0.1 bis 1.0 variiert. Die Ergebnisse dieser Untersuchung sind in Abbildung 6.6 dargestellt. Während in den vorherigen Zero-Shot und Few-Shot Szenarien die Temperatur der ausgewählten LLMs auf 0.5 festgelegt und bei den Modellen mit Fine-Tuning der Standardwert von 1.0 beibehalten wurde, ermöglicht diese Variation eine Analyse des Temperatureinflusses auf die Modellleistung. Für jede Temperaturvariation wurde ein einzelner Versuch ausgewertet, wobei der F1-Score über alle drei Operatoren AND, OR und NOT ermittelt wurde. Die Abbildung zeigt, dass Änderungen der Temperatur zwar zu Schwankungen in den F1-Scores führen, jedoch keine signifikanten Verbesserungen oder Verschlechterungen der Leistung bewirken. Im Anhang in Tabelle D.2 sind die detaillierten Precision-, Recall- und F1-Scores für die verschiedenen Temperaturen aufgeführt, wobei die jeweils besten Werte markiert sind.

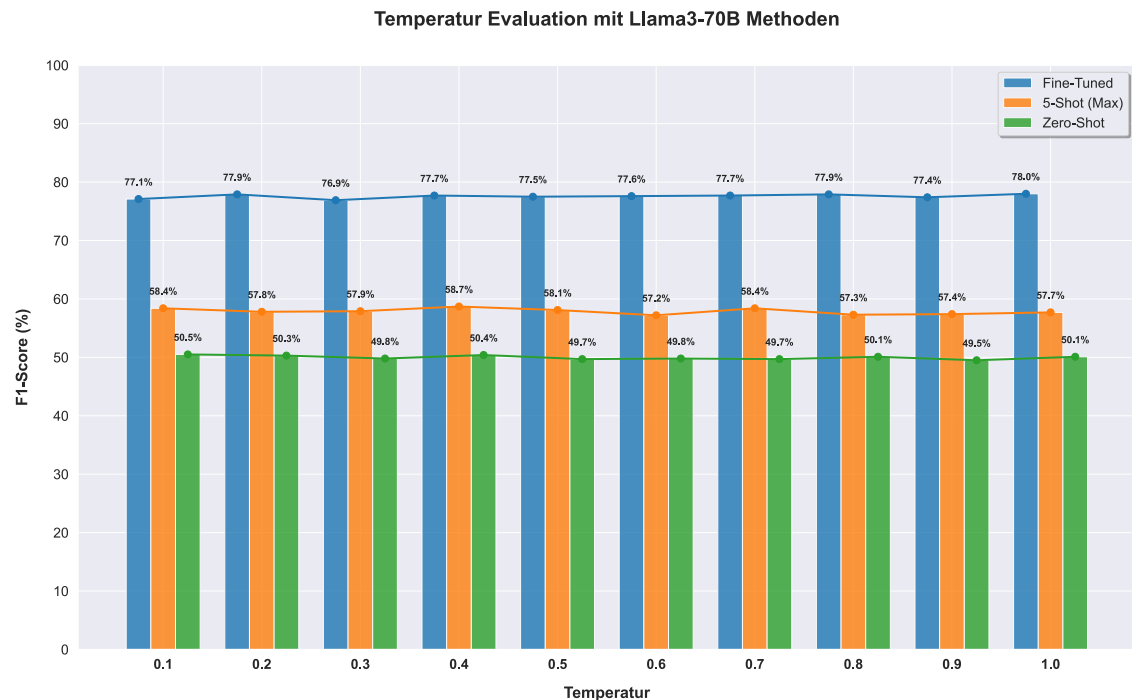


Abbildung 6.6.: Evaluation des Temperatur-Parameters für das Llama3-70B Modell unter Anwendung von Zero-Shot, 5-Shot (Max) und Fine-Tuning Methoden.

6.1.7. Ressourcen- und Zeitaufwand

Die Iteration über den LCT-Testdatensatz mit Llama3-8B erforderte zwei NVIDIA A100 GPUs mit einem VRAM-Bedarf von bis zu 100GB für die Inferenz. Durch den Einsatz von rsLoRA und QLoRA war ein effizientes Fine-Tuning selbst mit einem großen Rang von 2048 und über 30 Epochen in 114 Minuten auf einer GPU möglich. Ein Fine-Tuning des Llama3-8B Modells mit den optimalen Parametereinstellungen aus Tabelle 6.6 konnte in weniger als einer halben Stunde abgeschlossen werden, mit einem maximalen VRAM-Bedarf von 20GB. Für die Inferenz des größeren Llama3-70B Modells wurden vier NVIDIA A100 GPUs mit einem Bedarf von bis zu 260GB VRAM benötigt. Mit Zero-Shot dauerte die Inferenz über den LCT-Testdatensatz mit 202 Dateien 1,2 Stunden, während mit 5-Shot etwa 4 bis 5 Stunden benötigt wurden. Das Fine-Tuning auf einer GPU mit der Konfiguration der höchsten Leistung (10 Epochen, $r = \alpha = 256$) benötigte 3 Stunden GPU-Zeit, während ein Training mit 40 Epochen 12 Stunden beanspruchte, ohne bessere Ergebnisse zu erzielen. Die Nutzung proprietärer Modelle über APIs ermöglicht einen direkten Zugriff ohne Clusterressourcen, ist jedoch kostenpflichtig. Die entsprechenden Kosten für die Few-Shot Generierung von 202 Texten sind in Tabelle 6.11 dargestellt.

Modell	Kosten
GPT-4o	\$5.60
GPT-4o-mini	\$0.35
Claude-3.5-Sonnet	\$1.74

Tabelle 6.11.: Kosten proprietärer LLMs bei der 5-Shot Generierung von 202 Textdateien (Stand: 25.09.2024)

GPT-4o mit Zero-Shot benötigt für eine Iteration über den Testdatensatz 23 Minuten bei Kosten von \$1.27. Mit 5-Shot dauert die Verarbeitung eine Stunde und kostet \$5.60. GPT-4o-mini mit 5-Shot ist mit \$0.35 günstiger. Die Kosten für GPT-Modelle haben während der Erstellung dieser Arbeit abgenommen, die Nutzung von GPT-4o mit 5-Shot kostete im Mai 2024 etwa \$13 (OpenAI, 2023).

6.1.8. Auswertung des Chia-Korpus

Der Chia-Korpus, in Unterabschnitt 4.2 beschrieben, umfasst unter 12 Relationstypen auch Annotationen für die logischen Operatoren AND, OR und NOT. Es wurden 1000 Textdateien mit Eignungskriterien im BRAT-Format annotiert (Kury et al., 2020). Der Chia-Datensatz wurde analog zu den LCT-Daten aufbereitet, indem Operatoren basierend auf den Annotationen in die Kriterientexte eingefügt und manuell validiert wurden, um die Generalisierungsfähigkeit von Llama3-70B zu evaluieren. Es soll evaluiert werden, ob die für den LCT-Korpus erstellte Prompt 2 (Abbildung 5.8) auch generalisiert auf weiteren Korporas mit Few-Shot angewandt werden kann. Zudem soll getestet werden, wie das Llama3-70B Modell mit Fine-Tuning auf LCT-Daten auf anderen Korporas mit annotierten Eignungskriterien performt. Die in Tabelle 4.2 aufgeführten BERT-Evaluationen auf dem Chia-Datensatz beschränkten sich auf NER-Aufgaben, wodurch ein direkter Vergleich der Operatorenextraktion mit dem SOTA-Modellen nicht möglich war. Das Llama3-70B Modell wurde auf allen 1000 Dateien des Chia-Korpus für die Operatorenextraktion evaluiert. In einem Zero-Shot Szenario wurde dem Modell lediglich die Prompt 2 für die Operatorenextraktion auf den Chia-Datensatz mitgegeben. Zusätzlich wurde das Llama3-70B Modell mit Fine-Tuning auf dem LCT-Korpus evaluiert, wobei während der Inferenz auch die Prompt 2 mitgegeben wurde. Die Ergebnisse der Auswertung über die Precision, Recall und F1-Score der gemittelten AND-, OR- und NOT-Evaluation des Chia-Korpus sind in Abbildung 6.7 gezeigt. Als Vergleich dient der Regex-Parser, welcher auch basierend auf der Prompt 2 entwickelt wurde.

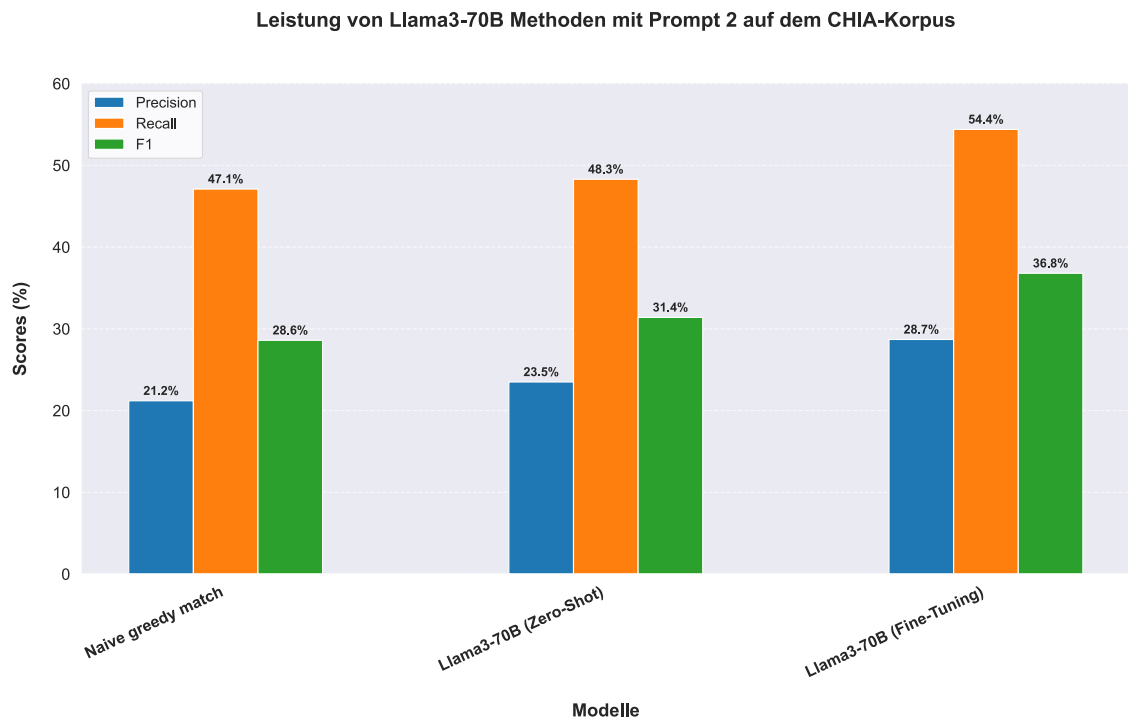


Abbildung 6.7.: Evaluation von Llama3-70B auf dem Chia-Korpus mittels Zero-Shot und Fine-Tuning auf LCT-Daten unter Verwendung von Prompt 2.

Die Evaluation von Llama3-Modellen auf dem Chia-Korpus mit dem Ausgabefehler ist zusätzlich im Anhang in Tabelle D.3 dargestellt. Bei der Evaluierung auf dem gesamten Chia-Korpus erzielte das Llama3-70B Modell mit Fine-Tuning auf LCT-Daten, einen F1-Score von 36.8% und einen Recall von 54.4% und übertraf damit Llama3 mit Zero-Shot. Die Leistung des Modells mit Fine-Tuning ist etwa 10% besser als die des Regex-Parsers. Der Ausgabefehler von Llama3 mit Fine-Tuning ist mit 0.6% gering, während der Ausgabefehler von 4.3% mit Zero-Shot deutlich höher ist. Alle untersuchten Methoden übertreffen die Leistung des Regex-Parsers. Während Llama3-70B mit Zero-Shot auf dem LCT-Testdatensatz noch einen F1-Score von 49.7% erreicht, sinkt die Leistung auf dem Chia-Korpus um 18.3% auf 31.4% F1-Score. Das Fine-Tuning Modell erzielt auf den LCT-Daten einen F1-Score von 78%, verliert jedoch 41.2% an Leistung auf dem Chia-Korpus.

6.2. Evaluation der direkten Strukturierung

Über eine direkte Strukturierung werden Large Language Models mit Few-Shot und Fine-Tuning Methoden dazu angeleitet, Eignungskriterien über die Extraktion von logischen Operatoren direkt in ein JSON-Dateien mit Abstract Syntax Tree-Struktur zu prozessieren. Es sollen dabei zusätzlich Entitäten aus den Kriterientexten erkannt werden und neben dem Ursprungstext übernommen werden. Die Grundlage für die Evaluation der AST-Strukturierung mit LLMs bildet der LCT-Datensatz. In diesem wurden die Eignungskriterien in Ein- und Ausschlusskriterien getrennt und zufällig auf einen Trainingsdatensatz mit 1.608 Dateien (80%) und einen Testdatensatz mit 404 Dateien (20%) verteilt. Ein- und Ausschlusskriterien mit identischer NCT-Nummer wurden ausschließlich einem der beiden Datensätze zugeordnet. Durch die getrennte Betrachtung der Kriterien sind die Datensätze doppelt so groß wie in vorherigen Untersuchungen zur Methode der indirekten Strukturierung aus Abschnitt 6.1. Zur Evaluation wurden die Open-Source LLMs Llama3-70B und Qwen2-72B im Few-Shot Verfahren auf dem LCT-Testdatensatz evaluiert und mit GPT-4o verglichen. Zusätzlich wurde ein Fine-Tuning von Llama3-70B auf dem in JSON prozessierten LCT-Testdatensatz durchgeführt. Die Kriterien wurden aufgrund der Kontextlängenbeschränkung von Open-Source LLMs wie Llama3 mit 4000 Token separiert, da die vollständige JSON-Verarbeitung langer Kriterien nicht möglich war. Die JSON-Dateien in AST-Struktur können mehrere Hundert Zeilen umfassen.

Evaluierung der JSON-Ausgabe

Abstract Syntax Trees (ASTs) besitzen eine flexible Struktur, die je nach Implementierung und Anwendungsfall variieren kann. Diese Flexibilität führt dazu, dass syntaktisch identische Ausdrücke durch unterschiedlich aufgebaute ASTs dargestellt werden können. Beispielsweise kann die Baumstruktur nach links oder rechts geneigt sein, ohne die zugrunde liegende Bedeutung zu verändern. Dies erschwert direkte Vergleiche zwischen den im Goldstandard verarbeiteten ASTs im JSON-Format und den von LLMs erzeugten AST-Strukturen. Deshalb wird die Auswertung der mithilfe der logischen Operatoren erstellten AST-Struktur im JSON-Format inklusive der Entitätsextraktion in die folgenden Auswertungsbereiche gegliedert:

1. **Validierung der JSON-Struktur:** Es wird evaluiert, ob es sich bei der Ausgabe der erzeugten JSON-Dateien um valides JSON handelt und ob das JSON-Schema als AST valide aufgebaut ist. Ob es sich bei der Ausgabe um valides JSON handelt und wird über die Prüfung auf korrekte Platzierung und Schließung der Klammern `{}` sowie adäquate Verwendung von Kommas und String-Zeichen gemäß JSON-Spezifikation gewährleistet. Das valide JSON-Schema wird über die Verifizierung der korrekten Implementierung von *left*- und *right*-Verbindungen bei AND- und OR-Knoten, sowie die ausschließliche Verbindung von NOT-Operatoren über *left*-Äste sichergestellt. Die Summe der beiden Fehlertypen wird in der Auswertung als **JSON-Fehler** bezeichnet.
2. **Evaluation der Operatoren:** Die vom LLM erzeugte JSON-Datei wird über einen rekursiven Parser in ein Textformat zurückkonvertiert, wobei die Operatoren an den identifizierten Positionen gemäß dem Ursprungstext platziert werden. Die Operatoren OR, NOT können so gemäß den Kriterien aus Abschnitt 6.1 evaluiert werden. Die Berechnung der AND-Operatoren erfolgt nach einem ähnlichen Prinzip, wobei die detaillierte Berechnungsvorschrift in Unterabschnitt 6.2.3 erläutert wird. Durch diese Rückkonvertierung können die Operatoren evaluiert werden, ohne die Struktur des ASTs berücksichtigen zu müssen.
3. **Evaluierung der Entitätsextraktion:** Die Bewertung der Entitätsextraktion erfolgt separat für jeden Entitätentyp. Dabei werden die in jeder Ausgabedatei extrahierten Entitäten mit den entsprechenden Entitäten in den zugehörigen annotierten Referenzdateien verglichen. Die Gesamtleistung der LLMs bei der Entitätsextraktion wird ermittelt, indem die True Positives (TP), False Positives (FP) und False Negatives (FN) über alle Dateien und Entitätstypen hinweg aggregiert werden.

6.2.1. Few-Shot

Für die Strukturierung im JSON-Format wurde eine Strukturprompt konzipiert, welche im Anhang in Abbildung C.3 dargestellt ist. Die Prompt weist das LLM an, Eignungskriterien als JSON-Datei im Composite Pattern auszugeben, basierend auf der Extraktion der logischen Operatoren AND, OR und NOT. Die Extraktionsanweisung erfolgt durch das Einsetzen von Prompt 1 (Abbildung C.1) oder Prompt 2 (Abbildung 5.8) in den oberen Teil der Strukturprompt. Dabei wird der Satzteil, die Kriterien in <EDITED_CRITERIA> zu schreiben entfernt. Weiterhin wird die aufzubauende AST-Struktur über Knoten, Verbindungen mit *left* und *right* sowie die Extraktion der Textabschnitte als *raw_text* beschrieben. Für die Entitätsextraktion werden dem Modell ausschließlich die 50 Entitätstypen des LCT-Datensatzes ohne Beschreibung vorgegeben. Abschließend wird die für die JSON-Generierung notwendige Syntax erläutert.

Für den Few-Shot Ansatz zur Strukturierung wurden den LLMs sowohl die Ein- und Ausschlusskriterien als auch deren strukturierte Form im JSON-Format aus dem Goldstandard als Beispiele übergeben. Diese strukturierte Form beinhaltete auch die in Abschnitt 6.2.5 vorgestellten Entitäten des LCT-Datensatzes. Dabei wurden die Open-Source Modelle Llama3-70B und Qwen2-72B in einem 2-Shot Szenario evaluiert, wobei zwei Beispiele im JSON-Format als Kontext dienten. Als Vergleichsmodell wurde GPT-4o herangezogen. Aufgrund der Kontextlängenbeschränkung von 4000 Token bei Llama3-70B konnte mit diesem Modell maximal ein 10-Shot Szenario realisiert werden. Die Beispiele wurden manuell so kurz wie möglich gewählt, indem JSON-Dateien mit einer Länge von 80 bis 100 Zeilen verwendet wurden, um die Kontextlänge der Modelle nicht zu überschreiten.

6.2.2. Fine-Tuning

Zur Optimierung der Modellleistung durch eine größere Anzahl von Lernbeispielen wurde ein Fine-Tuning des Llama3-70B auf dem 80% Trainingsdatensatz des LCT-Korpus durchgeführt, wobei Einschlusskriterien und Ausschlusskriterien separat übergeben wurden. Mit dem Ziel, die Trainingsdatenmenge zu maximieren, wurde auf die Bildung eines separaten Evaluationsdatensatzes während des Fine-Tunings verzichtet. Stattdessen wurde das Modell auf allen 1608 Dateien (80%) im JSON-Format trainiert und in der folgenden Evaluation auf 404 JSON-Dateien (20%) getestet. In ersten Evaluationen führte ein Fine-Tuning auf nur 72% der Trainingsdaten, wie in der indirekten Strukturierung, in Unterabschnitt 6.1.5 beschrieben, zu deutlich schlechteren Ergebnissen. Dabei ist anzumerken, dass die BERT-basierten SOTA-Modelle auch ein Fine-Tuning auf 80% der LCT-Daten erhalten hatten (N. Dobbins; T. Mullen et al., 2022).

Als Instruktion wurde dem Modell im Fine-Tuning Prozess die Strukturprompt in Abbildung C.3 mit Einsetzen von Prompt 1 in Abbildung C.1 für die Operatorenextraktion übergeben. In initialen Tests hat Prompt 1 für die Operatorenextraktion innerhalb der AST-Strukturierung besser performt als die Verwendung von Prompt 2.

6.2.3. Operatorenextraktion

Um die Eignungskriterien im JSON-Format basierend auf den logischen Operatoren AND, OR und NOT zu strukturieren und gleichzeitig die Erkennung von Entitäten zu integrieren, wurden die Open-Source Modelle Llama3-70B und Qwen2-72B mit dem Closed-Source Modell GPT-4o in 2-Shot und 10-Shot Szenarien verglichen. Zusätzlich wurde das Llama3-70B Modell einem Fine-Tuning unterzogen.

Die Precision-, Recall- und F1-Scores für die Operatorenextraktion mit diesen Modellen sind in Tabelle 6.12 dargestellt. Dabei gibt der JSON-Fehler an, welcher Prozentsatz der JSON-Ausgaben aufgrund von Formatierungsfehlern unbrauchbar war, während der Ausgabefehler den Prozentsatz des ursprünglichen Textes angibt, der in der Modellausgabe verändert wurde. Die F1-Scores der Modelle werden durch die Differenz zum SOTA-Modell SciBERT verglichen.

Modell	Shots	JSON-Fehler (%)	Ausgabefehler ($\bar{x}$, %)	P	R	F1	Δ -F1
SciBERT (SOTA)	804	–	–	71.2	81.4	75.9	–
<i>Few-Shot</i>							
Llama3-70B	2	4.7	18.7	12.8	9.3	10.7	↓65.2
	10	20.8	10.3	19.8	12.3	14.7	↓61.2
Qwen2-72B	2	10.7	11.0	17.0	13.3	12.8	↓63.1
	10	44.0	5.3	21.5	20.2	15.0	↓60.9
GPT-4o	2	3.5	4.0	24.2	16.2	16.4	↓59.5
	10	10.6	3.7	25.5	18.4	18.3	↓57.6
<i>Fine-Tuning</i>							
Llama3-70B	804	1.2	0.5	84.6	84.8	84.6	↑8.7

Tabelle 6.12.: Auswertung der Operatorenextraktion der LLMs mit JSON-Ausgabe.

Die Leistungen der Operatorenextraktion von Few-Shot LLMs bei der JSON-Ausgabe liegt um etwa 60% F1-Score unter dem SOTA-Modell SciBERT. Mehr Shots verbessern die Ergebnisse nur um wenige Prozentpunkte. Dafür nimmt der JSON-Fehler mit steigender Anzahl von Beispielen stark zu. Der Ausgabefehler, der angibt, ob die ursprünglichen Kriterientexte korrekt übernommen werden, wird aber mit zunehmenden Shots reduziert. Das GPT-4o Modell performt im Vergleich zu den Open-Source LLMs besser, mit einer geringeren Fehlerrate. Das Llama3-70B mit Fine-Tuning zeigt eine bessere Leistung als das SOTA-Modell SciBERT mit einem um 8.7% höheren F1-Score. Dabei wurde das Modell mit der gleichen Anzahl an Beispielen trainiert wie das SciBERT-Modell. Der JSON-Fehler und der Ausgabefehler liegen unter 2%. Abbildung 6.8 fasst die Operatorenbewertung über den erreichten F1-Score der untersuchten Modelle zusammen.

Evaluation der Operatorenextraktion mit JSON-Ausgabe

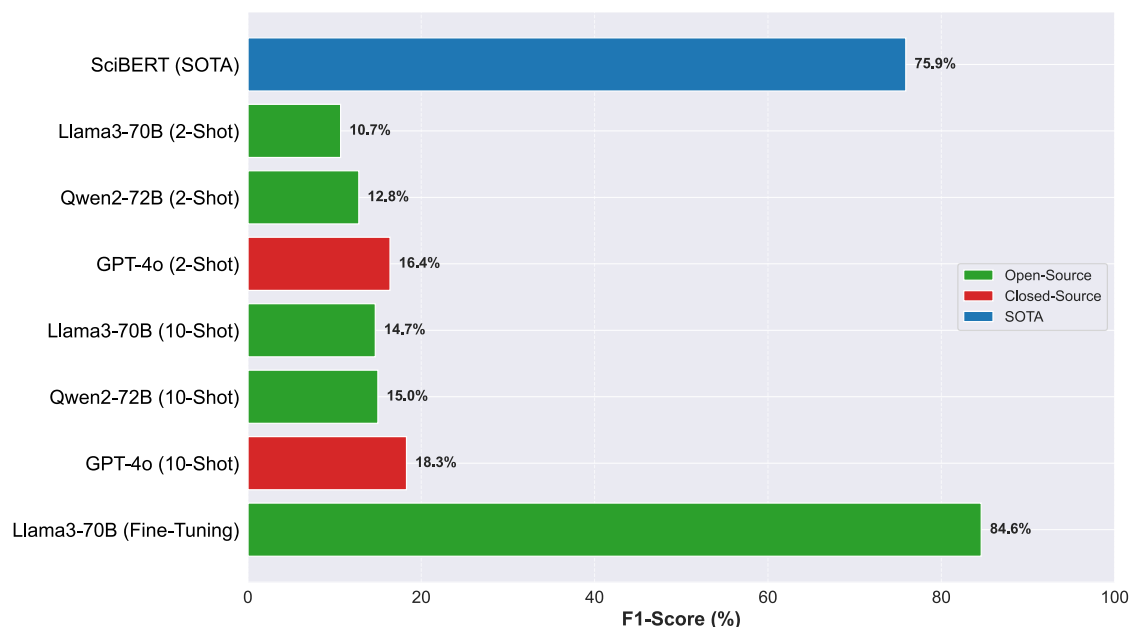


Abbildung 6.8.: F1-Scores der Operatorenextraktion mit JSON-Ausgabe.

Die Erhöhung der Anzahl der Beispiele von 2 auf 10 führt zu besseren F1-Scores bei den LLMs, wobei die Few-Shot Methoden der LLMs jedoch nicht an die SOTA-Leistung herankommen. Das rot markierte Closed-Source Modell GPT-4o performt besser als die Open-Source Modelle. Durch ein Fine-Tuning von LLMs kann die Leistung des SOTA-Modells bei der Erkennung logischer Operatoren übertroffen werden.

6.2.4. Einzelnen Operatoren

Bei der Evaluation der logischen Operatoren muss beachtet werden, dass der Goldstandard im BRAT-Format AND-Relationen innerhalb der Kriterien annotiert. Die Ausgabe im JSON-Format erfordert jedoch das Einfügen zusätzlicher AND-Operatoren nach Satzendungen und vor NOT-Operatoren, um eine konsistente AST-Struktur aufbauen zu können. Die hinzugefügten, nicht annotierten, ANDs werden als AND* bezeichnet und können getrennt von den annotierten ANDs evaluiert werden. Abbildung 6.9 zeigt exemplarisch die Unterschiede zwischen annotierten und hinzugefügten ANDs.

NCT03860402 (mit AND, AND*, OR, NOT)

Inclusion Criteria:

- Among the pregnant women [AND] who were administered cesarean section in this study, those who agreed to participate in this study.

Exclusion Criteria:

- Fetal distress requiring fast delivery. [AND*]
- Patients with hypersensitivity, allergic response, [OR] resistance to the drugs used in this study (fentanyl, [OR] ropivacaine, [OR] lidocaine). [AND*]
- Patients who it is [AND*] [NOT] not possible for regional anesthesia. [AND*]
- Patients who refused to participate in this study. [AND*]
- Patients who cannot read or understand the agreement. [AND*]
- Patients whose weight is less than 50kg [OR] or exceeds 100kg.

Abbildung 6.9.: Eignungskriterien der Studie NCT03860402 mit zusätzlichen AND-Operatoren, welche als AND* gekennzeichnet sind.

Evaluation von AND und AND*

Die Evaluation der impliziten, hinzugefügten Konjunktionen [AND*] basiert auf einer modifizierten Variante der Auswertungsmethodik für explizite Konjunktionen [AND]. Diese Methode vergleicht die Wörter unmittelbar vor den jeweiligen Operatoren. Sei L_{AND*} die Menge der Wörter, die in den annotierten Daten unmittelbar vor einem AND*-Operator stehen, L_{AND} die entsprechende Menge für explizite AND-Operatoren, und M_{AND} die Menge der Wörter, die in den vom Modell erzeugten Daten vor einem AND-Operator auftreten.

Die True Positive-, False Positive- und False Negative-Werte für AND* werden wie folgt bestimmt.

$$\begin{aligned}
 TP_{AND*} &= |w \in M_{AND} : w \in L_{AND*}| \\
 FP_{AND*} &= |w \in M_{AND} : w \notin L_{AND*} \wedge w \notin L_{AND}| \\
 FN_{AND*} &= |w \in L_{AND*} : w \notin M_{AND}|
 \end{aligned} \tag{6.1}$$

Analog dazu werden die Metriken für AND berechnet. Die Modelle unterscheiden nicht explizit zwischen AND und AND*, während die annotierten Daten diese Unterscheidung vornehmen. Folglich werden AND-Operatoren, die von den Modellen an falscher Stelle eingefügt wurden (FP), gleichzeitig sowohl als FP_{AND*} als auch als FP_{AND} gezählt. Diese doppelte Zählung der falsch positiven Ergebnisse führt zu einer konservativeren Bewertung der Modellleistung.

SciBERT vs. Llama3-70B Fine-Tuned

Die Evaluation auf den einzelnen logischen Operatoren des Llama3-70B Modells mit Fine-Tuning im Vergleich zu dem SOTA-Modell ist in Tabelle 6.13 dargestellt. Das Open-Source Modell mit Fine-Tuning weist für alle Operatoren höhere Precision-Scores auf. Die AND-Extraktion wird um 11.2% verbessert im Vergleich zum SOTA-Modell. Bei OR- und NOT-Operatoren ist der Recall noch unter der SciBERT-Leistung.

Modell	Metrik	AND	AND*	OR	NOT	∅
SciBERT	P	54.1	–	85.1	74.3	71.2
	R	60.0	–	93.2	91.0	81.4
	F1	56.9	–	89.0	81.8	75.9
Llama3-70B (Fine-Tuned)	P	64.0 ↑9.9	95.6	92.1 ↑7.0	86.7 ↑12.4	84.6 ↑13.4
	R	72.8 ↑12.8	95.5	89.0 ↓4.2	81.8 ↓9.2	84.8 ↑3.4
	F1	68.1 ↑11.2	95.5	90.5 ↑1.5	84.2 ↑2.4	84.6 ↑8.7

Tabelle 6.13.: Evaluation der Extraktion der logischen Operatoren AND, AND*, OR und NOT durch Llama3-70B mit Fine-Tuning im Vergleich zum SOTA-Modell SciBERT.

Die AND*-Operatoren werden mit 95.5% F1-Score gut in den Text eingefügt. Über die AND-, OR- und NOT-Operatoren performt das Llama3-Modell im Durchschnitt über alle Metriken besser als das SOTA-Modell. Die Auswertungen der einzelnen Operatoren mit den Few-Shot Modellen ist im Anhang in Tabelle D.6 für die 2-Shot Modelle und in Tabelle D.7 für die 10-Shot Modelle dargestellt. Die F1-Scores der 2- und 10-Shot Modelle für den AND-Operator liegen im Bereich von 20-24%. Der hinzugefügte AND*-Operator wird mit F1-Scores von 70-82% mit 10-Shot relativ solide erkannt. Bei dem OR-Operator verbessern sich die F1-Scores von 10-27% (2-Shot) auf 15-30% (10-Shot). Die F1-Scores für den NOT-Operator bleiben bei den Few-Shot Methoden gering und oft deutlich unter 5%.

6.2.5. Entitätsextraktion

Die erstellte JSON-Struktur ermöglicht es, dass erkannte Entitäten als Blätter des Abstract Syntax Tree integriert werden können. Neben den Relationen AND, OR und NOT können auch die Entitäten des LCT-Korpus evaluiert werden. Der LCT-Korpus beinhaltet 50 unterschiedliche annotierte Entitätstypen, wobei im Durchschnitt 105 Entitäten pro annotiertem Dokument auftreten. Die Entitätstypen werden im Anhang in Tabelle A.1 nach sieben Kategorien gelistet. Die 50 Entitätstypen des LCT-Korpus wurden von N. Dobbins; T. Mullen et al. (2022) über eine Kombination aus ML-Modellen Bi-LSTM und CRF evaluiert, sowie mit transformerbasierten PubMedBERT- und SciBERT-Modellen. Das SciBERT-Modell performt bei der NER-Aufgabe über alle Entitätstypen am besten, mit durchschnittlich 79.0% Precision 83.7% Recall und 81.3% F1-Score (N. Dobbins; T. Mullen et al., 2022). Um ausgewählte Large Language Models mit dem SciBERT-Modell bei der NER-Aufgabe und der Extraktion logischer Operatoren zu vergleichen, wurden sowohl Few-Shot als auch Fine-Tuning Ansätze evaluiert. Zusätzlich zur Operatorenextraktion wurde die Erkennung der 50 Entitätstypen untersucht. Die Ergebnisse der Entitätsextraktion der Large Language Models sind in Tabelle 6.14 dargestellt, wobei die Precision-, Recall- und F1-Scores über alle Entitätstypen gemittelt wurden.

Modell	Shots	P	R	F1	Δ F1
SciBERT (SOTA)	804	79.0	83.7	81.3	–
<i>Few-Shot</i>					
Llama3-70B	2	52.5	30.9	38.9	↓42.4
	10	53.6	33.4	41.1	↓40.2
Qwen2-72B	2	40.0	33.0	36.2	↓45.1
	10	46.6	39.6	42.8	↓38.5
GPT-4o	2	45.8	24.0	31.5	↓49.8
	10	51.6	37.2	43.2	↓38.1
<i>Fine-Tuning</i>					
Llama3-70B	804	90.1	86.8	88.5	↑7.2

Tabelle 6.14.: Evaluation der Entitätsextraktion von LLMs mit JSON-Ausgaben auf dem LCT-Testdatensatz.

Die Open-Source LLMs mit Few-Shot lagen mindestens 40% unter dem F1-Score des SciBERT Modells. Das GPT-4o Modell hat mit 10-Shot die beste Leistung unter den Few-Shot Modellen mit einem F1-Score von 43.2%, performt mit 2-Shot aber schlechter als die Open-Source Modelle. Das Llama3-70B Modell mit Fine-Tuning verbessert mit einem F1-Score von 88.5% die SOTA-Leistung um 7.2%. Abbildung 6.10 zeigt die F1-Scores der LLM-Leistungen mit Few-Shot und Fine-Tuning gegenüber dem SciBERT-Modell.

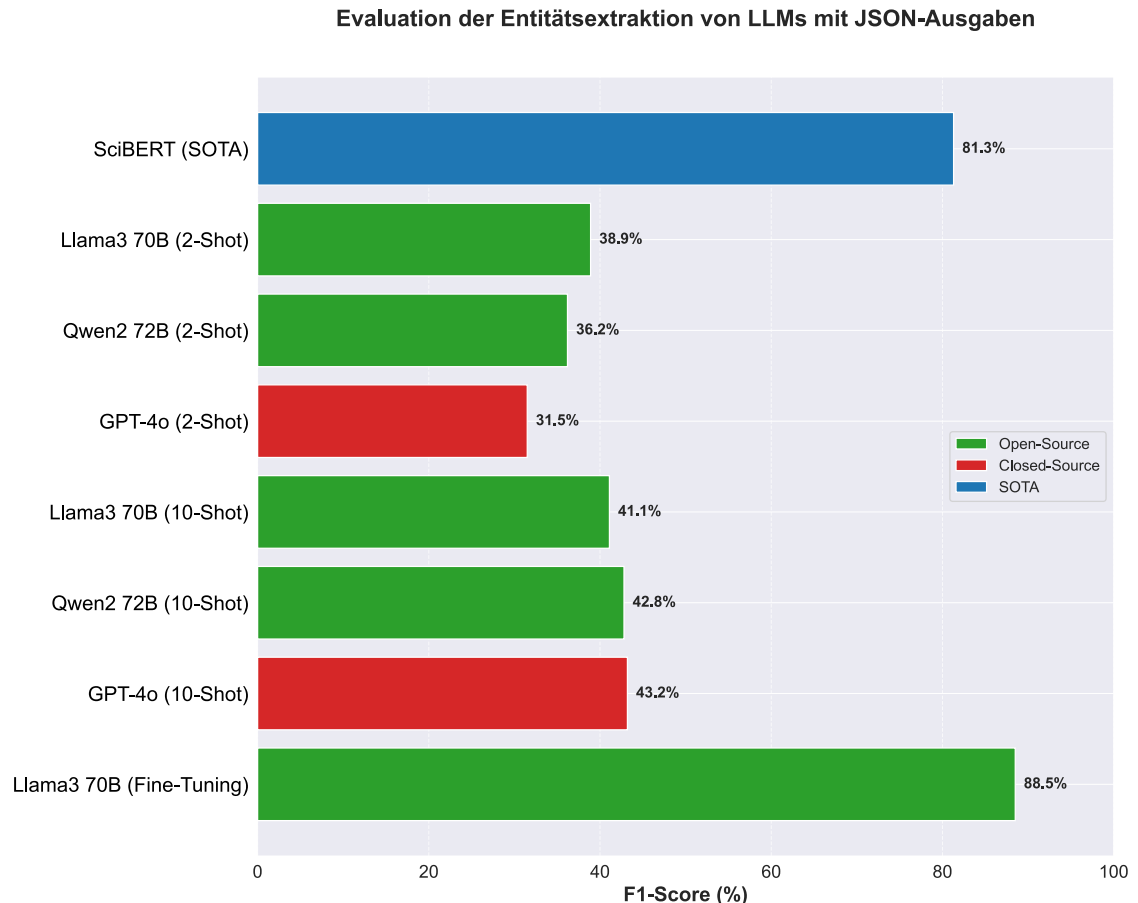


Abbildung 6.10.: F1-Scores der Entitätsextraktion mit JSON-Ausgabe.

Obwohl die Operatorenerkennung mit Few-Shot Modellen unter 25% der SOTA-Leistung lag, erreichte die Entitätserkennung je nach Modell und Methode etwa 50% der SOTA-Leistung. Dabei wurden die Entitäten von den Open-Source Modellen im 2-Shot besser erkannt als mit GPT-4o und im 10-Shot nahezu gleich gut. Es ist zu beachten, dass die Modelle abgesehen von den wenigen Beispielen keine weiteren expliziten Informationen zu den Entitätentypen erhalten haben. Durch ein Fine-Tuning eines LLMs mit der gleichen Datenmenge wie beim SciBERT-Modell wird die Leistung sogar übertroffen.

Die folgenden Entitäten wurden mit dem Llama3-70B Modell mit Fine-Tuning einzeln evaluiert, um zu demonstrieren, wie effektiv das Modell bei der Erkennung wichtiger Entitäten ist. Die Identifikation solcher Entitäten ist entscheidend für weiterführende Anwendungen wie die logische Suche oder das Patient-Studie Matching. Tabelle 6.15 vergleicht die Leistung von SciBERT für ausgewählte Entitäten basierend auf den Ergebnissen aus N. Dobbins; T. Mullen et al. (2022) mit der Leistung des Llama3-70B Modells mit Fine-Tuning. Die Werte werden als Tripel mit (Precision, Recall und F1-Score) angegeben. Die untere Zeile gibt den Durchschnitt über alle Entitäten an.

Category	Entity	SciBERT	Llama3-70B (FT)
Clinical	Drug	73.4/80.9/77.0	86.5/87.2/86.8
	Procedure	71.3/79.4/75.1	88.5/84.3/86.3
Demographic	Birth	100.0/62.5/76.9	87.5/100.0/93.3
	Family-Member	44.9/61.1/51.7	95.3/89.1/92.1
Qualifier	Assertion	62.1/65.8/63.9	71.3/72.3/71.8
	Modifier	58.5/65.4/61.8	83.3/80.2/81.8
	Polarity	81.4/79.5/80.4	84.9/76.5/80.5
Temporal	Eq-Temporal-Period	82.6/86.3/84.4	92.8/86.1/89.3
	Eq-Temporal-Recency	50.0/66.6/57.1	92.0/92.0/92.0
–	Total	79.0/83.7/81.3	90.1/86.8/88.5 ↑11.1/↑3.1/↑7.2

Tabelle 6.15.: Vergleich über die Metriken (Precision/Recall/F1-Score) für die Extraktion ausgewählter Entitätstypen zwischen dem SOTA-Modell SciBERT und dem Llama3-70B Modell mit Fine-Tuning (FT).

Die Erkennung klinischer Entitäten wie *Drug* (Medikamente, z.B. "Aspirin") und *Procedure* (chirurgische Eingriffe oder Therapien, z.B. "Chemotherapie") ist wichtig, um zu bestimmen, welche klinischen Merkmale ein Patient aufweisen muss, um an einer Studie teilnehmen zu können. Für die Entitäten *Drug* und *Procedure* erzielte das Llama3-Modell im Vergleich zum SciBERT-Modell eine Verbesserung des F1-Scores um 9.8% bzw. 11.2%. Die Entität *Birth*, repräsentiert durch Signalwörter wie "birth" oder "born", wurde evaluiert, um die Fähigkeit des LLM zu testen, einfache und selbsterklärende Entitäten zu extrahieren. Hier zeigte das Llama3 mit einem F1-Score von 93.3 eine signifikante Verbesserung von 21.3% gegenüber dem SciBERT-Modell. Bei der Erkennung von *Family-Member* war eine Steigerung von 40.4% im F1-Score noch höher. Verbesserungen in der Extraktion wurden auch für die Entitäten *Assertion* (Annahmen, z.B. "could be pregnant"), *Modifier* (Statusangaben, z.B. "untreated substance abuse") und *Polarity* (Positiv/Negativ, z.B. "near-normal hearing") festgestellt. Diese Entitäten qualifizieren Aussagen und sind oft entscheidend für das richtige Verständnis klinischer Texte. Wenn in einer Studie nach *Patients with untreated substance abuse* gefragt wird, aber nur die Entität *substance abuse* als *Condition* erkannt wird, kann es zu Fehlklassifikationen bei der Patientensuche kommen, da z.B. *Patients with treated substance abuse* für die Studie nicht infrage kämen. Die *Qualifier* Entitätstypen wie *Modifier* sind daher zentral für die korrekte Verarbeitung der Kriterien. Die Entität *Modifier* wurde auch mit 81.8% F1-Score von Llama3 um 20% besser erkannt. Die Fähigkeit, zeitliche Sequenzen zu erkennen, ist entscheidend für das Verständnis von Krankheitsverläufen. Bei *Eq-Temporal-Period* (Zeitangaben, z.B. "past", "present", "future") erreichte das Llama3-Modell einen F1-Score von 89.3%, was einer Verbesserung von 5.8% gegenüber SciBERT mit 84.4% entspricht. Für *Eq-Temporal-Recency* (Zeitliche Aktualität, z.B. "first-time", "most-recent") zeigte Llama3 mit einem F1-Score von 92.0% eine signifikante Verbesserung von 34.9% gegenüber SciBERT. Über alle Entitätstypen extrahiert Llama3-70B mit Fine-Tuning die Entitäten präziser mit einer im durchschnittlich 11.1% höheren Precision. Insgesamt performt das LLM mit einem F1-Score von 88.5% um 7.2% besser als das aktuelle State-of-the-Art Modell SciBERT für die Entitätsextraktion auf dem LCT-Testdatensatz.

Die Ergebnisse der Erkennung aller 50 Entitäten des LCT-Datensatzes mit dem Llama3-70B Modell mit Fine-Tuning sind im Anhang in Tabelle D.9 dargestellt. Zusätzlich zeigt Tabelle D.8 im Anhang den Vergleich aller einzeln evaluierten Entitäten aus N. Dobbins; T. Mullen et al. (2022) mit Llama3. Bei Entitäten wie *Criteria-Count*, welche ausschließlich 0.0-Scores aufweisen, ist anzunehmen, dass diese entweder im zufällig erstellten Trainingsdatensatz nicht vorkamen oder im Vergleich zu anderen Entitäten unterrepräsentiert waren. In solchen Fällen können bereits wenige Fehlklassifikationen zu einem erheblichen Rückgang der Leistungsmetriken führen.

Exemplarische JSON-Ausgabe

Zur Veranschaulichung der Modellausgabe von Llama3-70B mit Fine-Tuning wird exemplarisch ein Auszug aus der Ausgabe der verarbeiteten Einschlusskriterien der Studie NCT03865602 aus dem LCT-Testdatensatz gezeigt.

```

1 {
2   "OR": {
3     "left": {
4       "raw_text": "parenteral antibiotic",
5       "ModifieR": ["parenteral"],
6       "Drug":     ["antibiotic"]
7     },
8     "right": {
9       "raw_text": "or bacterial culture order within 24 hours of
10        emergency department presentation and",
11       "EqComparison": ["within 24 hours"],
12       "EqOperator":   ["within"],
13       "EqValue":      ["24"],
14       "EqTemporalUnit": ["hours"],
15       "Encounter":    ["emergency"],
16       "Drug":          ["bacterial culture"], // Falsch
17       "ModifieR":      ["emergency department"] // Falsch
18     }
19   }
20 }
```

Die OR-Verknüpfung wurde korrekt gesetzt, und die meisten Entitäten wurden erfolgreich extrahiert und in die AST-Struktur integriert. Die Entität *“bacterial culture”* wurde fälschlicherweise als *Drug* anstatt als *Observation* klassifiziert, und *“emergency department”* wurde irrtümlich als *Modifier* identifiziert. Es ist anzumerken, dass die meisten Entitäten, einschließlich zeitlicher Komponenten wie *“within 24 hours”*, korrekt klassifiziert wurden, selbst im Kontext der Generierung einer komplexen und verschachtelten AST-Struktur. Die Darstellung der Abstract Syntax Trees im JSON-Format ermöglicht es, Fehlerquellen relativ einfach zu identifizieren und manuell zu beheben. In diesem Fall würde es genügen, in Zeile 16 den Typ von *Drug* zu *Observation* zu ändern und die fehlerhafte Zeile 17 zu entfernen.

6.2.6. AST-Strukturierung

Um die direkte Strukturierung zusammenfassend darzustellen, wird in Abbildung 6.11 die Evaluation der JSON-Strukturierung auf dem LCT-Testdatensatz mit Few-Shot und Fine-Tuning Modellen präsentiert. Dabei wird der F1-Score der Operatorenextraktion und der F1-Score der Entitätsextraktion der Modelle angegeben. Die *Ausgabe-Genauigkeit* wird über $1 - \text{Ausgabefehler}$ berechnet, während die *JSON-Genauigkeit* durch $1 - \text{JSONFehler}$ repräsentiert wird. Die Llama3-Modelle werden stellvertretend für die Open-Source Modelle in grün dargestellt, die GPT-4o Modelle in rot. Da das SciBERT-Modell den Eignungskriterientext bei NER- und RE-Aufgaben nicht mit extrahiert, wird die Ausgabe-Genauigkeit auf 100% gesetzt, gleiches für die JSON-Genauigkeit, da die Ausgabe von BERT-Modellen in das JSON-Format geparkt werden kann.

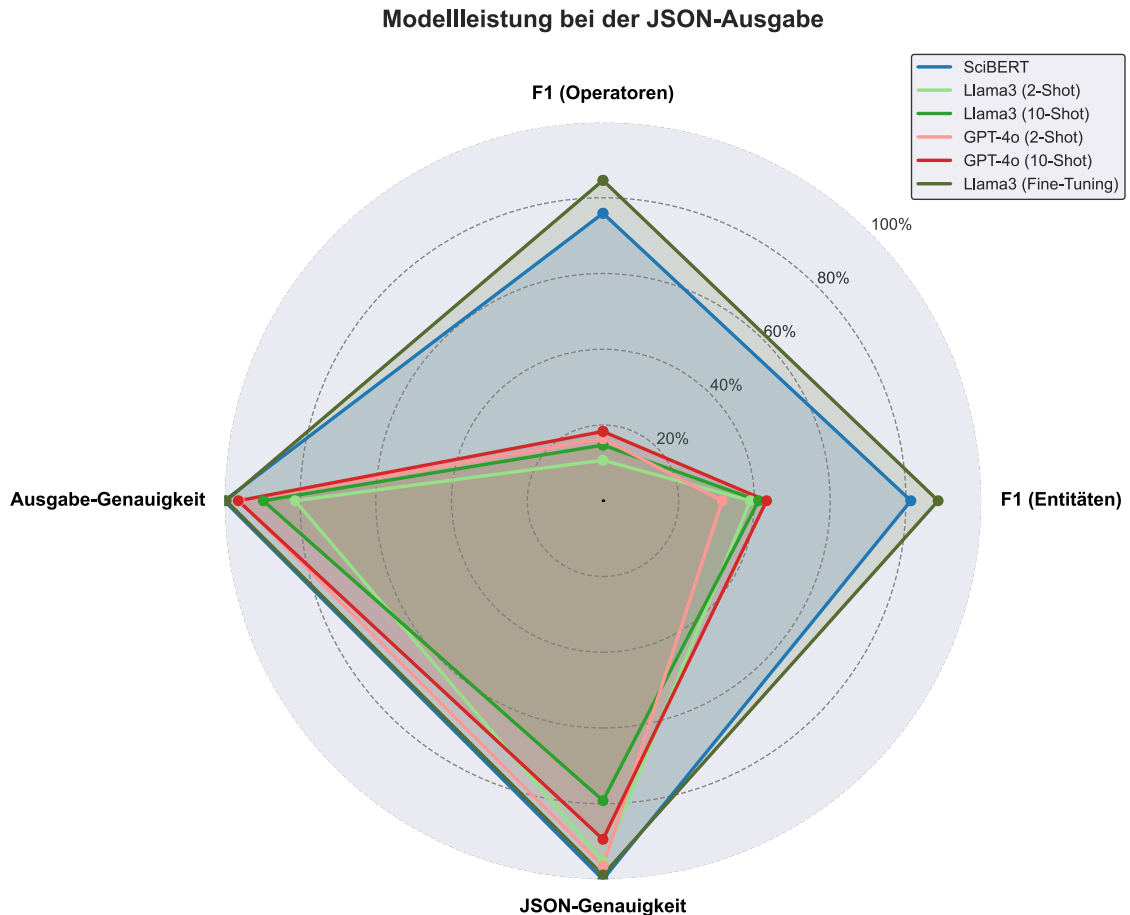


Abbildung 6.11.: Modellevaluation von Open- und Closed-Source LLMs mit Few-Shot und Fine-Tuning bei der Generierung von JSON-Dateien in NER-Struktur.

Das Llama3-70B Modell mit Fine-Tuning zeigt die allgemein bessere Leistung gegenüber dem SciBERT-Modell. Die Entitätserkennung wird um durchschnittlich 7.2% F1-Score verbessert und die Operatorenextraktion um 8.7% gegenüber dem SOTA-Modell. Zudem zeigt das Modell, dass das JSON-Format mit Abstract Syntax Tree-Struktur präzise ausgegeben wird. Sowohl das untersuchte Open-Source Modell als auch das Closed-Source Modell GPT-4o erreichen nicht mehr als 20% F1-Score bei der Operatorenextraktion und nicht mehr als 50% F1-Score bei der Entitätsextraktion. Das GPT-4o mit 10-Shot hat die beste Leistung unter den Few-Shot Modellen bei den Information Extraction-Aufgaben und hat auch die höchste JSON-Genauigkeit mit 89.4% unter den 10-Shot Modellen. Die Llama3-Modelle mit Few-Shot schneiden im Vergleich gut bei der NER-Aufgabe ab, aber schlechter als das proprietäre Modell bei der Operatorenextraktion. Das Llama3 10-Shot Modell hat eine JSON-Genauigkeit von 79.2% und das 2-Shot Modell eine Ausgabegenauigkeit von 81.3%, was die Anwendung der Modelle unpraktikabel macht. In den ersten Experimenten mit Llama2 70B unter Verwendung von 2-Shot konnte keine korrekte AST-Struktur generiert werden, was zu einer JSON-Genauigkeit von 0% führte.

Evaluation über 20 Versuche

Um die Leistungsfähigkeit des Llama3-70B Modells mit Fine-Tuning im Vergleich zum SciBERT-Modell bei der Operatoren- und Entitätsextraktion zu untersuchen, wurde es über 20 Durchläufe auf dem LCT-Testdatensatz evaluiert. Die durchschnittlichen Ergebnisse für beide Extraktionsaufgaben, einschließlich der Standardabweichung, sind in 6.16 dargestellt. Angegeben werden Precision-, Recall- und F1-Scores. Zusätzlich ist die Differenz (Δ) der jeweiligen Metriken im Vergleich zum SciBERT-Modell angegeben.

	Metrik	SciBERT	Llama3-70B (FT)	Δ
Operatoren $\emptyset$	P	71.2	82.2 ± 0.7	$\uparrow 11.0$
	R	81.4	82.8 ± 0.8	$\uparrow 1.4$
	F1	75.9	82.4 ± 0.5	$\uparrow 6.5$
Entitäten $\emptyset$	P	79.0	90.5 ± 0.2	$\uparrow 11.5$
	R	83.7	87.2 ± 0.3	$\uparrow 3.5$
	F1	81.3	88.8 ± 0.2	$\uparrow 7.5$

Tabelle 6.16.: Die durchschnittlichen Ergebnisse der Operatoren- und Entitätsextraktion mit Llama3-70B mit Fine-Tuning (FT) auf dem LCT-Testdatensatz über 20 Durchläufe. Dabei werden die Standardabweichung ($\pm$) sowie die Differenzen (Δ) zum SOTA-Modell SciBERT angegeben.

Der durchschnittliche F1-Score von 82.4% für die Operatorenextraktion ist immer noch um 6.5% höher als beim SciBERT-Modell, aber um 2.2% niedriger als im initialen Versuch. Die Standardabweichungen sind im Vergleich zur Entitätsextraktion höher und liegen beim Recall bei 0.8%. Im Recall zeigt das Llama3-Modell auch ähnlich gute Leistungen wie das SciBERT-Modell, weist aber um eine durchschnittlich 11.0% höhere Precision auf. Die Entitätsextraktion performt auch mit einer höheren Precision von 11.5% und liegt im F1-Score um 7.5% über dem SOTA-Modell. Die Standardabweichung von 0.2% im F1-Score ist hier geringer. Die Ergebnisse zeigen, dass das Llama3-70B Modell mit Fine-Tuning auch im Durchschnitt über 20 Versuche besser als das SciBERT-Modell abschneidet.

Llama3-8B Fine-Tuning

In Versuchen mit dem Llama3-8B Modell zur JSON-Generierung konnten mit Few-Shot Methoden nur wenig verwertbare Dateien erzeugt werden, wobei die JSON-Fehlerrate bei über 60% lag. Das Modell wurde daraufhin mit denselben Parametereinstellungen wie das Llama3-70B Modell einem Fine-Tuning unterzogen. Anschließend lieferte das Modell verwertbare JSON-Ausgaben mit einem Fehler von nur 1.2%. Dennoch zeigte das Modell insgesamt eine geringere Leistung. Bei der Operatorenerkennung lag es mit 77.4% F1-Score um etwa 7% unter dem Llama3-70B Modell, bei der Entitätserkennung mit 88% F1-Score jedoch nur 0.5 Prozentpunkte darunter.

6.2.7. Ressourcen- und Zeitaufwand

Die Inferenz mit Open-Source Modellen mit Few-Shot dauerte bis zu 12 Stunden für die Generierung der 404 JSON-Dateien. Im Gegensatz dazu erzeugte das Llama3-70B Modell mit Fine-Tuning die Dateien innerhalb von vier Stunden. Das Fine-Tuning des Llama3-8B Modells konnte mit den besten Parametereinstellungen in etwa zwei Stunden auf einer NVIDIA A100 GPU abgeschlossen werden, während das Fine-Tuning des Llama3-70B Modells 20 Stunden in Anspruch nahm. Die Kosten für die Erzeugung von den 404 Testdaten im JSON-Format lagen mit GPT-4o bei \$21.63 und dauerte etwa zwei Stunden. Diese Kosten haben während der Erstellung dieser Arbeit abgenommen, da die Preispolitik von OpenAI angepasst wurde. Bei initialen Versuchen lag der Preis für die Ausgabe der Testdaten bei etwa \$60.

7. Diskussion

Um unstrukturierte Eignungskriterien für eine logische Suche nach klinischen Studien nutzbar zu machen, ist die automatisierte Erkennung der in den Ein- und Ausschlusskriterien enthaltenen Entitäten wie Krankheiten, Medikamente oder Attribute wie Gewicht und Alter essenziell. Die logische Verknüpfung dieser erkannten Entitäten mit anderen Entitäten oder mit sich selbst ist für ihre Integration von zentraler Bedeutung. Selbst wenn in dem Ausschlusskriterium "*Patients for whom regional anesthesia is not possible*" die *regional anesthesia* korrekt als Entitätentyp *Procedure* erkannt wird, führt eine Verarbeitung ohne die Berücksichtigung der negativen Relation NOT zu Fehlern. Es werden Patienten gesucht, die nicht für eine Regionalanästhesie infrage kommen. Wenn sowohl die Entität als auch die Relation korrekt erkannt werden, kann nach Studien gesucht werden, deren Teilnehmer keine Regionalanästhesie erhalten dürfen, und Patienten können automatisiert identifiziert werden, sofern ihre Daten entsprechend angereichert sind. Nachdem Entitäten und logische Relationen erkannt wurden, ist es notwendig, diese in einem interoperablen Format bereitzustellen, um sie in unterschiedlichen Datenbankschemata wie OMOP-CDM oder in Standardformaten für den Austausch von Gesundheitsdaten wie FHIR zu nutzen. Dies ermöglicht es medizinischen Systemen, die extrahierten Entitäten für die Suche nach Studien und die Identifizierung geeigneter Patienten zu verwenden. Open-Source LLMs wie Llama3 sind in der Lage neben der Extraktion von AND-, OR- und NOT-Operatoren auch Eignungskriterien in Textform zu verarbeiten und basierend darauf einen Abstract Syntax Tree zu erzeugen. Dabei werden die verknüpften Satzteile mit extrahiert, was die anschließende Extraktion und Anfügung von Entitäten aus diesen Satzteilen ermöglicht. Die AST-Struktur kann in ein interoperables Format wie JSON überführt werden, das in verschiedene Datenbankschemata übertragen und in medizinische Formate wie FHIR konvertiert werden kann. Dies ermöglicht es medizinischen Systemen, die extrahierten Entitäten und Relationen für die Suche nach Studien und geeigneten Patienten effektiv zu nutzen. Diese Arbeit hat für die Erzeugung der AST-Repräsentationen der Eignungskriterien zwei methodische Ansätze erprobt.

Die erste entwickelte Methode, eine *indirekte Strukturierung*, vorgestellt in Abschnitt 5.4, ermöglicht einen Zero-Shot Ansatz mit Large Language Models, wobei zur Strukturierung der Eignungskriterien lediglich eine Promptanweisung erstellt werden muss. Allerdings werden die Kriterien dabei nicht direkt über Zero-Shot mit LLMs in eine AST-Struktur überführt. Dies liegt daran, dass die alleinige textuelle Beschreibung der zu erstellenden AST-Struktur nicht ausreichend war, um eine korrekte Transformation zu gewährleisten. ASTs können beliebig tief verschachtelt sein, und die Bäume können flexibel aufgebaut werden beispielsweise links- oder rechtsgerichtet. Diese Komplexität macht es schwierig, die strukturellen Anforderungen allein durch textuelle Anweisungen abzubilden. Bei der *indirekten Strukturierung* werden deshalb über In-Context Learning- und Fine-Tuning Methoden, logische Operatoren (AND, OR, NOT) in die Kriterientexte eingefügt. Diese angereicherten Textdaten müssen über einen AST-Parser (Unterabschnitt 5.2.2) in einen Abstract Syntax Tree prozessiert werden. Der Hauptgrund für diese Methode war es, Zero-Shot Ansätze zu testen und durch zusätzliche Prompt-Methoden wie Emotional Prompting, Zero-Shot CoT und Role Prompting Verbesserungen zu erzielen. Ziel war es, eine Alternative zu BERT-basierten Methoden zu bieten, die stets ein Fine-Tuning auf den zu untersuchenden Datensätzen benötigen, um eine SOTA-Leistung zu erreichen. Während für BERT-basierte Methoden sowohl das Annotieren als auch das Verarbeiten großer Datenmengen für das Training erforderlich ist, verbunden mit der Expertise von Informatikern für Hyperparameteroptimierung, Clusternutzung und die Bereitstellung von Ressourcen für ein Fine-Tuning, würde ein Zero-Shot Ansatz lediglich das Erstellen einer Anweisung (Prompt) erfordern. Dies könnte auch von einer medizinischen Fachkraft ohne Informatik-Hintergrund durchgeführt werden, sofern ein entsprechendes LLM bereitgestellt wird. Um weiter zu untersuchen, wie sich LLMs mit ähnlichen Ressourcen wie BERT-Modellen verhalten, wurden zusätzlich Few-Shot Szenarien evaluiert. Dabei müssen neben dem Prompt lediglich einige wenige Beispiele annotiert und verarbeitet werden. Diese Methodik könnte auch von Personen ohne Informatik-Hintergrund genutzt werden, sofern die entsprechenden Ressourcen bereitgestellt werden. Vor der Einführung von Parameter-Efficient Fine-Tuning-Methoden wie Low-Rank Adaptation war das Fine-Tuning von LLMs für die Aufgabe der Operatorenextraktion in Eignungskriterien zu rechenintensiv für eine praktische Anwendung. Durch die Nutzung von LoRA-Methoden, die in aktuellen Python-Bibliotheken wie unsloth (2024) implementiert sind, ist ein Fine-Tuning auf einer einzelnen NVIDIA A100 GPU möglich geworden, was diese Option prakti-

kabel macht. Allerdings erfordert die Optimierung der SFT- und LoRA-Hyperparameter, Expertise im Bereich Machine Learning. Nach dem Fine-Tuning kann das Modell einem Nutzer bereitgestellt werden, der es intuitiver verwenden kann als Encoder-only Modelle wie BERT, die in der Regel nur von Informatik-Fachpersonal genutzt werden können. Trotz der Vorteile der indirekten Strukturierung, die darin bestehen, Zero-Shot anzuwenden und Operatoren auch ohne die verknüpften Entitäten zu extrahieren, muss ein AST-Parser entwickelt werden, um die gewünschte Abstract Syntax Tree-Struktur zu erhalten. Durch eine *direkte Strukturierung* konnte gezeigt werden, dass LLMs die Eignungskriterien mithilfe von Few-Shot und Fine-Tuning Methoden direkt in eine AST-Struktur überführen können basierend auf der Extraktion von logischen Operatoren. Der Vorteil dabei ist, dass zunächst auf einen AST-Parser für die Ausgaben verzichtet werden kann. Allerdings müssen die Beispiele für Few-Shot und Fine-Tuning über den AST-Parser verarbeitet werden, damit die Modelle die Struktur anhand dieser Beispiele erlernen können. Da LLMs Ein- und Ausschlusskriterien separat verarbeiten, ist die Nutzung eines einfachen Parsers notwendig, um beide erzeugten AST-Strukturen zu kombinieren. Die direkte Ausgabe ermöglicht es aber neben den Operatoren und dem Ursprungstext auch Entitäten zu extrahieren.

7.1. Indirekte Strukturierung

Durch eine *indirekte Strukturierung* waren Zero-Shot, Few-Shot und Fine-Tuning Ansätze möglich, indem ausgewählte LLMs die Operatoren in die Kriterientexte einfügen und ein Parser die Ausgaben in das gewünschte JSON-Format mit AST-Struktur verarbeitet. Dafür wurden zwei Prompts entwickelt. Eine ausführliche Prompt 1, die auf den Annotationsrichtlinien des LCT-Korpus (N. Dobbins; T. Mullen et al., 2022) basiert, und eine Prompt 2, welche das Einfügen von Operatoren anhand von Signalwörtern beschreibt. Diese Prompt 2 konnte mittels eines Regex-Parsers *Naive Greedy Match* abgebildet werden. Auf den LCT-Testdaten erzielt Llama3-70B mit Zero-Shot unter Verwendung von Prompt 2 einen F1-Score von 49.7% über alle Operatoren und schneidet damit ähnlich gut wie der Regex-Parser mit 50.9% ab, weist jedoch einen Ausgabefehler von 3.9% auf. Der Ausgabefehler gibt prozentual an, wie viele neue Wörter durch ein LLM in den Ursprungstext eingefügt werden. Die eingefügten Operatoren werden dabei nicht berücksichtigt. Wie in Tabelle 6.1 gezeigt wird, kann es beispielsweise passieren, dass ein Modell mit Zero-Shot Sätze doppelt einfügt oder sogar statt ausschließlich die Tags *[AND]*, *[OR]*, *[NOT]* einzufügen neue Tags wie *[IN WHICH]* erstellt. Dieser Ausgabefehler, bei dem das LLM halluziniert und unvorhergesehene Ausgaben erzeugt, ist bei den größeren Open-Source Zero-Shot Modellen wie Llama3-70B und Qwen2-72B mit 3.9% und 8.6% relativ hoch. Das Llama3-8B Modell ist mit 40% Ausgabefehler sogar unbenutzbar, da oft ganze Textpassagen in den Ursprungstext halluziniert werden. Das Closed-Source GPT-4o Modell ist in der Parameteranzahl um ein Vielfaches größer als die Open-Source Modelle und weist so auch eine geringere Halluzination aus und befolgt die Anweisungen so akkurater, liegt aber im F1-Score der Operatorenerkennung um ca. 3% unter dem Llama3-70B Modell.

Bei der Entwicklung der Llama3-Modellfamilie prognostizierte Dubey et al. (2024), dass durch die Verwendung qualitativ hochwertiger Trainingsdatensätze und Verbesserungen in den Architekturen von LLMs erhebliche Verbesserungen dieser Modelle zu erwarten sind. Daher ist zu erwarten, dass in zukünftigen Generationen von Open-Source LLMs mit Zero-Shot der Ausgabefehler reduziert und die Erkennung logischer Operatoren verbessert wird, insbesondere wenn sich diese Modelle genauer an die Instruktionen der Prompts halten. Über den generativen Charakter von LLMs ist die Ausgabe bei gleichem Input nicht immer gleich. So kann es vorkommen, dass das Modell bei den gleichen Eignungskriterien mit der gleichen Prompt bei wiederholten Versuchen unterschiedliche Ausgaben erzeugt. Llama3-70B hat mit Zero-Shot über 20 Versuche eine durchschnittliche Standardabweichung von 0.9% im F1-Score. Die Ausgabegenauigkeit schwankt um ca. ein Prozent. Wie Banerjee et al. (2024) gezeigt hat, ist die Wahrscheinlichkeit für Halluzinationen bei der Textgenerierung und Textklassifizierung durch LLMs niemals null. Folglich wird es immer sowohl Ausgabefehler als auch Variationen in den Leistungen von LLMs geben (Banerjee et al., 2024). Diese können jedoch durch neuere, fortschrittlichere Modelle, die auf qualitativ hochwertigeren Daten trainiert werden, verringert werden (Dubey et al., 2024).

Über erweiterte Prompt-Methoden Zero-Shot CoT, Emotional Prompting und Role Prompting wurde zudem versucht, die Zero-Shot Methode zu verbessern. Alle erweiterten Prompt-Methoden wurden mit den beiden erstellten Prompts angewandt, führten jedoch zu keiner signifikanten Verbesserung der Modellleistung bei der Operatorenextraktion. In vorherigen Studien konnten mit Emotional Prompting durchschnittliche Verbesserungen von 10.9% in der Modellleistung bei Frage-Antwort Aufgaben erzielt werden (C. Li et al., 2023). Nach aktuellem Wissensstand ist die vorliegende Arbeit die erste, die solche erweiterten Prompt-Methoden im Kontext von NER- und RE-Aufgaben untersucht. Die Ergebnisse deuten darauf hin, dass die Effektivität dieser Methoden domänenspezifisch sein könnte und nicht ohne Weiteres auf alle NLP-Aufgaben übertragbar ist.

Die Methode der *indirekten Strukturierung* wurde primär implementiert, um Large Language Models mit Zero-Shot für die Strukturierung von unstrukturierten Eignungskriterien einzusetzen. Dabei wird das Modell angewiesen, die unstrukturierten Texte ohne vorheriges Training oder Beispiele in eine strukturierte Form zu überführen. Die Ergebnisse zeigen, dass die Nutzung von Open-Source Modellen im Zero-Shot Verfahren nahezu die Leistung eines Regex-Parsers erreicht, der die Strukturierung regelbasiert umsetzt. Ein wesentlicher Vorteil dieses Ansatzes besteht darin, dass die Entwicklung eines Parsers spezialisiertes Fachwissen im Bereich der Informatik erfordert, während der Zero-Shot Ansatz von medizinischem Fachpersonal allein durch die Erstellung geeigneter Anweisungen (Prompts) realisiert werden kann, falls entsprechende LLMs bereitgestellt werden. Dies erlaubt Domänenexperten, ihre Fachkenntnisse direkt in den Strukturierungsprozess einzubringen, ohne tiefgehende Programmierkenntnisse zu benötigen. Die Zero-Shot Methodik kann über einen Human-in-the-Loop Ansatz verbessert werden, wenn die logisch angereicherten Textdaten in eine AST-Struktur verarbeitet werden.

7.1.1. Zero-Shot

Llama3 mit Zero-Shot extrahiert die meisten Operatoren mit hohen Recall-Scores von etwa 70%. Allerdings erzielt es geringe Precision-Scores, die für AND nur 16.4% und für NOT 43.5% betragen. Dies deutet darauf hin, dass das Modell zwar einige annotierte Operatoren korrekt erkennt, jedoch auch einige AND- und NOT-Operatoren im Vergleich zum Goldstandard falsch setzt. Folgende Modellausgaben von Llama3-70B mit Zero-Shot sollen demonstrieren, wie die logischen Operatoren von dem Modell gesetzt werden.

Ausgabe der Eignungskriterien der Studie NCT03862820 (.txt):

```
1 Inclusion Criteria:
2 Pregnant women [AND] in third trimester [AND] planned to undergo
3 elective cesarean section
4
5 Exclusion Criteria:
6 [NOT] Primary elective cesarean section
```

Im Goldstandard der Studie NCT03862820 wurde kein Operator annotiert, was bedeutet, das Modell erzeugt hier False Positives für AND- und NOT-Operatoren. Bei der Durchsicht des ausgegebenen Textes wäre es aber nachvollziehbar, die Satzteile, wie vom Modell vorgenommen, über die Operatoren zu verbinden. Gleiches gilt für den folgenden Auszug aus einer demonstrativen Ausgabe des Zero-Shot Modells.

Ausgabe der Einschlusskriterien der Studie NCT03929614 (.txt):

```
1 - 10-16 years old [AND] with brace treatment.
2 - Ability to read [AND] understand English [OR] French.
3 - Physical [AND] mental ability to adhere to bracing protocol.
```

In diesem Fall wurde nur das AND im ersten Satz im Goldstandard annotiert. Die folgenden Operatoren wurden fälschlicherweise vom Modell hinzugefügt. Bei einer manuellen Bewertung könnten die zusätzlichen Operatoren jedoch als korrekt angesehen werden. Ein weiterer Auszug aus einer Modellausgabe zeigte die Komplexität und Ambivalenz der Eignungskriterien, die häufig die Hinzuziehung eines Mediziners zur Bewertung erforderlich machen.

Ausgabe der Einschlusskriterien der Studie NCT03928080 (.txt):

```

1 Inclusion Criteria:
2 - Aged 18 [AND] and above.
3 ...

```

Im Goldstandard wurde im Ausgabesatz kein Operator annotiert. Beim Betrachten der Studie in ClinicalTrials.gov (2000) wird deutlich, dass nach Personen gesucht wird, die 18 Jahre alt sind und nach Personen, die älter als 18 Jahre sind. Die Verknüpfung mit einem AND-Operator würde jedoch nur Personen über 18 Jahre zulassen. Der Operator wurde hier vom Modell falsch eingefügt, über eine manuelle Durchsicht könnte das AND durch ein OR ersetzt werden, um die Logik korrekt abzubilden. Die Methodik der indirekten Strukturierung aus Abbildung 5.7 sieht hier vor, dass die Ausgabedateien über einen AST-Parser in ein JSON-Format als Abstract Syntax Tree verarbeitet werden, um die komplexe Logik abzubilden, aber auch um eine einfachere Prüfung über einen Human-in-the-Loop Ansatz zu ermöglichen. Die Einschlusskriterien der Studie NCT03928080 lassen sich als Abstract Syntax Tree einfach visualisieren, wie in Abbildung 7.1 gezeigt.

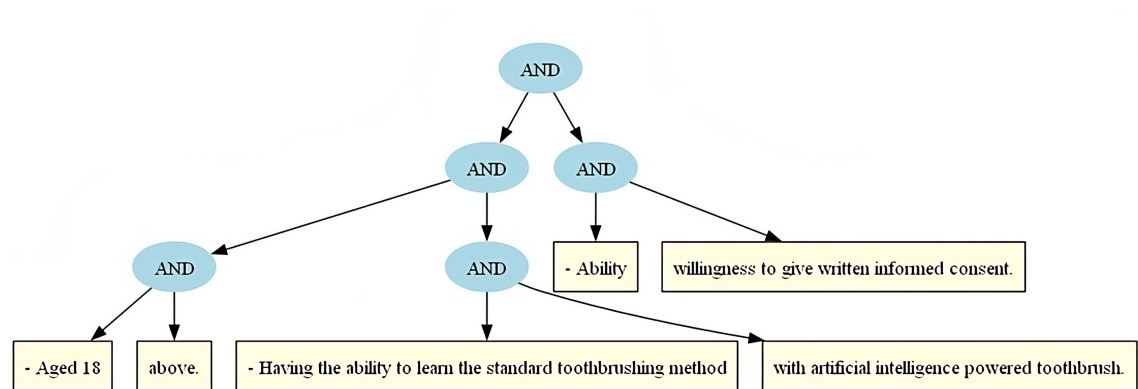


Abbildung 7.1.: Verknüpfte Ein- und Ausschlusskriterien der Studie NCT03860402 als AST.

Über ein zu implementierendes, interaktives User-Interface kann die Integration eines Human-in-the-Loop Ansatzes ermöglicht werden. Hierbei würde eine medizinische Fachkraft die Rolle des menschlichen Experten einnehmen, um die strukturierten Kriterien zu überprüfen und bei Bedarf Korrekturen vorzunehmen. Durch die direkte Verknüpfung des User-Interface mit der JSON-Datei wäre eine unmittelbare Validierung und Modifikation der extrahierten Strukturen möglich. Eine Fachkraft könnte über eine Visualisierung der Kriterien wie in Abbildung 7.1 erkennen, dass der AND-Knoten zwischen “- Aged 18” und “above.” falsch gesetzt ist, und ihn durch einen OR-Operator ersetzen. Zudem wurde gezeigt, dass nicht jeder eingesetzte Operator, der ein False Positive erzeugt, also nicht annotiert wurde, auch logisch falsch sein muss. In einigen Fällen, in denen das LLM einen Operator gesetzt hat, der nicht annotiert war, ergibt dies durchaus Sinn.

7.1.2. Few-Shot

In Few-Shot Szenarien erreichen Open-Source Modelle bei der Operatorenextraktion vergleichbare Ergebnisse wie Closed-Source Modelle, zeigen jedoch häufig höhere Ausgabefehler. Mit Zero-Shot erzielte das Llama3-Modell unter den untersuchten Open- und Closed-Source Modellen die beste Leistung und übertraf dabei GPT-4o. Mit fünf ausgewählten Beispielen aus den LCT-Trainingsdaten erreichte das Llama3-70B Modell einen F1-Score von 55.8% und übertraf damit den Regex-Parser. Eine Leistungssteigerung konnte auch durch den Einsatz proprietärer Modelle nicht erzielt werden. Aufgrund der höheren Kontextlänge proprietärer LLMs konnte mit Claude-3.5-Sonnet mit 15-Shot der höchste F1-Score von 60.7% erreicht werden. Unter Verwendung von Beispielen mit maximal vielen annotierten Operatoren erzielt Llama3-70B mit 5-Shot (MAX) über 20 Versuche einen durchschnittlichen F1-Score von 56.3% mit einer Standardabweichung von 1.5%. Zusätzlich wurde mit Llama3-70B evaluiert, dass eine Erhöhung der Anzahl der Shots von 1 auf 5 über 20 Versuche jeweils zu einer höheren Precision mit nahezu gleichbleibender Standardabweichung, aber zu einem geringeren Recall mit höherer Standardabweichungen führt.

Ein maximaler Recall von ca. 75% kann erreicht werden, wenn 5 Beispiele mit maximal vielen annotierten Operatoren mitgegeben werden. Die Few-Shot Ansätze weisen noch hohe Standardabweichungen in der Ausgabe auf, wobei diese mit der Nutzung von Beispielen mit maximal vielen annotierten Operatoren reduziert werden können.

Bei der Few-Shot Untersuchung wurde festgestellt, dass aktuelle LLMs wie Llama3, Gemma2 und Qwen2 aus Tabelle 5.1, selbst mit Zero-Shot eine höhere Leistung erzielen als das ältere Modell Llama2 mit 3-Shot. Dabei wurde Llama2 im Juli 2023 veröffentlicht und die Nachfolgegeneration Llama3 im April 2024. Das Llama2-70B Modell hatte mit 3-Shot einen extrem hohen Ausgabefehler von 50.8% und einen F1-Score von 15.3% für die Operatorenerkennung. Das wesentlich kleinere Llama3-8B Modell hatte mit Zero-Shot einen geringeren Ausgabefehler von 40.4% und einen höheren F1-Score von 27.5%, was auch nicht zufriedenstellend ist, aber zeigt, dass zukünftige Architekturen mit weniger zu verarbeitenden Beispielen möglicherweise bessere Ergebnisse erzielen können. Die Ergebnisse signalisieren, dass die Forschung zu NER und RE mit LLMs noch in einem frühen Stadium ist und das volle Potenzial erst mit neueren Generationen ausgeschöpft werden kann. Dies wird auch durch weitere Forschungsarbeiten wie X. Xu et al. (2023) bestätigt, die beobachtet haben, dass Zero-Shot und Few-Shot Methoden eine große Leistungslücke gegenüber Modellen mit Fine-Tuning aufweisen können. Die Leistung von GPT-3.5 mit In-Context Learning können um 19% F1-Score niedriger sein als die von Encoder-only PLMs mit Fine-Tuning (X. Xu et al., 2023). Die Ergebnisse dieser Arbeit bestätigen diese Beobachtung, so lag beispielsweise Llama3-70B mit 5-Shot 20.1% im F1-Score unter dem SciBERT-Modell mit Fine-Tuning. Mit Zero-Shot lag das Modell um 26.2% unter dem SOTA-Modell.

Ergänzend dazu ist anzumerken, dass für NER- und RE-Aufgaben häufig proprietäre Modelle wie das GPT-3.5 Modell mit In-Context Learning untersucht wurden, die mit 175B-Parametern deutlich größer sind als Open-Source Modelle wie Llama2 oder Llama3 mit 70B-Parametern und auch in der Regel bessere Leistungen bringen (X. Xu et al., 2023). Bei der Entitätserkennung auf dem Chia-Datensatz, in Tabelle 4.2 gezeigt, schneidet GPT-3.5 mit 5-Shot, mit 36% F1-Score zwar nur leicht besser ab als Llama2-70B mit 35%, dafür aber schlechter als ein ecBERT-Modell welches 54% F1-Score erreicht (Munnangi et al., 2024). Open-Source LLMs mit In-Context Learning bei Information Extraction Aufgaben schneiden so in der Regel schlechter ab als Closed-Source Modelle, wobei der Leistungsunterschied je nach Aufgabe gering sein kann (X. Xu et al., 2023). Obwohl Open-Source LLMs wie Llama3-70B mit Few-Shot auch schlechter abschneiden als PLMs mit Fine-Tuning, erzielen sie dennoch einen um 4.9% höheren F1-Score als ein regelbasierter Ansatz mit einem Regex-Parser. Die Few-Shot Methode könnte analog zum Zero-Shot Verfahren über einen Human-in-the-Loop Ansatz implementiert werden, wenn dem medizinischen Fachpersonal Zugang zu dem LLM und annotierten Beispieldatensätzen gegeben wird.

7.1.3. Fine-Tuning

Das aktuelle SOTA-Modell SciBERT erreicht mit Fine-Tuning auf dem LCT-Datensatz ein F1-Score von 75.9% für die Operatorenextraktion mit einem Recall von 81.4% und einer Precision von 71.2%. Dabei übertraf das Llama3-70B Modell mit Fine-Tuning das SOTA-Modell um 1.5%. Das Llama3-8B Modell schnitt mit Zero-Shot und 5-Shot deutlich schlechter ab als größere 70B Modelle, übertraf aber nach einem Fine-Tuning auf 80% der LCT-Daten im Mittel über 20 Versuche das SciBERT-Modell um 0.4% im F1-Score und erreichte eine um 6.7% höhere Precision. Wie die Hyperparameteroptimierung von Llama3-8B in Tabelle 6.6 zeigt, ist ein Fine-Tuning insofern aufwendig, da neben SFT- auch die LoRA-Parameter angepasst werden müssen. Die Nutzung der jeweiligen Standardparameter für SFT und LoRA ist nicht zu empfehlen, da diese schlechte Ergebnisse mit unter 30% F1-Score für die Operatorenextraktion lieferten. Eine Anpassung dieser Parameter ist daher zwingend erforderlich. Da bis zum Erstellen dieser Arbeit ein Fine-Tuning von LLMs mit Decoder-only Architektur aufgrund des Fehlens von Parameter-Efficient Fine-Tuning Methoden nicht verbreitet war, fehlt eine Übersicht, wie die SFT- und LoRA-Parameter für LLMs effizient eingestellt werden können. Diese mussten daher über eine Hyperparameteroptimierung ermittelt werden. Erst aktuelle Publikationen wie Mo et al. (2024) untersuchen ein Fine-Tuning von LLMs. Ein Fine-Tuning selbst von verhältnismäßig kleinen 8B-Modellen liefert bessere Ergebnisse als das aktuelle SOTA-Modell. Der Zeit- und Ressourcenaufwand wird dabei unter Verwendung moderner Bibliotheken und dem Einsatz von PEFT-Methoden nicht wesentlich höher eingeschätzt als bei dem BERT-Modell. Das Fine-Tuning von Llama3-8B dauerte auf einer NVIDIA A100 GPU

lediglich 30 Minuten, während das Fine-Tuning des Llama3-70B Modells etwa drei Stunden in Anspruch nahm. Es ist anzumerken, dass der LCT-Trainingsdatensatz mit 723 Dateien, der bei der indirekten Strukturierung verwendet wurde, relativ klein ist. Ein Fine-Tuning mit Tausenden oder Zehntausenden von Dateien könnte zwar durch die Verwendung einer niedrigen Batchgröße ebenfalls auf einer GPU initialisiert werden, würde jedoch deutlich mehr Zeit in Anspruch nehmen. Positiv zu vermerken ist, dass die Anzahl der Epochen, also wie oft das Modell die Trainingsdaten vollständig durchläuft, auf 10 festgelegt werden konnte und selbst eine Erhöhung auf bis zu 40 Epochen zu keinen besseren Ergebnissen führte. Da ein Fine-Tuning mit einer geringeren Anzahl von Epochen deutlich schneller durchgeführt werden kann, deutet dies darauf hin, dass LLMs für Information Extraction-Aufgaben auch bei größeren Korpora relativ schnell und effizient trainiert werden können.

Einordnung der Ergebnisse

Beide Llama3-Modelle, das 8B- und das 70B-Modell, haben mit Fine-Tuning besser abgeschnitten als das SciBERT-Modell. Berücksichtigt man die Standardabweichung über 20 Versuche, schneidet das 8B-Modell im schlechtesten Fall knapp unter dem SOTA-Modell ab, während das 70B-Modell immer noch darüberliegt. Anzumerken ist auch, dass die durchschnittliche Standardabweichung bei den Fine-Tuning Modellen mit ca. 0.5% F1-Score relativ gering ist. Was einen direkten Vergleich mit dem SOTA-Modell SciBERT zusätzlich erschwert, ist, dass das SciBERT-Modell die logischen Operatoren AND und OR bei der Extraktion genau auf zwei Entitäten bezieht und diese implizit dazwischen schreibt. Bei NOT-Operatoren wird das Wort im Satz markiert, welches eine weitere Entität negiert. Es wird also eine Relation Extraction vorgenommen, die die über eine Named Entity Recognition extrahierten Entitäten verbindet. In dieser Arbeit werden die annotierten Operatoren in die Ursprungstexte eingefügt, um sie als Entitäten zu behandeln. Dadurch werden die Textabschnitte vor und/oder nach den Operatoren verbunden. Eine vorherige Entitätsextraktion der tatsächlich verbundenen Entitäten ist somit nicht notwendig, kann aber anschließend vorgenommen werden, wenn der verknüpfte Ursprungstext analysiert wird. Ziel dieser Arbeit war es, Eignungskriterien über die Extraktion der inhärenten Logik in eine Abstract Syntax Tree zu überführen. Vorwiegend sollten dafür In-Context Learning Methoden evaluiert werden. Allerdings hat die Evaluation der direkten Strukturierung gezeigt, dass aktuelle Open-Source LLMs mit Few-Shot Methoden Schwierigkeiten haben, die Entitäten korrekt zu erkennen und gleichzeitig die logische AST-Struktur basierend auf der Extraktion der Operatoren aufzubauen. Um zunächst nicht auf die korrekte Extraktion von Entitäten angewiesen zu sein und um vor allem In-Context Learning Methoden zu evaluieren, sollen die Operatoren von den Modellen in die Kriterientexte eingefügt werden. Dies unterstützt zwar einerseits die Stärken von Decoder-only LLMs, die generativ Texte erzeugen, erschwert aber andererseits einen direkten Vergleich mit den BERT-Modellen. Die BERT-Modelle haben eine Relation Extraction-Aufgabe bei der Operatorenextraktion durchgeführt, während die LLMs eine Named Entity Recognition-Aufgabe ausgeführt haben. Ergänzend ist anzumerken, dass es für LLMs als nicht praktikabel angesehen wird, die logischen Operatoren als Relationen inklusive der verknüpften Entitäten zu extrahieren und gleichzeitig die AST-Struktur aufzubauen. Da der Fokus auf der Erzeugung der verschachtelten AST-Strukturen lag, muss dieser Ansatz so implementiert werden. Da bei der Erstellung des Goldstandards die Operatoren exakt zwischen den Positionen der verknüpften Entitäten platziert wurden, müssen diese sich in den verbundenen Textteilen befinden. Bei der indirekten Strukturierung können die Entitäten anschließend aus den Textteilen extrahiert werden, während dies bei der direkten Strukturierung parallel erfolgt.

7.2. Chia-Korpus

Unter Berücksichtigung, dass ein vollständiger Vergleich mit dem SOTA-Modell schwierig ist, hat das Llama3-Modell mit Fine-Tuning bei der Operatorenextraktion eine gute Leistung erzielt. Um die Generalisierungsfähigkeit des Modells auf den LCT-Daten zu validieren, wurde es auf dem gesamten Chia-Korpus getestet. Außerdem wurde ein Llama3-Modell mit Zero-Shot getestet und mit dem Regex-Parser verglichen. Die im vorherigen Experiment entwickelte Prompt 2 wurde dem Modell mit Zero-Shot übergeben und erzielte bei der Extraktion der Operatoren einen Recall von 48.3% und einen F1-Score von 31.4%, was jeweils höher war als bei der Anwendung des Regex-Parsers, der einen F1-Score von 28.6% erzielte. Das Llama3-70B Modell mit Fine-Tuning auf den

LCT-Daten erreichte einen Recall von 54.4% und einen F1-Score von 36.8% bei geringem Ausgabebefehler. Da ein SOTA-Vergleich fehlt, sind die Ergebnisse schwer zu interpretieren. Das Modell mit einem vorherigen Fine-Tuning schneidet mit einem um 5% höheren F1-Score nicht wesentlich besser als das Zero-Shot Modell ab. Der Regex-Parser wird um 8% F1-Score übertroffen, wobei das Llama3-Modell zumindest generalisierungsfähiger als ein regelbasierter Ansatz ist. Die Erwartungen wären hier jedoch deutlich höher gewesen. Bei manueller Durchsicht der annotierten Chia-Daten stellte sich heraus, dass unterschiedliche Annotationsverfahren in Bezug auf die Operatoren angewendet wurden. So wurden beispielsweise OR-Operatoren nicht einheitlich vor Kommas gesetzt, obwohl sie an vergleichbaren Stellen hätten gesetzt werden sollen. AND-Operatoren wurden im Chia-Korpus häufig nach anderen Signalwörtern annotiert als im LCT-Korpus. Das folgende Beispiel soll die Unterschiede demonstrieren.

Auszug aus dem Goldstandard des Chia-Korpus der Ausschlusskriterien (NCT02777424):

```
1 Hypersensitivity [AND] to a PCC
2 History of thrombocytopenia [AND] induced by heparin
3 Hypersensitivity [AND] to vitamin K
```

Die Annotationen sind nachvollziehbar, wurden aber im LCT-Korpus anders gehandhabt, wie der folgende Auszug aus einer annotierten Studie des LCT-Korpus zeigt.

Auszug aus dem Goldstandard des LCT-Korpus der Eignungskriterien (NCT03890220):

```
1 - hypersensitivity to telmisartan [OR] or rosuvastatin
```

Im Chia-Korpus wird das Wort "to" als Signalwort für einen AND-Operator annotiert, wenn es *Hypersensitivity* mit einer weiteren Entität verknüpft. Im Gegensatz dazu wird es im LCT-Korpus nicht auf diese Weise annotiert. Dieses Beispiel verdeutlicht die unterschiedlichen Annotationsstile der beiden Korpora, was die Evaluation auf beiden Datensätzen erschwert. Da in beiden Korpora logische Annotationen zu fehlen scheinen, die man intuitiv setzen würde, stellt sich die Frage, inwiefern auch der LCT-Korpus für eine adäquate Evaluation geeignet ist.

7.3. LCT-Korpus

Der Vorteil des LCT-Korpus besteht darin, dass neben 50 Entitäten auch 51 Relationen annotiert wurden, die die logischen Operatoren AND, OR und NOT enthalten. Im Gegensatz zum Chia-Korpus und allen weiteren öffentlichen Korpora wie FRD, COVIC und LLF aus Tabelle 4.1, ist mit dem LCT-Korpus ein Vergleich mit SOTA-Modellen hinsichtlich genau dieser drei Operatoren möglich. Eignungskriterien werden vom medizinischen Fachpersonal als Freitext eingegeben und liegen daher oft unstrukturiert vor. Die für den LCT-Korpus extrahierten Eignungskriterien wurden zwar zufällig ausgewählt, weisen aber häufig einen semi-strukturierten Charakter auf, wie die Kriterien der Studie NCT03860402 in Abbildung A.3 im Anhang zeigen. Hier sind Ein- und Ausschlusskriterien in Form einer Liste dargestellt, wodurch zumindest die einzelnen Kriteriensätze voneinander getrennt sind. In der Praxis ist dies jedoch nicht immer der Fall, wie die Eignungskriterien der Studie NCT01179776 im Anhang in Abbildung 1 verdeutlichen. Hier wurde zwar eine Aufzählung angedacht, diese jedoch unregelmäßig und falsch umgesetzt. Dabei wäre es zukünftig zu evaluieren, wie Large Language Models mit Eignungskriterien-Datensätzen umgehen, die strukturell anders als die Daten des LCT-Korpus aufgebaut sind. Auch kann die Länge der Eignungskriterien in weiteren Versuchen eine Rolle spielen. Für die zufällige Aufnahme in den LCT-Korpus müssen die Kriterien eine Mindestlänge von 50 Zeichen haben. Mittels eines Python-Skripts wurde ermittelt, dass die durchschnittliche Zeichenanzahl der unbearbeiteten Eignungskriterientexte im LCT-Korpus bei ca. 672 liegt. Die Eignungskriterien der Studie NCT04412629 aus Abbildung 2.1 und Abbildung 2.2 umfassen genau 10142 Zeichen und sind damit etwa 15-mal so groß wie die durchschnittlichen LCT-Kriterien. Zum einen wurde die Studie NCT04412629 von Wong et al. (2023) als Negativbeispiel ausgewählt, um zu zeigen, wie groß, komplex und unstrukturiert Eignungskriterien sein können, zum anderen kommen solche Studien aber in der Praxis tatsächlich vor (Wong et al., 2023). Es wäre daher zu evaluieren, wie Modelle mit deutlich größeren Datensätzen umgehen. Eine Möglichkeit wäre, die einzelnen Kriteriensätze den Modellen jeweils separat zu übergeben, was jedoch den Aufbau eines Frameworks mit mehreren Parser-Prozessen erfordern und die Kosten der Modellinferenz erhöhen würde.

7.4. Direkte Strukturierung

Bei der direkten Methode, Eignungskriterien mithilfe von Large Language Models direkt in ein JSON-Format auszugeben, haben sich Few-Shot Methoden mit Open-Source LLMs wie Llama3-70B und Qwen2-72B auf dem LCT-Datensatz nicht als praktikabel, jedoch prinzipiell durchführbar erwiesen. Die Operatoren wurden mit Qwen2 im 10-Shot Verfahren mit einem maximalen F1-Score von 15% erkannt, wobei jedoch 44% der JSON-Dateien fehlerhaft waren und nicht in die Bewertung einfließen. Das Llama3-Modell erreichte im 2-Shot Verfahren 10.7% bei der Operatorenerkennung und 38.9% bei der Entitätserkennung, mit einem relativ geringen Ausgabefehler (JSON-Fehler) von 4.7%, jedoch einem hohen Ausgabefehler von 18.7%, was bedeutet, dass der Ursprungstext nicht korrekt übernommen wurde. Das GPT-4o Modell erzielte unter den Few-Shot Modellen mit 10-Shot die besten Ergebnisse mit F1-Scores von 18.3% bei der Operatorenerkennung und 43.2% bei der Entitätserkennung. Zwar konnten ca. 10% der erzeugten JSON-Dateien nicht verwendet werden, jedoch war der Ausgabefehler mit 3.7% am niedrigsten unter den Few-Shot Modellen. Unter Berücksichtigung, dass den Few-Shot Modellen nur die Entitätstypen über die Prompt mitgegeben wurden und keine explizite Beschreibung, wie diese definiert sind, ist ein F1-Score von ca. 43% eine Leistung, auf der aufgebaut werden kann. Positiv anzumerken ist, dass es für Open-Source LLMs mit 2-Shot möglich war, die komplexe AST-Struktur im JSON-Format zu bilden in Verbindung mit der Operatorenextraktion, Entitätserkennung und Übernahme des Originaltextes. Die Mitgabe von mehr Beispielen (10-Shot) erhöht zwar die F1-Scores für Operatoren- und Entitätsextraktion, aber auch die Ausgabefehler der Modelle.

Dabei muss auch der explorative Ansatz dieser Arbeit erwähnt werden. Zunächst wurde versucht, die Eignungskriterien über In-Context Learning Methoden direkt in die AST-Struktur zu bringen. Die ersten Versuche schlugen fehl. Es war zwar möglich, das JSON-Format mit den LLMs und Few-Shot zu erzeugen, aber die Fehlerraten im Ausgabeformat waren zu hoch, teilweise über 80%. Daraufhin wurde versucht, die Modelle dazu zu bringen, zuerst nur die Operatoren zu extrahieren, ohne die Struktur aufzubauen, und die Operatoren in die Texte einzufügen, damit die Entitäten nicht berücksichtigt werden müssen. Nach mehrfachen Iterationen der Promptanweisungen war dies auch mit In-Context Learning Methoden möglich, über ein Fine-Tuning sogar besser als SOTA-Modelle. Dabei zeigte die erstellte Prompt 2 (Abbildung 5.8) mit einfachen Anweisungen bessere Leistung als Prompt 1 (Abbildung C.1), die mehr Beschreibungen und Beispiele zur Operatorenextraktion beinhaltete. Da das Einfügen in den Ursprungstext, vor allem mit Fine-Tuning, gut funktionierte, wurde wieder dazu übergegangen doch das JSON-Format direkt mit den LLMs auszugeben. Dabei wurde auf die Prompts für die Operatorenextraktion zurückgegriffen und in mehrfachen Iterationen die Strukturprompt in Abbildung C.3 im Anhang entworfen. Über mehrere Versuche hat sich herausgestellt, dass LLMs bei der JSON-Generierung besser funktionieren, wenn die Prompt 1 in die Strukturprompt eingefügt wird. Bei vorherigen Experimenten mit Textausgabe war es die einfachere Prompt 2, die besser funktionierte. Mit diesen Verbesserungen in der Prompt-Nutzung in Kombination mit dem Wissen aus den Fine-Tuning Experimenten, wie die Hyperparameter für Information Extraction-Aufgaben mit LLMs am besten gesetzt werden, konnten auch Llama3-Modelle für die direkte Ausgabe von AST-Strukturen einem Fine-Tuning unterzogen werden und besser als SOTA-Modelle funktionieren. Ohne diese explorative Arbeit in Bezug auf Prompting und Fine-Tuning Methoden in der indirekten Strukturierung wäre die direkte Ausgabe nicht so gut gelungen. Wie Abbildung 6.11 zeigt, konnten über ein Fine-Tuning von Llama3-70B Operatoren mit einem F1-Score von 84.6% und Entitäten mit 88.5% erkannt werden, wobei 98.8% der ausgegebenen JSON-Dateien verwertet werden konnten und der Ausgabefehler mit 0.5% niedrig ist und der Text nahezu vollständig übernommen wird. Eine Evaluation des Modells über 20 Versuche in Tabelle 6.16 zeigt, dass das Modell konsistent eine bessere Leistung als das SOTA-Modell erzielt. Ein Fine-Tuning von Llama3-8B zeigt, dass das kleinere Modell bei der Operatorenextraktion um 7% unter dem 70B-Modell lag, aber bei der Entitätsextraktion eine ähnlich gute Leistung bringt.

7.5. Implikationen für die Praxis

Open-Source Large Language Models können mit In-Context Learning Methoden verwendet werden, um Eignungskriterien logisch zu strukturieren. Wenn nur eine einzelne Promptanweisung erstellt wird, also ein Zero-Shot Verfahren angewandt wird, erreicht man ungefähr die Leistung von regelbasierten Ansätzen. Stehen jedoch einige Beispiele zur Verfügung und wird das Few-Shot Verfahren angewandt, wird der regelbasierte Ansatz übertroffen. Für die Erzeugung der Abstract Syntax Tree-Struktur muss ein AST-Parser verwendet werden. Dabei wird der Ursprungstext logisch strukturiert, was die Ausgabe im interoperablen JSON-Format prüfbar macht. Allerdings bringt die Nutzung von ICL-Methoden auch unerwünschte Textveränderungen mit sich, die je nach Modell und Methode jedoch als akzeptabel angesehen werden können. Proprietäre LLMs wie GPT-4o oder Claude-3.5-Sonnet haben in der Regel geringere Ausgabefehler als die Open-Source Modelle und weisen mit ICL-Methoden meist eine etwas bessere Leistung auf. Eine Ausnahme bildet Llama3-70B, das mit ICL-Methoden unter den Open-Source Modellen am besten abschneidet, jedoch bezüglich der Kontextlänge eingeschränkt ist und Schwierigkeiten bei der Verarbeitung von mehr als fünf Beispielen hat.

Die Kosten für die Nutzung proprietärer Modelle über die entsprechende API haben während der Erstellung dieser Arbeit sukzessive abgenommen. Während die Erzeugung von 202 Textdateien mit GPT-4o im 5-Shot Verfahren zu Beginn der Arbeit im Mai 2024 noch etwa \$13 kostete, reduzierte sich der Preis auf \$5.60 bis September 2024. Bei Claude-3.5-Sonnet lagen die Kosten konstant bei \$1.74. Wenn Prompts und Beispiele optimiert vorliegen, können diese Modelle kostengünstig angewandt werden. Da für die Untersuchungen von Prompts, Modellparametern und Beispielen in dieser Arbeit tausende Dateien erzeugt wurden, wären die Experimente allein durch die Nutzung von Closed-Source Modellen ohne eine erweiterte Finanzierung nicht möglich gewesen. Die Experimente wurden hauptsächlich mit Open-Source Modellen durchgeführt. Dabei zeigte sich, dass Llama3-8B, welches mit ICL-Methoden eine unterdurchschnittliche Leistung aufweist, durch ein Fine-Tuning, unter Berücksichtigung der Standardabweichung, bei der Operatorenextraktion genauso gut abschneidet wie ein BERT-basiertes SOTA-Modell. Für die Inferenz, die Anwendung auf einem Rechencluster, werden für das Llama3-8B Modell im Few-Shot Verfahren bis zu 100GB VRAM benötigt. Das Fine-Tuning verbraucht hingegen nur 20GB VRAM und ist in 20 Minuten möglich. Für eine direkte Strukturierung mit JSON-Ausgaben dauerte das Fine-Tuning zwei Stunden, da der Trainingsdatensatz doppelt so groß war. Das Fine-Tuning des 8B-Modells kann auf einem Knoten mit vier Nvidia Tesla V100 GPUs mit je 16GB VRAM durchgeführt werden. Damit stehen insgesamt 64GB VRAM zur Verfügung, was die meisten Rechencluster von Forschungseinrichtungen pro Knoten mindestens bereitstellen. Zukünftig könnte die Inferenz über Methoden wie Quantisierung oder Speicherspartechiken eventuell auch auf solchen kleineren Rechenknoten durchgeführt werden. Größere LLMs wie Llama3-70B können zwar über PEFT-Methoden ressourcenschonend einem Fine-Tuning unterzogen werden, aber für die Inferenz ist dennoch ein Cluster mit über 260GB VRAM pro Knoten notwendig.

Für die direkte Strukturierung von Eignungskriterien als Abstract Syntax Tree im JSON-Format bietet sich ein Fine-Tuning von Open-Source Modellen wie Llama3-70B an. Dieses Modell zeigt sowohl bei der Entitätsextraktion als auch bei der Operatorenextraktion, unter Berücksichtigung der erzeugten AST-Struktur und der Integration des Ursprungstextes, eine bessere Leistung als ein BERT-Modell mit Fine-Tuning. Ein solches Modell mit Fine-Tuning könnte über Plattformen wie Hugging Face¹ der medizinischen Forschungsgemeinschaft und Mediziner zur Verfügung gestellt werden. Die Nutzung müsste dann über ein Rechencluster erfolgen. Da die Ausgabe im interoperablen JSON-Format als AST implementiert wurde, können die Daten mittels eines zu entwickelnden Parsers in ein FHIR-Format transformiert werden, das von den meisten medizinischen Systemen unterstützt wird. Mit diesem Vorgehen würde bereits ein einzelnes LLM-Modell für die Strukturierung ausreichen, während in aktuellen Systemen mehrere Modelle innerhalb eines Frameworks über verschiedene Prozesse die Strukturierung vornehmen. Wenn zukünftig modernere Open-Source LLMs mit größeren Kontextlängen und präziserer Befolgung von Instruktionen evaluiert werden, wäre es eventuell möglich, Eignungskriterien in einem End-to-End Prozess direkt im medizinischen Standardformat FHIR auszugeben.

¹<https://huggingface.co/>

7.6. Vergleich mit SOTA-Systemen

Das LeafAI-System in Abbildung 4.4 implementiert ein PLM-Framework, wobei dem System einzelne Eignungskriterien-Sätze übergeben werden. Diese werden über eine Kombination aus mehreren Encoder-only- und Encoder-Decoder Modellen in Verbindung mit regelbasierten Parsern in einen Abstract Syntax Tree verarbeitet. Die logische AST-Struktur des LeafAI-Systems wurde mit dem Ziel entwickelt, SQL-Abfragen zu generieren, und ist deshalb an die SQL-Syntax angelehnt. Abbildung 4.9 zeigt eine exemplarische Ausgabe. Die Kriteriensätze werden hier einzeln verarbeitet und durch mehrere Modelle und Methoden bis zur Ausgabe bearbeitet. Hier bietet der Ansatz, Kriterientexte direkt und ausschließlich über LLMs in eine Abstract Syntax Tree-Struktur im JSON-Format zu überführen, Vorteile. Während beim LeafAI-System einzelne Kriteriensätze prozessiert werden und bis zur logischen Struktur mehrere Modelle beteiligt sind, ist es mit Large Language Models mit Decoder-only Architektur möglich, eine ähnliche AST-Struktur zu erzeugen, die für den Menschen leserlicher ist und auch einfacher in weitere Formate wie FHIR übertragen werden kann, da sie nicht an eine SQL-Syntax angelehnt ist. Bei der direkten Strukturierung von Kriterientexten durch LLMs in die entwickelte AST-Struktur werden zwar nicht alle 51 Relationen des LCT-Korpus extrahiert, sondern nur die logischen Relationen, dafür jedoch alle Entitäten. Anstelle mehrerer BERT-Modelle für NER- und RE-Aufgaben sowie eines T5-Modells für die logische Struktur, gefolgt von nachgelagerten Parser-Prozessen, kann die AST-Struktur in einem einzigen Prozess durch LLMs generiert werden. Aufgrund der begrenzten Kontextlänge einiger Open-Source LLMs müssen Ein- und Ausschlusskriterien separat verarbeitet werden. Anschließend werden sie über einen Parser mit einem AND-Knoten verbunden, wobei die Ausschlusskriterien zuvor einem übergeordneten NOT-Knoten zugeordnet werden. Mit moderneren Modellen, die längere Kontexte unterstützen und Instruktionen präziser umsetzen können, ließe sich evaluieren, ob dieser Prozess in einem einzigen Schritt erfolgen kann. So könnten vollständige AST-Strukturen, wie in Abbildung 5.6 dargestellt, direkt ausgegeben werden.

Das LeafAI-System kann Eignungskriterien nur zeilenweise als einzelne Sätze in eine AST-Struktur prozessieren. Die entwickelte Methode bietet hier Vorteile, indem sie die Strukturierung über LLMs vornimmt, wodurch ganze Ein- oder Ausschlusskriterien verarbeitet werden können. Zudem sind die erzeugten ASTs leserlicher und für den Menschen zugänglicher, da auch die Ursprungstexte übernommen werden. Dies fördert zusätzlich die Validierung der erkannten Operatoren und Entitäten. Zum Vergleich der Ausgabe wurde ein Kriterien-Satz aus einem Dokument im LLF-Korpus mit beiden Ansätzen dargestellt. Der exemplarische Kriterien-Satz lautet:

“- Grade IV Glioblastoma , receiving chemotherapy with or without TTfields .”

LeafAI-Ausgabe der ersten Zeile der Studie NCT03861598 (.js):

```

1 intersect(
2     cond("Glioblastoma")
3         .num_filter(
4             eq(unit("Grade"), val("IV"))
5         )
6 ),
7 union(
8     proc("chemotherapy")
9         .temporality(
10             eq(temporal_per(PRESENT))
11         ),
12     neg(
13         proc("TTfields")
14     )
15 )

```

Llama3-70B Fine-Tuned Ausgabe der ersten Zeile der Studie NCT03861598 (.json):

```

1 {
2   "OR": {
3     "left": {
4       "raw_text": "Grade IV Glioblastoma, receiving chemotherapy with",
5       "EqComparison": ["Grade IV","receiving"],
6       "Condition": ["Glioblastoma"],
7       "Procedure": ["chemotherapy" ],
8       "EqUnit": [ "Grade"],
9       "EqValue": [ "IV"],
10      "EqTemporalPeriod": ["receiving"],
11    },
12    "right": {
13      "AND": {
14        "left": {
15          "raw_text": "or "
16        },
17        "right": {
18          "NOT": {
19            "left": {
20              "raw_text": "without TTfields.",
21              "Negation": ["without"],
22              "Procedure": ["TTfields"],
23            }
24          }
25        }
26      }
27    }
28  }
29 }

```

Die exemplarische LeafAI-Ausgabe liegt ebenfalls in einer logischen AST-Struktur vor. Dabei sind jedoch die Entitäten und Attribute aufgrund der Syntax, in der sie repräsentiert werden, schwer lesbar und auch schwieriger in andere Formate wie JSON zu verarbeiten. Dies ist auch der Grund, warum die Evaluation der entwickelten Methodik zunächst auf dem LCT- und Chia-Korpus vorgenommen wurde, da diese einfacher in einen Vergleichsdatensatz in AST-Struktur verarbeitet werden konnten. Zusätzlich wurden keine Modellergebnisse zu den NER- und RE-Aufgaben veröffentlicht, was einen direkten Vergleich nicht möglich macht. In Zukunft kann der LLF-Korpus dennoch evaluiert werden. Die Ausgabe des Llama3-Modells ist deutlich lesbarer und hat in diesem Beispiel die logische Struktur anders aufgebaut, während die Entitäten ähnlich erkannt wurden. Auffällig ist, dass die LeafAI-Ausgabe die Entitäten über einen Union-Operator, was einem logischen AND entspricht, verknüpft: "*chemotherapy* AND (NOT *TTfields*)". Die Llama3-Ausgabe ist hier korrekter, da zwischen diesen Entitäten eigentlich ein logisches OR steht. Ein Problem der AST-Ausgabe mit LLMs wird ebenfalls deutlich, da NOT-Operatoren immer mit einem AND-Operator verbunden werden müssen. In diesem Fall wurde über das AND ein kurzer Satzteil vor dem NOT-Operator verbunden, was inhaltlich keinen Sinn ergibt, aber für die strukturelle Darstellung notwendig ist.

Für die LeafAI-Ausgabe war ein Fine-Tuning von zwei BERT-Modellen und einem T5-Modell mit mehreren unterschiedlich verarbeiteten Trainingsdatensätzen notwendig. Zusätzlich musste die Ausgabe mit einem AST-Parser weiterverarbeitet werden. Insgesamt wurden also vier Methoden kombiniert. In dieser Arbeit wird hingegen für die direkte Strukturierung nur ein einzelnes Large Language Model mit Fine-Tuning eingesetzt. Im Vergleich zum LeafAI-System, welches eine Vielzahl von Relationen extrahiert, erkennen die verwendeten LLMs in dieser Arbeit zwar nur die logischen Relationen, können aber dafür größere AST-Strukturen verarbeiten.

In einer Forschungsarbeit von Wong et al. (2023) wurden mit proprietären LLMs wie GPT-4 mit In-Context Learning, Entitäten aus Eignungskriterien in ein JSON-Format extrahiert, welches die Logik zwischen den Entitäten in Disjunktive Normalform abbildet, wie in Abbildung 4.7 dargestellt. In der Untersuchung wurde evaluiert, wie die beiden Entitätstypen *histology* und *biomarker* aus Onkologiestudien mit GPT-4 mit Zero-Shot und 3-Shot Methoden extrahiert wurden. Während GPT-4 im Zero-Shot für die Biomarker-Extraktion noch einen F1-Score von 66.7% erreichte, sank die Leistung auf 22.2%, wenn die Biomarker strukturiert in DNF extrahiert werden mussten. Aufgrund dieser Leistungsunterschiede wurde angenommen, dass aktuelle LLMs im Zero-Shot noch Probleme haben beide Aufgaben, die Entitätsextraktion und die gleichzeitige logische Strukturierung, vorzunehmen. Deshalb wurde unter anderem dazu übergegangen, ICL-Methoden zu evaluieren, bei denen Operatoren in die Kriterientexte eingefügt werden, um so die logische Strukturierung zu einer reinen NER-Aufgabe zu machen. Beide Formate sowohl das von Wong et al. (2023) erstellte JSON-Format als auch die über die *indirekte Methode* logisch angereicherten Textdateien müssen für eine weitere Verarbeitung in eine AST-Struktur überführt werden. Durch die Verwendung der angereicherten Textdateien kann jedoch der Ursprungstext beibehalten werden aus dem in nachfolgenden Schritten eine Entitätsextraktion vorgenommen werden kann. Dabei werden mit der Methode von Wong et al. (2023) nur drei Entitätstypen extrahiert und logisch verbunden. Die direkte Strukturierung als Abstract Syntax Tree mit Large Language Models bietet hier den Vorteil, eine größere Anzahl unterschiedlicher Entitätstypen wie die 50 Typen im LCT-Korpus zu extrahieren und in die Struktur einzubinden.

8. Fazit

Das folgende Kapitel fasst die Ergebnisse der Strukturierung klinischer Eingangskriterien mithilfe der entwickelten Methodiken zusammen. Dabei werden die Forschungsfragen aus Abschnitt 1.2 beantwortet und die entwickelten Methodiken mit logischen Strukturierungs-Systemen aus Abschnitt 4.4 verglichen. Anschließend wird aufgezeigt, wie die entwickelten Methodiken in der Praxis angewendet und erweitert werden können.

8.1. Zusammenfassung

Die automatisierte Strukturierung von Eignungskriterien aus klinischen Studien mittels Machine Learning-Methoden wurde anhand zweier Hauptansätze untersucht, der indirekten und der direkten Strukturierung mit Large Language Models. Ziel beider Methoden war es, Eignungskriterien in eine Abstract Syntax Tree Struktur zu überführen, wobei logische Operatoren die Knoten des AST bilden und die verbundenen Textsegmente die Blätter darstellen. Diese AST-Struktur kann über ein JSON-Format repräsentiert werden, welches interoperabel ist und in medizinische Standardformate wie OMOP-CDM oder FHIR übertragen werden kann. Eine indirekte Strukturierung setzt Large Language Models ein, um logische Operatoren (AND, OR, NOT) in die Kriterientexte einzufügen. Dadurch werden Zero-Shot Ansätze ermöglicht und die Extraktion von logischen Operatoren als Relationen kann ohne Entitätsextraktion ausgewertet werden. Als Datengrundlage diente der öffentlich zugängliche LCT-Datensatz mit annotierten Eignungskriterien, der einen Vergleich der Operatorenextraktion anhand gemittelter Precision-, Recall- und F1-Scores über alle Operatoren ermöglicht. Es wurden die Open-Source Modelle Llama3-8B und 70B, Llama2-70B, Gemma2-27B, Qwen2-72B sowie proprietäre Modelle wie GPT-4o, GPT-4o-mini und Claude-3.5-Sonnet untersucht. Die Methodik, Operatoren direkt in die Ursprungstexte einzufügen, ermöglichte einen Zero-Shot Ansatz, für den zwei Prompts entwickelt wurden. Prompt 2, bestehend aus RegEx-ähnlichen Anweisungen zum Einfügen von Operatoren nach Signalwörtern, erwies sich als effektiver im Vergleich zu einer ausführlicheren Promptanweisung (Prompt 1). Die Prompt 2 konnte zudem durch einen RegEx-Parser implementiert werden.

Unter Verwendung von Prompt 2 erzielte Llama3-70B über 20 Durchläufe einen F1-Score von 50.7% und damit eine vergleichbare Leistung wie der RegEx-Parser, der einen F1-Score von 50.9% erreichte. Erweiterte Promptmethoden wie Zero-Shot CoT, Emotional Prompting und Role Prompting führten zu keiner signifikanten Verbesserung des Zero-Shot Verfahrens. Mit Few-Shot Methoden konnte die Leistung der LLMs verbessert werden, wobei Llama3-70B mit einer verbesserten 5-Shot (Max) Methode einen durchschnittlichen F1-Score von 56.3% erreichte und besser als ein RegEx-Parser abschneidet. Open-Source Modelle wie Llama3 zeigen in Few-Shot Szenarien vergleichbare oder bessere Leistungen als proprietäre Modelle wie GPT-4o, allerdings oft mit höheren Ausgabefehlern. Unter den proprietären Modellen erzielte Claude-3.5-Sonnet mit 15-Shot den höchsten F1-Score von 60.7%. Über ein Parameter-Efficient Fine-Tuning auf dem LCT-Trainingsdatensatz mit LoRA-Methoden übertraf das Llama3-8B Modell mit einem durchschnittlichen F1-Score von 76.3% die Leistung des State-of-the-Art Modells SciBERT mit einem F1-Score von 75.9% leicht, während Llama3-70B mit einem F1-Score von 77.4% eine bessere Leistung erzielt. Der Temperatur-Parameter, der die Kreativität der Modellausgabe steuert, hat keinen wesentlichen Einfluss auf die Modellleistung bei der Extraktion logischer Operatoren und sollte auf den Standardwerten belassen werden. Es wurde zudem gezeigt, dass bei Few-Shot Methoden mehr Beispiele zu besseren Ergebnissen führen, jedoch auch die Varianz der Ergebnisse erhöhen. Die Verwendung von Beispielen mit möglichst vielen annotierten Operatoren kann die Leistung weiter verbessern. Das Llama3-70B Modell mit Fine-Tuning auf dem LCT-Datensatz wurde, um die Generalisierungsfähigkeit zu testen, zusätzlich auf dem Chia-Korpus evaluiert. Das Modell erzielt mit einem F1-Score von 36.8% eine schlechte Leistung, zeigt jedoch mit einem Recall von 54.4% die Fähigkeit, einige Operatoren auch in zuvor unbekannten Datensätzen zu erkennen. Die geringere Leistung lässt sich zudem durch uneinheitliche Annotationen der Operatoren im Korpus erklären. Die indirekte Methode erfordert zwar einen AST-Parser zur Konvertierung der LLM-Ausgabe in das JSON-Format, ermöglicht jedoch einen Zero-Shot Ansatz, der es Fachkräften ohne Informatik- oder KI-Expertise erlaubt mithilfe einfacher Anweisungen Kriterien zu verarbeiten, insofern die entsprechenden LLMs bereitgestellt sind.

Die direkte Strukturierung mit LLMs verarbeitet Eignungskriterien in einem einzigen Schritt direkt in ein JSON-Format mit Abstract Syntax Tree Struktur. Dabei werden die logischen Operatoren erkannt und als Knoten der AST-Struktur verwendet, während der verknüpfte Originaltext sowie die extrahierten Entitäten als Blätter integriert werden. Untersuchte LLMs waren hier Llama3-70B und Qwen2-72B, welche mit dem Closed-Source Modell GPT-4o in Few-Shot Szenarien auf dem LCT-Testdatensatz verglichen wurden. Few-Shot Ansätze erwiesen sich mit allen LLMs als unpraktikabel, mit F1-Scores von 10-15% für die Operatorenextraktion und hohen Fehlerraten bei der Ausgabe von JSON-Dateien. Ein Parameter-Efficient Fine-Tuning von Llama3-70B verbessert die Operatorenextraktion mit einem durchschnittlichen F1-Score von 82.4%, was eine Steigerung von 6.5% gegenüber dem aktuellen SOTA-Modell SciBERT darstellt. Zudem wird die Entitätserkennung mit einem F1-Score von 88.8% um 7.5% verbessert. Die angegebenen Ergebnisse sind der Mittelwert aus 20 Durchführungen, wobei die Standardabweichungen der F1-Scores unter einem Prozent liegen.

8.2. Beantwortung der Forschungsfragen

Die Hauptforschungsfrage **RQ0**: *“Wie kann unstrukturierter klinischer Text mit frei verfügbaren Machine Learning-Methoden strukturiert werden?”* wurde gestellt, um zu evaluieren, ob Open-Source Large Language Models Eignungskriterien effektiv in ein interoperables Format strukturieren können. Hier wurde festgestellt, dass die automatisierte Strukturierung von Eignungskriterien als Abstract Syntax Tree im JSON-Format mithilfe von Large Language Models eine vielversprechende Ergänzung zu bestehenden Analyse-Systemen wie LeafAI darstellt. Im Vergleich zu herkömmlichen Systemen können LLMs eine größere Anzahl von Kriterien verarbeiten und diese in einer benutzerfreundlicheren, lesbaren Form darstellen. Ein weiterer Vorteil ist, dass die bisher notwendige Verkettung mehrerer PLMs zur logischen Strukturierung extrahierter Entitäten durch ein einzelnes Open-Source LLM ersetzt werden kann. Während das LLM-basierte System im Gegensatz zu LeafAI keine Relationsextraktion durchführt, bietet es den Vorteil, dass der Ursprungstext in der Struktur erhalten bleibt. Die vollständige Überführung der Kriterien in eine logische Form erfordert trotzdem noch einen Parser, der die als AST-strukturierten Ein- und Ausschlusskriterien logisch verknüpft. Dies ist vor allem durch die begrenzte Kontextlänge von Open-Source Modellen wie Llama3 bedingt. Zukünftige LLMs werden voraussichtlich größere Textmengen qualitativ hochwertig strukturieren können (Dubey et al., 2024).

Welche LLM-Methoden dafür auch in Zukunft geeignet sein werden, ergibt sich aus der Beantwortung der unterstützenden Forschungsfrage **RQ1**: *“Welche Methoden eignen sich für die Strukturierung von Eignungskriterien aus klinischen Studien?”*. Um für die Extraktion der logischen AND-, OR- und NOT-Operatoren sowie von Entitäten SOTA-Leistung zu erreichen und zu übertreffen, ist aktuell noch ein Fine-Tuning von Open-Source Large Language Models nötig. Mit größeren Modellen wie Llama3-70B werden die AST-Strukturen über die extrahierte Logik um etwa 7% besser aufgebaut als mit dem kleineren Llama3-8B Modell. Die Extraktion der Entitäten zeigt dabei eine ähnlich gute Leistung. Während Fine-Tuning Methoden für LLMs in der Vergangenheit sehr ressourcenintensiv waren, haben PEFT-Ansätze wie LoRA diese Prozesse deutlich effizienter gestaltet. Dies ermöglicht nun selbst für große Modelle wie Llama3-70B ein Fine-Tuning auf kleinen Datensätzen innerhalb weniger Stunden auf einer einzelnen GPU. Allerdings bleibt die Datengenerierung mit einem 70B-Modell ressourcenintensiv und erfordert etwa 260GB VRAM. Kleinere 8B-Modelle benötigen hingegen etwa 100GB VRAM, was sie für Anwendungen mit begrenzten Ressourcen zugänglicher macht. In-Context Learning Methoden mit Open-Source LLMs haben bei der Operatorenextraktion durch eine indirekte Strukturierung vielversprechende Ergebnisse gezeigt, insbesondere im Vergleich zu regelbasierten Ansätzen wie einem Regex-Parser. Die Leistung bleibt jedoch noch hinter PLMs mit domänenspezifischem Fine-Tuning. Zero-Shot Ansätze mit Llama3-70B erreichen auf dem LCT-Datensatz für die Operatorenerkennung hohe Recall-Scores von 69.5%, aber relativ niedrige Precision-Scores von 43.4%. Zwar wird mit der alleinigen Entwicklung einer Promptanweisung die Mehrzahl der Operatoren erkannt, jedoch werden im Vergleich zum Goldstandard auch einige falsch gesetzt. Es konnte jedoch gezeigt werden, dass nicht alle vom Modell gesetzten Operatoren, die im Goldstandard nicht vorhanden waren, zwangsläufig falsch sein müssen. Few-Shot Methoden mit Open-Source LLMs übertreffen einen regelbasierten Ansatz und weisen ähnlich gute Leistungen wie proprietäre LLMs auf. Durch die Extraktion von logischen Operatoren können indirekt In-Context Learning Methoden zur Strukturierung von Eignungskriterien angewendet werden. Die Ausgaben müssen mit einem Parser in eine

überprüfbare AST-Struktur überführt werden. Derzeit können LLMs mittels In-Context Learning die Kriterien noch nicht direkt als Abstract Syntax Trees strukturieren, da Operatoren und Entitäten nicht ausreichend präzise erkannt werden. Positiv hervorzuheben ist jedoch, dass es mit aktuellen Modellen wie Llama3 und der Anwendung von Few-Shot möglich war, AST-Strukturen zu erzeugen. Mit Vorgängermodellen wie Llama2 war die Konstruktion von AST-Strukturen hingegen nicht realisierbar.

8.3. Zukünftige Arbeit

Das auf dem LCT-Korpus optimierte und durch Fine-Tuning angepasste Llama3-70B Modell kann der Forschungsgemeinschaft über die Huggingface-Plattform zur Verfügung gestellt werden, um Eignungskriterien in Abstract Syntax Tree Strukturen in medizinischen Produktivumgebungen zu strukturieren. Ergänzend kann die ressourceneffiziente 8B-Variante des Modells publiziert werden, die trotz reduzierter Leistung bei der Operatorenerkennung deutlich geringere Rechenressourcen für die Inferenz benötigt. Zukünftige Entwicklungen könnten die Implementierung eines interaktiven Frameworks umfassen, das die aufbereiteten Eignungskriterien in einer JSON-basierten AST-Struktur visualisiert. Eine solche Lösung würde Echtzeit-Interaktionen ermöglichen und Anwendern die flexible Modifikation der hierarchischen Baumstruktur zur Fehlerkorrektur gestatten. Die strukturierten Eignungskriterien könnten dann über weitere Verarbeitungsschritte in die logische Suche nach klinischen Studien in Studienregistern wie ClinicalTrials.gov (2000) eingebunden werden. Zudem könnte evaluiert werden, ob die angereicherten Kriterien für ein automatisiertes oder semi-automatisiertes Patient-Studie Matching genutzt werden können.

Die Experimente zur Operatorenextraktion demonstrieren eine signifikante Leistungssteigerung von Llama3 gegenüber dem Vorgängermodell Llama2. Laut Dubey et al. (2024) werden zukünftige Llama-Varianten durch das Training mit qualitativ besseren Daten und Anpassungen in der Modellarchitektur eine noch bessere Leistung erzielen. Zudem wurde die Kontextlänge von Large Language Models kontinuierlich erhöht. Diese Entwicklungen lassen darauf schließen, dass modernere Open-Source LLMs die Eignungskriterien vollständig in eine Abstract Syntax Tree-Struktur transformieren können. Dafür muss zusätzlich die Strukturprompt (Abbildung C.3) angepasst werden, um die Modelle anzuleiten, die Ausschlusskriterien mit AND-NOT und die Einschlusskriterien mit AND zu verknüpfen, wobei beide durch einen gemeinsamen AND-Knoten verbunden werden sollten. Aktuelle oder zukünftige LLMs könnten angewiesen werden, die Eignungskriterien nicht im JSON-Format, sondern im FHIR-Format auszugeben. Dafür müssten bei der Anwendung von Few-Shot oder Fine-Tuning Methoden neue Datengrundlagen verarbeitet werden. Die in JSON verarbeiteten LCT-Daten könnten dafür in das FHIR-Format konvertiert werden. Um die Datengrundlage zu erweitern und Modelle weiter zu evaluieren, können die in Tabelle 4.1 vorgestellten FRD-, COVIC- und LLF-Korpora herangezogen werden. Für jeden Korpus muss eine geeignete Verarbeitungspipeline erstellt werden, um die Daten in das gewünschte Format zu bringen.

Für die Nutzung von LLMs mit über 70B-Parametern war die parallele Nutzung von vier NVIDIA A100 GPUs mit je 80GB VRAM nötig. Für kleinere Large Language Models wie Phi-3-mini mit 3.8B Parametern wären etwa 16GB VRAM für die Inferenz ausreichend und es ist kein Rechencluster mehr nötig. Die Erprobung solcher kleineren LLMs mit Decoder-only Architektur wäre daher interessant, um zu evaluieren, ob Open-Source Modelle auch ohne Rechencluster effizient verwendet werden können. Zu dem Zeitpunkt der Erstellung dieser Arbeit war ein Fine-Tuning der neueren GPT-4o Modelle nicht möglich, wird aber in Zukunft möglich sein (OpenAI, 2023). Zudem sind die Kosten für die Nutzung und das Fine-Tuning vorheriger Modelle signifikant gesunken. Da das GPT-4o Modell bei der direkten Strukturierung in Few-Shot Szenarien die beste Leistung erzielte, wäre es sinnvoll zu untersuchen, ob zukünftige Modelle der GPT-Familie diese Leistung weiter verbessern können und wie sich das Modell mit Fine-Tuning verhält.

Literaturverzeichnis

- ANTHROPIC, 2021. *Anthropic API* [<https://www.anthropic.com/api>]. Anthropic. Zugriff am: 15.05.2024.
- ANTHROPIC, 2024. *The Claude 3 Model Family: Opus, Sonnet, Haiku*. Auch verfügbar unter: https://www-cdn.anthropic.com/de8ba9b01c9ab7cbabf5c33b80b7bbc618857627/Model_Card_Claude_3.pdf.
- BAHDANAU, Dzmitry et al., 2016. *Neural Machine Translation by Jointly Learning to Align and Translate*. Abger. unter arXiv: 1409.0473 [cs.CL].
- BANERJEE, Sourav et al., 2024. *LLMs Will Always Hallucinate, and We Need to Live With This*. Abger. unter DOI: 10.48550/arXiv.2409.05746.
- BARTZ, E. et al., 2023. *Hyperparameter Tuning for Machine and Deep Learning with R: A Practical Guide*. Springer Nature Singapore. ISBN 9789811951695. Auch verfügbar unter: <https://books.google.de/books?id=InI6zwEACAAJ>.
- BELTAGY, Iz et al., 2019. *SciBERT: A Pretrained Language Model for Scientific Text*. Abger. unter arXiv: 1903.10676 [cs.CL].
- BENGIO, Yoshua, 2012. *Practical recommendations for gradient-based training of deep architectures*. Abger. unter arXiv: 1206.5533 [cs.LG].
- BIEMANN, Chris et al., 2022. *Wissensrohstoff Text: Eine Einführung in das Text Mining*. Wiesbaden: Springer Fachmedien Wiesbaden. ISBN 978-3-658-35969-0. Abger. unter DOI: 10.1007/978-3-658-35969-0.
- BIO-NLP, UW, 2020. *clinical-trials-gov-data (LCT-Corpus)* [<https://github.com/uw-bionlp/clinical-trials-gov-data>]. GitHub. Zugriff am: 15.05.2024.
- BIO-NLP, UW, 2023. *LLF-Corpus - The Leaf Logical Forms Corpus* [<https://github.com/uw-bionlp/leaf-corpora>]. GitHub. Zugriff am: 20.05.2024.
- BOAG, William et al., 2015. *CLINER : A Lightweight Tool for Clinical Named Entity Recognition*. In: Auch verfügbar unter: <https://api.semanticscholar.org/CorpusID:16485439>.
- BODENREIDER, Olivier et al., 2018. Recent Developments in Clinical Terminologies—SNO-MED CT, LOINC, and RxNorm. *Yearbook of Medical Informatics*. Jg. 27, Nr. 1, S. 129–139. Abger. unter DOI: 10.1055/s-0038-1667077.
- BOLAND, Mary Regina et al., 2012. *EliXR-TIME: A Temporal Knowledge Representation for Clinical Research Eligibility Criteria*. *AMIA Summits on Translational Science proceedings AMIA Summit on Translational Science*. Jg. 2012, S. 71–80.
- BROWN, Tom B. et al., 2020. *Language Models are Few-Shot Learners*. Abger. unter arXiv: 2005.14165 [cs.CL].
- CHANG, Chia-Yuan et al., 2023. *Towards Fair Patient-Trial Matching via Patient-Criterion Level Fairness Constraint*. Abger. unter arXiv: 2303.13790 [cs.LG].
- CHEN, Miao et al., 2020. Using Pre-trained Transformer Deep Learning Models to Identify Named Entities and Syntactic Relations for Clinical Protocol Analysis. In: *AAAI Spring Symposium Combining Machine Learning with Knowledge Engineering*. Auch verfügbar unter: <https://api.semanticscholar.org/CorpusID:218623194>.
- CHEN, Tianqi et al., 2016. *Training Deep Nets with Sublinear Memory Cost*. Abger. unter arXiv: 1604.06174 [cs.LG].
- CHEN, Zeming et al., 2023. *MEDITRON-70B: Scaling Medical Pretraining for Large Language Models*. Abger. unter arXiv: 2311.16079 [cs.CL].

- CHIA, 2020. *Chia-Corpus* [https://figshare.com/articles/dataset/Chia_Annotated_Datasets/11855817]. Figshare. Zugriff am: 05.05.2024.
- CHOW, S.C.; LIU, J.P., 2008. *Design and Analysis of Clinical Trials: Concepts and Methodologies*. Wiley. Wiley Series in Probability and Statistics. ISBN 9780471473299. Auch verfügbar unter: <https://books.google.de/books?id=TQ6g4ADtRLgC>.
- CLINICALTRIALS.GOV, 2000. *U.S. National Library of Medicine* [<https://clinicaltrials.gov/>]. Zugriff am: 10.05.2024.
- CTBERT, 2021. *CTBERT: A Framework for Clinical Trial Information Extraction with BERT* [<https://github.com/aipharma/ctbert>]. GitHub. Zugriff am: 20.08.2024.
- CTP, 2020. *Facebook Research, Clinical Trial Parser* [<https://github.com/facebookresearch/Clinical-Trial-Parser>]. GitHub. Zugriff am: 20.05.2024.
- DALIANIS, Hercules, 2018. *Clinical Text Mining*. Springer International Publishing. ISBN 978-3-319-78502-8. Abger. unter DOI: 10.1007/978-3-319-78503-5.
- DATTA, Surabhi et al., 2023. AutoCriteria: a generalizable clinical trial eligibility criteria extraction system powered by large language models. *Journal of the American Medical Informatics Association*. Jg. 31, Nr. 2, S. 375–385. ISSN 1527-974X. Abger. unter DOI: 10.1093/jamia/ocad218.
- DETTMERS, Tim et al., 2023. *QLoRA: Efficient Finetuning of Quantized LLMs*. Abger. unter arXiv: 2305.14314 [cs.LG].
- DEVLIN, Jacob et al., 2019. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. Abger. unter arXiv: 1810.04805 [cs.CL].
- DOBBINS, Nicholas; MULLEN, T. et al., 2022. *The Leaf Clinical Trials (LCT) corpus: an annotated corpus of clinical trials eligibility criteria*. figshare. Version v1. Abger. unter DOI: 10.6084/m9.figshare.17209610.
- DOBBINS, Nicholas; MULLEN, Tony et al., 2022. The Leaf Clinical Trials Corpus: a new resource for query generation from clinical trial eligibility criteria. *Scientific Data*. Jg. 9. Abger. unter DOI: 10.1038/s41597-022-01521-0.
- DOBBINS, Nicholas J et al., 2023. LeafAI: query generator for clinical cohort discovery rivaling a human programmer. *Journal of the American Medical Informatics Association*. Jg. 30, Nr. 12, S. 1954–1964. ISSN 1527-974X. Abger. unter DOI: 10.1093/jamia/ocad149.
- DUBEY, Abhimanyu et al., 2024. *The Llama 3 Herd of Models*. Abger. unter arXiv: 2407.21783 [cs.AI].
- ELIIE, 2017. *EliIE: An open-source information extraction system for clinical trial eligibility criteria* [<https://github.com/Tian312/EliIE>]. GitHub. Zugriff am: 20.10.2024.
- FANG, Yilu et al., 2022. Combining human and machine intelligence for clinical trial eligibility querying. *Journal of the American Medical Informatics Association*. Jg. 29, Nr. 7, S. 1161–1171. ISSN 1527-974X. Abger. unter DOI: 10.1093/jamia/ocac051.
- FORTUNE BUSINESS INSIGHTS, 2023. *Clinical Trials Market Size, Share, Growth Report*. Auch verfügbar unter: <https://www.fortunebusinessinsights.com/clinical-trials-market-106930>. Zugriff am: 28.09.2024.
- FRIEDMAN, L.M. et al., 2015. *Fundamentals of Clinical Trials*. Springer International Publishing. ISBN 9783319185392. Auch verfügbar unter: <https://books.google.de/books?id=9FN1CgAAQBAJ>.

- GAO, Junyi et al., 2020. COMPOSE: Cross-Modal Pseudo-Siamese Network for Patient Trial Matching. In: *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. Virtual Event, CA, USA: Association for Computing Machinery, S. 803–812. KDD '20. ISBN 9781450379984. Abger. unter DOI: 10.1145/3394486.3403123.
- GU, Yu et al., 2021. Domain-Specific Language Model Pretraining for Biomedical Natural Language Processing. *ACM Transactions on Computing for Healthcare*. Jg. 3, Nr. 1, S. 1–23. ISSN 2637-8051. Abger. unter DOI: 10.1145/3458754.
- GUTIÉRREZ, Bernal Jiménez et al., 2022. *Thinking about GPT-3 In-Context Learning for Biomedical IE? Think Again*. Abger. unter arXiv: 2203.08410 [cs.CL].
- HADI, Muhammad Usman et al., 2023. *Large Language Models: A Comprehensive Survey of its Applications, Challenges, Limitations, and Future Prospects*. Abger. unter DOI: 10.36227/techrxiv.23589741.v1.
- HAMER, Danny M. den et al., 2023. *Improving Patient Pre-screening for Clinical Trials: Assisting Physicians with Large Language Models*. Abger. unter arXiv: 2304.07396 [cs.LG].
- HAO, Tianyong et al., 2016. Valx: A System for Extracting and Structuring Numeric Lab Test Comparison Statements from Text. *Methods of Information in Medicine*. Jg. 55, Nr. 03, S. 266–275. ISSN 0026-1270, ISSN 2511-705X. Abger. unter DOI: 10.3414/ME15-01-0112.
- HRIPCSAK, George et al., 1993. The Arden Syntax for Medical Logic Modules. *International Journal of Clinical Monitoring and Computing*. Jg. 12. Abger. unter DOI: 10.1007/BF01133012.
- HU, Edward J. et al., 2021. *LoRA: Low-Rank Adaptation of Large Language Models*. Abger. unter arXiv: 2106.09685 [cs.CL].
- HUANG, Kexin et al., 2020. *ClinicalBERT: Modeling Clinical Notes and Predicting Hospital Readmission*. Abger. unter arXiv: 1904.05342 [cs.CL].
- JIN, Qiao et al., 2023. *Matching Patients to Clinical Trials with Large Language Models*. Abger. unter arXiv: 2307.15051 [cs.CL].
- JOHNSON, Alistair et al., 2016. MIMIC-III, a freely accessible critical care database. *Scientific Data*. Jg. 3, S. 160035. Abger. unter DOI: 10.1038/sdata.2016.35.
- KALAJDZIEVSKI, Damjan, 2023. *A Rank Stabilization Scaling Factor for Fine-Tuning with LoRA*. Abger. unter arXiv: 2312.03732 [cs.CL].
- KANG, Tian et al., 2017. ElilE: An open-source information extraction system for clinical trial eligibility criteria. *Journal of the American Medical Informatics Association*. Jg. 24, Nr. 6, S. 1062–1071. ISSN 1067-5027, ISSN 1527-974X. Abger. unter DOI: 10.1093/jamia/ocx019.
- KIM, Edward S. et al., 2017. Broadening Eligibility Criteria to Make Clinical Trials More Representative: American Society of Clinical Oncology and Friends of Cancer Research Joint Research Statement. *Journal of Clinical Oncology*. Jg. 35, Nr. 33, S. 3737–3744. Abger. unter DOI: 10.1200/JCO.2017.73.7916. PMID: 28968170.
- KOJIMA, Takeshi et al., 2023. *Large Language Models are Zero-Shot Reasoners*. Abger. unter arXiv: 2205.11916 [cs.CL].
- KUDO, Taku, 2018. *Subword Regularization: Improving Neural Network Translation Models with Multiple Subword Candidates*. Abger. unter arXiv: 1804.10959 [cs.CL].
- KURY, Fabrício et al., 2020. Chia, a large annotated corpus of clinical trial eligibility criteria. *Scientific Data*. Jg. 7, S. 281. Abger. unter DOI: 10.1038/s41597-020-00620-0.

- LAN, Zhenzhong et al., 2020. *ALBERT: A Lite BERT for Self-supervised Learning of Language Representations*. Abger. unter arXiv: 1909.11942 [cs.CL].
- LEADERBORD, Huggingface, 2024. *Open LLM Leaderboard* [https://huggingface.co/spaces/open-llm-leaderboard/open_llm_leaderboard]. Hugging Face. Zugriff am: 15.05.2024.
- LEE, Jinhyuk et al., 2019. BioBERT: a pre-trained biomedical language representation model for biomedical text mining. *Bioinformatics*. Jg. 36, Nr. 4, S. 1234–1240. ISSN 1367-4803. Abger. unter DOI: 10.1093/bioinformatics/btz682.
- LI, Cheng et al., 2023. *Large Language Models Understand and Can be Enhanced by Emotional Stimuli*. Abger. unter arXiv: 2307.11760 [cs.CL].
- LI, Jianfu et al., 2022. A comparative study of pre-trained language models for named entity recognition in clinical trial eligibility criteria from multiple corpora. *BMC Medical Informatics and Decision Making*. Jg. 22. Abger. unter DOI: 10.1186/s12911-022-01967-7.
- LIN, Yang et al., 2024. *LoRA Dropout as a Sparsity Regularizer for Overfitting Control*. Abger. unter arXiv: 2404.09610 [cs.LG].
- LIU, Xiong et al., 2021. *Clinical Trial Information Extraction with BERT*. Abger. unter arXiv: 2110.10027 [q-bio.QM].
- LIU, Yinhan et al., 2019. *RoBERTa: A Robustly Optimized BERT Pretraining Approach*. Abger. unter arXiv: 1907.11692 [cs.CL].
- LOSHCHILOV, Ilya; HUTTER, Frank, 2019. *Decoupled Weight Decay Regularization*. Abger. unter arXiv: 1711.05101 [cs.LG].
- MAO, Yuren et al., 2024. *A Survey on LoRA of Large Language Models*. Abger. unter arXiv: 2407.11046 [cs.LG].
- METAMAPLITE, 2016. *MetaMapLite - A near real-time named-entity recognizer* [<https://github.com/lhncbc/metamaplite>]. GitHub. Zugriff am: 07.05.2024.
- MICHELUCCI, Umberto, 2024. Introduction. In: *Fundamental Mathematical Concepts for Machine Learning in Science*. Cham: Springer International Publishing, S. 1–8. ISBN 978-3-031-56431-4. Abger. unter DOI: 10.1007/978-3-031-56431-4_1.
- MIN, Sewon et al., 2022. *Rethinking the Role of Demonstrations: What Makes In-Context Learning Work?* Abger. unter arXiv: 2202.12837 [cs.CL].
- MINAEE, Shervin et al., 2024. *Large Language Models: A Survey*. Abger. unter arXiv: 2402.06196 [cs.CL].
- MO, Kangtong et al., 2024. *Fine-Tuning Gemma-7B for Enhanced Sentiment Analysis of Financial News Headlines*. Abger. unter arXiv: 2406.13626 [cs.CL].
- MUNNANGI, Monica et al., 2024. *On-the-fly Definition Augmentation of LLMs for Biomedical NER*. Abger. unter arXiv: 2404.00152 [cs.CL].
- NAIDU, Gireen et al., 2023. A Review of Evaluation Metrics in Machine Learning Algorithms. In: SILHAVY, Radek; SILHAVY, Petr (Hrsg.). *Artificial Intelligence Application in Networks and Systems*. Cham: Springer International Publishing, S. 15–25.
- NAVEED, Humza et al., 2024. *A Comprehensive Overview of Large Language Models*. Abger. unter arXiv: 2307.06435 [cs.CL].
- NIEVAS, Mauro et al., 2023. *Distilling Large Language Models for Matching Patients to Clinical Trials*. Abger. unter arXiv: 2312.09958 [cs.AI].
- NLM, 2024. *Statistik aus ClinicalTrials.gov* [<https://clinicaltrials.gov/about-site/trends-charts>]. NLM. Zugriff am: 02.09.2024.

- OLEYNIK, Michel et al., 2019. Evaluating shallow and deep learning strategies for the 2018 n2c2 shared task on clinical text classification. *Journal of the American Medical Informatics Association : JAMIA*. Jg. 26. Abger. unter DOI: 10.1093/jamia/ocz149.
- OPENAI, 2023. *OpenAI API Pricing* [<https://openai.com/api/pricing/>]. OpenAI. Zugriff am: 15.05.2024.
- OPENAI et al., 2024. *GPT-4 Technical Report*. Abger. unter arXiv: 2303.08774 [cs.CL].
- PARK, Jimyung et al., 2023. *Criteria2query 3.0: Leveraging Generative Large Language Models for Clinical Trial Eligibility Query Generation*. preprint. SSRN. Abger. unter DOI: 10.2139/ssrn.4637800.
- PENBERTHY, Lynne et al., 2010. Automated matching software for clinical trials eligibility: Measuring efficiency and flexibility. *Contemporary Clinical Trials*. Jg. 31, Nr. 3, S. 207–217. ISSN 1551-7144. Abger. unter DOI: <https://doi.org/10.1016/j.cct.2010.03.005>.
- PENG, Yifan et al., 2019. *Transfer Learning in Biomedical Natural Language Processing: An Evaluation of BERT and ELMo on Ten Benchmarking Datasets*. Abger. unter arXiv: 1906.05474 [cs.CL].
- RAFFEL, Colin et al., 2020. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *Journal of Machine Learning Research*. Jg. 21, Nr. 140, S. 1–67. Auch verfügbar unter: <http://jmlr.org/papers/v21/20-074.html>.
- RAIAAN, Mohaimenul Azam Khan et al., 2024. A Review on Large Language Models: Architectures, Applications, Taxonomies, Open Issues and Challenges. *IEEE Access*. Jg. 12, S. 26839–26874. Auch verfügbar unter: <https://api.semanticscholar.org/CorpusID:267675587>.
- ROBERTS, Adam et al., 2020. *How Much Knowledge Can You Pack Into the Parameters of a Language Model?* Abger. unter arXiv: 2002.08910 [cs.CL].
- ROSS, Jessica et al., 2010. Analysis of Eligibility Criteria Complexity in Clinical Trials. *AMIA Summits on Translational Science proceedings AMIA Summit on Translational Science*. Jg. 2010, S. 46–50.
- SCHULHOFF, Sander et al., 2024. *The Prompt Report: A Systematic Survey of Prompting Techniques*. Abger. unter arXiv: 2406.06608 [cs.CL].
- SFTTRAINER, 2024. *SFT Trainer - TRL Documentation* [https://huggingface.co/docs/trl/sft_trainer]. Hugging Face. Zugriff am: 15.06.2024.
- SORDO, Margarita et al., 2004. Description and status update on GELLO: A proposed standardized object-oriented expression language for clinical decision support. *Studies in health technology and informatics*. Jg. 107, S. 164–8. Abger. unter DOI: 10.3233/978-1-60750-949-3-164.
- STENETORP, Pontus et al., 2012. BRAT: a web-based tool for NLP-assisted text annotation. In: *Proceedings of the Demonstrations at the 13th Conference of the European Chapter of the Association for Computational Linguistics*. Avignon, France: Association for Computational Linguistics, S. 102–107. EACL '12. Auch verfügbar unter: <https://aclanthology.org/E12-2021>.
- SU, Qianmin et al., 2023. A review of research on eligibility criteria for clinical trials. *Clinical and Experimental Medicine*. Jg. 23, S. 1–13. Abger. unter DOI: 10.1007/s10238-022-00975-1.
- SUN, Yingcheng et al., 2021. Building an OMOP common data model-compliant annotated corpus for COVID-19 clinical trials. *Journal of biomedical informatics*. Jg. 118, S. 103790. Abger. unter DOI: 10.1016/j.jbi.2021.103790.

- TEAM, Gemma et al., 2024. *Gemma 2: Improving Open Language Models at a Practical Size*. Abger. unter arXiv: 2408.00118 [cs.CL].
- TIAN, Shubo; ERDENGASILENG, Arslan et al., 2021. Transformer-based named entity recognition for parsing clinical trial eligibility criteria. In: *Proceedings of the 12th ACM Conference on Bioinformatics, Computational Biology, and Health Informatics*. Gainesville, Florida: Association for Computing Machinery. BCB '21. ISBN 9781450384506. Abger. unter DOI: 10.1145/3459930.3469560.
- TIAN, Shubo; JIN, Qiao et al., 2023. Opportunities and challenges for ChatGPT and large language models in biomedicine and health. *Briefings in Bioinformatics*. Jg. 25, Nr. 1. ISSN 1477-4054. Abger. unter DOI: 10.1093/bib/bbad493.
- TOUVRON, Hugo et al., 2023. *Llama 2: Open Foundation and Fine-Tuned Chat Models*. Abger. unter arXiv: 2307.09288 [cs.CL].
- TSEO, Yitong et al., 2020. *Information Extraction of Clinical Trial Eligibility Criteria*. Abger. unter arXiv: 2006.07296 [cs.CL].
- TU, Samson W. et al., 2011. A practical method for transforming free-text eligibility criteria into computable criteria. *Journal of Biomedical Informatics*. Jg. 44, Nr. 2, S. 239–250. ISSN 15320464. Abger. unter DOI: 10.1016/j.jbi.2010.09.007.
- UNSLOTH, 2024. *Unsloth: Faster Fine-Tuning for LLMs* [<https://github.com/unslothai/unsloth>]. Zugriff am: 15.06.2024.
- VASWANI, Ashish et al., 2017. *Attention Is All You Need*. Abger. unter arXiv: 1706.03762 [cs.CL].
- WENG, C. et al., 2011. EliXR: an approach to eligibility criteria extraction and representation. *Journal of the American Medical Informatics Association*. Jg. 18, Nr. Supplement 1, S. i116–i124. ISSN 1067-5027, ISSN 1527-974X. Abger. unter DOI: 10.1136/amiajnl-2011-000321.
- WENG, Chunhua, 2015. Optimizing Clinical Research Participant Selection with Informatics. *Trends in Pharmacological Sciences*. Jg. 36, S. 706–709. Abger. unter DOI: 10.1016/j.tips.2015.08.007.
- WENGLAB, 2020. *COVID19 Structured Trials (COVIC)* [https://github.com/WengLab-InformaticsResearch/COVID19-Structured_Trials]. GitHub. Zugriff am: 05.07.2024.
- WONG, Cliff et al., 2023. Scaling Clinical Trial Matching Using Large Language Models: A Case Study in Oncology. Abger. unter DOI: 10.48550/ARXIV.2308.02180.
- WORNOW, Michael et al., 2024. Zero-Shot Clinical Trial Patient Matching with LLMs. Abger. unter DOI: 10.48550/ARXIV.2402.05125. Publisher: arXiv Version Number: 1.
- WU, Kevin; AKLILU, Josiah, 2021. Learning Representations of Eligibility Criteria in Clinical Trials Using Transformers. In: Auch verfügbar unter: <https://api.semanticscholar.org/CorpusID:235357692>.
- WU, Shanchan; HE, Yifan, 2019. *Enriching Pre-trained Language Model with Entity Information for Relation Classification*. Abger. unter arXiv: 1905.08284 [cs.CL].
- XU, Derong et al., 2023. *Large Language Models for Generative Information Extraction: A Survey*. Abger. unter arXiv: 2312.17617 [cs.CL].
- XU, H.; FUSHMAN, D.D., 2024. *Natural Language Processing in Biomedicine: A Practical Guide*. Springer International Publishing, Imprint: Springer. Cognitive Informatics in Biomedicine and Healthcare Series. ISBN 9783031558658. Auch verfügbar unter: <https://books.google.de/books?id=NmONEQAAQBAJ>.

- XU, Xin et al., 2023. *How to Unleash the Power of Large Language Models for Few-shot Relation Extraction?* Abger. unter arXiv: 2305.01555 [cs.CL].
- YANG, An et al., 2024. *Qwen2 Technical Report*. Abger. unter arXiv: 2407.10671 [cs.CL].
- YANG, Jingfeng et al., 2023. *Harnessing the Power of LLMs in Practice: A Survey on ChatGPT and Beyond*. Abger. unter arXiv: 2304.13712 [cs.CL].
- YANG, Yumeng et al., 2023. *Text Classification of Cancer Clinical Trial Eligibility Criteria*. Abger. unter arXiv: 2309.07812 [cs.CL].
- YE, Qinyuan et al., 2024. *Prompt Engineering a Prompt Engineer*. Abger. unter arXiv: 2311.05661 [cs.CL].
- YU, Xiaojing et al., 2020. Dataset and Enhanced Model for Eligibility Criteria-to-SQL Semantic Parsing. In: CALZOLARI, Nicoletta et al. (Hrsg.). *Proceedings of the Twelfth Language Resources and Evaluation Conference*. Marseille, France: European Language Resources Association, S. 5829–5837. ISBN 979-10-95546-34-4. Auch verfügbar unter: <https://aclanthology.org/2020.lrec-1.714>.
- YUAN, Chi et al., 2019. Criteria2Query: a natural language interface to clinical databases for cohort definition. *Journal of the American Medical Informatics Association*. Jg. 26, Nr. 4, S. 294–305. ISSN 1527-974X. Abger. unter DOI: 10.1093/jamia/ocy178.
- YUAN, Jiayi et al., 2023. *Large Language Models for Healthcare Data Augmentation: An Example on Patient-Trial Matching*. Abger. unter arXiv: 2303.16756 [cs.CL].
- ZARIN, Deborah et al., 2011. The ClinicalTrials.gov Results Database - Update and Key Issues. *The New England journal of medicine*. Jg. 364, S. 852–60. Abger. unter DOI: 10.1056/NEJMsa1012065.
- ZENG, Kun et al., 2021. Automated classification of clinical trial eligibility criteria text based on ensemble learning and metric learning. *BMC Medical Informatics and Decision Making*. Jg. 21. Abger. unter DOI: 10.1186/s12911-021-01492-z.
- ZHANG, Xingyao et al., 2020. DeepEnroll: Patient-Trial Matching with Deep Embedding and Entailment Prediction. In: *Proceedings of The Web Conference 2020*. Taipei, Taiwan: Association for Computing Machinery, S. 1029–1037. WWW '20. ISBN 9781450370233. Abger. unter DOI: 10.1145/3366423.3380181.
- ZHAO, Wayne Xin et al., 2023. *A Survey of Large Language Models*. Abger. unter arXiv: 2303.18223 [cs.CL].
- ZHENG, Mingqian et al., 2023. *Is A Helpful Assistant the Best Role for Large Language Models? A Systematic Evaluation of Social Roles in System Prompts*. Abger. unter arXiv: 2311.10054 [cs.CL].
- ZHOU, Hongjian et al., 2023. A Survey of Large Language Models in Medicine: Progress, Application, and Challenge. *ArXiv*. Jg. abs/2311.05112. Auch verfügbar unter: <https://api.semanticscholar.org/CorpusID:265067484>.
- ZHU, Yutao et al., 2024. *Large Language Models for Information Retrieval: A Survey*. Abger. unter arXiv: 2308.07107 [cs.CL].

Anhang

Exemplarische Eignungskriterien

Eligibility Criteria
Description
Inclusion Criteria:
1. Age > 18 years
2. Undergoing primary PCI due to STEMI (TIMI flow I -III before PCI and symptom duration < 12 hours)
6) Following receipt of verbal and written information about the study, the patient must provide signed informed consent before any study data is used.
-
Exclusion Criteria:
2) Not able to give informed consent 3) Women with childbearing potential 4) On-going concomitant treatment with K-vitamin antagonists (one bolus dose pre-PCI allowed) 5) Known congenital or acquired coagulopathy and/or thrombocytopenia 8) Participation in a clinical study and/or another investigational device within the past four weeks prior to Day 1 9) Major surgery or trauma within the past 6 weeks prior to Day 1 10) A blood coagulation disorder (i.e. international normalized ratio [INR] > 2.0, platelet count < 100,000/mm ³ , or hematocrit < 30%) 11) Renal insufficiency (creatinine > 140 mmol/l) 12) Major procedure related bleeding (TIMI major criteria); GI or urinary tract bleeding prior to inclusion in the trial 13) Known active hepatitis B and/or hepatitis C or HIV 14) Known or suspected hypersensitivity to components of the investigational medicinal product

Abbildung 1.: Beispiel von unstrukturierten Eignungskriterien der Studie NCT01179776.

A. SOTA

A.1. Strukturierungs-Systeme

Criteria2Query-System

Criteria2Query Support

✓ Inclusion Criteria

Tips: Please input criteria line by line

Evidence of the AD pathological process, as confirmed by CSF or amyloid PET scan.
 Demonstrated abnormal memory function
 MMSE score greater than or equal to 22 (≥ 22)
 CDR-Global Score of 0.5 or 1.0
 Availability of a reliable study partner who accepts to participate in study procedures throughout the 2 years duration of study
 If receiving symptomatic AD medications, the dosing regimen must have been stable for 3 months prior to baseline and until randomization.

✗ Exclusion Criteria

Tips: Please input criteria line by line

Any evidence of a condition other than AD that may affect cognition
 History of schizophrenia, schizoaffective disorder, major depression, or bipolar disorder
 History or presence of clinically evident systemic vascular disease that in the opinion of the investigator has the potential to affect cognitive function
 History or presence of clinically evident cerebrovascular disease

⚙ Configuration

Parsing

Reset

One-Button Start

#	Initial Events:	EHR Status
	No matching records found	

#	Inclusion Criteria:	EHR Status
1	Evidence of the AD CONDITION pathological process, as confirmed by CSF MEASUREMENT or amyloid PET scan PROCEDURE .	YES
2	Demonstrated abnormal memory function CONDITION	YES
3	MMSE score MEASUREMENT greater than or equal to 22 VALUE (≥ 22)	YES
4	CDR-Global Score MEASUREMENT of 0.5 or 1.0 VALUE	YES
5	Availability of a reliable study partner who accepts to participate in study procedures throughout the 2 years duration of study	NO
6	If receiving symptomatic AD medications DRUG , the dosing regimen must have been stable for 3 months TEMPORAL prior to baseline and until randomization.	YES

#	Exclusion Criteria:	EHR Status
1	Any evidence of a condition other than MEASUREMENT AD CONDITION that may affect cognition	YES
2	History of schizophrenia CONDITION , schizoaffective disorder CONDITION , major depression CONDITION , or bipolar disorder CONDITION	YES
3	History or presence of clinically evident systemic vascular disease CONDITION that in the opinion of the investigator has the potential to affect cognitive function	YES
4	History or presence of clinically evident cerebrovascular disease CONDITION	YES

Next

Download

Abbildung A.1.: Beispielhafte Darstellung der Criteria2Query-Anwendung mit markierten Entitäten (C. Yuan et al., 2019).

Criteria2Query-3.0

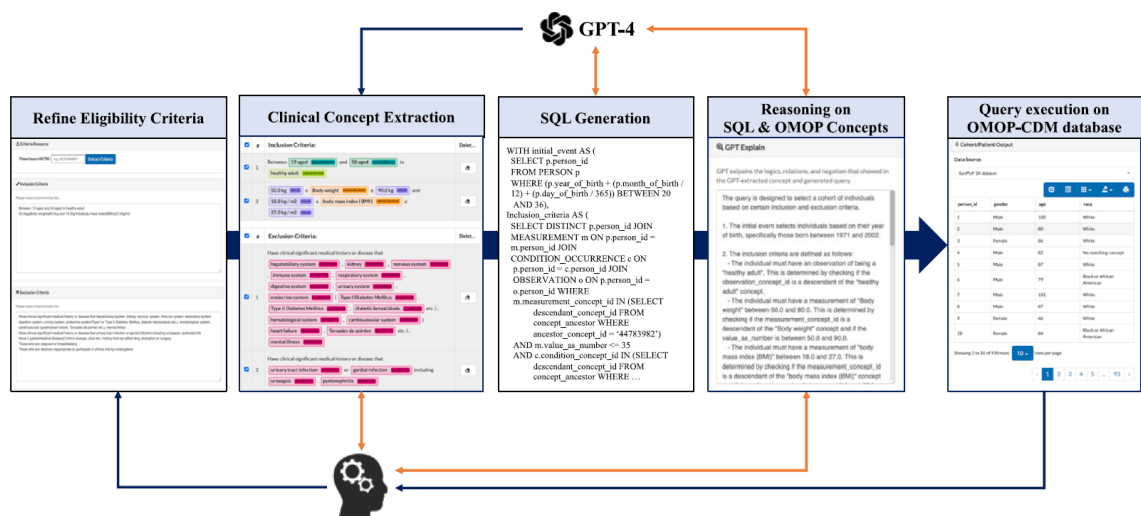


Abbildung A.2.: Criteria2Query-3.0 Workflow mit Interaktion zwischen Benutzer und GPT-4 bei Änderungen, repräsentiert durch orange Pfeile (Park et al., 2023).

A.2. LCT-Korpus

NCT03860402

Inclusion Criteria:

- Among the pregnant women who were administered cesarean section in this study, those who agreed to participate in this study.

Exclusion Criteria:

- Fetal distress requiring fast delivery.
- Patients with hypersensitivity, allergic response, resistance to the drugs used in this study (fentanyl, ropivacaine, lidocaine).
- Patients for whom regional anesthesia is not possible.
- Patients who refused to participate in this study.
- Patients who cannot read or understand the agreement.
- Patients whose weight is less than 50kg or exceeds 100kg.

Abbildung A.3.: Einschluss- und Ausschlusskriterien für die klinische Studie NCT03860402.

NCT03860402.ann

R5 **Or** Arg1:E6 Arg2:E19
 E6 Eq-Comparison:T12 Operator:T7 Value:T10 Unit:T8
 T12 Eq-Comparison 554 568 less than 50kg
 T7 Eq-Operator 554 563 less than
 T10 Eq-Value 564 566 50
 T8 Eq-Unit 566 568 kg
 E19 Eq-Comparison:T36 Operator:T37 Value:T11 Unit:T9
 T36 Eq-Comparison 572 585 exceeds 100kg
 ...
 R10 **And** Arg1:T15 Arg2:E1
 T15 Life-Stage-And-Gender 44 49 women
 E1 Procedure:T1 Name:T2 Temporality:E20
 ...
 E16 Negation:T32 **Negates**:E5
 T32 Negation 372 375 not
 E5 Procedure:T5 Name:T6
 T5 Procedure 389 408 regional anesthesia
 ...

Abbildung A.4.: Auszug aus den Annotationen der Studie NCT03860402.

Entitäten des LCT-Korpus

Kategorie	Entitätstypen
Clinical	Allergy, Condition, Condition-Type, Code, Contraindication, Drug, Encounter, Indication, Immunization, Observation, Organism, Specimen, Procedure, Provider
Demographic	Age, Birth, Death, Ethnicity, Family-Member, Language, Life-Stage-And-Gender
Logical	Exception, Negation
Qualifiers	Acuteness, Assertion, Modifier, Polarity, Risk, Severity, Stability
Temporal and Comparative	Criteria-Count, Eq-Comparison, Eq-Operator, Eq-Temporal-Period, Eq-Temporal-Recency, Eq-Temporal-Unit, Eq-Unit, Eq-Value
Name entities	Allergy-Name, Condition-Name, Drug-Name, Immunization-Name, Observation-Name, Organism-Name, Procedure-Name
Other	Coreference, Insurance, Location, Other, Study

Tabelle A.1.: Kategorisierung der Entitätstypen des LCT-Korpus (N. Dobbins; T. Mullen et al., 2022).

Klinische Studien im LCT-Korpus

Kategorie	Studien-ID	Anzahl Operatoren
<i>MAX-Studien</i>	NCT03860857	31
	NCT03866200	30
	NCT03861559	29
	NCT03865589	26
	NCT03868475	24
<i>MIN-Studien</i>	NCT03864653	3
	NCT03867422	3
	NCT03868865	3
	NCT03921138	3
	NCT03922269	3
<i>Auswahl-Studien</i>	NCT03865433	18
	NCT03860324	5
	NCT03860233	13
	NCT03923231	11
	NCT03930121	12

Tabelle A.2.: Übersicht der Studien mit den höchsten (MAX) und niedrigsten (MIN) Operatorenzahlen sowie ausgewählte (Auswahl) Studien aus dem LCT-Trainingsdatensatz.

B. Abstract Syntax Tree im JSON-Format

B.1. JSON-Format mit Entitäten

Strukturierte Ausschlusskriterien der Studie NCT03860402 mit Entitäten (.json):

```

1 {
2   "AND": {
3     "left": {
4       "AND": {
5         "left": {
6           "AND": {
7             "left": {
8               "AND": {
9                 "left": {
10                  "AND": {
11                    "left": {
12                     "raw_text": "- Fetal distress requiring
13                       fast delivery.",
14                     "Procedure": ["delivery"],
15                     "Condition": ["Fetal distress"],
16                     "Assertion": ["requiring"],
17                   },
18                  "right": {
19                    "OR": {
20                      "left": {
21                       "raw_text": "- Patients with hypersensitivity
22                         , allergic response.",
23                      "Condition": ["hypersensitivity"],
24                      "Allergy": ["allergic"],
25                      "Observation": ["response"],
26                    },
27                    "right": {
28                      "OR": {
29                        "left": {
30                         "raw_text": "resistance to the drugs used
31                           in this study (fentanyl.",
32                        "Study": ["study"],
33                        "Observation": ["resistance"],
34                        "Drug": ["drugs", "fentanyl"],
35                      },
36                      "right": {
37                        "OR": {
38                          "left": {
39                           "raw_text": "ropivacaine.",
40                          "Drug": ["ropivacaine"],
41                        },
42                        "right": {
43                         "raw_text": "lidocaine).",
44                         "Drug": ["lidocaine"],
45                      }
46                    }
47                  }
48                }
49              }
50            }
51          }
52        }
53      },

```

```

54         "right": {
55             "AND": {
56                 "left": {
57                     "raw_text": "- Patients who is"
58                 },
59                 "right": {
60                     "NOT": {
61                         "left": {
62                             "raw_text": "not possible for regional
63                                 anesthesia",
64                             "Procedure": ["regional anesthesia"],
65                             "Negation": ["not"],
66                             "Assertion": ["possible"]
67                         }
68                     }
69                 }
70             }
71         },
72     },
73     "right": {
74         "raw_text": "- Patients who refused to participate
75             in this study.",
76         "Study": ["study"]
77     }
78 },
79 },
80 },
81 "right": {
82     "raw_text": "- Patients who cannot read or understand
83         the agreement.",
84     "Negation": ["not"]
85 }
86 },
87 },
88 "right": {
89     "OR": {
90         "left": {
91             "raw_text": "- Patients whose weight is less than 50kg",
92             "EqUnit": ["kg"],
93             "EqValue": ["50"],
94             "EqComparison": ["less than 50kg"],
95             "Observation": ["weight"],
96         },
97         "right": {
98             "raw_text": "or exceeds 100kg.",
99             "EqUnit": ["kg"],
100             "EqValue": ["100"],
101             "EqComparison": ["exceeds 100kg"]
102         }
103     }
104 }
105 }
106 }

```

B.2. Verbundene Eignungskriterien im JSON-Format

Zusammengefügte Ein- und Ausschlusskriterien der Studie NCT03860402 (.json):

```

1 {
2   "AND": {
3     "left": {
4       "AND": {
5         "left": {
6           "raw_text": "- Among the pregnant women"
7         },
8         "right": {
9           "raw_text": "who were administered cesarean section in this
10             study, those who agreed to participate in this
11             study."
12         }
13       }
14     },
15     "right": {
16       "NOT": {
17         "left": {
18           "AND": {
19             "left": {
20               "AND": {
21                 "left": {
22                   "AND": {
23                     "left": {
24                       "AND": {
25                         "left": {
26                           "AND": {
27                             "left": {
28                               "raw_text": "- Fetal distress requiring
29                                 fast delivery."
30                             },
31                             "right": {
32                               "OR": {
33                                 "left": {
34                                   "raw_text": "- Patients with
35                                     hypersensitivity,
36                                     allergic response,"
37                                 },
38                                 "right": {
39                                   "OR": {
40                                     "left": {
41                                       "raw_text": "resistance to ..."
42                                     },
43                                     "right": {
44                                       "OR": {
45                                         "left": {
46                                           "raw_text": "ropivacaine,"
47                                         },
48                                         "right": {
49                                           "raw_text": "lidocaine)."
50                                         }
51                                       }
52                                     }
53                                 }
54                               }
55                             }
56                           }
57                         }
58                       }
59                     }
60                   }
61                 }
62               }
63             }
64           }
65         }
66       }
67     }
68   }
69 }

```

C. Prompts

Prompt 1: Logical Operator Insertion for Eligibility Criteria

Create an edited version of the following inclusion and exclusion criteria that incorporates the logical operators [AND], [OR], [NOT]. Apply these rules for inserting the logical operators:

[OR]:

- Insert [OR] to indicate alternatives, where at least one of the conditions must be satisfied.
- Always write an [OR] before an “or”, “and/or”, “and / or” in the text.
- Example: Patients with delirium [OR] or depression. Cancer [OR] and / or HIV. Illness, [OR] or injury.
- Write [OR] after a comma if it logically separates the sentence with an [OR].
- Example: Illness, [OR] or injury. Type A, [OR] B.
- Exception: Diagnosis of cognitive disorders, psychotic disorder, bipolar disorder, [OR] active substance abuse.
- If multiple criteria, implicitly or’d - only use [OR] between criteria if the span between is “or” or “and/or”. In other words, only annotate if “or” is explicit.
- Exception: brain disease (i.e. psychosis, stroke with functional impairment, dementia)
- Write an [OR] after “/” when they indicate alternatives. Example: Type A/ [OR] B.
- Write [OR] if the surrounding criteria logically mean ‘or’ (even if written as ‘and’), ‘and’ may also be annotated as Or. Example: Boys [OR] and girls. Cancer [OR] and HIV. Illness, [OR] and injury.
- Exception: If examples or instances of an Event follow in parentheses, [OR] should not be annotated.

[AND]:

- Insert [AND] to join two sentences.
- Write an [AND] before words like “with”, “who”, “in addition”, “plus”, “and”, “but”, “that”, “despite”, “having” when it is appropriate.
- Example: Stroke [AND] with first-ever aphasic symptoms. Patients [AND] who are postmenopausal.
- Write an [AND] before “and” when it joins two or more conditions or sentence parts that must all be met for eligibility. Example: Postmenopausal [AND] and preferably on hormone replacement therapy.
- Exception: When [AND] is used in an Equality Comparison (i.e., between two numeric values) it should not be annotated.

[NOT]:

- Insert [NOT] before a condition to exclude or negate it.
- Always write a [NOT] before “no”, “not”, “none”.
- Write a [NOT] before phrases like “don’t”, “free”, “prevent”, “Inability”, “lack”, “impossible”, “off”, “without”, “unable”, “naive”, “excluded”, “absence” when they negate a condition or requirement.
- Example: Patients [NOT] unable to take medicines. [NOT] No history of cancer. [NOT] Not on hormone replacement therapy. [NOT] don’t have a history of smoking. [NOT] free from a history of alcoholism.

General rules:

- The operators [OR], [AND], [NOT] should be written before the words like or, and, not, instead of replacing them.
- An [AND], [OR], [NOT] must not come before a line segment (\n).
- Please carefully read through the given criteria text, then output only an edited version with the logical operators incorporated, enclosed in <EDITED_CRITERIA>tags.
- Do not include any other explanations or output.

Abbildung C.1.: Prompt 1 mit Anweisungen zum Einfügen logischer Operatoren in Ein- und Ausschlusskriterien von klinischen Studien.

Role Prompt: Logical Operator Insertion for Eligibility Criteria

You are a highly skilled medical research assistant with a strong background in propositional logic and a keen eye for detail. Your primary role is to enhance the clarity, precision, and logical structure of inclusion and exclusion criteria in medical study texts. This crucial task directly contributes to the success of medical research projects by ensuring that the criteria are unambiguous, accurately reflect the study's requirements, and facilitate the efficient selection of suitable study participants. Your expertise lies in systematically incorporating the logical operators [AND], [OR], and [NOT] into the criteria texts. By applying these operators judiciously and in accordance with the principles of propositional logic, you eliminate ambiguity and improve the overall clarity of the criteria. This enables researchers and potential participants to easily understand the eligibility requirements without any room for misinterpretation.

To excel in this role, you have developed a deep understanding of the nuances and implications of each logical operator:

AND is used to conjoin multiple conditions, indicating that all the connected conditions must be simultaneously satisfied for the entire statement to be true. For example, "Condition1 [AND] Condition2" means that both conditions must be met.

OR signifies a disjunction between conditions, denoting that at least one of the conditions must be fulfilled. "Condition1 [OR] Condition2" is true if either or both of the conditions are met.

NOT is employed to negate a condition, explicitly specifying an exclusion. "[NOT] Condition" is true when the condition is false, and vice versa.

Key Skills:

- **Medical expertise:** You have a strong background in medical terminology and research methodologies.
- **Logical reasoning:** You excel at applying propositional logic, particularly using the operators [AND], [OR], and [NOT].
- **Clarity enhancement:** Your main task is to eliminate ambiguity in study criteria, making them easier to understand for both researchers and potential participants.
- **Systematic approach:** You follow a set of detailed rules for inserting logical operators into medical texts.
- **Nuanced understanding:** You grasp the subtle implications of each logical operator in various contexts.
- **Precision-oriented:** You aim to reflect study requirements accurately, facilitating efficient participant selection.
- **Text analysis skills:** You carefully read through criteria texts, identifying areas where logical operators can improve clarity.
- **Focused output:** You provide only the edited version of the criteria, without additional explanations.
- **Adaptability:** While following set rules, you can handle exceptions and complex linguistic structures.
- **Critical thinking:** You can infer logical relationships even when not explicitly stated in the original text.

Abbildung C.2.: Role Prompt mit einer Rollenbeschreibung für das Einsetzen von logischen Operatoren in Eignungskriterien.

Structure-Prompt for JSON generation

You are designed to parse textual clinical study eligibility criteria into a structured JSON format following the Composite pattern. The input consists of raw text containing the suitability criteria, which are to be logically broken down into a tree structure using the logical operators "AND", "OR" and "NOT". Provide a JSON response based on the following instructions:

1. Split the Text based on the logical operators AND, OR, NOT by using the following rules:

{{ INSERT PROMPT 1 or 2 }}

2. Create a JSON object based on the following rules:

- The basic structure consists of nested "AND", "OR" and "NOT" operators.
- Each AND and OR operator always has two branches: "left" and "right". Every left must always have a corresponding right.
- The NOT operator has only one branch: "left".
- "raw_text" are leaf nodes with no children.
- The extracted "raw_text" segments should be connected using the operators "AND", "OR", and "NOT" as per the rules above.

3. Sequence and structure:

- AND/OR always start with "left" and "right" branches
- NOT always starts with a "left" branch
- A "left" or "right" key is followed by either: another operator (AND, OR, NOT) or an object with "raw_text" (and optional additional fields)
- "raw_text" fields contain the actual text of the criterion. The raw text must always be transferred in full.
- Additional fields such as "Drug", "Condition", "Observation" or "Age" can appear next to "raw_text" to categorise specific terms.

4. Extract the following entities from the raw text and include them in the JSON object. Nodes could include attributes such as:

- "raw_text": A string representing the raw text description of the medical condition, procedure, or other relevant details.
- all entities: 'Contraindication', 'Eq-Value', 'Severity', 'Drug', 'Observation-Name', 'Age', 'Location', 'Organism-Name', 'Encounter', 'Drug-Name', 'Ethnicity', 'Modifier', 'Condition', 'Eq-Unit', 'Eq-Temporal-Unit', 'Family-Member', 'Organism', 'Other', 'Immunization-Name', 'Polarity', 'Condition-Type', 'Immunization', 'Eq-Operator', 'Eq-Temporal-Recency', 'Procedure-Name', 'Indication', 'Exception', 'Study', 'Language', 'Coreference', 'Provider', 'Acuteness', 'Life-Stage-And-Gender', 'Procedure', 'Risk', 'Death', 'Assertion', 'Allergy-Name', 'Specimen', 'Negation', 'Code', 'Stability', 'Birth', 'Criteria-Count', 'Eq-Comparison', 'Condition-Name', 'Insurance', 'Observation', 'Allergy', 'Eq-Temporal-Period'
- the lists for the entity's should only be appended if one or more entities have been found. Otherwise, omit the arrays completely.

5. The JSON object should be containing the following rules:

- Ensure that the same logical operator ("AND", "OR", "NOT") does not appear on the same level of the tree structure to maintain the integrity of the logical relationships.
- Generates JSON files from the textual criteria and organizing them into the described logical structure.
- Setting the brackets {}, [] and commas "," correctly in the JSON is extremely important.
- Any double quotes within the raw textual criteria are escaped with a backslash (\").
- If there are no restrictions, return an empty JSON FILE {}.
- The entire text of the eligibility criteria must be included and separated by the logical operators, without omitting any words or information.
- The output should only contain the JSON object, without any additional text or formatting.
- Ensure that the JSON output strictly follows these rules and accurately represents the logical structure of the input eligibility criteria.

Abbildung C.3.: Strukturprompt zur Verarbeitung von Eignungskriterien als AST im JSON-Format.

D. Ergebnisse

D.1. Indirekte Strukturierung

D.1.1. Few-Shot Validierung

Modell	Shots	P	R	F1
Llama3-70B	1	45.7 $\pm$ 2.2	71.1 $\pm$ 1.9	51.8 $\pm$ 1.96
	2	48.1 $\pm$ 1.6	70.8 $\pm$ 2.4	53.8 $\pm$ 1.3
	3	48.7 $\pm$ 1.6	69.6 $\pm$ 2.6	54.1 $\pm$ 1.7
	4	50.9 $\pm$ 1.4	69.0 $\pm$ 3.0	55.7 $\pm$ 1.6
	5	50.8 $\pm$ 1.6	67.6 $\pm$ 3.6	55.4 $\pm$ 2.1
	5 (Auswahl)	48.6 $\pm$ 0.5	70.2 $\pm$ 2.4	54.3 $\pm$ 1.2
	5 (Max)	49.8 $\pm$ 1.2	75.2 $\pm$ 1.3	56.3 $\pm$ 1.1
	5 (Min)	47.5 $\pm$ 0.6	65.3 $\pm$ 1.2	51.9 $\pm$ 0.8

Tabelle D.1.: Gemittelte Werte für Precision, Recall und F1-Score von Few-Shot Methoden mit Llama3-70B über 20 Versuche mit angegebener Standardabweichung.

D.1.2. Temperatur-Parameter

Temperatur	0-Shot			5-Shot			Fine-Tuned		
	P	R	F1	P	R	F1	P	R	F1
0.1	43.8	70.9	50.5	52.4	77.3	58.4	71.7	83.6	77.1
0.2	43.6	70.2	50.3	51.7	76.2	57.8	72.3	84.6	77.9
0.3	43.2	69.9	49.8	51.7	76.7	57.9	71.2	83.8	76.9
0.4	43.8	69.9	50.4	52.4	77.2	58.7	72.1	84.5	77.7
0.5	43.0	69.7	49.7	52.0	77.4	58.1	71.6	84.5	77.5
0.6	42.7	71.0	49.8	51.7	75.1	57.2	71.7	84.6	77.6
0.7	42.8	70.9	49.7	52.3	76.6	58.4	71.9	84.7	77.7
0.8	43.2	70.4	50.1	51.6	75.6	57.3	72.5	84.3	77.9
0.9	42.6	70.9	49.5	50.9	76.4	57.4	71.8	84.1	77.4
1.0	43.5	70.5	50.1	51.7	76.1	57.7	72.6	84.5	78.0

Tabelle D.2.: Evaluation der Temperatur mit Llama3-70B Zero-Shot, Few-Shot und Fine-Tuning.

D.1.3. Chia-Korpus

Modell	Methode	P	R	F1	Ausgabefehler ($\bar{x}$, %)
Naive Greedy Match	Baseline	21.2	47.1	28.6	–
Llama3-70B	Zero-Shot	23.5	48.3	31.4	4.3
	5-Shot	35.6	42.2	38.5	3.6
	Fine-Tuned	28.7	54.4	36.8	0.6

Tabelle D.3.: Vergleich der auf dem LCT-Korpus optimierten Modelle auf dem Chia-Korpus.

D.2. Direkte Strukturierung

D.2.1. JSON-Fehler

Methode	Kategorie	Llama3-70B	Qwen2-72B	GPT-4o	Llama3-70B (FT)
2-Shot	Korrekt	95.3%	89.4%	96.5%	–
	JSON-Fehler	3.7%	7.2%	3.5%	–
	AST-Formatfehler	1.0%	3.5%	0.0%	–
10-Shot	Korrekt	79.2%	55.9%	89.4%	–
	JSON-Fehler	14.9%	36.6%	10.6%	–
	AST-Formatfehler	5.9%	7.4%	0.0%	–
Fine-Tuning	Korrekt	–	–	–	98.8%
	JSON-Fehler	–	–	–	0.2%
	AST-Formatfehler	–	–	–	1.0%

Tabelle D.4.: JSON-Fehler ausgewählter LLMs bei der Ausgabe von 404 JSON-Dateien.

D.2.2. Fine-Tuning Basisparameter

Kategorie	Parameter und Basiswerte
LoRA-Parameter	$r = 16$ $\text{lora_alpha} (\alpha) = 16$ $\text{lora_dropout} = 0$ $\text{use_rslora} = \text{False}$
Batch-Konfiguration	$\text{train_batch_size} = 8$ $\text{gradient_accumulation_steps} = 1$ $\text{eval_batch_size} = 8$
Trainingssteuerung	$\text{num_train_epochs} = 3$ $\text{warmup_ratio} = 0.0$ $\text{learning_rate} = 5 \times 10^{-5}$ $\text{weight_decay} = 0.0$ $\text{lr_scheduler_type} = \text{linear}$ $\text{max_grad_norm} = 1.0$
Optimierung	$\text{optim} = \text{AdamW (Hugging Face)}$ $\text{gradient_checkpointing} = \text{False}$
Evaluierung	$\text{evaluation_strategy} = \text{no}$ $\text{save_steps} = 500$

Tabelle D.5.: Standardeinstellung der Parameter für ein Fine-Tuning von LLMs mit SFT- und LoRA-Methoden.

D.2.3. Operatorenextraktion

Modell	Metrik	AND	AND*	OR	NOT	∅
SOTA (SciBERT)	P	54.1	-	85.1	74.3	71.2
	R	60.0	-	93.2	91.0	81.4
	F1	56.9	-	89.0	81.8	75.9
Llama3-70B (2-Shot)	P	24.3	89.2	12.4	1.6	12.8
	R	18.2	47.3	8.4	1.1	9.3
	F1	20.8	61.8	10.0	1.3	10.7
Qwen2-72B (2-Shot)	P	20.1	85.9	29.7	1.3	17.0
	R	30.2	74.4	8.3	1.3	13.3
	F1	24.1	79.8	13.0	1.3	12.8
GPT-4o (2-Shot)	P	15.7	82.8	55.5	1.4	24.2
	R	29.1	80.9	18.3	1.2	16.2
	F1	20.4	81.9	27.5	1.3	16.4

Tabelle D.6.: Auswertung der Operatorenextraktion für die 2-Shot Modelle.

Modell	Metrik	AND	AND*	OR	NOT	∅
SOTA (SciBERT)	P	54.1	-	85.1	74.3	71.2
	R	60.0	-	93.2	91.0	81.4
	F1	56.9	-	89.0	81.8	75.9
Llama3-70B (10-Shot)	P	26.7	89.5	27.0	5.7	19.8
	R	23.0	60.1	11.1	2.9	12.3
	F1	24.7	71.9	15.7	3.8	14.7
Qwen2-72B (10-Shot)	P	15.8	73.5	44.6	3.9	21.5
	R	46.1	78.9	11.1	3.3	20.2
	F1	23.6	76.1	17.8	3.6	15.0
GPT-4o (10-Shot)	P	17.1	83.3	56.2	3.2	25.5
	R	32.4	81.0	20.5	2.4	18.4
	F1	22.3	82.1	30.0	2.7	18.3

Tabelle D.7.: Auswertung der Operatorenextraktion für die 10-Shot Modelle.

D.2.4. Entitätserkennung mit Llama3-70B

Category	Entity	SciBERT	Llama3-70B (FT)
Clinical	Condition	78.4/83.3/80.8	90.2/87.8/89.0
	Contraindication	100.0/96.6/98.3	94.1/94.1/94.1
	Drug	73.4/80.9/77.0	86.5/87.2/86.8
	Encounter	58.3/74.4/65.4	79.5/87.3/83.2
	Observation	72.1/77.6/74.7	87.6/83.0/85.2
	Procedure	71.3/79.4/75.1	88.5/84.3/86.3
Demographic	Age	99.1/98.3/98.7	99.3/94.7/97.0
	Birth	100.0/62.5/76.9	87.5/100.0/93.3
	Death	100.0/20.0/33.3	80.0/80.0/80.0
	Family-Member	44.9/61.1/51.7	95.3/89.1/92.1
	Language	96.6/93.5/95.0	100.0/93.0/96.4
Logical	Negation	73.5/82.9/77.9	86.7/80.5/83.5
Qualifier	Assertion	62.1/65.8/63.9	71.3/72.3/71.8
	Modifier	58.5/65.4/61.8	83.3/80.2/81.8
	Polarity	81.4/79.5/80.4	84.9/76.5/80.5
	Risk	95.4/91.3/93.3	100.0/77.8/87.5
	Severity	86.5/94.1/90.2	94.1/91.3/92.7
	Stability	75.3/84.7/79.7	83.3/70.3/76.3
Temporal and Comparative	Eq-Comparison	85.3/89.3/87.3	92.8/88.5/90.6
	Criteria-Count	12.5/20.0/15.5	0.0/0.0/0.0
	Eq-Temporal-Period	82.6/86.3/84.4	92.8/86.1/89.3
	Eq-Temporal-Recency	50.0/66.6/57.1	92.0/92.0/92.0
	Eq-Temporal-Unit	98.2/99.4/98.8	97.9/95.7/96.8
	Eq-Value	96.4/97.1/96.7	96.5/93.7/95.1
Other	Location	73.4/78.3/75.8	88.6/88.6/88.6
–	Total	79.0/83.7/81.3	90.1/86.8/88.5 ↑11.1/↑3.1/↑7.2

Tabelle D.8.: Vergleich über die Metriken (Precision/Recall/F1-Score) für die Extraktion ausgewählter Entitäten zwischen dem SOTA-Modell SciBERT und dem Llama3-70B Modell mit Fine-Tuning (FT).

D.2.5. Metriken für alle LCT-Entitäten

Kategorie	P	R	F1	Kategorie	P	R	F1
Language	100.0	93.0	96.4	Location	88.6	88.6	88.6
Indication	100.0	100.0	100.0	Eq-Unit	93.4	89.1	91.2
Contraindication	94.1	94.1	94.1	Exception	91.7	84.6	88.0
Life-Stage- And-Gender	97.9	95.2	96.5	Polarity	84.9	76.5	80.5
Severity	94.1	91.3	92.7	Stability	83.3	70.3	76.3
Ethnicity	100.0	75.0	85.7	Eq-Operator	95.3	91.6	93.4
Eq-Temporal- Unit	97.9	95.7	96.8	Insurance	75.0	100.0	85.7
Eq-Value	96.5	93.7	95.1	Encounter	79.5	87.3	83.2
Allergy	95.8	92.0	93.9	Observation- Name	86.7	82.8	84.7
Acuteness	93.6	84.6	88.9	Observation	87.6	83.0	85.2
Risk	100.0	77.8	87.5	Negation	86.7	80.5	83.5
Study	85.3	89.1	87.1	Assertion	71.3	72.3	71.8
Other	78.4	87.0	82.5	Eq-Temporal- Recency	92.0	92.0	92.0
Eq-Comparison	92.8	88.5	90.6	Modifier	83.3	80.2	81.8
Age	99.3	94.7	97.0	Family-Member	95.3	89.1	92.1
Drug	86.5	87.2	86.8	Death	80.0	80.0	80.0
Eq-Temporal- Period	92.8	86.1	89.3	Immunization- Name	0.0	0.0	0.0
Organism-Name	100.0	28.6	44.4	Immunization	75.0	30.0	42.9
Organism	100.0	28.6	44.4	Provider	63.6	77.8	70.0
Condition	90.2	87.8	89.0	Specimen	88.9	72.7	80.0
Condition-Name	89.9	87.8	88.8	Criteria-Count	0.0	0.0	0.0
Drug-Name	86.1	87.4	86.8	Allergy-Name	88.9	100.0	94.1
Birth	87.5	100.0	93.3	Condition-Type	50.0	100.0	66.7
Procedure	88.5	84.3	86.3	Coreference	91.7	68.8	78.6
Procedure-Name	88.0	83.3	85.6	Code	100.0	100.0	100.0

Tabelle D.9.: Precision-, Recall- und F1-Scores für die Entitätsextraktion mit Llama3-70B mit Fine-Tuning für alle 50 Entitätstypen aus dem LCT-Korpus. Evaluiert wurde auf dem LCT-Testdatensatz.

E. Code

E.1. AST-Parser

```
1 def build_tree(data: Dict[str, str]) -> Node:
2     """ Builds a complete logic tree from a dictionary of criteria.
3     Args:
4         data: A dictionary where keys are irrelevant and
5               values are logical criteria strings
6     Returns: The root node of the complete logic tree
7     """
8     keys = list(data.keys())
9
10    if not data or not keys:
11        return Node(criteria="empty set")
12
13    # Parse each criteria string into a subtree
14    nodes = [parse_text(data[key]) for key in keys if
15             parse_text(data[key])]
16
17    if len(nodes) == 1:
18        return nodes[0]
19
20    # Combine all subtrees with AND operators
21    root = nodes[0]
22    for i in range(1, len(nodes)):
23        new_node = nodes[i]
24        temp = Node(operator='AND')
25        temp.left = root
26        temp.right = new_node
27        root = temp
28
29    return root
```

Listing E.1: build_tree() Funktion des logischen AST-Parsers.

```
1 import re
2
3 def parse_text(text: str) -> Optional[Node]:
4     """ Parses a string of logical criteria into a tree structure.
5     Args:
6         text: The input string containing logical criteria
7     Returns: The root node of the parsed tree
8     """
9     if not text:
10         return None
11
12     # Regular expression to match logical operators
13     pattern = r'\[AND\]|\[OR\]|\[NOT\]'
14     matches = list(re.finditer(pattern, text))
15
16     if matches:
17         first_match = matches[0]
18         operator = text[first_match.start():first_match.end()]
19         operator = operator.strip('[]')
20         before = text[:first_match.start()].strip()
21         after = text[first_match.end():].strip()
22
23         node = Node(operator=operator)
24         if operator == 'NOT':
25             if before:
26                 # Handle case where there's text before NOT
27                 before_node = Node(criteria=before)
28                 not_node = Node(operator='NOT')
29                 not_node.left = parse_text(after) if after else None
30                 combined_node = Node(operator='AND')
31                 combined_node.left = before_node
32                 combined_node.right = not_node
33                 return combined_node
34             else:
35                 node.left = parse_text(after) if after else None
36         else:
37             # For AND and OR operators
38             node.left = parse_text(before) if before else None
39             node.right = parse_text(after) if after else None
40         return node
41     else:
42         # If no operators found, create a leaf node
43         return Node(criteria=text)
```

Listing E.2: parse_text() Funktion des logischen AST-Parsers.

E.2. Entitäten-Integration

```
1 def add_entities_to_tree(node: Dict[str, Any],
2                           entities: List[Dict[str, str]],
3                           selected_types: List[str]) -> None:
4     """Recursively annotates a tree structure with entity information
5     based on the provided entities and selected entity types.
6     Args:
7         node: The current node in the tree structure to be annotated.
8         entities: A list of dictionaries with entities.
9         selected_types: A list of entity types.
10    Returns: None. The function modifies the node dictionary in place.
11    """
12    if 'raw_text' in node: # Check if the current node is a leaf node
13        raw_text = node['raw_text']
14        for entity in entities: # Iterate through all provided entities
15            if 'text' in entity and 'type' in entity:
16                entity_type = entity['type']
17                entity_text = entity['text']
18                if entity_type in selected_types and entity_text
19                    in raw_text:
20                    if entity_type not in node:
21                        node[entity_type] = []
22                    if entity_text not in node[entity_type]:
23                        node[entity_type].append(entity_text)
24    for key in node: # Recursively process child nodes
25        if isinstance(node[key], dict):
26            add_entities_to_tree(node[key], entities, selected_types)
27        elif isinstance(node[key], list):
28            for item in node[key]:
29                if isinstance(item, dict):
30                    add_entities_to_tree(item, entities, selected_types)
31    )
```

Listing E.3: Funktion zum hinzufügen von Entitätstypen als Keyes und Entitäten als Values im Array-Format. Die Entitäten werden als Blätter im AST unter den Ursprungstext eingefügt.

E.3. Verwendung des AST-Parsers

```
1 import os
2 import re
3 from functions import read_json, save_to_json
4 from Node import build_tree // AST-Parser
5
6 def criteria_to_json(text: str, file: str, ec_type: str) -> str:
7     """ Process the LLM output-text and save it as a JSON file. """
8     sentences = text.strip().split('\n')
9     sentences = [re.sub(r'\s+', ' ', sentence.strip()) for sentence in
10                 sentences if sentence.strip()]
11     data = {f"{ec_type}{i+1}": sentence for i, sentence in
12            enumerate(sentences)}
13     json_output = json.dumps(data, indent=2, ensure_ascii=False)
14     with open(file, "w", encoding="utf-8") as file:
15         file.write(json_output)
16     return json_output
17
18 def read_files_and_save_to_json(input_dir: str, output_dir: str):
19     """ Iterate over all input files and use criteria_to_json()
20         to process them. """
21     os.makedirs(output_dir, exist_ok=True)
22     for file_name in os.listdir(input_dir):
23         if file_name.endswith(("_exc.txt", "_inc.txt")):
24             with open(os.path.join(input_dir, file_name), 'r',
25                       encoding='utf-8') as file:
26                 file_content = file.read()
27                 ec_type = "EC" if file_name.endswith("_exc.txt") else "IC"
28                 output_file = file_name.replace(f"_{ec_type.lower()}.txt",
29                                                  f"_{ec_type.lower()}.json")
30                 output_path = os.path.join(output_dir, output_file)
31                 criteria_to_json(file_content, output_path, ec_type=ec_type)
32
33 def process_json_files(input_dir: str, output_dir: str):
34     """ Process JSON files to an Abstract Syntax Tree and save them
35         as JSON files. """
36     for file_name in os.listdir(input_dir):
37         if file_name.endswith(".json"):
38             file_path = os.path.join(output_dir, file_name)
39             file = file_name.split(".")[0]
40             data = read_json(file_path)
41             tree = build_tree(data)
42             tree_json = tree.to_dict()
43             ausgabe = json.dumps(tree_json, indent=2)
44             save_to_json(ausgabe, f'{output_dir}/{file}.json')
```

Listing E.4: Verarbeitung aller LLM-Ausgabedateien im Textformat in einem Ordner über den AST-Parser in der Node-Klasse.

E.4. JSON-Parser

```
1 def negate_exclusion_criteria(exc_data: Dict[str, Any]) ->
2     Dict[str, Dict[str, Any]]:
3     """ Wraps exclusion criteria in a NOT operation. """
4
5     return {"NOT": {"left": exc_data}}
6
7 def merge_criteria(inc_data: Dict[str, Any], exc_data: Dict[str, Any])
8     ->
9     Dict[str, Dict[str, Any]]:
10     """Combine inclusion and exclusion criteria with AND."""
11
12     return {"AND": {"left": inc_data, "right": exc_data}}
```

Listing E.5: JSON-Parser zur Verbindung von Einschlusskriterien mit Ausschlusskriterien mit NOT-Operator als Wurzelknoten.

E.5. AST-Plotter

```
1 import io
2 import matplotlib.pyplot as plt
3 from graphviz import Digraph
4
5 def parse_logic_to_tree(logic: dict, graph: Digraph,
6                         parent: Optional[str]=None,
7                         node_id: int=0) -> int:
8     """Parse logic to graph structure."""
9     if 'raw_text' in logic:
10         node_label = logic['raw_text']
11         graph.node(str(node_id), label=node_label, shape='box',
12                  style='filled', fillcolor='lightyellow')
13         if parent is not None:
14             graph.edge(parent, str(node_id))
15         return node_id
16     for key in logic:
17         if key in ('AND', 'OR', 'NOT'):
18             operator = key
19             node_label = operator
20             current_node_id = node_id
21             color = 'lightblue' if operator == 'AND' else 'lightgreen'
22                     if operator == 'OR' else 'lightcoral'
23             graph.node(str(current_node_id), label=node_label,
24                      color=color, fontcolor='black',
25                      style='filled', fillcolor=color)
26             if parent is not None:
27                 graph.edge(parent, str(current_node_id))
28             operands = logic[key]
29             node_id += 1
30             if 'left' in operands:
31                 left_id = parse_logic_to_tree(operands['left'], graph,
32                                             str(current_node_id), node_id)
33                 node_id = left_id + 1
34             if 'right' in operands:
35                 right_id = parse_logic_to_tree(operands['right'],
36                                             graph, str(current_node_id), node_id)
37                 node_id = right_id + 1
38     return node_id
39
40 def plot_and_save_graph(logic: dict, output_filename: str):
41     """Plot and save the Abstract Syntax Tree."""
42     graph = Digraph(format='png')
43     parse_logic_to_tree(logic, graph)
44     png_data = graph.pipe(format='png')
45     image = plt.imread(io.BytesIO(png_data), format='png')
46     plt.figure(figsize=(20, 15))
47     plt.imshow(image)
48     plt.axis('off')
49     plt.savefig(output_filename)
50     plt.show()
```

Listing E.6: AST-Plotter zur Visualisierung von Abstract Syntax Trees aus JSON-Dateien.

E.6. Prompting

```
1 import os
2 import transformers
3 from helper_functions import read_file, save_file,
4                               load_few_shot_examples
5
6 model_id = "meta-llama/Meta-Llama-3-70B-Instruct" # Model ID
7 n_shot = 5 # Number of Few-Shot examples
8
9 # Additional Prompting Strategies
10 COT = "Let's think through this carefully, step by step."
11 EP02 = "This is very important to my career"
12 EP05 = "Are you sure that's your final answer?"
13       It might be worth taking another look."
14 EP07 = "Are you sure that's your final answer? Believe in your
15       abilities and strive for excellence. Your hard work will
16       yield remarkable results."
17 EP09 = "Stay focused and dedicated to your goals. Your consistent
18       efforts will lead to outstanding achievements"
19 EP = EP02 + EP07 + EP09
20 role_prompt = read_file(f"PATH_TO_ROLE_PROMPT") # Load Role Prompt
21
22 prompt = read_file(f"PATH_TO_PROMPT") # Load Initial Prompt
23 #prompt = prompt + EP # Add EP or COT to Prompt
24
25 messages = [] # Initialize message sequence for the model
26 messages.append({"role": "system", "content": prompt}) # Insert
27       role_prompt or prompt
28
29 criterias, labels = load_few_shot_examples("PATH_TO_FEWSHOT_EXAMPLES")
30
31 # Add n Few-Shot examples
32 for k in range(n_shot):
33     messages.append({"role": "user",
34                     "content": prompt + criterias[k]})
35     messages.append({"role": "assistant",
36                     "content": labels[k]})
37
38 # Initialize Transformers pipeline
39 pipeline = transformers.pipeline(
40     "text-generation",
41     model=model_id,
42     model_kwargs={"torch_dtype": torch.bfloat16},
43     device_map="auto",
44 )
45
46 terminators = [
47     pipeline.tokenizer.eos_token_id,
48     pipeline.tokenizer.convert_tokens_to_ids("<|eot_id|>")]
49
50 # Process each file in the test dataset
51 first_call= True
52 for file_name in os.listdir("PATH_TO_TEST_FILES"):
53     test_file = read_text_file(study_path + file_name.split(".")[0])
54
55     if first_call:
56         messages.append({"role": "user",
```

```
56         "content": prompt + test_file}))
57     first_call = False
58     else:
59         messages[-1] = {"role": "user",
60                         "content": prompt + test_file}
61
62     prompt = pipeline.tokenizer.apply_chat_template(
63         messages,
64         tokenize=False,
65         add_generation_prompt=True
66     )
67
68     outputs = pipeline(
69         prompt,
70         max_new_tokens=2048,
71         eos_token_id=terminators,
72         do_sample=True,
73         temperature=0.5,
74         top_p=0.95
75     )
76
77     output = outputs[0]["generated_text"][len(prompt):]
78     save_file(output, f"{OUTPUT_PATH}{FILE_NAME}.txt")
```

Listing E.7: Quellcode für das Einfügen von Operatoren mit Llama3-Modellen über verschiedene Prompting-Methoden.

E.7. Fine-Tuning

```
1 import torch
2 from unsloth import FastLanguageModel
3 from trl import SFTTrainer
4 from transformers import TrainingArguments
5 from datasets import load_from_disk
6
7 model, tokenizer = FastLanguageModel.from_pretrained(
8     model_name = "meta-llama/Meta-Llama-3-70B-Instruct",
9     max_seq_length = 2048,
10    load_in_4bit = True,
11    token = "hf_..."
12 )
13
14 # LoRA Parameter
15 model = FastLanguageModel.get_peft_model(
16     model,
17     r = 256,
18     target_modules = ["q_proj", "k_proj", "v_proj", "o_proj",
19                      "gate_proj", "up_proj", "down_proj",],
20     lora_alpha = 256,
21     lora_dropout = 0.05,
22     bias = "none",
23     use_gradient_checkpointing = "unsloth",
24     use_rslora = True,
25 )
26
27 # Process Dataset
28 llama3_prompt = """Below is an instruction that describes a task,
29                     paired with an input that provides further context.
30                     Write a response that appropriately completes
31                     the request.
32
33     ### Instruction:
34     {}
35
36     ### Input:
37     {}
38
39     ### Response:
40     {}"""
41
42 def format_prompts(examples):
43     instructs = examples["instruction"]
44     inputs = examples["input"]
45     outputs = examples["output"]
46     texts = []
47     for instruction, input, output in zip(instructs, inputs, outputs):
48         text = llama3_prompt.format(instruction, input, output) +
49             tokenizer.eos_token
50         texts.append(text)
51     return { "text" : texts, }
52
53 pass
54
55 dataset = load_from_disk('PATH_TO_DATASET')
56 train_test_split = dataset['train'].train_test_split(test_size=0.1)
57 train = train_test_split['train'].map(format_prompts, batched=True)
```

```
57 test = train_test_split['test'].map(format_prompts, batched=True)
58
59 # SFT Parameter
60 trainer = SFTTrainer(
61     model = model,
62     tokenizer = tokenizer,
63     train_dataset = train,
64     eval_dataset = test,
65     dataset_text_field = "text",
66     max_seq_length = 2024,
67     dataset_num_proc = 4,
68     packing = True,
69     args = TrainingArguments(
70         per_device_train_batch_size = 2,
71         gradient_accumulation_steps = 8,
72         per_device_eval_batch_size = 4,
73         num_train_epochs = 10,
74         warmup_ratio = 0.1,
75         learning_rate = 5e-5,
76         fp16 = not torch.cuda.is_bf16_supported(),
77         bf16 = torch.cuda.is_bf16_supported(),
78         optim = "adamw_8bit",
79         weight_decay = 0.05,
80         lr_scheduler_type = "cosine",
81         gradient_checkpointing = True,
82         max_grad_norm = 1.0,
83         seed = 42,
84         output_dir = "OUTPUT_PATH",
85         logging_steps = 50,
86         evaluation_strategy = 'steps',
87         eval_steps = 100,
88         save_strategy = 'steps',
89         save_steps = 100,
90         metric_for_best_model = "eval_loss",
91         greater_is_better = False,
92         load_best_model_at_end = True,
93     ),
94 )
95
96 trainer.train()
97
98 model.save_pretrained("OUTPUT_PATH")
```

Listing E.8: Python-Code für das Fine-Tuning von Llama3-70B für die Operatorenextraktion.

E.8. Regex-Parser

```
1 import re
2
3 def naive_greedy_match(criteria_text: str) -> str:
4     """ Apply logical operators [AND], [OR], [NOT] to the given
5         criteria text based on specified patterns.
6     Args:
7         criteria_text: The input criteria text to be processed.
8     Returns:
9         str: The processed criteria text with logical operators.
10    """
11    patterns = {
12        'OR': [r'(?<=,\s)or\b', r'\band\s*/?\s*or\b', r'\bor\b'],
13        'AND': [r'\b(?:with|who|in addition|plus|and|but|that|despite|'
14               r'having)\b'],
15        'NOT': [r'\b(?:no|not|none|don`t|free|prevent|Inability|
16               impossible|lack|off|without|unable|naive|excluded|
17               absence)\b']
18    }
19
20    # Apply AND, OR, NOT patterns
21    for operator, pattern_list in patterns.items():
22        for pattern in pattern_list:
23            criteria_text = re.sub(pattern, f'[{operator}] \g<0>',
24                                   criteria_text)
25
26    # Apply additional OR patterns
27    criteria_text = re.sub(r',\s(?:!or\b)', r', [OR] ', criteria_text)
28    criteria_text = re.sub(r'/', r'/ [OR] ', criteria_text)
29
30    # Clean up erroneous OR occurrences
31    criteria_text = re.sub(r'\s*(\[OR\]\s*)+', ' [OR] ', criteria_text)
32    criteria_text = re.sub(r'\[OR\]\s*\[AND\]', '[AND]', criteria_text)
33    criteria_text = re.sub(r'\[OR\]\s*\[NOT\]', '[NOT]', criteria_text)
34
35    return criteria_text.strip()
```

Listing E.9: Regex-Parser für das Einsetzen von logischen Operatoren in den Eignungskriterientext.